

残酷共学 · 增量笔记

自 2026-08-16 23:02 BJT 后新增 91
篇

本次增量 · 带索引

点击目录条目可跳转

数据来源：intensivecolearn.ing 活动公开打卡

生成时间：2026-08-17 23:02 (BJT)

目录

主题章节 → 笔记索引 (点击可跳转)

01 AMM机制 (11 篇)	6
笔记 001: 链上套利 Day 10 笔记	6
笔记 002: Day 12 笔记 : 稳定币套利 —— 最"简单"也最难的套利	16
笔记 003: 今日案例 : TUT 代币暴涨暴跌的 CEX-DEX 价差套利	18
笔记 004: Crypto CEX—DEX、跨链库存套利, 以及充提/事件驱动套利: 系统性研究	19
笔记 005: August 16, 2026 · Day 12 / 21 · 时间分配: 先动手 60min → 后	49
笔记 006: 做了个面向新币 TGE 的链上流动性监控平台, 基于 TypeScript、React、Node.js	52
笔记 007: Day 12 打卡 链上套利残酷共學	52
笔记 008: 1. 什么是跨所套利	53
笔记 009: 学习笔记 : 如何查询链上 LP 的价格范围与手续费	56
笔记 010: 今日学习心得 :	61
笔记 011: Day 12 当前市场身份与盘口配对	61
02 LI.FI与跨链 (14 篇)	62
笔记 012: 链上套利残酷共学打卡 - 2026-08-16 (Day 12)	62
笔记 013: LI.FI 學習筆記 : 從交易意圖到合約安全驗證 (B1–B6)	63
笔记 014: 【Day 10 打卡】2026/8/14	75
笔记 015: 1\ 实操目标本阶段目标是完成从API接入到小额闭环验证的完整流程, 重点验证「价差 ≠ 利润」这一 . . .	76
笔记 016: 今天终于把 Hermes Agent 的配置问题基本处理完了, Terminal 和 Desktop	77
笔记 017: 8月16号\	77
笔记 018: 8月15日 (周六) —— 自动化测验 : 用Python脚本批量获取报价	78
笔记 019: 今天查阅了 LI.FI 官方 Transaction Status Tracking 文档。跨链交易	80
笔记 020: 今天重新梳理套利系统的核心 : 不同套利的收益逻辑虽然不同, 但发现机会的流程高度相似。真正决定系统价值的 . .	80
笔记 021: D14 (8/17) 第二周复盘 : 工作流跑通真实候选 + ARC 链研究	81
笔记 022: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践, Agent 仅负责	82

笔记 023: 鏈路統一的七個教訓	83
笔记 024: 1、监控系统发现套利机会，做了4笔交易，2笔成功（利润为0），2笔失败，损失1个点左右。	85
笔记 025: 套利基础学习打卡⑧ Inventory 与资金预部署：为什么真正的跨交易所套利不是“买完再转过去卖”	85
03 MEV与抢跑 (9 篇)	103
笔记 026: Day 10 (8/16) 打卡	103
笔记 027: Day 13 链上套利残酷共学	104
笔记 028: Day 13 (08/17) 学习总结，素材来自今日同学精华：	105
笔记 029: Day 11 (8/15) MEV 供应链：对手是谁	105
笔记 030: 链上套利残酷共学 · Day 13 打卡草稿：OEV 回收与 Chainlink SVR	105
笔记 031: 三角套利的核心，是利用三个交易对之间短暂出现的汇率失衡，通过连续三次兑换，最后回到最初的资产，并且最	108
笔记 032: 残酷共学打卡：原子 DEX 套利，也要先赢下区块排序	108
笔记 033: Day 10 MEV、Mempool、Searcher、Bundle 与 Block Buildin	110
笔记 034: 数据库：	140
04 做市与LP (6 篇)	142
笔记 035: 学习记录	142
笔记 036: 今日学习：Lighter 取消免费账户做市积分，本质为创收而非抑刷——周末美股无波动时该平台仍成交近	143
笔记 037: Day 13：全自動套利系統架構 v1.2	143
笔记 038: markdown	148
笔记 039: 学习记录	149
笔记 040: 参考	149
05 其他 (1 篇)	150
笔记 041: 1\、闭环资产验证 (M1 验收项)	150
06 套利框架 (31 篇)	150
笔记 042: 课程 Day 11 暂停旧课程路径，重新定义研究问题	150
笔记 043: markdown	151
笔记 044: 打卡笔记：周末效应与杠杆 ETF 偏离规律（监控第 20-22 次）	151

笔记 045: 本地做了下面这些测试，逐渐熟悉所有流程	152
笔记 046: Day 9 比價工具邁向多鏈：一個地址寫錯的教訓，與「單一事實來源」重構	152
笔记 047: 分析 Gas、滑点与手续费	153
笔记 048: 具体任务	153
笔记 049: D09 稳定币套利	154
笔记 050: 	154
笔记 051: 今天做了什么	155
笔记 052: Day 13 2026.08.17 Monad 成本模型：为什么 gasLimit 不能乐观	155
笔记 053: 探讨在 cex-dex 套利中我一直存疑的问题，Day 11：【股票/RWA】	156
笔记 054: 今天结束时的真实状态	156
笔记 055: CEX—CEX 套利赚的是同一枚币在两所的可成交价差，属于库存套利，非原子。	158
笔记 056: https://github.com/ZeroNullNone/onchain_arbitrage_	158
笔记 057: 资金费对冲:从零到「我该从哪开始」	158
笔记 058: RobinHood Lighter ↔ Lighter 双市场价差套利：可行性与复现路线	164
笔记 059: 2026-08-17 學習打卡	168
笔记 060: 链上套利残酷共学 · Day 12 心得	169
笔记 061: Day 12：两次 Quote 只差 2 秒，但回程条件在预计到达时已过去 1,078 秒。	169
笔记 062: 2026-08-17 共学 Day 13 深度案例研究2：资金费率套利 投入约 1.	170
笔记 063: Day 13 观测缓冲重算净收益门槛	170
笔记 064: 链上套利共学 · Day13 打卡笔记 (08/17)	171
笔记 065: Day 13 MEV 生态分层、原子交易与对冲	171
笔记 066: Day 13 (08-17)：90 篇增量笔记批次消化 + 执行层两连	172
笔记 067: 2026-08-17 学习打卡	172
笔记 068: day13	173
笔记 069: Day12 跨所、跨期、展期套利	173
笔记 070: Day 10 中場盤點：九天學到的東西，拼成一張可用的地圖	174
笔记 071: 【Day 17 打卡 · 2026-08-17】试跑迭代与信号收敛	175

笔记 072: markdown	175
07 学习计划 (5 篇)	175
笔记 073: 链上套利残酷共学 · Day 11 心得	175
笔记 074: Day 12 LTV、NAV 与池子深度：为什么扩大规模不是越多越好（链上套利共学）	176
笔记 075: Day 13 — 资金管理与风险控制	176
笔记 076: Day 12 · 2026-08-17 · 把周末两天积压的小坑一次清掉:Day 11 自我订正、I	176
笔记 077: 链上套利残酷共学 2026-08-17 打卡	180
08 实盘与数据 (7 篇)	180
笔记 078: 2026-08-16 — Day 12(Week 2 第 5 天):ADR 扩散 vs 收敛 + G	180
笔记 079: 技术核心：严重依赖程序量化系统（Python/Go/Rust）、低延迟 WebSocket 行情推	181
笔记 080: Day 13 · 2026-08-17 · Paper Trading 方案：把策略变成可检验的规则	182
笔记 081: 链上套利共学 Day 13 Gate CrossEx 跨所统一账户 + 开源本地交易终端	183
笔记 082: 链上套利残酷共学 · Day 13 打卡（2026-08-17）· 8/15+8/16 增量笔记精华	184
笔记 083: 新兴 Crypto 套利机会：Tokenized Stock × Hyperliquid HIP-3	186
笔记 084: 2026-08-17 — Day 13(Week 2 第 6 天):Dune 数据管道上手 + Ga	205
09 风险与安全 (7 篇)	206
笔记 085: binance三角套利延迟优化	206
笔记 086: Day 12 · 把机会评分变成可审计的决策记录	208
笔记 087: 2026-08-16 学习打卡	209
笔记 088: 今天最大的进展，是把研究从“固定历史路径是否还能成立”，推进到了“出现信号的新候选路径，是否值得独立...	
笔记 089: Day 11 (8/17) 打卡	211
笔记 090: 学习笔记：模块 5 第 2 天，安全执行框架	211
笔记 091: Day 12 · 把机会评分变成可审计的决策记录	212

笔记 001: 链上套利 Day 10 笔记

作者 : Harper | 时间 : 2026-08-16 15:20 UTC | 字数 : 5169 | 评分 : 90

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利 Day 10 笔记

今日主题 : Price Impact、Slippage 与 Minimum Amount Out

今天主要学习 :

Price Impact (价格冲击)

Slippage (滑点)

Price Impact 和 Slippage 的区别

为什么套利特别害怕滑点

minimumAmountOut 是什么

为什么 minimumAmountOut 不满足时要 Revert

如何把这些概念放进完整的套利系统

一、Price Impact 是什么？

Price Impact :

你的交易本身改变了 AMM 池子的价格。

AMM 使用 :

$x \times y = k$

例如池子 :

300,000 USDC

100 ETH

初始价格 :

$300,000 \div 100$

$\backslash =$

3000 USDC / ETH

如果我进行一笔很大的买入 :

投入大量 USDC

↓

拿走大量 ETH

池子变成 :

USDC ↑

ETH ↓

为了维持 :

$x \times y = k$

ETH 的池内价格就会上升。

因此 :

我自己买得越多,我自己的交易越会把价格推高。

这就是 Price Impact。

二、为什么大额交易的平均成交价格会变差？

假设开始时 :

ETH = 3000 USDC

你开始大量买 ETH :

第一部分 :

≈ 3000

第二部分：

≈ 3010

第三部分：

≈ 3020

第四部分：

≈ 3030

所以你不可能让全部 ETH 都按照：

3000 USDC

成交。

最终：

平均成交价格

\>

交易开始时价格

原因就是：

你的交易沿着 AMM 曲线执行，同时不断改变池子价格。

三、Price Impact 的本质

记住：

Price Impact

\=

你的交易本身造成的价格变化

所以：

Trade Size ↑

↓

Price Impact ↑

↓

平均成交价格恶化

通常：

池子越小，大额交易造成的 Price Impact 越明显。

四、Slippage 是什么？

Slippage：

中文：

滑点

简单理解：

预期成交结果和实际成交结果之间的偏差。

例如：

Quote 告诉你：

10000 USDC

→

3.30 WETH

但是实际执行：

10000 USDC

→

3.25 WETH

实际得到的 Token 少于预期。

这就是滑点造成的实际成交偏差。

五、Price Impact 和 Slippage 的区别

这是今天最重要的知识之一。

Price Impact

重点：

你的交易本身对价格造成的影响。

例如：

开始：

ETH = 3000

你进行大额买入

↓

池子价格：

3000

↓

3010

↓

3020

↓

3030

这是你的交易自己造成的。

Slippage

重点：

预期成交结果和最终实际成交结果之间的偏差。

例如：

预计：

3.30 ETH

实际：

3.25 ETH

两者差异就是实际成交中的滑点。

六、可以这样简单记

Price Impact

\=

我的交易把价格推走了多少

Slippage

\=

我原本预期的成交结果和最终实际成交结果差多少

二者相关，但不是同一个概念。

七、为什么套利特别害怕 Slippage？

套利本身赚的通常就是：

很小的价格差。

例如：

DEX A：

3000 USDC

DEX B：

3020 USDC

价差：

3020 - 3000

\=

20 USDC

如果执行过程中因为价格变化损失：

15 USDC

剩下：

20 - 15

\=

5 USDC

再扣：

DEX Fee

Gas

Flash Loan Fee

其他成本

最后可能：

亏损

所以：

有价差 ≠ 有利润。

八、套利真正应该计算什么？

不能只看：

DEX A Price

vs

DEX B Price

应该考虑：

价格差

\-

Price Impact

\-

DEX Fee

\-

Slippage影响

\-

Flash Loan Fee

\-

Gas

\-

其他成本

\=

Expected Net Profit

真正需要关注的是：

Expected Net Profit (预计净利润)

九、为什么需要 minimumAmountOut？

假设：

10000 USDC

↓

DEX A

Quote 告诉我们：

预计得到：

3.30 WETH

如果智能合约完全不设置保护：

“你给我多少 WETH 都行。”

那么市场突然变化时，可能出现：

3.30 WETH

↓

3.25 WETH

↓

2.90 WETH

智能合约仍然可能继续执行。

这对于套利机器人非常危险。

十、什么是 minimumAmountOut ？

minimumAmountOut ：

这次 Swap 我最低允许自己得到多少 Token。

例如：

Quote ：

3.30 WETH

设置：

minimumAmountOut ：

3.27 WETH

意思：

至少给我 3.27 WETH。

十一、minimumAmountOut 的执行逻辑

如果实际得到：

3.29 WETH

因为：

$3.29 > 3.27$

所以：

交易成功

如果实际得到：

3.25 WETH

因为：

$3.25 < 3.27$

不满足最低要求：

Revert

于是：

这笔交易回滚。

十二、minimumAmountOut 是一种交易保护机制

可以把它理解成：

给 Swap 设置一道底线。

例如：

Quote ：

3.30 WETH

最低接受：

3.27 WETH

那么：

实际 ≥ 3.27

→ Success

实际 < 3.27

→ Revert

十三、它和 Day 8 的 Revert 有什么关系？

Day 8：

利润不满足 Minimum Profit

↓

Revert

Day 10：

实际 Output

<

minimumAmountOut

↓

Revert

所以智能合约可以同时设置多个保护条件：

Swap A

↓

检查 minimumAmountOut

↓

Swap B

↓

检查 minimumAmountOut

↓

归还 Flash Loan

↓

检查 Minimum Profit

↓

Success

只要关键条件不满足：

Revert

十四、完整套利流程已经越来越完整

目前学到这里：

发现 DEX A 和 DEX B 存在价差

↓

确认 Token

↓

确认 Contract Address

↓

处理 Decimals

↓

检查流动性

↓

获取 Quote

↓

确定 Trade Size

↓
计算 Price Impact
↓
考虑 Slippage
↓
考虑 DEX Fee
↓
考虑 Flash Loan Fee
↓
考虑 Gas
↓
计算 Expected Net Profit
↓
达到执行标准
↓
Flash Loan
↓
DEX A Swap
↓
检查 minimumAmountOut
↓
DEX B Swap
↓
检查 minimumAmountOut
↓
归还 Flash Loan
↓
检查 Minimum Profit
↓
Success

如果关键条件不满足：

Revert

十五、Price Impact 在套利中的作用

假设：

DEX A：

3000

DEX B：

3030

看起来：

价差：

30

但是如果你投入大量资金：

Trade Size ↑

↓
DEX A Price Impact ↑
↓

买入成本变高

同时：

Trade Size ↑

↓

DEX B Price Impact ↑

↓

卖出价格变低

最终：

套利空间

↓

被自己的交易逐渐吃掉

所以：

套利机器人不能只寻找最大资金，而要寻找最佳交易规模。

这就是我们昨天开始接触的：

Optimal Trade Size

十六、为什么不能无限放大 Flash Loan？

错误想法：

价差 = 1%

借 100万

→

赚1万

借 1000万

→

赚10万

借1亿

→

赚100万

实际情况：

交易规模增加

↓

Price Impact增加

↓

成交价格恶化

↓

套利空间减少

↓

利润增长放缓

↓

达到最佳规模

↓

继续扩大

↓

利润下降

↓

甚至亏损

所以：

Flash Loan 解决的是“本金不够”，不是“套利一定赚钱”。

十七、今天的核心公式

1. AMM

$$x \times y = k$$

2. 套利净利润

Net Profit

\=

Final Output

\-

Principal

\-

DEX Fees

\-

Flash Loan Fee

\-

Gas

\-

其他执行成本

3. Swap 最低输出保护

Actual Amount Out

≥

Minimum Amount Out

满足：

Success

不满足：

Revert

十八、今天最重要的几个概念

概念

含义

Price Impact

交易本身对池子价格造成的影响

Slippage

预期成交结果与实际成交结果之间的偏差

Trade Size

交易规模

Minimum Amount Out

Swap 最低允许获得的 Token 数量

Expected Net Profit

扣除各种成本后的预计净利润

Revert

条件不满足时回滚交易

Optimal Trade Size

能使净利润最大化的交易规模

十九、今日理解题答案

题目：

Quote：

10000 USDC

→

3.30 WETH

设置：

minimumAmountOut：

3.27 WETH

实际执行：

3.25 WETH

因为：

$3.25 < 3.27$

所以正确答案：

B. Revert

原因：

Actual Amount Out

<

Minimum Amount Out

交易不满足最低输出要求，因此智能合约应该拒绝这次 Swap。

Day 10 一句话总结

Price Impact 是交易本身改变 AMM 池子价格造成的影响，Slippage 是预期成交结果与实际成交结果之间的偏差。套利利润本来就很薄，因此必须考虑 Price Impact、Slippage、DEX Fee、Gas 和 Flash Loan Fee，并通过 minimumAmountOut 和 Minimum Profit 等条件保护交易，条件不满足就 Revert。

目前学习路线

AMM

↓

$x \times y = k$

↓

Liquidity Pool

↓

DEX

↓

Price

↓

Quote

↓

Price Impact

↓

套利

↓

Atomic Transaction

↓

Revert

↓

Flash Loan

↓

Slippage

↓

Minimum Amount Out

↓

Optimal Trade Size

这条线已经开始从“理解 DeFi”进入真正的套利策略与执行机制了。

笔记 002: Day 12 笔记：稳定币套利 —— 最“简单”也最难的套利

作者：GeoGuO | 时间：2026-08-16 15:25 UTC | 字数：5463 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 笔记：稳定币套利 —— 最“简单”也最难的套利

日期：2026 年 8 月 16 日

进度：理解了稳定币套利的特殊之处 + 用 The Graph 查了 Curve 稳定币池的币种比例 + 算了一笔 USDC/USDT 互换的盈亏 + 深入理解了“脱锚是暂时的还是永久的”这个致命问题 + 搞懂了 Curve 的 AMM 曲线和 Uniswap 的本质区别

一、稳定币套利为什么“简单”又“难”

说它简单，是因为道理太直白了：USDC 和 USDT 都是稳定币，理论上都应该严格等于 1 美元。如果某天 USDC 只值 0.998 美元、USDT 值 1.001 美元，你只要把打折的 USDC 换成 USDT 再按溢价卖出，就赚了 0.3%。没有任何复杂路径，不需要跨链，不需要闪电贷，两笔 swap 就完事。

说它难，是因为这个“简单”背后藏着两个致命的陷阱：第一，价差极小（通常 < 0.5%），导致你必须动用大资金才能赚到有意义的绝对收益，而大资金又放大了风险暴露。第二，也是最要命的——你凭什么认定“USDC 折价”是暂时的市场噪音，而不是 USDC 真的要脱锚了？如果是后者，你的“套利”就是“接飞刀”，接的是正在下落的刀。

Day 12 的核心，就是搞清楚这两件事：怎么算稳定币套利的账，以及怎么判断脱锚的性质。

二、稳定币套利的账怎么算

先算一个理想情况下的例子，体会一下“小价差 × 大资金”的逻辑。

假设 USDC 当前价格 \$0.998，USDT 当前价格 \$1.001。你用 10 万 USDC 换成 USDT，再卖出 USDT 换成美元。

第一步：10 万 USDC 换 USDT。因为 USDC 便宜、USDT 贵，如果按 1:1 的锚定价换，你能换到 10 万 USDT。但市场上 USDC 是折价的，所以实际兑换会有偏差。简化理解：如果有一个池子按真实市场价定价，10 万 USDC 换到的 USDT 数量 = $100000 \times (0.998 / 1.001) \approx 99700$ USDT。

第二步：99700 USDT 换成美元。USDT 溢价， $99700 \times 1.001 \approx 99800$ 美元。

等等，这样算下来 10 万 USDC → 99800 美元，反而亏了 200。这个例子说明，如果只是“把折价的币换成溢价的币”，中间没有额外的价差来源，你赚不到钱——市场已经把折价和溢价都反映在兑换比例里了。

真正能赚钱的稳定币套利，赚的不是“USDC 和 USDT 之间的价差”本身，而是“某个池子里的定价暂时偏离了它们各自的真实市场价”。也就是说，机会出现在某个 Curve 稳定币池因为大额赎回导致内部比例失衡、池子里的 USDT 相对 USDC 被临时低估的时候。这时候你用 USDC 从这个失衡的池子里换 USDT，换到的比例比“真实市场价”更划算——这才是套利。

所以稳定币套利的正确姿势是：盯住稳定币池（尤其是 Curve 的池子），当池子里的币种比例严重失衡时，池子的 AMM 定价会偏离 1:1 的锚定价，这时池子内的兑换比例就产生了套利空间。

三、Curve 的稳定币池：为什么它和 Uniswap 不一样

Day 1 学的 Uniswap 用的是恒定乘积公式 $x \times y = k$ 。这个公式对波动资产（ETH/USDC）很合适，但对稳定币（USDC/USDT）就不友好——因为恒定乘积公式意味着价格波动时会有巨大的滑点，而稳定币价格本应该几乎不动。

Curve 专门为稳定币设计了一种不同的 AMM 曲线，核心思想是：在价格接近 1:1 的区间内，曲线非常平坦（几乎零滑点），让稳定币之间的兑换可以接近 1:1 地完成；只有当币种比例严重失衡时，曲线才开始变陡，价格才显著偏离锚定价。

这个设计的意义在于：正常状态下，Curve 池子里的 USDC 换 USDT 几乎就是 1:1，滑点极小，手续费极低（通常 0.04%）。这让 Curve 成为稳定币流动性的核心基础设施——所有人都在 Curve 上换稳定币，因为这里最便宜、最滑点最小。

但对套利者来说，Curve 池子的关键信号是“币种比例”。一个健康的 3pool（USDC/USDT/DAI 三种币）里，三种币的比例应该是大致均衡的。如果某天 USDC 占了 60%、USDT 只占 15%、DAI 占 25%，说明有大量的人用 USDT 和 DAI 换走了 USDC——市场在抢购 USDC（可能因为某个事件让大家更信任 USDC 或者更不信任其他稳定币）。这时候池子的 AMM 定价会把 USDC 的价格推高（因为池子里 USDC 变少了，物以稀为贵），把 USDT 的价格压低。

如果你判断这个失衡是暂时的，你就可以反向操作：用 USDC 换 USDT，等池子比例恢复后，USDT 会升值回锚定价，你再换回来，赚取差价。

四、动手：查 Curve 3pool 的币种比例

用 The Graph 查 Curve 的 3pool。Curve 有自己的 subgraph (Curve Finance 官方部署)，查 USDC/USDT/DAI 池子的当前 balances。

查询思路 (示意)：

```
{
  pools(where: { name_contains: "3pool" }) {
    id
    name
    balances
    coins
  }
}
```

返回的 balances 数组是池子里每种币的数量 (最小单位)。把它换算成美元，就能看出三种币各自占多少比例。

假设查到当前 3pool 的状态 (示意数字)：

USDC 余额：6000 万 (占 60%)

USDT 余额：1500 万 (占 15%)

DAI 余额：2500 万 (占 25%)

总池子：1 亿

这个比例严重失衡了——USDC 占了 60%，远超 1/3 的均衡位置。说明大量 USDT 和 DAI 被换成了 USDC，USDC 在池子里是“被抢购”的状态。

接下来我站在当前比例下算一笔账：用 10000 USDC 在这个池子里能换多少 USDT？

因为池子严重失衡 (USDC 太多、USDT 太少)，AMM 曲线已经偏离了平坦区，进入了陡峭区。用 USDC 换 USDT 的兑换比例不再是 1:1，而是 USDT 相对 USDC 变得更贵了。假设算出来 10000 USDC 只能换到 9800 USDT (USDT 溢价 2%)。

这个 2% 的溢价，就是“池子失衡”造成的偏离。如果我认为这个失衡是暂时的 (比如过几天那波抢购 USDC 的恐慌过去了，池子会恢复平衡)，那么现在用 USDC 换 USDT 是划算的——等池子恢复后，9800 USDT 能换回 9800 USDC，加上你持有的 USDT 本身回到锚定价，你赚的就是那 2% 的溢价差。

但注意：如果这 2% 的溢价不是暂时的，而是 USDT 真的出了问题 (脱锚了)，那你用 USDC 换 USDT 就是往火坑里跳——USDT 可能继续跌。

五、致命问题：脱锚是暂时的还是永久的？

这是稳定币套利和所有其他套利最大的区别。DEX 间套利的价差，你可以比较确定地说“这是暂时的，套利者会把它抹平”。但稳定币的折价/溢价，可能是暂时的市场噪音，也可能是永久性的脱锚信号。区分这两者，是稳定币套利生死攸关的判断。

两个真实的教训：

UST/LUNA (2022 年 5 月)：UST 是算法稳定币，本应锚定 1 美元。当它开始脱锚时，很多人以为“这是暂时的，套利一下”，于是 0.9 美元买入 UST，期望它回归 1 美元。结果 UST 一路崩盘到 0.01 美元，那些“套利者”血本无归。这是稳定币套利历史上最惨烈的一课——脱锚可以是不可逆的死亡螺旋。

USDC (2023 年 3 月，硅谷银行倒闭)：USDC 因为部分储备金存在硅谷银行而短暂脱锚，一度跌到 0.87 美元。但这次脱锚是暂时的——Circle 声明储备金安全，USDC 很快回归 1 美元。那些在 0.87 买入 USDC 的人赚了约 15% 的“套利”收益。这是成功的稳定币套利案例。

同样是“稳定币跌了”，一个是死亡螺旋，一个是黄金坑。区别在哪？

区别在于“脱锚的根本原因”。UST 脱锚是因为它的抵押品设计有缺陷 (用不稳定的 LUNA 抵押)，是结构性的、不可逆的。USDC 脱锚是因为一个外部事件 (银行倒闭) 造成的暂时性恐慌，Circle 的实际储备是充足的，是可逆的。

所以判断脱锚性质的核心问题是：这个稳定币的抵押品 (或者它维持锚定的机制) 是否依然完好？如果是，脱锚就是暂时的，可以套利；如果抵押品出了问题、或者锚定机制本身失效了，脱锚就是永久的，套利等于自杀。

这个判断需要你真正理解每个稳定币的运作机制，而不是只看价格——价格告诉你“它现在折价了”，但不会告诉你“它为什么折价、还会不会回来”。

六、稳定币套利的三重成本

算一笔完整账 (回到 Curve 3pool 失衡的例子，用 10000 USDC 换 USDT，USDT 溢价 2%)：

毛收益 = 2% 溢价 × 10000 = \$200 (假设池子恢复平衡后 USDT 回到 1:1)

扣协议费：Curve 的稳定币池 swap fee 通常是 0.04%，两笔 (换过去再换回来) 约 \$8。

扣 Gas：在 Ethereum 主网上一笔 Curve swap 的 Gas 可能 \$15-50，两笔 \$30-100。在 Arbitrum 上的 Curve 池子 (如果存在) Gas 便宜得多，两笔可能 \$1-2。

扣滑点：因为池子已经失衡，你的交易本身会进一步加剧失衡，实际成交价可能比报价差

0.1%-0.5%（取决于你的交易规模占池子的比例）。10000 相对 1 亿的池子，冲击约 0.01%，可忽略。但如果你是 100 万资金，冲击就是 0.1% 量级。

扣资金占用成本：稳定币套利通常需要持仓几天到几周等待池子恢复，这期间资金占用成本（Day 3 学的）不可忽略。10 万 $\times$ 4% 年化 $\div$ 365 $\times$ 7 天 $\approx$ \$77（一周）。

扣脱锚风险溢价：这是最不确定的一项。如果判断错了（脱锚是永久的），损失不是 2%，而是 20% 甚至 90%。

净收益 = 200 - 8 - 30（Gas 取主网中间值）- 77（一周资金占用）- 风险溢价 = 约 \$85 减去风险溢价。

这个账算下来，稳定币套利在理想情况下（脱锚是暂时的、判断正确）能赚几十到几百美元，但风险是判断错误可能亏掉本金的大半。盈亏比并不如表面那么诱人。而且这还是 10 万资金的量级——如果只有 1 万，毛收益才 \$20，扣完各种成本基本是负的。

七、今天的三个核心收获

第一，稳定币套利赚的不是“稳定币之间的价差”，而是“池子失衡导致的定价偏离”。健康的稳定币之间没有套利空间（市场已经定价好了）。套利机会出现在某个池子因为大额赎回而内部比例失衡、AMM 定价暂时偏离锚定价的时候。

第二，Curve 是理解稳定币套利的钥匙。它的 AMM 曲线专门为稳定币设计——锚定价附近几乎零滑点，失衡时曲线变陡。币种比例（balances）是判断池子是否失衡、偏离有多大的核心指标。

第三，脱锚性质的判断比价差计算重要一百倍。UST（死亡螺旋）和 USDC（黄金坑）的对比说明：同样的“折价”，结果可能天差地别。判断脱锚是暂时的还是永久的，要看抵押品和锚定机制是否完好，而不是看价格本身。这是稳定币套利里唯一真正重要的风险判断。

八、今天的三个问题

1. Curve 的池子失衡到什么程度才值得套利？有没有一个量化的阈值（比如某个币占比超过 45% 就是失衡信号）？

2. 除了 Curve，还有哪些稳定币池子可以监控？Balancer 的稳定币池、以及各个借贷协议里的稳定币利率，是不是也能提供套利信号？

3. 怎么用链上数据判断一个稳定币的“锚定机制是否完好”？能不能查它的抵押品储备、赎回机制、或者链上的发行/销毁事件来辅助判断？

九、稳定币套利这条路的现实判断

稳定币套利是六种套利类型里，看起来门槛最低（道理最简单）但实际风险最隐蔽的一种。它不需要速度（不像三明治攻击和三角套利），不需要复杂算法，理论上普通人都能参与。

但它的陷阱在于：你面对的不是“价差会不会消失”（DEX

间套利的问题），而是“这个币会不会归零”（稳定币套利的问题）。前者输的是时间，后者输的是本金。UST 的教训摆在那里——无数“聪明人”在 UST 脱锚时抄底套利，最后一起被埋了。

对个人来说，稳定币套利最稳妥的参与方式可能是：只在极端明确的、可逆的脱锚事件里介入（比如大机构背书、储备金可验证、只是暂时恐慌），并且严格控制仓位（用你亏得起的小钱），永远做好“判断错了就认栽”的准备。而不是把它当成一个可以反复薅的“固定收益”。

Day 13 会学清算——那又是一个“确定性奖励但竞争惨烈”的赛道。稳定币套利和清算的共同点是：道理都很直白，但“知道”和“能安全地做到”之间，隔着一整套风险控制的能力。

笔记 003: 今日案例：TUT 代币暴涨暴跌的 CEX-DEX 价差套利

作者：Zhou Junjie | 时间：2026-08-16 15:29 UTC | 字数：1114 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日案例：TUT 代币暴涨暴跌的 CEX-DEX 价差套利

分析 TUT（Tutorial，BSC）在 8 月 9 日左右的波动带来的套利空间。

波动概况（pump & dump）

8 月 7 日 \$0.0365 $\rightarrow$ 8 月 9 日最高 \$0.3056（8.4 倍）

然后崩回 8 月 14 日 \$0.0286（-90%）

套利核心：8 月 9 日 15:00 的「核爆一小时」

| 场所 | 最高 | 最低 | 单小时振幅 | 成交量 |

|-----|-----|-----|-----|

| 币安（CEX 订单簿） | 0.3056 | 0.0913 | 234% | \$4750 万 |

| PancakeSwap（DEX AMM） | 0.2949 | 0.1036 | 184% | \$791 万 |

CEX 和 DEX 在这一小时严重脱节，价差最大 13.5%。

价差来源的本质

1. 订单簿 vs AMM 响应速度差：暴跌时币安砸穿到 0.0913，PancakeSwap 的 AMM 滞后在 0.1036
2. 流动性不对称：DEX 成交量只有 CEX 的 1/6，天然反应慢

三层套利空间

1. CEX-DEX 价差套利 (13.5%)：币安低价买 → PancakeSwap 高价卖，但需提前备好双边库存（临时提币来不及）
2. 跨 DEX 套利：PancakeSwap vs Uniswap（BSC 同链，Uniswap 流动性仅 PCS 2%，脱节更严重，可闪电贷）
3. MEV 夹子套利：234% 振幅 + 巨量市价单，是夹子攻击的完美猎物

与前两个案例的对照

| 案例 | 定价脱节来源 |

|-----|-----|

| Solana MEV (EURC) | 低流动性池定价错误 |

| sNUSD (Morpho) | 协议估值 vs 市场价 |

| TUT (本次) | 订单簿 vs AMM 响应速度差 |

三者本质相同：同一资产在不同场所的定价脱节，只是脱节原因和套利工具不同。

关键收获

1. 波动套利找「定价脱节」，量化用同时时间戳 K 线对比 (onchainos + Binance API)
2. 成交量对比揭示流动性不对称 (DEX/CEX 比值越低，滞后越严重)
3. 理论价差 ≠ 实际利润：受池子深度和提币延迟限制
4. pump & dump 里方向性抄底必套牢，只有吃价差的套利者赚钱

笔记 004: Crypto CEX—DEX、跨链库存套利，以及充提/事件驱动套利：系统性研究

作者：SimonQuant | 时间：2026-08-16 15:56 UTC | 字数：18230 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Crypto CEX—DEX、跨链库存套利，以及充提/事件驱动套利：系统性研究

一、核心结论

本研究主要覆盖四类策略：

1. CEX—DEX 库存套利
2. 跨链库存套利
3. 实时 Bridge 跨链套利
4. 充提状态 / 公告事件驱动套利

最重要的结论是：

预置库存套利应该是整个体系的核心；实时跨桥和充提事件套利更适合作为增强策略。

原因很简单：

如果交易时还需要：

买币 → 提币 → 等链上确认 → 等交易所充值到账 → 再卖币

那么你实际上承担了整个转账期间的价格风险。

如果两边提前拥有库存：

低价市场买入 + 高价市场同时卖出

则交易过程基本可以做到市场中性，之后再慢慢进行库存再平衡。

因此：

预置库存套利 > 实时搬币套利

这里的“>”表示作为长期系统化策略的优先级，而不是单笔利润大小。

置信度：高。

第二个重要结论：

真正困难的地方通常不是发现价差，而是执行、库存、状态监控和异常处理。

一个成熟系统需要同时管理：

CEX Order Book

DEX Quote

RPC

Gas

Bridge
Deposit Status
Withdrawal Status
Wallet
Nonce
API
Inventory
Reconciliation
Risk Limit
Incident Response

所以真正有价值的系统不是：

Spread Scanner

而是：

Market Data + Execution + Inventory + Transfer State + Risk + Reconciliation

置信度：高。

第三个结论：

网页上看到的价差不能直接理解为利润。

真正的利润应该是：

净利润

可执行价格差

CEX 手续费

DEX 手续费

Gas

Bridge 费用

提币手续费

滑点

对冲成本

资金占用成本

失败概率损失

尾部风险准备

所以真正应该监控的是：

Executable Net Edge

而不是：

Displayed Spread

置信度：极高。

二、套利机会分类

1. CEX—DEX 库存套利

最典型的结构：

Binance：

USDT 100,000\

ETH 20

Arbitrum：

USDC 80,000\

ETH 15

假设：

Uniswap ETH = 3,000

Binance ETH = 3,018

那么：

Uniswap 买入 ETH

同时：

Binance 卖出 ETH

毛价差：

$(3018 - 3000 \times (1 + 0.02)) / 3000$

$\approx 0.6\%$

也就是：

60 bps

但是最终还需要扣掉：

Binance 手续费

Uniswap LP Fee

DEX Price Impact

Gas

MEV

滑点

交易完成后：

Binance：

ETH 减少

Arbitrum：

ETH 增加

于是产生 Inventory Imbalance。

之后再通过：

Bridge

CEX 提币

反向套利

低 Gas 时段

重新平衡库存。

这种策略的关键点是：

跨链不在交易关键路径上。

交易：

秒级

库存再平衡：

分钟、小时甚至更久

这两个过程应该完全分离。

置信度：高。

三、跨链库存套利

例如：

Arbitrum：

ETH = 3000

Base：

ETH = 3020

如果两个链上都提前拥有：

ETH + USDC

那么可以同时：

Arbitrum：

买 ETH

Base：

卖 ETH

完成套利。

之后再通过 Bridge :

Base ETH → Arbitrum ETH

或者通过其他资金流恢复库存。

这里的核心思想仍然是 :

交易 != 跨链

跨链只是 :

Rebalancing。

如果把两者绑定 :

买 ETH\

↓ \

Bridge\

↓ \

等待\

↓ \

卖 ETH

就会产生非常大的 Latency Risk。

置信度 : 高。

四、实时 Bridge 套利

实时 Bridge 套利结构例如 :

USDC Arbitrum\

↓ \

买 ETH\

↓ \

Bridge ETH\

↓ \

Base\

↓ \

卖 ETH\

↓ \

USDC

假设 :

原始价差 = 100 bps

Bridge 总成本 = 15 bps

DEX + Gas = 20 bps

那么表面净 Edge :

$100 - 15 - 20 = 65$ (tel:100%20-%2015%20-%2020)

\= 65 bps

但最大的问题不是手续费,而是 :

Bridge Latency。

假设 :

T = 30 秒

那么真正利润为 :

PnL

目标市场 T 秒后的价格

当前买入价格

成本

也就是 :

PnL

$P_B(t+T)$

$P \backslash A(t)$

Cost

而不是：

$P \backslash B(t)$

$P \backslash A(t)$

Cost

因此实时 Bridge 套利实际上承担：

短时间方向风险。

波动率越高：

风险越大。

所以它通常应该要求：

更高的 Minimum Edge。

例如：

库存套利可能要求：

20 bps

实时 Bridge 套利可能需要：

50 / 100 / 200 bps

具体阈值取决于：

资产波动率

Bridge 延迟

P95 / P99 到账时间

失败率

交易深度

置信度：高。

五、充提状态套利

这是非常值得研究的一类机会。

它的本质是：

市场之间的资产流动突然被切断。

例如：

Binance 某个 Coin：

Deposit Disabled

但：

Trading 正常

那么新的 Token 无法被充值进 Binance。

如果外部价格：

10 USDT

而 Binance：

10.5 USDT

就出现：

5% Premium。

为什么？

因为：

外部 Token 无法进入 Binance。

所以 Binance 市场暂时成为一个：

半封闭市场。

如果你提前在 Binance 有 Token：

可以：

Binance 卖出 Token

同时：

其他市场买入 Token

锁定价差。

之后等：

Deposit Resume

再恢复库存。

这种套利最重要的不是：

“看到停充后再搬币。”

因为已经停充了。

真正的 Alpha 是：

提前拥有库存。

所以它实际上也是：

Inventory Arbitrage。

置信度：高。

六、Withdrawal Suspension 套利

另一种情况：

某交易所：

Withdrawal Disabled

但：

Deposit 正常

可能产生另一种价格结构。

例如：

大量用户无法把 Token 提出去。

这可能导致：

交易所内部 Token 折价。

原因：

Token 被困在交易所内部。

如果：

外部 Token = 10

交易所 Token = 9.5

那么：

交易所内部出现：

-5% Discount。

问题是：

你虽然可以买便宜 Token，

但无法提出。

所以这个价差不是免费的。

实际上你买的是：

Withdrawal Resume Option。

你的收益取决于：

提现什么时候恢复。

因此：

这类套利已经接近 Event Risk Trade。

而不是纯粹 Arbitrage。

置信度：高。

七、Deposit Suspension 和 Withdrawal Suspension 的差异

Deposit Disabled :

更容易产生交易所内部溢价。

因为：

外部 Token 无法流入。

Withdrawal Disabled :

更容易产生交易所内部折价。

因为：

内部 Token 无法流出。

简单理解：

Deposit Disabled :

供给被切断

→ 容易 Premium

Withdrawal Disabled :

出口被切断

→ 容易 Discount

这是非常重要的第一性原理。

置信度：中高，因为实际价格仍受持仓结构、做市商库存和衍生品市场影响。

八、事件套利

除了充提状态之外，还应该监控：

Listing

Delisting

Spot Pair Removal

Futures Delisting

Token Migration

Contract Swap

Mainnet Upgrade

Wallet Maintenance

Deposit Suspension

Withdrawal Suspension

Deposit Resume

Withdrawal Resume

Bridge Migration

Stablecoin Depeg

Network Upgrade

Hard Fork

Token Unlock

Airdrop Snapshot

这些事件的共同点是：

它们都会改变：

资产流动性。

所以套利机会往往并不是来自：

“价格预测。”

而是来自：

资金流动约束发生改变。

置信度：高。

九、事件状态机

建议对每个：

Exchange × Coin × Network

建立状态。

例如：

NORMAL

↓

DEGRADED

↓

DEPOSIT_DISABLED

↓

WITHDRAW_DISABLED

↓

BOTH_DISABLED

↓

RESUMING

↓

NORMAL

数据库记录：

exchange

coin

network

deposit_enabled

withdraw_enabled

event_type

official_event_time

local_detect_time

start_time

resume_time

source

confidence

最重要的字段之一：

local_detect_time

因为真正的 Alpha 不是：

公告什么时候发布。

而是：

你的系统什么时候发现。

例如：

Binance 12:00:00 发布公告

竞争对手：

12:00:01 发现

你：

12:00:20 发现

那么你的：

Information Latency

\= 19 秒。

对于 Event Arbitrage：

这可能已经决定胜负。

置信度：高。

十、CEX—DEX套利定价

最简单的 Raw Spread :

Raw Spread

$$(P_DEX - P_CEX) / P_CEX$$

但这个指标几乎不能直接交易。

真正需要计算 :

Executable Spread(q)

其中 q 为交易规模。

Executable Spread(q)

$\backslash = \backslash$

$$\backslash [\text{Sell VWAP}(q) - \text{Buy VWAP}(q)] \backslash$$

$/ \backslash$

Buy VWAP(q)

比如 :

你想交易 50,000 USDT。

CEX 卖价 :

10.00

但只有 :

5,000 USDT 深度。

真正买完 :

50,000 USDT

平均价格可能是 :

10.08。

那么真正价格不是 :

10

而是 :

10.08。

因此套利系统应该永远使用 :

VWAP(q)

而不是 :

Best Bid / Best Ask。

十一、完整收益函数

设 :

q = 交易规模

$P_sell(q)$ = 高价市场可执行 VWAP

$P_buy(q)$ = 低价市场可执行 VWAP

那么 :

Gross PnL(q)

$\backslash = \backslash$

$$q \times \backslash [P_sell(q) - P_buy(q)]$$

Net PnL :

Net PnL(q)

Gross PnL(q)

CEX Fee

DEX Fee

Gas

Bridge Cost

Withdrawal Cost

Slippage

Hedge Cost

Funding Cost

Borrow Cost

Capital Cost

Failure Cost

Tail Risk Reserve

可以简写成：

Net PnL(q)

Gross Edge(q)

Total Cost(q)

交易条件：

$\text{Net PnL}(q) > \text{Minimum Required Profit}$

或者用 bps：

$\text{Net Edge}(q) > \text{Min Edge}$

例如：

Gross Edge：

70 bps

CEX：

8 bps

DEX：

10 bps

Gas：

3 bps

MEV Buffer：

5 bps

Failure Buffer：

4 bps

Inventory Cost：

5 bps

总成本：

35 bps

最终：

Net Edge：

35 bps

十二、套利容量

套利最重要的变量之一：

q。

因为：

Edge 会随着 q 增加快速下降。

最大可交易规模：

$q_{\max}$

$\backslash = \backslash$

min(

买方市场深度，

卖方市场深度，

现金库存，

Token 库存，
交易所限额，
Bridge 限额，
提现限额，
风控限额

)

这是理解套利容量最核心的公式之一。

例如：

理论价差：

100 bps

交易 1,000 USDT：

净 Edge = 80 bps

交易 10,000：

净 Edge = 55 bps

交易 100,000：

净 Edge = 10 bps

交易 300,000：

净 Edge = -30 bps

所以：

100 bps 的机会

并不意味着：

你可以无限吃 100 bps。

十三、为什么套利容量是次线性的

主要有四个原因：

1. CEX Order Book Impact
2. DEX AMM Price Impact
3. Inventory Fragmentation
4. Risk Limit

因此：

$\text{Capital} \times 10$

通常不会：

$\text{Profit} \times 10$

更可能：

$\text{Capital} \times 10$

$\text{Profit} \times 3 \sim 6$

只是示意，不是固定比例。

置信度：高。

十四、AMM 套利

对于最简单的 Constant Product AMM：

$x \times y = k$

例如：

$x = \text{USDC}$

$y = \text{ETH}$

价格大致：

$P = x / y$

当你买 ETH：

USDC 储备增加

ETH 储备减少

于是 ETH 价格越来越贵。

因此：

买得越多：

Price Impact 越大。

所以 DEX 套利必须考虑：

Amount。

不能只看：

Spot Price。

真正需要的是：

DEX Quote(q)

十五、Uniswap V3 更复杂

Uniswap V3 不是简单：

$xy = k$

因为流动性被分布在不同 Tick。

所以：

实际 Price Impact

取决于：

当前 Tick 附近有多少 Active Liquidity。

因此两个池：

TVL 都是：

100M

并不意味着：

可交易深度一样。

真正重要的是：

当前价格附近的 Liquidity Density。

因此专业系统应该：

实时调用 Quoter

或者：

自己重建 Pool State。

置信度：高。

十六、DEX 执行风险

链上交易与 CEX 最大区别：

DEX Tx 在被确认之前：

不是最终成交。

流程是：

Sign

↓

Broadcast

↓

Mempool

↓

Builder / Validator

↓

Block

↓

Execution

在这个过程中：

价格可以变化。

所以需要设置：

Minimum Amount Out

或者：

Maximum Amount In

比如：

期望：

100 ETH → 300,000 USDC

设置：

minimumOut = 299,000

如果最终：

298,000

交易：

Revert。

但 Revert：

Gas 仍然损失。

所以：

Failure Cost != 0。

十七、MEV风险

如果交易进入公开 Mempool：

其他 Searcher 可以看到。

可能出现：

Front-run

Back-run

Sandwich

Copy Trade

Builder Competition

因此专业执行可能使用：

Private RPC

Flashbots

Private Relay

Builder Direct Submission

这也是为什么：

链上套利的护城河越来越偏向基础设施。

置信度：高。

十八、CEX订单执行

CEX 方面建议优先使用：

IOC

FOK

PostOnly

而不是无限制 Market Order。

IOC：

Immediate Or Cancel

可以成交多少成交多少。

剩余立即取消。

FOK :

Fill Or Kill

全部成交才成交。

否则全部取消。

PostOnly :

只做 Maker。

如果会立即成交 :

取消订单。

对于套利 :

IOC 很常见。

因为可以设 :

Limit Price

控制最大滑点。

十九、双腿执行风险

最典型风险 :

Leg A 成交

Leg B 没成交。

例如 :

Uniswap :

买 ETH 成功

Binance :

卖 ETH 失败

那么 :

你突然变成 :

Long ETH。

所以系统必须定义 :

Legging Policy。

例如 :

如果 A 成交 B 失败 :

立即 Market Hedge

或者 :

Futures Hedge

或者 :

Reduce Size

应该记录 :

Hedge Delay。

例如 :

25 ms

100 ms

500 ms

2 秒

因为 :

Hedge Delay

直接决定 :

Execution Tail Risk。

二十、库存管理

每一个：

Venue $\times$ Asset

都应该维护独立库存。

例如：

Binance:

USDT = 100,000\

ETH = 20

OKX:

USDT = 70,000\

ETH = 10

Arbitrum:

USDC = 60,000\

ETH = 15

Base:

USDC = 50,000\

ETH = 10

定义：

Target Inventory = I^*

Current Inventory = I

Inventory Deviation：

$D = I - I^*$

可以建立 Inventory Penalty：

$\text{Penalty} = \lambda \times D^2$

那么：

Required Edge

$\Delta =$

Base Edge\

$+$

$\lambda \times D^2$

直觉：

如果 Base 上已经有太多 ETH：

继续：

Arbitrum 买 ETH

Base 卖 ETH

会让 Base ETH 更少。

库存更加失衡。

因此：

只有出现更高 Edge

才允许继续交易。

二十一、库存再平衡

Rebalancing 不一定立即执行。

可以等待：

Gas 低

Bridge Fee 低

出现反向套利

交易所恢复充提

其他链产生反向流量

所以：

Rebalancing

本身也是：

Optimization Problem。

目标函数可以近似：

Minimize：

Bridge Cost



Price Risk



Inventory Penalty



Capital Cost

二十二、库存套利真正的资本成本

这是很多套利者忽略的成本。

假设总本金：

100,000 USDT

但是分散为：

Binance：

20k

OKX：

20k

Arbitrum：

20k

Base：

20k

Solana：

20k

真正能抓某一个具体机会的资金：

可能只有：

20k。

所以：

Nominal Capital != Effective Capital。

这叫：

Capital Fragmentation。

随着：

Exchange 数量 × Chain 数量 × Asset 数量

增加，

资金碎片化会越来越严重。

这是套利规模化的核心瓶颈之一。

置信度：高。

二十三、充提状态数据

必须建立：

Exchange × Coin × Network

级别的数据。

不能只记录：

BTC deposit enabled

而应该记录：

USDT ERC20

USDT TRC20

USDT Arbitrum

USDT Optimism

USDT Solana

USDT BSC

等。

每一条 Network 应记录：

depositEnabled

withdrawEnabled

withdrawFee

withdrawMin

withdrawMax

minConfirm

estimatedArrivalTime

busy

因为：

同一个资产不同链：

完全可能状态不同。

二十四、充提监控

建议监控：

Binance

OKX

Bybit

Coinbase

Gate

KuCoin

MEXC

Bitget

但初期没有必要覆盖所有交易所。

建议：

Binance + OKX + Bybit

先构建。

原因：

流动性较好

API 完整

币种多

充提状态数据丰富

置信度：中高。

二十五、事件监控

建议同时监控：

官方 API

官方 Announcement

官方 Status Page

官方 Telegram

官方 X

但是：

API / Status

应该作为主要机器信号。

Announcement：

作为解释层。

例如：

API：

withdrawEnable = false

那么：

立即标记：

WITHDRAW_DISABLED

不必等待公告。

如果稍后公告：

“Wallet Maintenance”

再把：

Reason

补进去。

二十六、事件套利的优先级

最值得研究：

1. Deposit Disabled
2. Withdrawal Disabled
3. Deposit Resume
4. Withdrawal Resume
5. Wallet Maintenance
6. Network Upgrade
7. Token Migration
8. Delisting

为什么 Resume 特别重要？

因为：

市场分割解除时：

价差通常会快速收敛。

所以：

Resume 本身就是一个：

Mean Reversion Catalyst。

例如：

某交易所：

ETH Premium = 2%

Deposit 恢复。

套利者开始：

外部买 ETH

充值交易所

卖出

于是：

Premium 快速收敛。

二十七、事件套利的时度

大致可以分：

毫秒~秒：

Order Book / MEV

秒~分钟：

CEX—DEX

分钟~小时：

Deposit / Withdrawal 状态

小时~天：

Maintenance / Delisting / Migration

截图中那种：

人工盯盘 + 节点 + 监控 + 跨链脚本 + 充提脚本

更像：

秒级~小时级套利

而不是：

纯 HFT MEV。

置信度：高。

二十八、Bridge系统

可以研究：

Across

LayerZero

Axelar

Stargate

Relay

LI.FI

deBridge

对于每一个 Bridge：

至少记录：

Origin Chain

Destination Chain

Token

Input Amount

Output Amount

Fee

Expected Time

Actual Time

P50

P95

P99

Failure Rate

Refund Time

TVL

Security Model

不要只比较：

Fee。

真正重要的是：

Fee + Latency + Tail Risk。

二十九、Bridge风险

Bridge 主要风险：

Smart Contract Risk

Validator Risk

Relayer Risk

Oracle Risk

Liquidity Risk

Message Verification Risk

Chain Finality Risk

Contract Upgrade Risk

Admin Key Risk

因此：

Bridge 风险不能简单表示成：

“手续费 5 bps”。

应该加入：

Expected Loss。

例如：

假设某 Bridge：

年度重大事故概率估计：

0.5%

事故发生时：

平均损失：

40%

那么：

粗略 Annual Expected Loss：

$0.5\% \times 40\%$

$\approx 0.2\%$

即：

20 bps / 年。

这只是示例模型，不代表真实风险概率。

三十、尾部风险

套利策略的收益分布往往：

大量小盈利



少数巨大亏损。

因此：

Sharpe 可能非常高。

但：

Tail Risk 也可能很高。

例如：

连续 500 笔：

+20 USDT

累计：

+10,000

一次 Bridge Exploit：

-50,000

策略直接亏损。

因此：

不能只看：

Sharpe

Win Rate

Profit Factor。

必须看：

Expected Shortfall

Max Drawdown

Worst Event

Stuck Capital Days

Incident Loss

建议特别关注：

ES99。

置信度：高。

三十一、资金需求

假设目标：

每月净赚 1,500 USDT。

如果净月收益率：

0.5%

本金：

300,000

1%

本金：

150,000

2%

本金：

75,000

3%

本金：

50,000

5%

本金：

30,000

10%

本金：

15,000

如果目标：

7,500 USDT / 月：

0.5%

1,500,000

1%

750,000

2%

375,000

3%

250,000

5%

150,000

10%

75,000

真正的问题不是这些数学。

而是：

能否长期实现：

2% / 3% / 5%

净收益，

同时控制：

尾部风险。

置信度：高。

三十二、示例：CEX—DEX套利

假设：

交易规模：

20,000 USDT

Gross Executable Edge：

45 bps

成本：

CEX：

8 bps

DEX + Price Impact：

12 bps

Gas：

4 bps

MEV / Latency Buffer：

5 bps

Failure Reserve：

2 bps

总成本：

31 bps

净 Edge：

45 - 31

\= 14 bps

利润：

$20,000 \times 0.14\%$

\= 28 USDT

如果一个月：

120 次

利润：

28×120

\= 3,360 USDT

假设平均在险资本：

120,000

月收益率：

2.8%

这是示例，不代表市场平均收益。

三十三、示例：充提状态套利

假设：

某币 Withdrawal Suspension。

局部市场出现：

80 bps Premium。

可以安全执行：

15,000 USDT。

交易成本 + Event Risk Buffer：

25 bps。

净 Edge：

80 - 25

\= 55 bps。

利润：

$15,000 \times 0.55\%$

\= 82.5 USDT。

如果一个月：

10 次。

利润：

825 USDT。

这说明：

充提状态套利非常适合：

Overlay Strategy。

但未必适合作为：

唯一策略。

置信度：高。

三十四、回测方法

这类套利绝对不能用：

1m OHLC

或者：

Close Price

直接回测。

必须使用：

Execution-Native Backtest。

CEX：

L2 Order Book

Snapshot

Delta

Sequence

Trades

DEX：

Pool State

Quote

Block State

Gas

Slippage

Route

Bridge :

Quote Time

Submit Time

Fill Time

Completion Time

Failure

Refund

Event :

Announcement Time

Detection Time

Deposit Status

Withdrawal Status

Resume Time

三十五、时间戳体系

至少记录 :

exchange_event_time

local_receive_time

decision_time

order_submit_time

order_ack_time

fill_time

block_time

tx_submit_time

tx_inclusion_time

bridge_start_time

bridge_complete_time

destination_credit_time

否则很容易产生 :

Look-Ahead Bias。

例如 :

公告发布时间 :

10:00:00

你的历史数据库记录 :

10:00:00

但实际上 :

API 抓到 :

10:00:12

如果回测按照 :

10:00:00

开始交易 ,

就凭空获得 :

12 秒 Future Information。

三十六、回测指标

建议至少统计 :

Raw Spread

Executable Spread
Net Spread
Fill Rate
Partial Fill Rate
Slippage
Hedge Delay
Transaction Failure Rate
Bridge Completion P50
Bridge Completion P95
Bridge Completion P99
Inventory Utilization
Capital Utilization
Turnover
Inventory Days Outstanding
Stuck Capital Days
Gross PnL
Net PnL
Max Drawdown
Sharpe
Sortino
ES95
ES99
Worst Incident

三十七、生产系统架构

建议：

CEX WebSocket\
DEX RPC\
Bridge API\
Announcement Feed\
Status API\
↓ \
Market State Bus\
↓ \
Opportunity Engine\
↓ \
Risk Engine\
↓ \
Execution Orchestrator\
↓ \
CEX Router + Onchain Router\
↓ \
Order / Transaction\
↓ \
Reconciliation\
↓ \
Inventory Ledger\
↓ \
Rebalancing Engine

三十八、机会引擎

核心逻辑：

1. 获取所有市场报价
2. 统一资产映射
3. 计算 VWAP
4. 计算手续费
5. 计算 Gas
6. 计算库存
7. 检查充提状态
8. 检查 Bridge
9. 计算 Net Edge
10. 检查 Risk Limit
11. Ranking
12. Execution

最终可以定义：

Opportunity Score

$\backslash =$

Net Edge

$\times$ Fill Probability

$\times$ Capital Efficiency

$\div$ Tail Risk

这只是一个抽象框架。

实际可以进一步建立：

Expected PnL：

$E[PnL]$

$\backslash = \backslash$

$P(fill) \times Profit$

●

$P(partial) \times Partial Loss$

●

$P(failure) \times Failure Loss$

●

Tail Risk Reserve

三十九、交易前检查

每次交易必须检查：

Market Data Freshness

CEX Book Sync

DEX Quote Freshness

Inventory

Gas

RPC Health

Deposit Status

Withdrawal Status

Bridge Health

Wallet Balance

Nonce

Stablecoin Risk

Exchange Exposure

Chain Exposure

Token Exposure

任何一个异常：

Reject Signal。

四十、Kill Switch

以下情况自动停止交易：

CEX WebSocket Desync

RPC Lag

Block Stall

Gas Spike

Exchange API Error Spike

Deposit Disabled

Withdrawal Disabled

Bridge Incident

Bridge Delay异常

Stablecoin Depeg

Inventory Limit Breach

Exchange Exposure Limit Breach

Private Key Risk

Unknown Token Contract Change

四十一、对账

每笔套利必须关联：

Signal ID

Quote ID

Order ID

Fill ID

Transaction Hash

Bridge ID

Destination Transaction

Inventory Change

PnL Entry

任何一个缺失：

状态：

UNRECONCILED

而不是：

TRADE COMPLETE。

这是非常重要的生产系统原则。

置信度：高。

四十二、风险限额

建议建立：

Exchange Limit

Chain Limit

Bridge Limit

Token Limit

Stablecoin Limit

Wallet Limit

Strategy Limit

Daily Loss Limit

Incident Loss Limit

例如：

Binance Exposure：

最大 30%

OKX：

20%

Arbitrum：

20%

Base：

15%

单 Bridge：

10%

单长尾 Token：

2%

这里只是示例。

实际参数必须根据资本、流动性和风险承受能力确定。

四十三、真正的护城河

这类套利中：

最容易复制：

Spread Scanner。

真正难复制的是：

1. 数据延迟
2. 状态监控
3. CEX执行
4. Onchain执行
5. MEV Protection
6. Inventory Management
7. Rebalancing
8. Risk Engine
9. Reconciliation
10. Incident Response

所以：

Alpha 会逐渐从：

“我知道哪里价差”

转变为：

“我可以比别人更可靠地把价差变成真实 PnL”。

置信度：高。

四十四、个人/小团队最合理的发展路线

第一阶段：

Binance



Arbitrum



Uniswap



ETH / USDC

先只做：

CEX—DEX Inventory Arbitrage。

目标：

把以下模块做对：

CEX Book

DEX Quote

Executable Spread

Inventory

Execution

PnL

Reconciliation

第二阶段：

加入：

Base

Optimism

开始：

Cross-Chain Inventory Arbitrage。

但：

Bridge 仍然只用于：

Rebalancing。

第三阶段：

加入：

Binance

OKX

Bybit

的：

Deposit / Withdrawal Monitor。

开始做：

Event Overlay。

第四阶段：

加入：

Across

LayerZero

Relay

等 Bridge。

开始研究：

Live Bridge Arbitrage。

不建议一开始：

几十个交易所

几十条链

几百个小币。

因为：

Operational Complexity

会迅速失控。

置信度：高。

四十五、最值得研究的 Alpha

如果从实际可持续性排序：

第一梯队：

CEX—DEX 预置库存套利

跨链预置库存套利

第二梯队：

Deposit / Withdrawal Event Arbitrage

Wallet Maintenance Arbitrage

Network Upgrade Arbitrage

第三梯队：

Bridge Live Arbitrage

第四梯队：

Long-tail Token

Migration

Depeg

Risky Asset Arbitrage

最后一类虽然：

Spread 最大，

但通常也是：

Tail Risk 最大。

四十六、对截图中团队模式的理解

截图中提到：

节点

监控

跨链脚本

充提脚本

人工盯盘

不依赖聚合器

这非常符合：

Inventory + Event Arbitrage

团队。

他们的优势可能不是：

高频预测模型。

而是：

更早发现机会



两边提前有库存



知道哪些链能走



知道哪些交易所能充提



快速执行



人工处理异常。

因此：

这种团队的真实核心资产更像：

Operational Knowledge Base。

而不是：

单个算法。

置信度：高。

四十七、最终理解

把整个领域压缩成一个公式：

套利利润

\=

价格错位

× 可执行规模

× 成功概率

●

直接交易成本

●

库存成本

●

延迟风险

●

失败风险

●

尾部风险

也可以写成：

Alpha

\=

Market Fragmentation

●

Information Advantage

●

Execution Advantage

●

Inventory Advantage

●

Operational Risk

●

Tail Risk

真正高质量的 Crypto 套利系统，不是在找：

“哪里价格不一样”。

而是在解决：

如何在不同交易所、不同链、不同资产库存、不同结算时间和不同风险状态下，把价格错位稳定转换成可重复、可扩展、可对账的净收益。

最终优先级：

CEX—DEX 预置库存套利

跨链预置库存套利

充提状态 / Event Arbitrage

实时 Bridge Arbitrage

高风险长尾币套利

笔记 005: August 16, 2026 · Day 12 / 21 · 时间分配：先动手 60min → 后

August 16, 2026 · Day 12 / 21 · 时间分配 : 先动手 60min → 后概念 40min → 打卡 20min

[!SUCCESS]

已还原一笔疑似 Ethereum 单链套利交易案例。交易执行成功并产生可见正向结算，但最终净利润无法确认。
本文只还原已发生的链上现金流和当前能够确认的成本边界，不把交易成功、实际 USDT 回转差额或 caller 收到 ETH 直接写成套利净利润。

学了什么

- 还原公开交易时，需要区分以下几种金额：
 - 实际 USDT 回转差额；
 - Gas ；
 - Builder 支付；
 - caller 收到的结算；
 - caller 扣除 Gas 和 Transaction Value 后的结算；
 - 策略层仍然未知的成本。
- 实际 Swap 输出已经包含协议费和确定性价格冲击。使用实际成交输出进行核算时，不能再次重复扣除协议费。
- 单笔交易日志可以显示实际输入、输出和结算，但不能完整确定：
 - 交易前 quote ；
 - 独立的价格冲击；
 - quote 到实际执行之间的滑点；
 - 延迟成本；
 - 策略层失败交易成本；
 - 资金占用成本。
- mempool (memory pool, 内存池/交易池) 是各节点本地维护的待处理交易集合，不是内容完全一致、统一的全球数据库。缺少交易到达各节点 mempool 的时间，就无法从本笔交易完整量化延迟成本。

做了什么

1. 固定案例参数

字段	内容
网络	Ethereum Mainnet
Tx Hash	0xc34b41a1d730d854bf75ec653b2fb3db6f007d5aae2aa9e1fabcd45f77afd80d
Etherscan	https://etherscan.io/tx/0xc34b41a1d730d854bf75ec653b2fb3db6f007d5aae2aa9e1fabcd45f77afd80d
区块	24099235
执行时间	2025-12-26 08:37:11 UTC
Status	Success
Gas Price	0.030509185 Gwei
Transaction Fee	0.000009674737146165 ETH
Transaction Value	63 wei, 即 0.000000000000000063 ETH

以上交易数据属于 user-reported : 由用户提供，并已由用户核对 Etherscan 页面、Swap 日志和结算记录；本会话未完成独立链上复核，因此不标记为 repo-verified 或本会话已实测。

2. 还原实际执行路径

1. Uniswap V3 :

1,078.372983 USDT → 1,079.356368216149115899 USDe

2. Fluid DEX :

1,079.356368216149115899 USDe → 1,078.496217 USDT

3. Uniswap V3 :

0.123234 USDT → 0.000042074585970909 WETH

这些数值属于用户已核对的实际 Swap 输入和输出，不是交易前 quote。

3. 复算实际 USDT 回转差额

1,078.496217 1,078.372983 = 0.123234 USDT

因此，两段 USDT → USDe → USDT 执行路径产生的实际 USDT 回转差额／名义差额为：

+0.123234 USDT

该数值不是最终价差收益，也不是最终净利润。

4. 复算结算关系

Builder 与 caller 收到的 ETH 合计：

text

0.000023294799655261

+ 0.000018779786315711

= 0.000042074585970972 ETH

WETH 输出加上 Transaction Value：

text

0.000042074585970909

+ 0.000000000000000063

= 0.000042074585970972 ETH

两组金额在算术上完全对应。

caller 扣除 Gas 和 63 wei 后：

text

0.000018779786315711

0.000009674737146165

0.000000000000000063

= 0.000009105049169483 ETH

0.000009105049169483 ETH 是 caller 在该核算口径下剩余的正向结算，不能直接称为完整八项成本后的套利净利润。

得到了什么结果

可见执行结果

交易状态为 Success；

两段 USDT 回转产生 +0.123234 USDT 的实际 USDT 回转差额／名义差额；

0.123234 USDT 随后换得 0.000042074585970909 WETH；

Titan Builder 收到 0.000023294799655261 ETH；

caller 收到 0.000018779786315711 ETH；

caller 扣除 Gas 和 63 wei 后的结算为 0.000009105049169483 ETH。

Builder 支付

caller 的 0.000009105049169483 ETH 是 caller 实收口径。Builder 支付已由 MEV Bot 从同一结算流中另行分配，因此在该 caller 结算口径下不得再次扣除；如果从策略总现金流计算，则必须将 Builder 支付单独列为执行成本。

手续费口径

两个 Uniswap V3 池子的手续费均为 0.01%；

Fluid DEX：当前配置为 0.01%，历史值得确认；

Fluid 在区块 24099235 的历史 pool fee 尚未独立确认；

实际 Swap 输出已经包含协议费和确定性价格冲击，因此本次不再重复扣除协议费；

已知实际输出，并不等于已经分别量化了协议费、价格冲击和报价后滑点。

案例结论

该交易成功执行并产生可见正向结算，但由于缺少交易前

quote、独立价格冲击、报价后滑点、延迟、失败交易和资金占用数据，最终净利润无法确认。

本案例证明了一笔交易曾经成功执行并产生可见正向结算，但不足以证明当时或现在存在可复现的净利润机会。

遇到什么失败 / 卡点

1. 本会话没有完成独立链上复核，交易数据仍属于 user-reported。
2. Fluid 当前配置为 0.01%，但区块 24099235 的历史配置尚未独立确认。
3. 缺少交易前 quote 和报价时间，无法比较 quote 与实际执行结果。
4. 缺少交易执行前的池状态，无法独立计算确定性价格冲击。
5. 实际 Swap 输出包含协议费和确定性价格冲击，但无法仅凭本笔交易日志将二者分别拆出。
6. 缺少报价后池状态变化证据，无法量化报价后滑点。
7. 缺少发现、报价、提交以及到达各节点 mempool 的时间，无法量化延迟成本。

8. 本笔交易虽然成功，但没有同类策略的失败交易样本，无法估算策略层失败概率和预期失败成本。

9. Fluid LogOperate 中 borrowAmount=0，只能说明本笔记录没有显示借款，不能推出资金占用成本为 0。

10. 资金来源和占用时间未知，无法量化资金占用成本。

八项成本状态

#	项目	数值或状态	证据类型	核算说明
1	实际 USDT 回转差额	+0.123234 USDT	user-reported 链上数据	+ 算术复算 实际 USDT 回转差额 / 名义差额，不是最终价差收益或净利润
2	Gas	0.000009674737146165 ETH	user-reported 链上证据	，本会话未独立复核 已在 caller 最终结算复算中扣除
3	协议费	Uniswap V3 两池均为 0.01% ; Fluid 当前配置为 0.01% , 历史值待确认	Uniswap 当前池配置已知 ; Fluid 当前配置已知、历史值 unknown	实际 Swap 输出已经包含协议费，不得重复扣除
4	桥费	不适用 不适用	所有操作均发生在 Ethereum Mainnet	
5	滑点和价格冲击	unknown unknown	缺少交易执行前池状态、交易前	quote 和报价后池状态变化证据，无法分别量化价格冲击与报价后滑点
6	延迟成本	unknown unknown	缺少发现、报价、提交及 mempool 到达时间	
7	失败交易成本	unknown unknown	本笔交易成功，但缺少同类策略的失败交易样本和失败概率	
8	资金占用成本	unknown unknown	borrowAmount=0 不能证明资金成本为 0 ; 资金来源和占用时间未知	

Builder 支付另列为执行成本：

执行成本	数值	核算说明
Titan Builder 支付	0.000023294799655261 ETH	必须单独披露；使用 caller 最终结算额时不得再次重复扣除
Transaction Value	63 wei	已在 caller 最终结算复算中扣除
caller 扣除 Gas 和 63 wei 后的结算	0.000009105049169483 ETH	可见正向结算，不等于完整八项成本后的最终净利润

最大不确定性是缺少交易执行前池状态、交易前 quote 和报价时间，因此无法分离协议费、确定性价格冲击、报价后滑点与延迟成本；同时缺少失败交易样本和资金占用证据，无法确认最终净利润。

- 下一步验证什么
1. 使用 archive RPC 查询区块 24099235 的 Fluid 历史 pool fee ；
 2. 获取交易执行前池状态；
 3. 获取交易前 quote 和报价时间；
 4. 分离协议费、价格冲击、报价后滑点和延迟成本；
 5. 收集同类策略的失败交易样本；
 6. 确认资金来源与占用时间。

笔记 006: 做了个面向新币 TGE 的链上流动性监控平台，基于 TypeScript、React、Node.js

作者：OxAllen | 时间：2026-08-17 00:33 UTC | 字数：278 | 评分：46
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

做了个面向新币 TGE 的链上流动性监控平台，基于 TypeScript、React、Node.js、viem、Prisma 和 PostgreSQL，支持 Base/BNB Chain 及 Uniswap V3/PancakeSwap V3。通过扫描链上事件实现流动性池发现、LP NFT Position 回填和流动性事件持久化，并设计统一协议适配层、钱包白名单过滤及手动覆盖机制，最终通过可视化面板展示项目流动性分布和同步状态。

目前只能监控钱包地址，看到相关钱包地址，需要添加进去监控，否则计算量太大，成本也高

笔记 007: Day 12 打卡 | 鏈上套利殘酷共學

作者：roy328line | 时间：2026-08-17 06:27 UTC | 字数：852 | 评分：73
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 打卡 | 鏈上套利殘酷共學

- A. 降門檻掃描
- MIN_POOL_TVL = \$50k (Day 11)：真實機會 0，OBSERVE 226
- MIN_POOL_TVL = \$10k (今天)：真實機會 0，OBSERVE 340

降低門檻只增加幻覺數量，沒有找到任何真實機會。

B. 幻覺偏差統計

對所有通過第一階段的候選跑 Quoter，統計 $\text{ratio} = \text{net_real} / \text{net_star}$ ：

OBSERVE (tick_aware=False)：樣本 15，ratio 中位數 -0.005 (幻覺 99.5%)，正 EV 條數 0

tick_aware=True 候選：樣本 31，ratio 中位數 -0.028 (幻覺 97.2%)，正 EV 條數 0

校正公式無效：ratio 標準差 (0.083) 遠大於中位數 (-0.028)，偏差來源不是固定乘數，而是每條路徑的池情況各異。沒有通用校正公式，Quoter 是唯一可信驗證方式。

v3

最接近正 EV 的路徑

USDT→USDC→DAI

net_star = \$1,570 (amm_out 估算)

net_real = -\$5.26 (Quoter 真實)

gas = \$5.50

距離正 EV 門檻 = \$10.76

池子若在某個時刻比現在稍微不平衡，可能就會出現真實正 EV 機會——這正是為什麼需要長期掃描。

今日核心發現

這兩天掃描窗口內，沒有真實的三角套利機會。可能原因：

1. 機會持續時間 < 1 block (12 秒)，掃描器太慢
2. 同一 block 被競爭者搶走
3. 當前市場波動性低，機會本來就少
4. 池覆蓋率不足 (漏了某些路徑)

Day 13 方向：連續掃描 + 記錄歷史

把每次 Quoter 驗證後的 net_real_usd 記錄到 DuckDB，連續掃描 1 小時，分析 net_real_usd 的時間分佈，找出曾經最接近正 EV 的時刻。

笔记 008: 1. 什么是跨所套利

作者：wmroar | 时间：2026-08-17 09:02 UTC | 字数：6092 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1. 什么是跨所套利

核心：一价定律 (Law of One Price)。同一种资产 (比如 ETH) 理论上在任何市场都该是一个价。但现实中，不同交易所/不同流动性池是分割的市场，信息传播、流动性深度、手续费各不相同，所以同一瞬间价格会短暂不一致。

跨所套利就是：在便宜的 venue 买入，在贵的 venue 卖出，赚取价差。因为它几乎不承担方向性风险 (同时买卖、金额对冲)，所以传统上被称为“无风险套利”，但链上实际并不完全无风险 (见第 7 节)。

便宜池: 1 ETH = 1000 USDC 贵池: 1 ETH = 1100 USDC\

—— 花 1000 USDC 买 1 ETH —— 在贵池卖掉 ETH 拿回 ~1091 USDC\

→ 毛利 ≈ 91 USDC

本仓库的 cross_dex 信号就是这类：同一币对 (A/B)，在 venue1 做 A→B，在 venue2 做 B→A。

1. 必须先懂的 AMM 基础

去中心化交易所 (DEX) 大多是 AMM (自动做市商)，价格不是挂单簿，而是由公式 + 池子里的储备量决定。

2.1 常数积公式 (Uniswap V2)

$x \setminus y = k$ (x 、 y 是两种代币的储备量， k 恒定) \

价格 $P = y / x$ (1 个 x 能换几个 y)

关键：价格会随你的交易量移动。你买得越多，价格越贵 (滑点)。所以套利利润不是线性的，不能用“价差 × 本金”去粗算。

本仓库 src/arb.js 里的精确成交额公式：

amtWithFee = amountIn \ 997 // 扣 0.3% 手续费\

out = (amtWithFee \ reserveOut) / (reserveIn \ 1000 + amtWithFee)

我用这个公式做了个具体算例 (1000 ETH 的池，A 池 1000 USDC/ETH，B 池 1100 USDC/ETH)：

本金	利润	利润率	\
\$100	\$9.32	9.32%	\
\$1000	\$91.24	9.12%	\
\$5000	\$413.16	8.26%	\
\$50000	-\$279	-0.56%	\

这就是最重要的第一课：套利本金不是越大越好。大额交易会便宜池的价格推到比贵池还贵，反而亏损。所以本项目对不同规模分档模拟（0.1% / 1% / 5% 储备）取最优，而不是固定用最大本金。

2.2 V3 的集中流动性

Uniswap V3 不再是全价格区间的常数积，而是流动性集中在某个价格区间（concentrated liquidity）。价格用 `sqrtPriceX96`（Q64.96 定点数）表示，liquidity 只在该区间生效。这意味着：

成交价无法像 V2 那样用简单公式算，要用 QuoterV2 合约的 `quoteExactInputSingle` 做精确报价；

池子的"单侧深度"也要用 `liquidity + sqrtPrice` 反推（见 `src/tokens.js` 的 `poolDepthUsd`）。

不同 fee tier（0.01% / 0.05% / 0.3% / 1%）是不同的池，价格可以不同，也构成套利机会。

1. 套利的两种基本形态

3.1 跨池 2-hop 套利 (cross_dex)

同一币对，两个 venue 价差：

USDC --(venue1: 便宜, 买 ETH)--> ETH --(venue2: 贵, 卖 ETH)--> USDC

两个 venue 可以是：V2↔V3、两个不同 DEX 的池子（Uniswap ↔ PancakeSwap）、V2↔Curve 等。本仓库 README 里也写了实测结论：V2↔V3 跨池价差是这条链的主要机会来源。

3.2 三角套利 (triangular_v3)

三个代币形成闭环，绕一圈回到起点更多：

$A \rightarrow B \rightarrow C \rightarrow A$

例: $USDC \rightarrow ETH \rightarrow ARB \rightarrow USDC$

利润来自三个池子的汇率乘积 $\neq 1$ 。本仓库的三角套利把 V2 + V3 + Curve 图合并成一张图，然后枚举 $A \rightarrow B \rightarrow C \rightarrow A$ 的所有组合做精确模拟。

| 直觉上两跳 (cross_dex) 和三跳 (triangular) 本质一样：都是"绕一圈回到起点，终点余额 > 起点余额"。

1. 利润怎么算：必须"精确成交额模拟"

这是本项目最强调的设计原则之一：不要用 USD 价差估算利润。因为：

1. 滑点会让实际成交价和"报价"差很多；
2. V2/V3/Curve 的定价机制完全不同，没法统一用简单价格；
3. 池子深度直接决定你能成交多少。

本仓库的做法 (`src/arb.js` 的 `resolveLegs`)：

V2 腿：本地用常数积公式算（快）；

V3 腿：批量调 `QuoterV2.quoteExactInputSingle`（真实报价）；

Curve 腿：调池子的 `get\_dy`（真实报价）。

然后算 $profit = 终点金额 - 起点金额$ ，再减去 gas 成本得到 `net\_profit\_usd`。

1. 闪电贷 (Flash Loan)：套利为什么"不需要本金"

链上套利最优雅的一点是原子性 + 闪电贷：

一个交易里完成：

1. 借入 token (闪电贷, 无抵押, 因为同笔交易内必须归还)
2. 走套利路径 swap
3. 归还本金 + 利息 (若有)
4. 剩余部分 = 利润, 转给执行者

如果套利失败/亏损, 整个交易 revert, 等于什么都没发生, 只亏 gas。本仓库用 Morpho Blue 的 flashLoan (零费率), 合约在 contracts/src/FlashArb.sol :

Morpho flashLoan 借入 → 多跳 swap → 归还本金 → 利润转给 executor

所以套利者唯一需要准备的是: 支付 gas 的少量 ETH (README 说约 \$0.02-0.05/笔), 不需要几百万本金。

1. 链上执行的三层保护

src/executor.js + 合约一起构成了防御:

1. eth_call 模拟: 发真实交易前先本地模拟, revert 就放弃 (省 gas) ;
2. 滑点保护 amountOutMin: 每跳 swap 都设最低成交额, 成交价被抢跑者移动就 revert ;
3. 本金差额检查 minProfit: 合约里检查 余额 >= 本金 + minProfit, 否则整体回滚。

执行流程: live 发现信号 → buildSteps 转成合约调用 → eth_call 模拟 → 发交易 → 等确认 → 从 Arbitrage 事件解析实际利润 → 推送到 Telegram/Webhook。

1. 真实世界的风险 (为什么不是"无风险")

风险	说明	本项目对策
抢跑/排序竞争	信号是瞬时的, 别人先成交就把价差抹平, 甚至让你的交易反向亏损	滑点保护、坏信号缓存、发交易前模拟
信号瞬时性	你检测到的价差可能下一秒就消失	事件驱动实时模式 (live) 而非轮询
gas 成本	每次尝试都要 gas, 失败只亏 gas	只对净利超过阈值的信号执行
薄池假信号	假币/灰尘级池子报价很诱人但根本成交不了	MIN_DEPTH_USD (默认 \$5000) 深度过滤
假币	链上大量冒充 USDC/USDT/WETH symbol 的假币	只信任白名单地址 + 合理价格区间 (src/tokens.js)
MEV 生态	公共内存池 (mempool) 里你的交易会被盯上	FCFS 排序、私密 RPC、合约保护兜底

1. 一个完整套利系统的组成（对照本项目）

- ① 发现 discovery.js 枚举所有 V2/V3/Curve 池（V2 全枚举、V3 用 PoolCreated 事件全枚举）\
- ② 定价 tokens.js 从白名单稳定币锚点 BFS 推导全图 USD 价\
- ③ 检测 arb.js 三角 + 跨池，精确成交额模拟\
- ④ 实时 live.js 订阅链上日志（V2 Sync / V3 Swap），本地增量更新、去抖重算\
- ⑤ 执行 executor.js 转信号 → eth_call 模拟 → 发交易\
- ⑥ 合约 FlashArb.sol 闪电贷 + 多跳 swap + 归还 + 利润提取\
- ⑦ 落库/告警 db.js / alert.js MySQL 存机会，Telegram/Webhook 推送

1. 建议的进一步学习路径

2. AMM 数学：吃透 $x/y=k$ 、getAmountOut 推导、滑点公式；再学 V3 的 sqrtPriceX96 和流动性区间。
3. 套利检测：读懂 src/arb.js 的 findCrossDex 和 findTriangularArb，重点看“为什么分三档本金规模模拟”。
4. 闪电贷：读懂 FlashArb.sol 的完整调用链，理解“原子性 + revert 兜底”。
5. 运行一下：按 README 起 MySQL、node src/index.js scan 初始化、node src/index.js live 看实时信号（EXECUTOR_ENABLED=0 时只打印不上链，很安全）。

需要的话，我可以下一步带你逐行精读 src/arb.js 的套利检测逻辑，或者跑一个最小可复现的 V2 跨池套利模拟（纯 Python，不连链），把上面第 4 节的利润计算完整走一遍。你想先深入哪一块？

笔记 009: 学习笔记：如何查询链上 LP 的价格范围与手续费

作者：wy2221 | 时间：2026-08-17 12:59 UTC | 字数：4224 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习笔记：如何查询链上 LP 的价格范围与手续费

一、核心思路

在 Uniswap V3、PancakeSwap V3、Uniswap V4 这类集中流动性 AMM 中，LP 不再只是简单地“往池子里放两个币”，而是为资金指定一个价格区间。

因此研究一个 LP 仓位，核心就是回答四个问题：

- 这个 LP 属于谁？
- 资金放在哪个价格范围？
- 现在还有多少流动性？
- 已经赚了多少钱？

这类信息原则上都可以直接从链上获得，不需要依赖 Uniswap、PancakeSwap 等官方前端。

二、先理解 Pool 和 Position 的区别

一个交易池可能是：

TOKENA / USDT

Fee = 0.25%

但同一个池子中可以存在成千上万个 LP 仓位，每个 LP 的范围不同。

例如：

LP A : \$10 ~ \$20

LP B : \$15 ~ \$16

LP C : \$12 ~ \$30

所以查询时要区分：

Pool 数据

代表整个池：

交易对

当前价格

TVL

交易量

Fee Tier

池子总手续费

Position 数据

代表某个具体 LP :

Owner

Token ID

tickLower

tickUpper

liquidity

未领取手续费

最容易犯的错误，就是把池子的手续费收入误认为某个 LP 的手续费收入。

三、LP 的 Token ID 是最重要的索引

在 Uniswap V3/V4、PancakeSwap V3 中，一个集中流动性仓位通常表现为 NFT。

因此查询逻辑可以理解为：

钱包地址

↓

发现 LP NFT

↓

Token ID

↓

查询 Position

↓

还原价格范围、资金、手续费

例如：

Token ID = 1079214

有了 Token ID，通常就能继续查询：

Owner

Pool

Token0

Token1

Fee

Tick Lower

Tick Upper

Liquidity

所以开发 LP 查询工具时，Token ID 是最自然的主键之一。

四、如何查询 LP 的价格范围

V3 类协议的 Position 通常保存：

tickLower

tickUpper

Tick 并不是直接的美元价格。

基本关系为：

$Price \approx 1.0001 \wedge tick$

再根据：

Token0 decimals

Token1 decimals

Token0 / Token1 方向

进行调整，最终得到：

最低价格

最高价格

例如链上返回：

tickLower = 287000

tickUpper = 294000

程序转换后可能显示：

LP Range：

\$16.20

~

\$18.70

然后获取当前 Pool 的 Tick：

currentTick

判断：

$\text{tickLower} \leq \text{currentTick} < \text{tickUpper}$

则说明：

LP 当前处于 Active Range。

如果当前价格离开范围，LP 一般不会继续获得正常的区间内交易手续费，而且资产会逐渐变成单边。

因此研究 LP 时，范围是否覆盖当前价格非常重要。

五、如何查询池子的手续费率

手续费率和 LP 收益不是一回事。

例如：

Fee Tier = 0.25%

表示交易者每交易一定金额，需要支付相应比例的手续费。

但这并不意味着：

LP 收益 = TVL × 0.25%

实际手续费来源是：

交易量 × Fee

然后再根据协议规则和 LP 的有效流动性份额进行分配。

所以一个 LP 的收益大致取决于：

交易量

×

手续费率

×

自己在有效区间中的流动性占比

×

在区间内的时间

这比单纯看 APR 更有意义。

六、如何查询“未领取手续费”

这是整个查询中比较容易出错的部分。

Position 通常会保存：

liquidity

feeGrowthInside0LastX128

feeGrowthInside1LastX128

tokensOwed0

tokensOwed1

但：

tokensOwed0

tokensOwed1

并不一定等于当前实时全部手续费。

因为新的手续费可能已经累计，但 Position 状态还没有被更新。

更准确的思想是：

当前未领取手续费

≈

历史已记录 tokensOwed

+

当前 FeeGrowth

-

上次记录的 FeeGrowth

简化表达：

Fee0 =

tokensOwed0

+

Liquidity

×

(FeeGrowthInsideNow

-

FeeGrowthInsideLast)

÷ 2¹²⁸

Token1 同理。

七、更简单的方法：模拟 Collect

如果只是开发查询工具，第一版没必要马上自己实现完整 Fee Growth 数学。

可以使用：

eth_call

模拟调用：

collect()

也就是：

假装现在领取全部可领取资产，但是不真正发送交易。

节点会返回：

Token0 可领取数量

Token1 可领取数量

这种方法的优点是：

简单

不花 Gas

不改变链上状态

容易和官方前端对账

因此比较合理的开发顺序是：

第一版：

模拟 Collect

第二版：

Fee Growth 独立计算

第三版：

两套结果互相校验

八、Collect 不一定全部都是“利润”

这里有一个非常重要的细节。

LP 执行：

Decrease Liquidity

以后，如果本金还没有 Collect，那么：

Collect 返回资产

可能包含：

手续费

+

因此不能简单认为：

$$=$$

否则会把本金计算成利润。

九、如果想查询“某个钱包的所有 LP”

更合理的是：

↓

↓

然后统一显示：

$\left| \begin{array}{c} \vdots \\ \text{---} \\ \vdots \end{array} \right| \left| \begin{array}{c} \vdots \\ \text{---} \\ \vdots \end{array} \right| \left| \begin{array}{c} \vdots \\ \text{---} \\ \vdots \end{array} \right| \left| \begin{array}{c} \vdots \\ \text{---} \\ \vdots \end{array} \right| \left| \begin{array}{c} \vdots \\ \text{---} \\ \vdots \end{array} \right|$

#1002	\$18 ~ 22	Out	\$5,000	\$6
-------	-----------	-----	---------	-----

这样就可以实现类似专业 LP 查询网站的功能。

进一步可以查询整个 Pool 的 Position。

思路：

↓

最终可以得到类似：

\$16 ~ 17 ☐☐☐

\$17 ~ 18

\$18 ~ 19

\$19 ~ 20 ■■

它可以帮助判断：

哪些价格附近竞争最激烈

自己的 LP 是否应该变窄或变宽

十一、推荐的最小查询架构

如果以后自己开发工具，我更建议：

查询界面

↓

后端服务

↙ ↘

Indexer RPC

发现仓位 核验链上状态

↘ ↙

Multicall

↓

统一结果

原则是：

Indexer 用来“找”，RPC 用来“确认”。

不要完全依赖 Subgraph，因为索引可能延迟；也不要什么都从 RPC 扫链，因为效率很低。

十二、最值得记住的几个原则

研究集中流动性 LP 时，可以把复杂问题压缩成：

Pool ≠ Position

Fee Tier ≠ LP 实际收益

Token ID = LP 仓位的重要索引

Tick = LP 价格范围的底层表达

Active Range 决定当前资金效率

tokensOwed ≠ 一定等于实时全部手续费

Collect 金额 ≠ 一定全部都是手续费

Indexer 负责发现

RPC 负责确认

最终，一个可靠的 LP 查询工具，本质上只需要解决：

找到 Position → 还原 Range → 判断 Active → 计算 Fees。

掌握这条链路以后，不管是查询自己的 LP、研究别人的 LP 范围，还是分析整个池子的流动性分布，底层思路其实都是一样的。

笔记 010: 今日学习心得：

作者：Chic | 时间：2026-08-17 13:59 UTC | 字数：670 | 评分：53

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习心得：

今天主要学习了如何从 Etherscan 复盘真实链上交易，不再只看交易金额，而是从交易状态、交易类型、资产流向和 Gas 成本几个维度去判断一笔交易。

我重点区分了 Success、Fail 和 Pending 三种状态。Success 代表交易成功执行，资产变化已经发生；Fail 代表交易虽然上链了，但执行失败，资产兑换没有完成，不过 Gas 仍然会被扣；Pending 则表示交易还没有最终确认，最大的风险是时间不确定，价格、Route 和滑点都可能发生变化。

今天还复盘了 Uniswap V4 Universal Router 的成功和失败交易。成功 Swap 里，Value 可能显示为 0 ETH，但真正的资产变化要看 ERC-20 Tokens Transferred；失败 Swap 里，V4TooLittleReceived 说明实际收到数量低于最低接收数量，触发了滑点保护。这个案例让我更清楚地理解了：失败交易本身也是套利和链上操作中必须计入的成本。

最后学习了疑似 Backrun 的判断方法。Backrun 不是单看一笔交易，而是看大额 Swap 之后，是否有人紧跟着利用价格偏差进行交易。判断时需要看交易顺序、同一区块或相近时间、相关资产路径，以及扣除 Gas 后是否可能有净收益。

今天最大的收获是：链上交易分析不能只看表面金额，而要看状态、资产流向、成本和交易之间的因果关系。接下来我需要进行 Quote 观察、Etherscan 复盘和风险判断合并成一张完整的套利观察表。

笔记 011: Day 12 | 当前市场身份与盘口配对

作者：shiyang07ca | 时间：2026-08-17 14:24 UTC | 字数：1281 | 评分：79

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 | 当前市场身份与盘口配对

今日实际完成

从 Lighter 和 Hyperliquid xyz 的当前公开目录发现 341 个永续市场；未默认请求全部盘口，只检查四个明确指定的原油市场。

保存完整市场身份：场所、产品类型、场所命名空间、完整交易标识和本地编号。当前四个入口为

Lighter

WTI/145、BRENT OIL/159，以及 Hyperliquid xyz:CL/110029、xyz:BRENT OIL/110049。

实现并运行只读 scan 命令。最新结果为：300 个 catalog_only、37 个因目录状态停止、4 个 ready_for_market_mapping、请求错误 0。

完成互动课程 lessons/0008-day12-market-identity.html，并通过新场景迁移：Lighter 盘口响应不含可靠 symbol

时，要把请求前构造的完整身份、market_id、请求参数和响应绑定，不能按并发返回顺序分配市场。

理解 Hyperliquid metaAndAssetCtxs 的位置语义：先验证 universe 和 contexts 长度，再按原始 index 配对，最后才过滤下架市场；只过滤一边会让后续市场静默取得相邻市场的上下文。

Day 12 冻结响应测试 14/14 通过；历史回归测试 27/27 通过。代码与学习记录已提交并推送。

今天最重要的判断

ticker ↔ market_id 是必要映射，但单独使用 ticker

仍不足以作为跨场所市场身份。程序必须保留完整身份，并让目录记录、请求参数和盘口响应共同指向它。

active + two_sided

只表示数据入口当前存在，可以进入下一步市场映射；它不证明两个场所的合约是同一经济对象，也不证明存在可执行套利收益。

证据

learning-records/0010-day12-current-market-identity.md

lessons/0008-day12-market-identity.html

research/runs/day12-scan.json

research/manifests/day12-universe.json

src/monte_arb/

tests/test_day12_scan.py

研究状态与下一步

学习状态：Day 12 已完成。

交易研究状态：Blocked / 继续核验。尚未证明 WTI 与 xyz:CL、两个 BRENT OIL 的单位、底层基准、合约月份、价格来源、交易时段和结算方式一致；也未验证目标数量成交、滑点、费用、资金费或退出。

Day 13 将核验上述经济对象字段。只有确认两个市场确实可比后，才讨论跨场所价差；本次没有发送任何订单。

02 LI.FI与跨链（14 篇）

笔记 012: 链上套利残酷共学打卡 - 2026-08-16 (Day 12)

作者：KuTear | 时间：2026-08-16 15:02 UTC | 字数：1427 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学打卡 - 2026-08-16 (Day 12)

今日主题：跨结算域套利库存管理、借贷协议影子监控与三层信号引擎架构

一、跨链与跨市场非原子结算域的库存管理（Inventory Balancing）与终局性防护

1. 独立结算域的非原子性冲突与解耦：

- 单链原子事务 vs 跨链异步时滞：单链内套利可通过智能合约将多跳兑换封装在单笔原子交易内，失败即全量回滚；而跨链/跨市场套利涉及相互独立的结算域，无法共享全局原子状态。若采用串行“源域买入 $\$ \rightarrow \$$ 跨链传输 $\$ \rightarrow \$$ 目标域卖出”模式，在跨链传输与终局性确认窗口期内会形成严重的单边方向性敞口与价格滑移风险。

- 预置双边库存与异步再平衡（Rebalancing）：采用两端常驻流动性库存的并发执行模型，在发现价差时两端同时发起本地原子交易完成即时锁利，将跨链桥仅作为中长周期低频的“库存再平衡通道”，彻底隔离跨链时滞对套利执行确定性的负面影响。

2. 聚合路由拆分与冲击成本控制（Price Impact Mitigation）：

- 智能订单路由（SOR）多池拆单：针对深度受限的流动性池，引入多源聚合拆分机制，将大额套利指令按边际价格最优原则分流至多个流动性来源，非线性平滑单池价格冲击（Price Impact）。

- 全摩擦扣减的保守到账约束：结合滑点容忍度与做市差价，在路由生成端严格计算扣除 Gas、跨协议手续费及预估滑点后的净有效到账量（minAmountOut），避免名义价差被深度吞噬。

二、 借贷协议与复合套利路径的影子监控与净值鉴别 (Shadow Monitoring & Net Settlement)

1. 伪套利信号与负债现金残余穿透甄别：

- 资本结构穿透与真实 Net Edge 审计：针对涉及借贷协议、闪电借贷及循环质押的复合策略，建立穿透式资产负债核算模型。严格区分“借贷产生的名义现金残余”与“零负债纯净利润”，对存在残留借贷债务或未结清抵押风险的路径执行一票否决。

2. 只读影子监控管道 (Read-only Shadow Pipeline)：

- 链上状态镜像与清算边界跟踪：构建基于区块级状态快照、事件监听与本地状态游标的只读影子分析系统，实时追踪预言机喂价漂移、清算触发阈值及动态质押率 (LLTV) 变化，确保复合套利执行路径始终处于安全风控边界之内。

三、 信号监控系统的分层解耦与高可用工程化 (Event-Driven Pipeline)

1. 监控引擎三层解耦架构：

- 订阅层 (Ingestion Layer)：采用多节点事件驱动 (WebSocket/Log Filter) 与轻量轮询冗余备份模式，有效应对单一 RPC 节点的频控限制、网络抖动与连接中断，保障底层数据源的高可用性。

- 触发层 (Filter & Rule Engine)：建立内存级多级过滤网关，在事件流入时完成无状态轻量初筛与成本覆盖边际评估，大幅减少高开销的复杂模拟调用。

- 通知与执行层 (Action & Dispatch Layer)：通过异步消息总线将有效套利信号分发至执行模块或风控告警通道，实现毫秒级信号响应与全链路日志追溯。

笔记 013: LI.FI 學習筆記：從交易意圖到合約安全驗證 (B1-B6)

作者：OIK1380 | 时间：2026-08-16 15:24 UTC | 字数：19826 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LI.FI 學習筆記：從交易意圖到合約安全驗證 (B1-B6)

日期：2026-08-16\

課程位置：階段 B「LI.FI 研究接口與對象體系」全部完成\

運行邊界：OBSERVE_ONLY；只讀 Quote，不授權、不簽名、不廣播交易

閱讀地圖

今天完成的是一條完整的 LI.FI 只讀研究鏈路：

text

先寫下自己的交易意圖 (B1)

→ 查 LI.FI 當前支援能力 (B2)

→ 看懂 Quote / Route / Step (B3)

→ 逐層核對真實 Quote (B4)

→ 交易提交後按狀態與鏈上證據對帳 (B5)

→ 驗證實際調用的合約與 calldata (B6)

本章最重要的總原則是：

LI.FI 返回的是候選路線，不是安全證明。任何一層無法驗證，都應輸出明確的 STOP 或 UNKNOWN，不能靠名稱、畫面或猜測放行。

本日章節

| 課次 | 主題 | 狀態 |

| :- | :----- | :- |

| B1 | 交易意圖、最小單位與本地預檢 | 完成 |

| B2 | /chains、/tokens、/tools、/connections | 完成 |

| B3 | Quote、Route、Step 與內部步驟 | 完成 |

| B4 | Request → action → estimate → transactionRequest | 完成 |

| B5 | /status、退款、部分完成與最終對帳 | 完成 |

| B6 | 合約地址、calldata、ABI、版本與升級 | 完成 |

一、本課要解決的問題

在請求 LI.FI Quote 以前，先由本地策略明確規定：

text

允許哪兩條鏈？

允許哪兩個真實代幣地址？

允許多少金額與多少滑點？

資產最後必須由誰接收？

當前只允許研究，還是允許執行？

LI.FI 的 Quote 是候選方案，不能反過來修改使用者的交易意圖。

二、人類金額與最小單位

USDC 的 decimals = 6 時：

text

$20 \text{ USDC} \times 10^6 = 20,000,000$

API 金額應使用最小單位整數字串：

json

```
{
  "fromAmount": "20000000"
}
```

不能寫成浮點數，也不能把 "20" 當成 20 USDC；在最小單位語義中，"20" 只代表 0.000020 USDC。

正確轉換順序：

text

驗證 chainId + tokenAddress

→ 從可信 Registry 讀取 decimals

→ 解析人類金額字串

→ 轉成最小單位整數

→ 檢查精度與金額上下限

若 USDC 輸入 20.0000009，因為它有 7 位小數，無法用 6 位最小單位精確表示。系統不能擅自四捨五入或截斷，應輸出：

text

STOP: AMOUNT_PRECISION_EXCEEDED

三、金額上下限

金額規則必須使用相同代幣、相同鏈、相同最小單位整數比較：

text

$\text{minAmountRaw} \leq \text{fromAmountRaw} \leq \text{maxAmountRaw}$

本課策略：

text

$\text{minAmountRaw} = "1000000" // 1 \text{ USDC}$

$\text{maxAmountRaw} = "20000000" // 20 \text{ USDC}$ ，包含邊界

| 輸入 | 原始整數 | 結果 |

| :----- | :-----: | :----- |

| 20 USDC | 20000000 | PASS |

| 20.000001 USDC | 20000001 | STOP: AMOUNT_LIMIT_EXCEEDED |

| 20.0000009 USDC | 無法精確表示 | STOP: AMOUNT_PRECISION_EXCEEDED |

四、研究門與執行門必須分開

4.1 Research Gate

決定是否允許發出只讀 Quote 請求，檢查：

鏈和代幣身份；

最小單位及金額上下限；

接收地址；

滑點上限；

請求是否符合 IntentPolicy。

研究門通過後，可以取得 Quote、估算成本和保存證據；不代表可以執行。

4.2 Execution Gate

執行前還需要檢查：

模式是否允許執行；

來源資產餘額；

來源鏈 Gas；

Allowance 與條件式 Approve；

Quote 新鮮度；

transactionRequest 字段；

最新模擬；
全口徑保守淨利潤。

示例：USDC 餘額 25、Ethereum ETH Gas 為 0、模式為 OBSERVE_ONLY。只讀 Quote 可以研究，但執行輸出：

text

RESEARCH_GATE = PASS

EXECUTION_GATE = STOP

STOP: EXECUTION_DISABLED

STOP: SOURCE_GAS_INSUFFICIENT

4.3 實際盈虧只屬於執行後對帳

text

Quote / 模擬 → 預計或保守利潤

鏈上回執 + 真實餘額變化 → 已實現利潤

沒有發送交易時：

text

REALIZED_PNL = N/A

不能把模擬利潤標記成實際利潤。

五、IntentPolicy 字段表

| 分組 | 字段 | 用途 |

| :- | :----- | :----- |

| 模式 | mode | OBSERVE_ONLY / Paper / Execution 等明確狀態 |

| 身份 | fromChainId、toChainId | 限定來源鏈和目標鏈 |

| 身份 | fromTokenKey、toTokenKey | 使用 chainId + tokenAddress，不能只用 symbol |

| 身份 | receiverAddress | 限定最終接收資產的地址 |

| 金額 | fromAmountRaw | 本次請求的最小單位整數字串 |

| 金額 | minAmountRaw、maxAmountRaw | 本策略允許的包含式上下限 |

| 風險 | slippageBps、maxSlippageBps | 本次滑點與策略最大滑點 |

| 風險 | minNetProfitRaw | 全成本後最低保守利潤 |

| 風險 | maxQuoteAgeSeconds | Quote 最大允許年齡 |

| 權限 | quoteAllowed | 是否允許只讀詢價 |

| 權限 | executionEnabled | 是否允許進入執行流程 |

| 權限 | signingAllowed | 是否允許簽名 |

| 權限 | broadcastAllowed | 是否允許廣播 |

| 證據 | policyVersion、createdAt | 確認每筆研究使用哪一版規則 |

六、接收地址與合約地址

必須區分：

text

action.toAddress / receiverAddress

= 目標鏈最終接收資產的錢包

transactionRequest.to

= 來源鏈實際調用的路由/橋/DEX 合約

如果策略接收地址是 R，而 Quote 的 action.toAddress = X：

text

STOP: RECEIVER_MISMATCH

transactionRequest.to 是經過 Registry 驗證、鏈和版本匹配的 LI.FI 合約 C，並不要求它等於錢包 R。不能只看合約名稱，必須驗證真實地址、鏈、用途和允許版本。

七、本地 STOP 規則

text

STOP: CHAIN_NOT_ALLOWED

STOP: TOKEN_NOT_WHITELISTED

STOP: AMOUNT_PRECISION_EXCEEDED

STOP: AMOUNT_BELOW_MINIMUM

STOP: AMOUNT_LIMIT_EXCEEDED
STOP: SLIPPAGE_LIMIT_EXCEEDED
STOP: RECEIVER_MISMATCH
STOP: EXECUTION_DISABLED
STOP: SOURCE_GAS_INSUFFICIENT

只讀 Quote 也不能放過身份、金額、滑點和接收地址錯誤。錯誤 Quote 會污染研究數據，也可能在後續被誤用為可執行方案。

八、B1 驗收結果

小節	狀態
B1-1 最小單位轉換	已完成
B1-2 Research / Execution 雙門控	已完成
B1-3 金額上下限與精度	已完成
B1-4 IntentPolicy 與接收地址	已完成

text

B1：完成

下一課：B2 /chains、/tokens、/tools、/connections

九、官方資料

LI.FI：Endpoint Specifications (<https://docs.li.fi/agents/reference/endpoint-specs>)

LI.FI：Concepts and Objects (<https://docs.li.fi/agents/concepts>)

LI.FI：Data Schemas (<https://docs.li.fi/agents/reference/schemas>)

十、B2：能力目錄與策略門控

B2 的核心不是「LI.FI 有什麼就全部使用」，而是分開四種判斷：

text

/chains → LI.FI 是否收錄這條鏈

/tokens → LI.FI 是否收錄這條鏈上的精確代幣地址

/tools → LI.FI 當前列出哪些 Bridge / Exchange 工具

/connections → 指定鏈與代幣之間是否存在潛在連接

每一項 LI.FI 能力檢查後，仍要通過本地 IntentPolicy：

text

LI.FI_SUPPORTED = YES

不等於

POLICY_ALLOWED = YES

10.1 /chains

學會區分：

text

STOP: CHAIN_NOT_ALLOWED

STOP: CHAIN_NOT_SUPPORTED_BY_LIFI

STOP: QUOTE_CHAIN_MISMATCH

10.2 /tokens

代幣身份必須使用 chainId + address，不能只看 symbol、name、coinKey 或價格：

text

STOP: TOKEN_NOT_WHITELISTED

STOP: TOKEN_NOT_SUPPORTED_BY_LIFI

STOP: QUOTE_TOKEN_MISMATCH

10.3 /tools

Route 中每一個工具都要檢查，不能只檢查主 Bridge。工具存在但不在本地 allowlist 時：

text

STOP: BRIDGE_NOT_ALLOWED

STOP: EXCHANGE_NOT_ALLOWED

STOP: TOOL_NOT_SUPPORTED_BY_LIFI

10.4 /connections

Connection 表示可以進入路線搜尋，不表示指定金額當前一定有 Quote：

text

精確連接不存在

→ STOP: CONNECTION_NOT_AVAILABLE

連接存在，但指定金額沒有Quote

→ STOP: QUOTE_NOT_AVAILABLE

CONNECTION_GATE = PASS 也不能證明流動性、成本、Quote 新鮮度、Gas、可執行性或利潤。

十一、B2 實踐快照

2026-08-16 已實際只讀查詢 LI.FI：

/chains?chainTypes=EVM

/tokens?chains=1,8453

/tools

/connections，並指定 Ethereum 官方 USDC → Base 官方 USDC

結果：兩條鏈及兩個官方 USDC 地址均被收錄；精確 connection 返回 1 條。沒有請求 Quote、沒有連接錢包、沒有 Approve、沒有簽名或發送交易。

完整帶時間戳的工具清單與驗收結果見：

2026-08-16 LI.FI B2 只讀能力快照 (./2026-08-16_LIFI_B2只讀能力快照.md)

十二、B2 驗收結果

| 小節 | 狀態 |

| :----- | :-- |

| B2-1 /chains | 已完成 |

| B2-2 /tokens | 已完成 |

| B2-3 /tools | 已完成 |

| B2-4 /connections | 已完成 |

| B2 實際只讀快照 | 已完成 |

text

B2：完成

下一課：B3 Quote / Route / Step 對象結構

十三、B3：Quote、Route、Step 與 Included Steps

13.1 本章問題

LI.FI 的返回資料有好幾層。如果把「內部路徑」誤認為「錢包要逐筆簽名的交易」，或只檢查頂層工具而忽略內部步驟，安全判斷都會出錯。

13.2 四個核心名詞

| 名詞 | 白話意思 | 主要用途 |

| :----- | :----- | :----- |

| Quote | 指定金額下的一個即時候選報價 | 取得一個較直接的可執行方案 |

| Route | 從多個候選中組成的一條完整路線 | 比較不同路線、工具和步驟 |

| Step | 路線中需要完成的一個主要動作 | 例如同鏈兌換或跨鏈 |

| Included Step | 某個 Step 內部包含的子動作 | 例如先收費，再把剩餘資產交給 Across |

資料關係可以理解為：

text

Route

└── Step 1

└── Included Step A

└── Included Step B

includedSteps 是內部執行路徑，不代表錢包一定要為每一個分別簽名。真正要簽多少筆，必須以需要提交的交易請求為準。

Included

Step

13.3 為甚麼不能只看頂層 Bridge

一條頂層顯示 Across 的路線，內部仍可能包含：

協議收費；

來源鏈 DEX 或聚合器操作；

授權目標；
跨鏈 Bridge；
目標鏈兌換或調用。

所以每個工具、代幣、鏈、金額和接收地址都要檢查，不能因為頂層工具在白名單就全部放行。

13.4 Step 連續性

相鄰步驟必須能夠接起來：

text

前一步輸出的鏈 = 後一步輸入的鏈
前一步輸出的代幣 = 後一步輸入的代幣
前一步輸出的金額 = 後一步實際使用的金額

金額即使只差一個最小單位，也不能擅自忽略。否則成本、收益率和報價比較就不再使用同一個本金口徑。

常用停止原因：

text

STOP: STEP_CHAIN_DISCONTINUITY
STOP: STEP_TOKEN_DISCONTINUITY
STOP: STEP_AMOUNT_DISCONTINUITY
STOP: BRIDGE_NOT_ALLOWED
STOP: EXCHANGE_NOT_ALLOWED
STOP: TOOL_NOT_SUPPORTED_BY_LIFI

13.5 B3 結論

text

頂層工具匹配
≠ 所有內部工具匹配
≠ 所有步驟連續
≠ 需要簽多筆交易
≠ 可以執行

十四、B4：從 Request 到 Transaction Request 的完整復核

14.1 四層資料的責任

層級	它回答的問題	檢查重點
Request / IntentPolicy	我原本想做甚麼？	鏈、代幣、金額、接收人、滑點、工具限制
action	LI.FI 理解成甚麼交易？	必須逐字段回對 Request
estimate	預計收到多少、花多少時間和費用？	toAmountMin、Fee、Gas、時間、Approve 地址
transactionRequest	錢包實際會向鏈上提交甚麼？	chainId、from、to、value、data

驗證方向永遠是：

text

本地 IntentPolicy
→ Request
→ action
→ estimate / includedSteps
→ transactionRequest

不能反過來讓 Quote 修改本地意圖。

14.2 transactionRequest.chainId 的正確含義

它表示「這一筆交易要提交到哪條鏈」，不是整條跨鏈路線的終點。

例如：

text

Ethereum USDC → Base USDC

跨鏈是從 Ethereum 開始，因此來源鏈交易應提交到 Ethereum：

text

Request.fromChain = 1
action.fromChainId = 1

transactionRequest.chainId = 1

如果這筆來源鏈交易卻要求提交到 Base (8453)，才是鏈不一致。跨鏈完成後的目標鏈行為要按其自己的 Step 或狀態證據判斷。

14.3 value 不是所有交易的「投入本金」

transactionRequest.value 表示這筆合約調用隨交易附帶多少來源鏈原生幣。

用 ERC-20 USDC 作為投入：fromAmount = 20 USDC，通常 value = 0；

直接投入原生 ETH：本金可能出現在 value；

中途把 ETH 包裝成 WETH：WETH 是 ETH 的 ERC-20 形態，不能再把同一筆 ETH 當成第二份投入；

value 的用途不明或與意圖不符：停止並查明，不能自行解釋。

因此原生幣投入、WETH 中間形態和 Gas 必須分開記帳，避免重複計算。

14.4 真實 20 USDC Quote 快照

2026-08-16 的只讀研究條件：

text

Ethereum Circle 官方 USDC

→ Base Circle 官方 USDC

金額：20 USDC

滑點：0.5%

Bridge：Across

Exchange：1inch

返回的主要結果：

| 項目 | 結果 |

| :----- | -----: |

| fromAmount | 20.000000 USDC |

| toAmountMin | 19.945438 USDC |

| 已包含費用 | 0.054562 USDC |

| 來源鏈 Gas 估值 | 0.0428 USD |

| 加 20% Gas Buffer | 0.05136 USD |

已包含費用不能重複扣除：

text

20.000000 - 0.054562 = 19.945438 USDC

暫按 1 USDC = 1 USD，保守結果為：

text

19.945438 - 20.000000 - 0.051360

= -0.105922 USD

所以這份 Quote 可以作為研究資料，但不能通過盈利門：

text

STOP: MIN_NET_PROFIT_NOT_MET

完整原始快照見：

B4 真實 Quote 只讀快照 (./2026-08-16_LIFI_B4真實Quote只讀快照.md)

14.5 真實 Quote 的 Included Steps

text

Step 1：feeCollection

20.000000 Ethereum USDC

→ 19.950000 Ethereum USDC

Step 2：Across

19.950000 Ethereum USDC

→ 19.945438 Base USDC

兩步的鏈、代幣與金額能接上，因此本次：

text

STEP_CONTINUITY_GATE = PASS

但路徑連續只表示資料結構能銜接，不表示合約安全、Quote 新鮮、餘額足夠或交易盈利。

14.6 本課掌握的成本口徑

保守淨結果應從最小收到量開始：

text

保守淨結果

= 目標資產最小收到價值

- 原始投入價值
- 所有未包含費用
- Gas × (1 + Buffer)
- 條件性 Approve Gas
- 目標鏈 Gas 準備成本
- 必要的庫存再平衡成本

只有 Quote 而沒有鏈上交易時：

text

REALIZED_PNL = N/A

十五、B5：/status、跨鏈狀態與最終對帳

15.1 為甚麼提交交易不等於完成

跨鏈不是單鏈原子交易。來源鏈交易成功，只能證明來源鏈那一步已執行；目標鏈可能仍在等待、部分完成、退款或需要人工恢復。

text

SUBMITTED

≠ 已上鏈

≠ 來源鏈成功

≠ 目標鏈到帳

≠ 整條路線成功

15.2 狀態的白話含義

| 狀態 | 白話解釋 | 正確處理 |

| :----- | :----- | :----- |

| NOT_FOUND | LI.FI 暫時沒有找到這筆狀態 | 先查來源鏈 receipt；不要盲目重發 |

| PENDING | 跨鏈仍在進行 | 繼續追蹤，不計已實現利潤 |

| DONE / COMPLETED | 系統認為完整完成 | 仍要核對目標鏈 receipt、日誌和餘額 |

| DONE / PARTIAL | 只有部分路徑完成 | 進入恢復與人工處理，不算完整成功 |

| DONE / REFUNDED | 資產已退回或退回完成 | 路線意圖失敗；按真實退款和 Gas 對帳 |

| FAILED | 路線被判定失敗 | 查明資產位置、退款狀態與已消耗成本 |

NOT_FOUND 可能只是索引延遲。直接重發可能造成重複交易，因此必須先查鏈上 receipt 和 nonce 狀態。

15.3 Receipt、Logs 與 Balance 各自證明甚麼

| 證據 | 能證明的事 | 不能單獨證明的事 |

| :----- | :----- | :----- |

| Receipt | 某條鏈上的交易成功或失敗、實際 Gas | 目標資產一定已正確到帳 |

| Logs | 合約發出了哪些事件 | 使用者可支配餘額一定增加 |

| Balance Delta | 錢包最終持有資產的淨變化 | 單獨解釋每個中間動作 |

日誌顯示 Transfer，但餘額沒有增加，可能因為同一筆交易後續又把代幣轉出、事件並非最終接收資產，或完整交易最終回滾。日誌是交叉驗證證據，不能代替最終餘額。

15.4 失敗交易的資產與 Gas

如果交易最終失敗：

狀態修改和 value 轉移會回滾；

已消耗的 Gas 不會退回；

因此可能出現「資產變化為零，只損失 Gas」。

記帳時應以交易前後真實餘額差為主，並用 receipt 的 gasUsed × effectiveGasPrice 計算實際 Gas。

15.5 退款不等於成功

如果投入資產最後退回：

text

資產被找回 = 是

原交易意圖完成 = 否

成本為零 = 不一定

仍可能損失來源鏈

Gas、Bridge/Relayer

費、退款費和時間成本。退款中的資產屬於「未決資產」，不能提前當作已回到可用餘額。

15.6 最終已實現盈虧

完成或終止後，按實際餘額變化計算：

text

已實現盈虧

= 所有最終收到資產的可回收價值

- 所有實際付出資產的價值

- 所有鏈的實際 Gas

- 未包含的協議費、退款費和再平衡成本

核心證據順序：

text

來源鏈 receipt

→ 目標鏈 receipt / Bridge 證據

→ 兩邊 logs

→ 交易前後 balance delta

→ 實際 Gas

→ 最終狀態與已實現盈虧

15.7 B5 的停止與未知狀態

text

STOP: STATUS_NOT_FINAL

STOP: PARTIAL_COMPLETION

STOP: REFUND_PENDING

STOP: SOURCE_TRANSACTION_FAILED

UNKNOWN: TARGET_RECEIPT_NOT_VERIFIED

UNKNOWN: BALANCE_RECONCILIATION_FAILED

十六、B6：合約地址、Calldata 與版本安全

16.1 本章為甚麼最容易產生假安全感

看到 `LI.FI` 的官方地址，只能證明「交易送到哪個入口」。真正會做甚麼，由指令、參數以及該指令當前對應的合約模組共同決定。

calldata

text

地址正確

≠ 程序一定存在

≠ 指令種類正確

≠ 指令參數正確

≠ 合約版本仍是已驗證版本

16.2 本章專有名詞

| 名詞 | 白話解釋 |

| :----- | :----- |

| chainId | 鏈的編號，例如 Ethereum = 1、Base = 8453 |

| EOA | 由私鑰控制的普通帳戶，通常沒有鏈上程序 |

| Contract | 部署在鏈上的程序 |

| Bytecode | 合約編譯後，鏈上真正執行的機器指令 |

| RPC | 用來向區塊鏈節點查詢資料的接口 |

| eth_getCode | 讀取某條鏈、某個地址上的程序 |

| Calldata | 交易交給合約的完整指令與參數 |

| Selector | calldata 最前面的 4 bytes，表示要調用哪種功能 |

| ABI | 把 calldata 翻譯成人能閱讀的函數與參數的說明書 |

| Diamond | LI.FI 的總入口／前台合約 |

| Facet | Diamond 後面實際處理某種功能的模組 |

| Registry | 本地批准的鏈、地址、用途、版本和證據清單 |

16.3 五層合約檢查

text

1. 鏈是否正確？
2. 地址是否與官方清單及本地 Registry 一致？
3. 該地址是否真的存在程序？
4. calldata 要調用甚麼指令？
5. 指令中的代幣、金額、鏈、接收人等參數是否符合 Intent？

任一層未知都不能放行。

16.4 transactionRequest.to 與 data

text

transactionRequest.to = 把交易送給哪個合約

transactionRequest.data = 要該合約做甚麼，以及使用甚麼參數

如果 Intent 的收款地址是 R，但 calldata 解碼後的收款地址是 X：

text

$R \neq X$

→ STOP: CALldata_RECEIVER_MISMATCH

這裡改變的是「收款人」，不一定是發送交易的錢包。

16.5 本日真實合約證據

本次只讀核對的 Ethereum LI.FI Diamond：

text

chainId = 1

address = 0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE

兩個獨立 Ethereum RPC 節點讀取的 runtime bytecode 完全一致：

text

程序長度：5176 bytes

本地 SHA-256 指紋：

72211202d3d54f9e8b4590855d8f8390f1fad2d55c2472a3e488b58ccb61c2e9

這個 SHA-256 只是本地比較程序是否相同的「指紋」，不是 Ethereum 原生的 Keccak code hash。

以上證據只能證明：

當時 Ethereum 的該地址存在程序；

兩個獨立節點返回相同 bytecode。

它不能單獨證明功能安全、參數正確或內部 Facet 沒有改變。

16.6 真實 Selector 驗證

B4 Quote 的 calldata selector：

text

0x1794958f

按 LI.FI 官方 ABI，它對應：

text

swapAndStartBridgeTokensViaAcrossV4

白話拆解：

swap：在來源鏈進行必要的代幣處理或兌換；

startBridge：開始跨鏈；

ViaAcrossV4：使用 Across V4。

這與 Quote 聲明的 Across 路線方向一致，但 selector 只證明「指令種類」。在解碼全部參數以前，仍然必須：

text

STOP: CALldata_PARAMETERS_NOT_VERIFIED

16.7 Calldata 必須核對的關鍵資料

不用先背完整 ABI，但至少也要檢查：

| 參數 | 要回答的問題 |

| :----- | :----- |

receiver	最終資產交給誰？
refundAddress	失敗退款退給誰？
sendingAsset	實際花的是哪個代幣？
receivingAsset	實際收到的是哪個代幣？
minAmount / fromAmount	合約實際使用多少本金？
destinationChainId	資產要去的鏈是否正確？
outputAmount	路徑使用的目標輸出量是否一致？
deadline	指令是否仍在有效時間內？
swapData	來源鏈內部調用了誰、授權給誰、花了多少？

每一項都要與 Request、action、estimate 和本地 IntentPolicy 交叉核對。

16.8 Diamond 升級風險

LI.FI Diamond 像固定前台，Facet 像後面的功能部門。升級時可能出現：

text

昨天：同一 selector → Across 模組 A

今天：同一 selector → Across 模組 B

Diamond 地址和 selector

可以都不變，但真正執行的程序已經換了。升級不代表新程序一定有問題；它代表「昨天的驗證不能自動證明今天的版本」。

因此發現版本或 Facet 改變時：

text

STOP: CONTRACT_VERSION_CHANGED

重新驗證新版本後，還要重新取得新 Quote，避免拿舊 Quote 配合新程序。

16.9 Registry 最低字段

text

chainId

diamondAddress

用途

selector

selector 對應的 facetAddress / 版本

已批准的程序指紋

ABI 來源與版本

驗證區塊與驗證時間

policyVersion

Registry 不是永久真理，而是一份有來源、有時間、有版本的批准記錄。

16.10 B6 常用停止原因

text

STOP: CONTRACT_NOT_FULLY_VERIFIED

STOP: CONTRACT_CODE_MISSING

STOP: CONTRACT_ADDRESS_MISMATCH

STOP: CONTRACT_VERSION_CHANGED

STOP: SELECTOR_NOT_ALLOWED

STOP: CALldata_PARAMETERS_NOT_VERIFIED

STOP: CALldata_RECEIVER_MISMATCH

STOP: CALldata_TOKEN_MISMATCH

STOP: CALldata_AMOUNT_MISMATCH

STOP: CALldata_DESTINATION_CHAIN_MISMATCH

STOP: CALldata_REFUND_ADDRESS_MISMATCH

十七、完整只讀安全檢查順序

下面是 B1–B6 合併後的固定流程：

text

1. IntentPolicy

└─ 明確鏈、代幣、金額、接收人、滑點、工具、模式

2. 本地 Request Gate

└─ 最小單位、精度、上下限、官方代幣白名單

3. LI.FI Capability Gate

└─ /chains、/tokens、/tools、/connections

4. Quote Object Gate

└─ Request ↔ action ↔ estimate

5. Route / Step Gate

└─ 所有工具、鏈、代幣和相鄰金額連續

6. Transaction Request Gate

└─ chainId、from、to、value、data

7. Contract / Calldata Gate

└─ 官方地址、鏈上程序、selector、ABI、參數、Facet 版本

8. Cost / Profit Gate

└─ toAmountMin、未包含 Fee、Gas Buffer、Approve、目標 Gas、再平衡

9. Execution Gate

└─ 當前仍為 OBSERVE_ONLY，所以固定停止

10. Status / Reconciliation

└─ 只有真的提交後才使用；receipt、logs、balance、實際 Gas 對帳

目前模式固定為：

text

QUOTE_ALLOWED = YES

EXECUTION_ENABLED = NO

SIGNING_ALLOWED = NO

BROADCAST_ALLOWED = NO

十八、今天修正的常見誤區

| 誤區 | 正確理解 |

| :----- | :----- |

| Connection 存在就一定有 Quote | Connection 只表示可能搜尋路線；指定金額仍可能無 Quote |

| LI.FI 支援就能使用 | 還要通過本地 Policy 和工具白名單 |

| symbol 是 USDC 就是官方 USDC | 必須核對 chainId + 官方合約地址 |

| 頂層 Across 合法，內部步驟就都合法 | 每個 Included Step 和工具都要獨立驗證 |

| transactionRequest.chainId 應等於目標鏈 | 它表示該筆交易實際提交的鏈；來源鏈跨鏈交易通常是來源鏈 |

| value 永遠等於全部投入 | value 只表示隨合約調用附帶的原生幣 |

| included: true 的費用還要再扣一次 | 已反映在輸出量的費用不能重複扣除 |

| SUBMITTED 就是成功 | 它只表示已提交，甚至可能還未被區塊收錄 |

| 日誌說收到就等於最終到帳 | 最終以餘額變化為主，日誌作交叉驗證 |

| REFUNDED 等於路線成功且沒有損失 | 只表示資產退回；原意圖失敗且可能已付費與 Gas |

| 合約地址正確就安全 | 還要驗證程序、calldata 指令、參數和版本 |

| selector 正確就能執行 | selector 只說明指令種類；參數才描述具體交易 |

| 地址沒變就代表合約沒變 | Diamond 的內部 Facet 可以升級 |

十九、章末檢查表

面對任意 LI.FI Quote，現在應能依次回答：

[] 這是不是我原本允許的鏈、代幣、金額和接收人？

[] 代幣是否使用官方地址，而不是只看名稱？

[] LI.FI 當前是否支援精確的鏈、代幣和連接？

[] Quote 是單一 Step 還是多步 Route？

[] 每個工具都在本地 allowlist 嗎？

[] 相鄰 Step 的鏈、代幣和金額能完全接上嗎？

[] action 是否完整回對 Request？

[] toAmountMin、Fee 和 Gas 是否避免漏算或重複計算？

[] transactionRequest.chainId/from/to/value/data 是否一致？

[] 合約地址是否有官方來源和鏈上程序證據？

[] selector 是否存在於批准的 ABI？

- [] calldata 中的收款人、退款人、代幣、金額和目標鏈是否一致？
- [] selector 對應的 Facet／版本是否仍是批准版本？
- [] Quote 是否新鮮，保守淨利是否達到門檻？
- [] 當前模式是否允許執行？本課答案固定為否。

二十一、官方參考資料

LI.FI : Concepts and Objects (<https://docs.li.fi/agents/concepts>)
LI.FI : Endpoint Specifications (<https://docs.li.fi/agents/reference/endpoint-specs>)
LI.FI : Get a Quote (<https://docs.li.fi/li.fi-api/li.fi-api/requesting-a-quote>)
LI.FI : Transaction Status Tracking (<https://docs.li.fi/introduction/user-flows-and-examples/transaction-status-tracking>)
LI.FI : Smart Contract Architecture (<https://docs.li.fi/introduction/lifi-architecture/smart-contract-overview>)
LI.FI : Smart Contract Addresses (<https://docs.li.fi/introduction/lifi-architecture/smart-contract-addresses>)
LI.FI 官方 Diamond ABI (<https://github.com/lifinance/lifi-contract-types/blob/main/dist/diamond.json>)
Circle : USDC Contract Addresses (<https://developers.circle.com/stablecoins/usdc-contract-addresses>)

笔记 014: 【Day 10 打卡】2026/8/14

作者：ed11555 | 时间：2026-08-16 15:27 UTC | 字数：934 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 10 打卡】2026/8/14

今日學習：初步整合 — 套利分析工具

完成內容

1. 整合 DexScreener 價格監控
2. 整合 LI.FI 跨鏈報價
3. 建立簡單套利分析腳本

實作代碼

python

import requests

步驟 1：獲取 DEX 價格

```
dex_resp = requests.get(
    "https://api.dexscreener.com/latest/dex/search",
    params={"q": "USDC/WETH"},
    headers={"User-Agent": "Mozilla/5.0"}
)
```

pairs = dex_resp.json().get("pairs", [])

步驟 2：計算 DEX 間價差

```
prices = [float(p["priceUsd"]) for p in pairs if p.get("priceUsd")]
spread = (max(prices) - min(prices)) / min(prices) * 100
print(f"DEX 間最大價差: {spread:.4f}%")
```

步驟 3：整合分析

毛利 = 價差 x 本金

淨利 = 毛利 - (Gas + 橋費 + 滑點)

```
print(f"假設 $10,000 本金，價差 {spread:.4f}%")
```

```
print(f"毛利: ${10000 * spread / 100:.2f}")
```

```
print(f"成本估算: Gas $5 + 橋費 $2 + 滑點 $10 = $17")
```

```
print(f"淨利: ${10000 * spread / 100 - 17:.2f}")
```

關鍵發現

- 整合後可快速判斷套利可行性
- 需要同時考慮 DEX 間價差 + 跨鏈成本
- 小資金在以太坊主網幾乎必然虧損

學習狀態

- 整合價格監控：
- 整合 LI.FI 報價：
- 完成分析腳本：

笔记 015: 1\.

实操目标本阶段目标是完成从API接入到小额闭环验证的完整流程，重点验证「价差 ≠ 利润」这一

作者：XCHA000 | 时间：2026-08-16 15:40 UTC | 字数：1297 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1\、 实操目标本阶段目标是完成从API接入到小额闭环验证的完整流程，重点验证「价差 ≠ 利润」这一核心认知，并建立可复现的操作记录习惯。 \ 不追求盈利，只追求流程跑通与数据完整。

2\、实操环境与准备工作已完成事项：

注册 LI.FI Partner Portal，获取 API Key

注册 Binance Web3 API（用于交叉验证）

新建独立小额测试钱包（金额控制在可承受全部损失范围内）

准备至少1个托管RPC（QuickNode/Alchemy等）

明确私钥管理原则：业务逻辑与私钥分离

工具清单：

数据层：LI.FI API（主）+ Binance Web3 API（辅）

行情辅助：DEX Screener

执行：手动/半自动（本阶段不引入全自动脚本）

记录：本地表格或简单数据库

3\、实操步骤与执行记录

步骤1：API连通性测试

调用 LI.FI /v1/keys/test 验证Key有效

完成一次同链 Quote 查询

完成一次跨链 Route 查询

结果：连通正常，返回数据完整

步骤2：机会筛选与全成本计算

选取测试路径：同链主流稳定币或ETH相关交易对（真实规模）

使用LI.FI获取有限规模报价

按顺序计算：滑点 + 协议费 + 预估Gas + 其他成本

初步结论：多数肉眼可见价差在真实规模下净利润接近0或为负

步骤3：小额闭环执行

执行金额：极小额度（可接受归零）

执行方式：手动确认 + 签名

记录内容：

预期净利润

实际到账金额

实际支付的全部费用

执行耗时

是否出现滑点超出预期、失败或状态变化

步骤4：复盘

对比「预期 vs 实际」差异

归因：报价时效、滑点、Gas波动、路径完整性等

记录失败或偏差原因（如有）

4\、关键发现与数据洞察

1. 价差幻觉普遍存在\

静态或小额报价显示的正利润，在真实规模顺序计算后往往大幅缩水甚至转负。

2. 成本项容易被低估\

Gas + 优先费、滑点、跨链等待成本是最容易漏算的部分。

3. 安全流程必须前置\

独立钱包 + 权限最小化从第一天就要执行，否则后续风险会指数上升。

4. 记录习惯比单次盈利更重要\

完整的日志（路径、规模、成本明细、时间戳、实际结果）是后续优化的唯一依据。

5\. 问题与风险记录

当前主要限制：API限流、手动执行速度慢、对非原子路径库存管理经验不足

已识别风险：私钥管理、报价时效性、公共RPC延迟

应对措施：严格小额、独立签名原则、优先使用私有通道（下一阶段）

6\. 下一步行动计划（第二阶段）

1. 每日/隔日进行机会扫描 + 7步检查流程练习（强制记录）

2. 完成至少10次小额闭环并建立简单复盘表格

3. 开始编写最小扫描脚本（查询 + 全成本计算 + 日志输出）

4. 研究并测试独立Signer方案

5. 评估是否引入Hermes辅助监控与归因

笔记 016: 今天终于把 Hermes Agent 的配置问题基本处理完了，Terminal 和 Desktop

作者：Yannis | 时间：2026-08-16 15:41 UTC | 字数：525 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天终于把 Hermes Agent 的配置问题基本处理完了，Terminal 和 Desktop App 都已经能够正常使用，所以今天开始把注意力从“怎么把工具装起来”转到“这个工具到底怎么参与套利研究”。

我之前对 Agent 的理解其实比较模糊，会下意识觉得它是不是一种更高级的聊天机器人。今天重新梳理之后发现，更准确的理解应该是：大模型本身负责理解任务和做判断，Hermes 给模型接上了网页搜索、浏览器、Terminal、代码执行、文件操作这些工具。这样它就不只是告诉我“应该怎么查”，而是可以自己去查资料、读取结果，再继续处理数据。Hermes

官方文档里也确实把这些能力拆成了不同的 toolset，可以根据任务决定开放哪些工具。

这和链上套利的关系也开始清楚了一点。套利本身不是“让 AI 猜哪个币会涨”，而是发现不同市场之间暂时存在的价格差，再判断这个价格差扣掉 Gas、DEX 手续费、滑点、跨链费用、时间成本和失败风险以后还有没有利润。所以 Hermes 更适合先承担前面的信息收集和数据处理工作，比如查询 LI.FI 的报价和路线、整理不同路径的成本，甚至之后用 Python 批量计算，而不是一开始就让它拿钱包自动交易。

笔记 017: 8月16号\

作者：Krystal6633 | 时间：2026-08-16 15:57 UTC | 字数：1284 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月16号\

\

四、把这些类型放回同一张套利地图

下面这张表不再重新发明一套分类，只把前面的类型放在同一张图上，方便以后遇到新案例时定位

翻译：第四章就是把前面讲的14种方法，重新整理成一张“大地图”。这张表没有发明新的分类，纯粹是为了让你以后看到一个机会时，能快速在地图上找到它属于哪一类。

（第三部分（14种套利类型）的“速查备忘录”或“便携小卡片”。）

打个比方：第三部分是“详细的菜谱”（教你鱼香肉丝怎么切、怎么腌、油温多少），而第四部分这张表是“点菜快查表”（只告诉你这道菜用什么肉、什么味型、容易糊在哪里）。

这个部分是把14种类型压缩成了每行只有3个关键信息的表格。

套利地图速查

跨中心化交易所：价差来源：两个订单簿的供需不同；

闭环：两边同时买卖，再平衡库存；

主要风险：深度、充提限制、交易所风险。

跨 DEX：价差来源：不同协议池的状态不同；闭环：同链逐腿交换；主要风险：状态变化、Gas、执行竞争。

CEX-DEX：价差来源：链上与交易所更新不同步；闭环：两边预放库存并同时成交；主要风险：两边不能一起回滚。

同一 DEX 不同池：价差来源：费率档、版本或路由不同；闭环：协议内部换入再换出；主要风险：价格影响和聚合器竞争。

三角与多跳：价差来源：多个相对价格没有闭合；闭环：回到起始资产；主要风险：每一步都会改变下一步报价。

跨链：价差来源：资金不能及时跨链流动；闭环：跨链或两边库存再平衡；主要风险：到账时间、桥和库存风险。

铸造与赎回：价差来源：市价偏离协议转换价值；闭环：mint、redeem、wrap
unwrap；主要风险：通道暂停、额度和等待时间。

或

现货与到期合约：价差来源：现在价格和未来结算价不同；闭环：到期交割或价差收敛；主要风险：保证金、借币和交割规则。

永续资金费率：价差来源：多空持仓需求不平衡；闭环：对冲价格方向并收取费率；主要风险：费率反转和清算。

利率与收益率：价差来源：市场、期限或收益结构不同；闭环：借贷、持有到期或转换；主要风险：收益变化、协议和流动性风险。

清算：价差来源：协议为偿债提供奖励；闭环：偿债、接收并卖出抵押物；主要风险：预言机、Gas 和机器人竞争。

拍卖：价差来源：风险资产通过竞价出售；闭环：竞拍后在外部市场变现；主要风险：成交不确定和卖出深度。

订单流与状态变化：价差来源：即将发生的交易改变状态；闭环：在状态变化后恢复价格；主要风险：排序、打包和执行竞争。

消息与事件结算：价差来源：可验证的转换或结算价值改变；闭环：转换、结算或互补头寸；主要风险：规则、资格、争议和流动性。

同一笔交易可能横跨多行，但归类时仍然只选择利润的主要来源。其他部分放进执行路径和风险说明里，不再重复制造新的“套利类型”。

笔记 018: 8月15日（周六）—— 自动化测验：用Python脚本批量获取报价

作者：Cool-CoCo | 时间：2026-08-16 15:58 UTC | 字数：2963 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月15日（周六）—— 自动化测验：用Python脚本批量获取报价

昨天手动测试了4个案例，效率太低了。今天的目标是写一个Python脚本，自动获取报价并保存到CSV文件。

一、脚本设计思路

核心功能：

1. 读取测试用例列表（从CSV或直接写在代码中）
2. 对每个用例，调用LI.FI API获取报价
3. 解析返回的JSON，提取关键字段
4. 将结果保存到CSV文件，方便后续分析

需要的库：

requests：发送HTTP请求

json：解析返回数据

csv：保存结果

time：添加时间戳和延迟

二、动手写脚本

步骤1：安装依赖

```
bash
```

```
pip install requests
```

步骤2：编写脚本

```
python
```

```
import requests
```

```
import json
```

```
import csv
```

```
import time
```

```
from datetime import datetime
```

```
LI.FI API基础URL
```

```
BASE_URL = "https://li.quest/v1/quote"
```

```
测试用例列表
```

```
test_cases = [
```

```
# (源链, 目标链, 源资产, 目标资产, 金额, 排序方式, 测试ID)
```

```
"""调用LI.FI API获取报价"""
```

```
params = {
```

```
"fromChain": from_chain,
```

```
"toChain": to_chain,
```

```
"fromToken": from_token,
```

```
"toToken": to_token,
```

```
"fromAmount": amount,
```

```
"order": order
```

```
}
```

```
try:
```

```
"""从返回数据中提取关键字段"""
```

```
if not data:
```

```
return None
```

```
# 提取Gas成本
```

```
gas_costs = data.get("gasCosts", [])
```

```
total_gas = sum(float(gas.get("amount", "0")) for gas in gas_costs) if gas_costs else 0
```

```
# 提取费用成本
```

```
fee_costs = data.get("feeCosts", [])
```

```
total_fees = sum(float(fee.get("amount", "0")) for fee in fee_costs) if fee_costs else 0
```

```
# 提取路径信息
```

```
steps = data.get("includedSteps", [])
```

```
tools = [step.get("tool", "unknown") for step in steps]
```

```
return {
```

```
}
```

```
执行测试
```

```
# 每个请求间隔1秒，避免被限流
```

```
time.sleep(1)
```

```
保存到CSV
```

三、运行脚本

```
bash
```

```
python lifi_scanner.py
```

预期输出：

正在测试 T1: ARB→OPT | USDC→USDC | 100000000
正在测试 T2: ARB→BASE | USDC→USDC | 100000000
正在测试 T3: ETH→ARB | USDC→USDC | 100000000
正在测试 T10: ARB→OPT | USDC→USDC | 10000000000
正在测试 T6: ARB→OPT | ETH→ETH | 100000000000000000
结果已保存到 lifi_quote_results.csv，共 5 条记录

四、查看结果

打开生成的CSV文件，你会看到类似这样的表格：

表格

testId	fromChain	toChain	fromToken	toToken	amount	toAmount	gasCosts	feeCosts	tools	estimatedDuration
T1	ARB	OPT	USDC	USDC	100000000	99950000	0.0000365	5.00	1inch → Across → Uniswap	30
T2	ARB	BASE	USDC	USDC	100000000	99954000	0.0000380	4.50	1inch → Stargate → Uniswap	45
T10	ARB	OPT	USDC	USDC	10000000000	9999500000	0.0000400	5.00	1inch → Across → Uniswap	30

五、今日总结

今天成功编写了一个自动化脚本，可以批量获取LI.FI报价：
核心功能：读取测试用例 → 调用API → 解析结果 → 保存到CSV
关键参数：从链、到链、资产、金额、排序方式
输出格式：CSV文件，方便导入Excel进行数据分析
下一步，明天可以开始分析这些数据，找找看有没有有趣的规律。

笔记 019: 今天查阅了 LI.FI 官方 Transaction Status Tracking 文档。跨链交易

作者：jason52012345-bot | 时间：2026-08-17 01:35 UTC | 字数：497 | 评分：49
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天查阅了 LI.FI 官方 Transaction Status Tracking 文档。跨链交易广播后，不应仅以源链交易已确认判断流程完成；可使用 GET /v1/status 查询转账状态，关键参数包括 txHash、bridge、fromChain 与 toChain。该接口返回状态及更细的 substatus，适合作为前端或自动化策略的后续跟踪入口。

文档列出的主状态包括 NOT_FOUND、PENDING、DONE 和 FAILED。其中 PENDING 表示流程仍在进行，DONE 才是终态成功信号；FAILED 则应进入失败处理，而不是继续把同一笔交易当作可执行机会。官方建议以约 10–30 秒的间隔轮询，直至得到 DONE 或 FAILED，避免高频无效请求。

对链上套利执行而言，报价、签名广播与状态确认是不同阶段：报价不能替代最终交付确认。实现时应保存本次路线对应的交易哈希和桥接、源/目标链上下文，并为未找到状态、等待中与失败分别设置明确的处理分支；只有确认完成后，才将资金可用性纳入下一步决策。

笔记 020: 今天重新梳理套利系统的核心：不同套利的收益逻辑虽然不同，但发现机会的流程高度相似。真正决定系统价值的

作者：dyhbrewer | 时间：2026-08-17 09:25 UTC | 字数：1590 | 评分：70
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天重新梳理套利系统的核心：不同套利的收益逻辑虽然不同，但发现机会的流程高度相似。真正决定系统价值的不是再增加一种套利公式，而是能否持续、及时、低成本地监控到异常，并把假机会快速过滤掉。

我将机会发现设计为“两级扫描 + 状态机”。

一、第一级：全市场低成本异常雷达

不能持续对“所有链 × 所有资产 × 所有平台 × 所有金额”请求完整 Route，这样延迟高、接口消耗大，也容易被限流。第一级只读取轻量数据，用于发现异常，不做最终收益判断。

主要数据源包括：

- CEX WebSocket：最优买卖价、订单簿深度、成交量、资金费率、Mark Price 和 Index Price。
- 链上新区块与事件：DEX Pool 的 Swap、Mint、Burn，储备变化、大额交易和 Gas。

3. 稳定币价格：不同链、DEX、CEX 上的锚定偏差。
 4. 借贷协议：仓位健康度、预言机价格、更新时间和接近清算的账户。
 5. 跨链桥：库存、费率、预计到账时间和大额资金流。
 6. 市场结构变化：新池、新交易对、新资产、流动性新增或撤出。
- 异常触发器可以是：跨市场中间价偏差超过阈值、Pool 短时间出现大额 Swap、稳定币明显偏离 1 美元、资金费率或基差进入历史极端、预言机价与市场价分离、桥两端库存快速失衡。

二、第二级：候选机会精确验证

只有第一级检测到异常后，才调用 LI.FI、DEX Quoter、CEX 完整订单簿等成本更高的接口，分别测试 100、500、1,000、5,000 USDC 等规模。

真实净收益应计算：

Net Profit

\= 可成交卖出收入

实际买入成本

Gas

桥费

DEX/CEX 手续费

滑点与 Price Impact

失败成本

延迟与资金占用成本。

验证时必须记录精确 Route、目标金额下的 VWAP、报价年龄、预计执行时间、成交概率、压力测试收益、单边失败最大损失和机会已经持续多久。不能用中间价或盘口第一档代替实际可成交价格。

三、为每条机会建立状态机

机会不应只有“有/没有”两个状态，而应经历：

detected：轻量雷达发现异常；

quoted：已取得目标金额的精确报价；

confirmed：连续多次验证仍然成立；

paper_filled：按当时可见数据完成模拟成交；

expired：价差衰减或报价过期；

failed：流动性、接口、模拟或执行条件失败。

每一次状态变化都保存时间戳和失败原因，这样才能统计机会从出现到消失的生命周期，而不是只保存成功样本。

四、不同数据采用不同频率

CEX 订单簿使用 WebSocket 实时更新；新区块和 DEX 事件按区块处理；高优先级候选每 1~5 秒重新报价；资金费率与基差每 10~30 秒检查；全链全资产粗筛每 1~5 分钟；协议参数和资产列表每几小时或每天更新。

因此，当前五分钟全链扫描适合发现结构性异常，但不足以捕获短周期可执行机会。真正需要提速的是异常触发后的候选验证，而不是无差别提高所有请求频率。

五、提醒与排序逻辑

只有同时满足以下条件才推送：连续

2~3

次确认、报价未过期、目标金额深度充足、压力测试后净收益超过阈值、概率加权收益为正、单边失败存在明确退出路径。

排序不再只看最大价差，而使用：

Opportunity Score

\= 压力净收益 × 成交概率 × 新鲜度

流动性、延迟、失败和库存风险惩罚。

今天的结论：全链雷达负责覆盖，事件触发器负责抢时间，精确报价器负责排除假机会，Paper

Trading

负责证明可以成交。下一步应优先把现有系统升级为“两级扫描

+

状态机

+

概率加权排序”，继续保持只读，不连接钱包、不签名、不自动交易。

笔记 021: D14 (8/17) 第二周复盘： workflow 跑通真实候选 + ARC 链研究

作者：peanutwen00 | 时间：2026-08-17 13:31 UTC | 字数：856 | 评分：77

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D14 (8/17) 第二周复盘：工作流跑通真实候选 + ARC 链研究

- 1) 第二周 (D8-D14) 复盘：从“看懂案例”到“能跑工作流”——5 个免 key 脚本 (Morpho 监控/LI.FI 同链+跨链报价/Aave 健康因子直查) + 案例库 7 案例 + 16 个知识页
- 2) 用真实仓位跑通六步工作流：实时扫 Base Aave 发现 3 个接近清算线仓位——0x5900c3b7 (\$1570 万债务 HF1.10)、0xce5a88cb (HF1.09)、0x1beb2cc4 (HF1.04 最接近)
- 3) 算清关键数字：闪电贷费 = 借入 $\times$ 0.05% (\$23.91 仓只需 \$0.01)；清算最大亏损 = gas $\approx$ \$0.15 (原子回滚，任何一步失败整笔交易自动还原)
- 4) 三仓全观望：小仓 $E=0.42 < 0.5$ 门槛不过线；大仓 $E \approx \$780$ 但 $P=0.001$ 不可捕获 (机器人垄断)；中仓竞争不明——“观望”本身是有价值的判断，把模糊感觉变成数字决策
- 5) 证据表落地：arbitrage-evidence-log.md 记录每次评估的 P/失败成本，滚动校准工作流参数
- 6) ARC 链研究 (周五分享会行动项)：ARC = Circle 机构级 L1, 9/16 主网上线，LI.FI 首日支持跨链；验证人 BlackRock/DTCC/Visa 级别；USDC 计价 gas (\$0.005) + 亚秒最终性；Aave+Morpho+Uniswap 首发即上线
- 7) ARC 套利切入点 (带数字)：① 跨链流动性价差 (\$10K 示例毛利 \$80) ② 稳定币兑换价差 (桥接 USDC 折价 0.3%) ③ 清算套利 (\$100K 仓示例毛利 \$4,257, 机器人真空期) ④ gas 优势 (失败成本 \$0.15 $\rightarrow$ \$0.005)
- 8) 下一步：D15 方向选择 (7 类机会定位 2-5 个主攻方向)；ARC 9/16 上线前持续关注 LI.FI/Aave/Morpho 部署公告

笔记 022: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责)

作者：geyu210 | 时间：2026-08-17 13:36 UTC | 字数：3143 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

(本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责格式排版与提交流程。)

今天不再把 1 USDC dust 探针继续重复，而是把固定输入提高到 1000 USDC，只研究 LI.FI 本身的 route 成本与来源证据。1000 USDC 仍只是研究名义金额，不代表实际持仓；它的好处是 25 bps 平台费和 gas 都已经进入可解释的美元尺度，又不需要连接钱包或请求交易数据。

固定口径：chain Ethereum mainnet / chainId 1；tokenIn canonical USDC；tokenOut canonical WETH；amountIn 1,000,000,000 raw USDC；slippage 0.5%；order CHEAPEST；rounds exactly 5, no retry/replacement；endpoint POST /advanced/routes only。

正式采样前先遇到一个输入失败：首批预设地址请求 5/5 返回 404 / token-not-found。这批失败被保留为输入校验证据，没有混入市场样本；但随后独立复核时，字节相同的全小写地址也能正常返回 routes，因此目前不能把失败归因于 checksum 或大小写。换用仓库中经过验证的 canonical token metadata 后，重新声明一批 5 轮有效实验。这里唯一能确定的是“首批输入未被服务端识别”，根因仍未证实，不能把瞬时相关性写成 API 规则。

有效批次结果如下。outputUSD-afterGas 只是 LI.FI 自己的 USD 估值减 gas 估值，用来比较同批 route；它不是净利润，也没有扣回程、失败缓冲或其他腿。

5 轮结果：CHEAPEST 始终是 Nordstern；max-tokenOut 在 1、2、5 轮是 KyberSwap，在 3、4 轮与 Nordstern 相同。KyberSwap 相对 Nordstern 多给约 0.000037826826772522 WETH，约 0.7169 bps token output，但额外 gas 约 \$0.0746，按 1000 USDC 名义约 0.746 bps。因此它在三轮中的 USD-after-gas 反而少约 \$0.0031，这解释了为什么最高 token output 没有成为 CHEAPEST。

不足 30 秒内，route 数量在 5–7 之间变化，max-tokenOut 来源也发生切换；Nordstern 自身输出振幅约 0.3235 bps。所以 route 必须和时间、输入、完整保存范围绑定，不能缓存成“某工具长期最优”的静态结论。

平台费与优惠敏感性：Nordstern 和 KyberSwap 的顶层聚合费用都只有 LI.FI Fixed Fee，percentage 0.0025 = 25 bps，amount 2,500,000 raw USDC = 2.5 USDC，amountUSD 约 \$2.4988，included true，feeSplit.platform 2,500,000 raw，feeSplit.integrator 0 raw。这说明当前请求没有额外 integrator fee，25 bps 全部归在 LI.FI platform component。这个 platform 字段标准化自 LI.FI 原始响应中的 feeSplit.lifiFee；components 只是同一笔总费用的归因，不能再加到总 fee 上；included=true 也表示它已经反映在 toAmount 中，不能从输出再次扣除。

本次 CHEAPEST gas 只有约 \$0.0306–0.0308，LI.FI 固定费约为它的 81 倍。于是组织者争取的降费不是小优化：如果优惠作用于这笔 platform fee，它对小边际策略的影响远大于本次 gas 波动。但“有优惠”仍需在未来响应的 feeSplit.platform 中验证，不能只根据口头方案或 route 标签假设已经生效。

可回放证据边界：今天把 lifi-route-query 升到 v2，并把以下证据纳入标准化输出：1) 批次级实际 endpoint、HTTP status、integrator、apiKeySent、服务器时间和三项 rate-limit header，不保存 key 或完整 headers；2) 每个展开 route step 的来源路径，这只能证明 LI.FI 的集成 tool，不能证明 1inch 内部是否使用 Aqua；3) fee description 与 platform/integrator components；4) 同一批保存 routes 的 max-token-output 与 outputUSD-gasUSD 比较；5) 总 route 数和实际输出 route 数，若 CLI limit 截断，比较会明确 warning，而不是把前三条误称为整批最优。

生产代码仍不隐式写文件。需要回放时，由用户显式用 `--limit 20` 把完整 versioned JSON 重定向到本地研究目录。今天的 canonical WETH 五轮样本来自一次性只读研究 harness；当前正式 CLI 固定输出 native ETH，只能复现 v2 证据结构，不能复现相同的 USDC/WETH 交易对。CLI 没有新增数据库、轮询、重试或交易入口。

五轮响应头从 `remaining=11994` 依次降到 `11990`，每轮只消耗一次请求，没有隐藏 `retry`；`limit=12000`，`reset` 从 `7152s` 降到 `7126s`。这既证明本次 `key` 被发送，也给出了该窗口真实套餐证据。`apiKeySent` 这个字段只陈述客户端行为，不能替代服务端认证结论。

今天的结论：1000 USDC 档再次说明最高 token output 不等于扣 gas 后最好；当前 25 bps LI.FI platform fee 比同批 gas 和路线间微小差异更重要，费率优惠确实值得登记和后续验证。但单向聚合报价仍不能独立产生套利结论：没有外部公允价、反向退出腿、可执行性和完整成本，所以今天只得到 route 成本排序证据，没有“可执行偏差”。

官方依据：LI.FI Rate Limits (<https://docs.li.fi/api-reference/rate-limits>)、LI.FI Advanced Routes (<https://docs.li.fi/api-reference/advanced/get-a-set-of-routes-for-a-request-that-describes-a-transfer-of-tokens>)、LI.FI Fees (<https://docs.li.fi/faqs/fees-monetization>)、LI.FI Concepts and Objects (<https://docs.li.fi/agents/concepts>)。

若下一天继续，只做一个数量敏感性实验：固定同一时间窗口，预先选择少量输入档位，比较 LI.FI platform fee、gas 和 route-source 切换分别怎样随金额变化；仍不调用交易端点。

笔记 023: 鏈路統一的七個教訓

作者：demon851113 | 时间：2026-08-17 13:57 UTC | 字数：5002 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

鏈路統一的七個教訓

coin-registry · 2026-08-17

成果：Binance 鏈名 144/144 · 合約地址座標 725/725 · 179 測試

把三套互不相通的鏈名收斂成一份正本。過程中每一條規則都是被真實資料打臉後才寫對的。

問題

同一條鏈，三個資料源三種寫法：

| 來源 | 寫法 |

| :----- | :----- |

| Binance | AVAXC |

| CoinMarketCap | Avalanche C-Chain |

| lz-bridge | avalanche |

系統看不出它們是同一條鏈，於是同一顆幣的合約地址散落在三個座標系裡，無法比對。

起點是純名字比對，只對得上 57/158 (CMC) 與 34/144 (BN)。

01 · 鏈是維度，合約地址是座標

這是整件事的地基。合約地址 (CA) 從來不是獨立的身分，它是 (鏈, 地址) 的複合鍵。同一串地址在不同鏈上是完全不同的東西。

所以順序不能反：鏈名先歸一 → CA 才有共同座標系 → 幣的身分才能比對。

我一開始倒過來做——先用

CA

錨定幣的身分，再回頭補鏈名對照。結果一批「對不上」的案例，根因其實只是鏈名沒歸一。補完鏈名，34 筆自動解決。

Rule：分層系統要照依賴順序做完一層再做下一層。跳層會讓上層在殘缺的地基上比對，錯誤看起來像資料問題，實際上是順序問題。

02 · 名字不是身分——鏈也適用

幣的身分早就定案不能用 symbol 當鍵（會撞名、會改名）。但我一開始還是拿鏈名當鍵，直到踩了兩次才醒。

實測：lz 的鏈表把 Aptos 也標成 `chainId: 1`——那是它自己的鏈內編號，與 Ethereum 撞號。我的 `chain_key_of()` 假設 `chainId` 全網唯一，於是 Aptos 覆蓋了 Ethereum，害 `binance/APT` 和 `cmc/Aptos` 被綁到 Ethereum，還標成強證據。

`chainId` 只在 EVM 內部唯一。非 EVM 鏈沒有全網統一的數字 id，得另找穩定鍵。

03 · 原生 gas 幣是非 EVM 鏈的天然鍵

使用者提出的規則，事後看是整個鏈路層最漂亮的一刀。

Binance

用原生幣代號當鏈名 (ADA)，CMC

用鏈全名 (Cardano)。用名字當鍵，同一條鏈必然被拆成兩條。但兩邊講的都是同一顆 gas 幣——那才是鏈的固有屬性。

ETH 被 73 條鏈當 gas ← EVM 側完全不能用

BTC 9 條

USDC 5 條

非 EVM 側：11 條鏈、11 種原生幣、零重複

使用者同時指定「EVM 系列要排除這個做法」——這個限定是必要的，數字證明得很徹底。

原生幣怎麼判：CMC 的 platform = None（原生幣不是任何鏈的代幣，所以沒有 platform）+ rank 最高那顆。實測 ADA→Cardano(rank 15)、ATOM→Cosmos(60)、NEAR(34) 全部正確。

Rule：找身分證時，找「這個東西固有的、外界改不動的屬性」。名字是別人給的標籤，會變；gas 幣是鏈自己的一部分。

04·寬鬆比對必然誤合，而且專挑最像的那組下手

原生幣配對我用了前綴比對，因為 LUNA 的原生幣叫「Terra」，想順便吃到 Terra Classic。

| 前綴比對 | 精確相等 |

| :----- | :----- |
| binance/LUNA (Terra 2.0) 吃到 cmc/Terra Classic (2022 崩盤後的舊鏈) → 把新鏈綁到舊鏈 | LUNA → k:luna TerraLUNC → k:lunc Terra Classic 配對數 11 → 9，少的正是矇到的 |

分叉鏈與新舊鏈的名字天然互為前綴（Terra/Terra Classic、Ethereum/EthereumPoW）。寬鬆比對不是偶爾出錯，是必然在最需要區分的地方出錯。

這與當初否決「全名相似度判同幣」是同一個道理——那次的反例是 NIGHT 與 Midnight。

05·不要拿自己的結論當自己的證據

三個循環論證，每一個都讓錯誤看起來像被驗證過。

| 錯誤 | 後果 |

| :----- | :----- |
| 拿「CMC 側名字對照的結果」當 CA 索引，再回頭升級 CMC 側 | Ethereum 有 244 條 CA 卻永遠停在弱證據 |
| 交叉互證時沒檢查雙方指向同一條鏈 | 用「這兩個名字是同一條鏈」的證據，去背書「那條鏈是 Ethereum」這個沒被驗證的結論 |
| 單一 CA 就指認鏈 | BSC 對到 7 條鏈——EVM 同地址跨鏈部署是常態 |

真正獨立的證據長這樣：兩個來源各自觀測到同一個合約。Binance 和 CMC 是獨立採集的，它們共有的 CA 不依賴任何名字對照。

06·同一條鏈的不同「層」必須拆開

使用者的判斷：充錯層會丟幣，這個風險比多幾條鏈嚴重得多。

KAVA CA = ukava ← Cosmos 原生層

KAVAEVM CA = 0x919c1c267b... ← EVM 空間

兩個原本都綁 id:2222

同類還有 CFX/CFXEVM、FIL/FILEVM、SEI/SEIEVM、AVAX/AVAXC、BNB/BSC，以及 BTC 的四種網路型態（主網／閃電／Ordinals／SegWit）。

判準不是名字，是地址格式——格式不同代表執行環境不同，提幣路徑就不同。

07·欄位會騙人，資料不會

vm_type 三處硬編碼：chainlist 那層寫死 evm、原生幣兩層寫死 non-evm。實測誤標 10 條。

標成 EVM 但地址是...

Injective erc20:0x... / ibc/...

Algorand 31566704（純數字資產 ID）

IOST token.iost

MANTRA amantia

標成非 EVM 但地址是 0x...

Oasis Sapphire · EthereumPoW · VeChain

改成用實際地址驗證（0x+40hex 過半才算），驗不出來就留空不猜。最終「標 EVM 但地址不是 0x」降到 0。

例外也得靠資料才看得出來：

NEO 的 script hash 剛好也是 0x+40hex，格式撞了但不是 EVM

Zilliqa 同時支援 zil1... 和 0x... 兩種編碼，是同一條鏈不該拆

這兩條只能人工標，而人工標記不被自動規則覆蓋。

自動化的邊界

使用者定的分工：「有問題的話應該讓我手動接手處理，這不是問題。」

這句話讓規則可以誠實——判不出來就交人工，不必為了覆蓋率而猜。

| 證據型態 | 數量 | 性質 |

| :----- | :- | :----- |

| ca_match | 58 | CA 精確命中，最硬 |

native_coin	50	原生幣配對（非 EVM）
chainlist	46	EVM 通用註冊表
manual	43	人工裁決
lz_name	81	只有名字相等（弱）

那 43 筆人工裁決是規則產不出來的：BTC 四種網路型態要不要拆、FIAT_MONEY 根本不是鏈、SEGWITBTC 指什麼。

清空重建實測：235/280 可重現，16 筆會判錯，29 筆人工裁決全丟。

所以人工裁決被當成資產保護——匯出成獨立 JSON 進版控（data/chain_verdicts.json），重跑時自動規則永不覆蓋 evidence='manual'。（INSERT OR REPLACE 會靜默洗掉，這點是實測才發現的。）

三句話

資料會告訴你規則錯在哪，前提是你真的去跑。

七條教訓沒有一條是想出來的，全是實測打臉後改的。looks_evm() 的測試案例直接用踩過的真實格式，下次改壞會立刻紅。

拒絕猜測是規則的功能，不是缺陷。

ETH 這個鏈名對到 72 條鏈時，正確行為是交人工——把它當待改進的覆蓋率缺口，才會寫出誤合的規則。

誤合不可逆，誤拆只是多點一次頭。

這條在幣的身分層定下的紅線，在鏈路層同樣成立——而且更嚴重：鏈綁錯，該鏈上所有 CA 的座標都錯。

相關檔案

路徑	用途
registry/cmc_rules.py	判定規則正本（純函數，可測）
scripts/build_chain_registry.py	自動建鏈路（I/O 層）
scripts/chain_verdicts.py	人工裁決匯出／匯入
data/chain_verdicts.json	人工裁決（應進版控）
tests/test_cmc_rules.py	回歸測試（每條對應一個踩過的坑）
~/Obsidian Vault/踩雷/鏈路統一-鏈名不是身分的七次踩坑.md	跨 harness 教訓

笔记 024: 1、监控系统发现套利机会，做了4笔交易，2笔成功（利润为0），2笔失败，损失1个点左右。

作者：Ethanlx | 时间：2026-08-17 14:16 UTC | 字数：303 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1、监控系统发现套利机会，做了4笔交易，2笔成功（利润为0），2笔失败，损失1个点左右。

2、目前的跨链套利&监控交易系统，有几个问题：

（1）首先，监控的是价差，会慢很多。升级思路是监控可以改变价差出现套利空间的大额交易（需要结合当前的价钱来看），再升级就是监控mempool的大额交易（这会有风险，因为可以被取消、替换、丢弃等风险）。

（2）其次，跨链的时间太长（2-10分钟），导致价差变化利润缩小或者变负。解决思路有两个：（1）有合约的token，使用合约进行对冲。（2）无合约的token，需要考虑聚合器、求解器（典型如li.fi的intents产品）。

笔记 025: 套利基础学习打卡⑧ | Inventory

与资金预部署：为什么真正的跨交易所套利不是“买完再转过去卖”

作者：cfanboy | 时间：2026-08-17 14:53 UTC | 字数：11628 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

套利基础学习打卡⑧ | Inventory 与资金预部署：为什么真正的跨交易所套利不是“买完再转过去卖”？

前七篇已经逐渐把套利从基础概念推进到了实际执行：

- ① Funding Rate：资金费率\
- ② Basis：基差与收敛\
- ③ Delta Neutral：方向风险对冲\
- ④ Trading Cost：从毛价差到净利润\
- ⑤ Liquidity：订单簿深度与套利容量\
- ⑥ Latency：机会为什么会迅速消失\

⑦ Leg Risk：两条腿应该怎么执行

今天继续研究跨交易所套利中另一个非常关键的问题：

Inventory（库存）

以前理解跨交易所套利时，我脑子里的流程非常简单。

假设：

Exchange A：

BTC = 100,000

Exchange B：

BTC = 101,000

那么：

A 买入 BTC

↓

从 A 提币

↓

链上转账

↓

BTC 到达 B

↓

B 卖出 BTC

↓

赚取 1,000 USDT 价差

数学上完全成立。

但真正研究套利执行以后会发现：

这种模式最大的问题，是“套利”和“转账”被绑定在了一起。

而套利机会可能只存在：

几十毫秒、几秒甚至几十秒。

链上转账却可能需要：

几十秒、几分钟甚至更长。

所以：

Transfer Time >> Opportunity Lifetime

等 BTC 真正到达 B：

101,000 的价格可能早就不存在了。

因此真正的跨交易所套利通常需要引入一个非常重要的概念：

Pre-positioned Inventory

也就是：

提前在不同交易场所部署资金和资产。

一、先看“买入 → 转账 → 卖出”为什么有问题

假设：

A：

BTC Ask = 100,000

B：

BTC Bid = 101,000

Spread：

1%

系统发现机会后：

TO :

A 买入 1 BTC。

然后 :

A Withdrawal

↓

Blockchain Transaction

↓

Network Confirmation

↓

B Deposit Confirmation

↓

B Account Credit

整个过程假设 :

10 分钟。

10 分钟以后 :

BTC 在 B 可以卖了。

但是 B 的价格可能已经变成 :

100,200

甚至 :

99,500。

原来的 :

1,000 USDT Spread

早已经不存在。

如果 :

$B = 99,500$

那么 :

A 买 :

100,000

B 卖 :

99,500

反而亏 :

500 USDT

还没有计算 :

Trading Fee

Withdrawal Fee

Network Fee。

所以这种模式其实并不是严格意义上的 :

锁定套利。

而是在 :

买入以后承担一段时间的 BTC 价格风险。

二、真正的问题是 Settlement Latency

这和上一篇研究的 Latency 有关系, 但又不完全一样。

上一篇主要讨论 :

Market Data → Decision → Order → Fill

这里讨论的是 :

Settlement Latency

也就是：

资产从一个市场真正转移到另外一个市场需要多长时间。

例如：

Exchange A

↓

Withdrawal

↓

Blockchain

↓

Confirmations

↓

Exchange B

↓

Deposit Credit

这个时间可能远远大于：

套利机会的生命周期。

因此：

如果套利必须等待资产转移以后才能完成第二条腿，那么就无法真正锁定价差。

三、解决方法：提前在两个交易所放资金

于是套利架构发生变化。

假设准备：

200,000 USDT

做 A / B 两个交易所之间的 BTC 套利。

可以提前配置：

Exchange A

100,000 USDT

[图片]

1 BTC

Exchange B

100,000 USDT

[图片]

1 BTC

也就是说：

两个交易所都提前准备：

BTC + USDT

这就是：

Inventory

四、套利机会出现以后，不再转账

假设：

A：

BTC Ask = 100,000

B：

BTC Bid = 101,000

此时：

A：

直接使用 USDT：

Buy 1 BTC

同时：

B：

直接使用已有 BTC：

Sell 1 BTC

于是：

A：

USDT ↓

BTC ↑

B：

BTC ↓

USDT ↑

两条腿可以：

几乎同时执行。

根本不需要等待：

A BTC → B BTC

的链上转账。

于是：

Settlement Latency

从：

套利执行阶段

被移到了：

Rebalancing 阶段。

这是今天最重要的一个理解。

五、套利和资金转移被彻底分开了

原来的模式：

发现机会

↓

Buy

↓

Transfer

↓

Wait

↓

Sell

↓

Profit

新的模式：

资金提前部署

↓

发现机会

↓

Buy A + Sell B

↓

立即锁定 Spread

↓

Inventory 发生变化

↓

以后再 Rebalance

也就是说：

Trading 和 Settlement 被解耦了。

这对于高频跨交易所套利非常重要。

套利发生的时候：

只负责交易。

资金转移：

以后再处理。

六、但是 Inventory 会越来越不平衡

问题马上又来了。

假设初始：

A

100,000 USDT

1 BTC

B

100,000 USDT

1 BTC

不断出现：

A 便宜

B 贵

于是系统连续执行：

A Buy

B Sell。

执行一次以后：

A

USDT：

100,000 → 0

BTC：

1 → 2

B

BTC：

1 → 0

USDT：

100,000 → 201,000

此时下一次如果又出现：

A 便宜

B 贵

我们已经：

无法继续套利。

为什么？

因为：

A 没有足够 USDT。

B 没有 BTC 可以卖。

这就是：

Inventory Imbalance

库存失衡。

七、所以套利系统必须有 Rebalancing

为了继续套利：

需要把库存恢复。

例如：

A：

BTC 太多

USDT 太少。

B：

BTC 太少

USDT 太多。

于是可以：

A → B：

转 BTC。

B → A：

转 USDT。

最终重新回到：

A

BTC + USDT

B

BTC + USDT

这个过程就是：

Inventory Rebalancing

库存再平衡。

八、但是不应该每套利一次就 Rebalance

这也是今天一个很重要的认识。

如果：

每赚一次价差

↓

立即：

BTC A → B

USDT B → A

那么又会产生：

Withdrawal Fee

Network Fee

Deposit Delay

Operational Risk。

例如：

每次套利赚：

50 USDT

但一次 Rebalance：

Withdrawal + Network Cost：

15 USDT

利润直接少：

30%。

如果连续套利 10 次：

每次都 Rebalance：

成本：

150 USDT。

如果：

10 次以后统一 Rebalance：

可能只需要：

15 ~ 30 USDT。

所以更合理的做法是：

允许 Inventory 在一定范围内漂移，然后批量 Rebalance。

九、可以设置 Inventory Threshold

例如定义：

每个交易所 BTC 目标库存：

1 BTC

允许范围：

0.3 ~ 1.7 BTC

那么：

0.8 BTC：

正常。

0.5 BTC：

正常。

0.35 BTC：

接近警戒。

0.2 BTC：

触发：

Rebalance。

于是系统不需要：

每一笔套利后都转账。

而是：

$\text{Inventory} < \text{Lower Threshold}$

或者：

$\text{Inventory} > \text{Upper Threshold}$

才进行再平衡。

这样可以显著减少：

Transfer Frequency。

十、甚至可以等待“反向套利”自然平衡

还有一个更有意思的方法。

假设连续出现：

A 便宜

B 贵。

于是：

A：

BTC 越来越多。

B：

BTC 越来越少。

但过一段时间：

市场出现反向价差：

A：

BTC = 101,000

B：

BTC = 100,000

于是：

B Buy

[图片]

A Sell。

这样：

B：

BTC ↑

A：

BTC ↓

库存开始：

自然恢复。

也就是说：

有时候并不需要链上转账。

可以通过：

Reverse Arbitrage

实现：

Natural Rebalancing。

这会进一步降低：

Withdrawal Fee

Network Fee

Settlement Risk。

十一、Inventory 本身就是一种资源

今天学习以后，我开始觉得：

Inventory 不应该只理解为：

账户里有多少钱。

更准确地说：

Inventory 是套利系统在某个市场方向上的“执行能力”。

例如：

A：

USDT = 1,000,000

BTC = 0

那么：

可以大量：

Buy BTC

但无法：

Sell BTC Spot。

B：

BTC = 10

USDT = 0

那么：

可以：

Sell BTC

但无法：

Buy BTC。

所以：

资金总量相同：

并不意味着套利能力相同。

真正重要的是：

资产分布。

十二、可以定义 Inventory Utilization

假设：

Exchange A：

100,000 USDT

1 BTC

但过去一天：

80% 的机会都是：

A Buy。

那么：

A 的 USDT：

会快速耗尽。

而 BTC：

不断增加。

这说明：

初始 50/50 配置可能并不是最优的。

可以根据历史数据统计：

$A \rightarrow B$ Opportunity Frequency

$B \rightarrow A$ Opportunity Frequency

然后动态决定：

Inventory Allocation。

例如：

如果：

A Buy / B Sell

占：

70%

那么可以考虑：

A：

更多 USDT。

B：

更多 BTC。

而不是机械：

50 / 50。

于是 Inventory Management 开始变成：

一个：

Optimization Problem。

十三、更多交易所会让问题迅速复杂起来

两个交易所：

$A \leftrightarrow B$

已经需要管理：

BTC / USDT。

如果变成：

Binance

Bitget

Gate

OKX

Bybit

五个市场。

每个交易所都有：

BTC

ETH

SOL

USDT

USDC

那么整个系统就会形成一个巨大的：

Inventory Matrix

类似：

Exchange	BTC	ETH	SOL	USDT
:-----	:-----	:-----	:-----	:-----
A 1.2	10	200	100K	
B 0.5	20	50	200K	
C 2.0	5	100	80K	
D 0.8	15	300	150K	

这时候系统不仅要问：

哪里存在 Spread？

还要问：

我在这个方向有没有足够 Inventory 执行？

于是 Opportunity Detection 应该变成：

Spread

[图片]

Liquidity

[图片]

Inventory

[图片]

Risk Limit

共同决定：

Executable Size。

十四、Inventory 会降低资金效率

预部署虽然解决了 Settlement Latency：

但付出了一个非常明显的代价：

Capital Efficiency。

假设：

只需要执行：

10,000 USDT

套利。

但为了等待不同方向机会：

可能需要：

A：

50,000 USDT + BTC

B：

50,000 USDT + BTC

C：

50,000 USDT + BTC

D：

50,000 USDT + BTC

大量资金长期：

Idle。

真正参与某一次套利的：

可能只有：

10,000 USDT。

所以：

预部署交易所越多：

机会覆盖率提高。

但：

资金利用率下降。

这又是一个 Trade-off。

十五、所以不能只计算 Profit / Trade

以后评价套利系统：

不能只看：

一笔赚：

100 USDT。

更应该看：

Return on Allocated Capital

例如：

系统 A：

每天赚：

1,000 USDT

但需要：

1,000,000 USDT

预部署资金。

Daily Return：

0.1%

系统 B：

每天赚：

600 USDT

但只需要：

200,000 USDT。

Daily Return：

0.3%

虽然：

A 绝对利润更高。

但：

B 的资金效率：

可能明显更高。

所以：

Capital Efficiency

也是套利系统非常重要的指标。

十六、预部署资金还引入 Counterparty Risk

资金放在交易所：

意味着：

资产实际上由交易所托管。

如果同时把大量资金分散到：

多个 CEX

就会产生：

Counterparty Risk

例如：

Exchange Insolvency

Withdrawal Suspension

Account Freeze

API Restriction

Security Incident。

所以：

预部署更多资金：

可以降低：

Settlement Latency。

但同时增加：

Exchange Exposure。

这又形成一个 Trade-off：

Execution Efficiency

vs

Counterparty Risk。

十七、Rebalancing 本身也应该计算成本

假设套利赚：

500 USDT

但为了恢复库存：

需要：

BTC Transfer：

20 USDT

USDT Transfer：

5 USDT

Gas：

3 USDT

其他费用：

2 USDT

那么：

Rebalance Cost：

30 USDT

这实际上也是：

套利成本。

所以第四篇的 Net Profit 公式应该进一步扩展：

Net Profit

\=

Spread Profit

[图片]

Funding

[图片]

Trading Fee

[图片]

Slippage

[图片]

Borrow Cost

[图片]

Execution Loss

[图片]

Rebalancing Cost

也就是说：

即使 Rebalance 发生在几个小时以后：

它仍然应该分摊到前面的套利交易中。

十八、Rebalance 也可以寻找最便宜路径

这又可以进一步扩展。

假设：

BTC 从 A → B：

Withdrawal Fee：

20 USDT。

但：

USDT 从 B → A：

只需要：

1 USDT。

那么是否一定需要：

直接 BTC Transfer？

不一定。

可能：

A：

卖一部分 BTC → USDT

B：

买 BTC

然后：

只转 Stablecoin。

或者：

通过第三个交易所 C

进行：

Inventory Redistribution。

甚至：

使用不同网络：

Ethereum

Arbitrum

Base

Solana

Tron

寻找：

更便宜、更快的 Rebalancing Route。

于是：

Rebalancing 本身也可以变成一个路径优化问题。

十九、跨链套利其实也是同样的思想

这个思路继续扩展到 DEX / Cross-Chain。

例如：

Arbitrum

Base

Solana

之间套利。

如果每次：

Arbitrum Buy

↓

Bridge

↓

Base Sell

那么：

Bridge Latency

可能让套利机会消失。

所以更成熟的方式仍然是：

Arbitrum：

预部署 Inventory

Base：

预部署 Inventory

Solana：

预部署 Inventory。

发现机会：

两边直接执行。

之后：

统一：

Cross-Chain Rebalancing。

所以：

CEX-CEX Arbitrage

和：

Cross-Chain Arbitrage

在系统设计上其实存在非常相似的思想：

交易发生在本地库存之间，跨市场资金移动主要负责事后再平衡。

二十、Inventory Manager 应该成为独立模块

如果以后真正设计套利系统，我觉得应该有一个专门的：

Inventory Manager

持续维护：

Exchange Balance



Asset Balance



Available Balance



Locked Balance



Target Inventory



Min Inventory



Max Inventory



Inventory Deviation



Rebalance Requirement。

例如：

Binance BTC

Target: 2.00 BTC

Current: 0.42 BTC

Minimum: 0.50 BTC

Status:

REBALANCE_REQUIRED

然后：

Rebalance Engine

寻找：

Lowest Cost Route。

二十一、套利 Scanner 也必须考虑 Inventory

以前 Scanner：

if spread > threshold:

trade

后来变成：

Spread



Liquidity



VWAP



Cost



Net Profit

现在还要继续增加：

Net Profit



Check Inventory



Calculate Executable Size

↓
Check Risk

↓
Execute

例如：

理论：

Profitable Depth：

100,000 USDT。

但：

A Available USDT：

20,000。

B Available BTC：

0.15 BTC。

那么真正：

Executable Size

可能只有：

15,000 USDT。

所以：

Market Capacity $\neq$ Your Capacity

这是今天非常重要的一点。

二十二、今天最大的认知变化

以前理解跨交易所套利：

Buy Cheap

↓

Transfer

↓

Sell Expensive

今天理解变成：

Pre-Fund

↓

Buy Cheap + Sell Expensive Simultaneously

↓

Lock Spread

↓

Inventory Drift

↓

Continue Trading

↓

Rebalance When Necessary

也就是说：

Transfer 不是套利交易本身。

更准确地说：

Transfer 是维持套利系统持续运行所需要的库存管理行为。

这两者应该彻底解耦。

二十三、八篇知识终于形成了一套系统

目前已经学习：

① Funding Rate

收益从哪里来？



② Basis

为什么价格会产生偏离？



③ Delta Neutral

怎么消除方向风险？



④ Trading Cost

理论利润真正能留下多少？



⑤ Liquidity

这个机会到底能交易多少钱？



⑥ Latency

机会能存在多久？



⑦ Leg Risk

两条腿怎么真正成交？



⑧ Inventory

怎样保证下一次机会出现时仍然有能力立即交易？

这时候一个真正的套利系统已经逐渐形成：

Market Data Engine



Opportunity Scanner



Order Book Simulator



Profit Engine



Risk Engine



Inventory Check



Execution Engine



Fill Monitor



Delta Hedge



Inventory Manager



Rebalancing Engine



PnL & Post-Trade Analysis

今天最值得记住的一句话：

真正的跨市场套利不是发现价差以后才搬资金，而是在机会出现之前就把资金准备好；交易负责捕获价差，Rebalancing 负责让系统拥有捕获下一次价差的能力。

下一篇研究方向

下一篇准备继续研究一个与今天 Inventory 紧密相关、同时非常容易被忽视的问题：

套利中的 Capital Efficiency：为什么“年化 30%”的套利策略，实际资金收益率可能只有 8%？

准备进一步研究：

Notional

↓

Margin

↓

Collateral

↓

Idle Capital

↓

Inventory

↓

Leverage

↓

Return on Capital

↓

Risk-Adjusted Return

把“一个套利机会赚多少钱”升级成：

为了赚这些钱，我到底占用了多少真正的资金？

这会把前面 Funding、Basis、Inventory 和资金管理真正连接起来。

总结：今天这篇核心逻辑也有较强的现实依据：跨交易所套利的朴素“买入→转账→卖出”模式会受到区块链结算延迟影响；研究指出，预先将资产作为库存放在不同 CEX，可以让低价市场买入与高价市场卖出同时发生，从而规避交易阶段的 settlement latency，但同时会增加资本占用和交易所托管/违约风险。实际套利实践中，也通常把交易与再平衡分开，不必每次交易后立即转账，而是让库存适度漂移后批量再平衡，以摊薄转账费用。

03 MEV与抢跑（9 篇）

笔记 026: Day 10 (8/16) 打卡

作者：littlefyer007-lgtm | 时间：2026-08-16 15:03 UTC | 字数：592 | 评分：62

来源：<https://intensivecollearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10 (8/16) 打卡

今日完成

分叉主网测试

- 用hardhat分叉复制Base主网到本地
- 读到真实链上的USDC符号和WETH名称
- impersonateAccount冒充大户（拿到30.46 USDC）
- 合约在分叉上部署成功
- 5个测试全部通过

Mempool监控

- 写了mempool-monitor.js：WebSocket监听待确认交易

- 验证了wss://base-rpc.publicnode.com可用
- 成功订阅newPendingTransactions
- 过滤DEX合约 (Uniswap V3 Router/Aerodrome)
- 识别swap函数签名

关键认知

- 分叉主网=复制真实链到本地，零风险测试
- impersonateAccount可以冒充大户借测试资金
- 公共节点限流，生产级监控需Alchemy/QuickNode
- 完整套利链路：mempool监控→发现价差→调合约→闪电贷→防三明治

技术栈全景

合约层(FlashArbitrage.sol)+防护层(滑点+tx.origin+Flashbots)+测试层(单元+分叉)+监控层(mempool-monitor.js)

明天

捡尸体策略，或MEV-Share/私有RPC (Base链防三明治)

笔记 027: Day 13 | 链上套利残酷共学

作者：HashFlow | 时间：2026-08-16 23:10 UTC | 字数：1227 | 评分：74

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 13 | 链上套利残酷共学

日期：2026.08.17 (周一)

主题：知识图谱 v5 整合 + 审核修正

一、知识图谱 v4 → v5 整合

把 Day 11 (扫描脚本) 和 Day 12 (MEV 价值流) 成果落进图谱：

新增板块：

第 5 节：扫描脚本 arbitrage-scanner.py (逻辑闭环 + 3 个数据真实性问题 + 实测结果)

第 7 节：MEV 价值流 (5 角色 Mock 瀑布 + 两条价值路径 + 归因账本)

更新板块：

五种套利的跨链行：验证数据换成扫描实测 (200 资产仅 APXUSD 脱锚有优势)

成本模型 ⑥⑦：标注「内部转付」；⑧-⑫：补页面原文

术语表：+5 个 MEV 术语

二、审核发现的问题 (2 处)

审核 v5 时发现术语定义不精确——又是「凭记忆简化官方定义」的错误：

1. MEV 定义：我写的是「区块构建者可提取的最大价值」，漏了 ethereum.org 官方定义的两个关键限定
2. front-running/sandwich/back-running：只有中文翻译「抢跑/夹子/背跑」，没有定义

三、修正 (查 ethereum.org 官方原文)

MEV 官方定义 (完整)：

"Maximal extractable value (MEV) refers to the maximum value that can be extracted from block production in excess of the standard block reward and gas fees by including, excluding, and changing the order of transactions in a block."

关键补漏：①「超出标准区块奖励和 Gas 费之外」②「通过包含/排除/改变交易顺序」

sandwich trading 官方定义：

searcher 观察 mempool 中的大额 DEX 交易，计算其价格影响，在目标交易前后各执行一单 (front-run + back-run 组合)

front/back-running：ethereum.org 未单独定义，诚实标注而非编造。

认知更新

审核自己写的知识图谱时，又一次发现「凭记忆简化定义」的问题——MEV

定义漏了两个关键限定。这印证了铁律：术语定义必须查官方原文，不能凭记忆写。

下一步

WBTC 无报价排查

扫更多资产 (当前 200/1280 多链) 找长尾机会

APXUSD 脱锚深挖

Backrun 清算模块 + private-bundles 模块 (Day 12 未完成部分)

笔记 028: Day 13 (08/17) 学习总结, 素材来自今日同学精华:

作者: cjdaocoin | 时间: 2026-08-17 01:00 UTC | 字数: 424 | 评分: 53

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 13 (08/17) 学习总结, 素材来自今日同学精华:

1. Web3Rason: 而且资金费在该组合下可能是收益不是成本 (一边恒 0.0000%/8h、另一边 +0.0071%/8h, 约 +7.8% APR)
2. clare ma: 以上交易数据属于 user-reported: 由用户提供, 并已由用户核对 Etherscan 页面、Swap 日志和结算记录; 本会话未完成独立链上复核, 因此不标记为 repo-verified 或本会话已实测
3. HashFlow: MEV 定义: 我写的是「区块构建者可提取的最大价值」, 漏了 ethereum.org 官方定义的两个关键限定
4. xzone911: 以太坊上你习惯「多给点 gas 反正用不完会退」, Monad 上成本 = 提交的 gasLimit × gasPrice, 估多了就是实打实多付

以上为今日从同学打卡中学到的知识点, 继续积累执行与验证能力。

笔记 029: Day 11 (8/15) | MEV 供应链: 对手是谁

作者: yixin | 时间: 2026-08-17 07:30 UTC | 字数: 1135 | 评分: 63

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 (8/15) | MEV 供应链: 对手是谁

一句话目标: 理解链上套利的市场结构。不懂这层就会一直看到机会却抢不到。

核心概念

1. PBS (提议者-构建者分离) 供应链:

用户/searcher → (bundle) → builder → (block) → relay → proposer/validator

- searcher: 找机会, 打包成 bundle, 出价
 - builder: 从众多 bundle 中挑选组合出价值最高的区块
 - relay: 中介, 保证 builder 不看到 proposer 的内容、proposer 不偷 builder 的区块
 - proposer/validator: 选择出价最高的区块
2. 你的出价就是你的成本: bundle 通过 priority fee 或 直接给 coinbase 转账 出价。竞争把出价推到接近毛利。Day 9 题 9.3 的机制细节就在这里。
 3. 公开 mempool = 你的意图在裸奔: 任何提交到公开 mempool 的套利交易都会被抢跑。所有严肃的链上套利必须走私有通道 (Flashbots Protect / builder 直连 / MEV-Share 等)。
 4. MEV 的分类 (要能区分, 因为经济性完全不同):
 - DEX 套利 (我们的主题): 修正价格, 中性偏正面
 - 清算: 修正风险, 正面
 - 三明治: 纯粹从用户身上抽税, 负面。不做, 理由不只是道德——它需要极强的基础设施, 且正在被 MEV-Share/私有 mempool 结构性地消灭。
 - 抢跑/尾随 (backrunning): 尾随大单是最主要的原子套利来源
 5. L2 的不同: 多数 L2 目前是中心化 sequencer + FCFS (先到先得), 所以竞争维度从「出价」变成「延迟」。这意味着:
 - L2 上原子套利拼的是网络延迟和地理位置, 这是一个你几乎不可能赢的战场
 - 但也意味着 L2 上非原子的、慢速的机会 (跨链、库存型) 竞争相对温和
 - 这条判断应该直接影响你的战场选择。
 6. 利润归属的现实: 据 Frontier Research、Flashbots 等的公开数据, 链上套利利润的大头长期流向 builder/validator, searcher 的净利润率很薄。结论: 单纯做标准化原子套利不是一个好生意, 除非你有结构性优势。

笔记 030: 链上套利残酷共学 · Day 13 打卡草稿: OEV 回收与 Chainlink SVR

作者: jinny | 时间: 2026-08-17 08:39 UTC | 字数: 3459 | 评分: 90

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 13 打卡草稿: OEV 回收与 Chainlink SVR

日期: 2026-08-17 (Asia/Shanghai)

主题: OEV 是什么、Chainlink SVR 如何把预言机更新触发的清算 MEV 回收给协议

边界：只读机制研究；没有扫描实时仓位、连接钱包、提交打卡或寻找可执行机会。

选题说明

今天课程 events API 仍为空。公开打卡集中在 Base Flashblocks/MEV、Monad gas、资金管理 Kelly、LI.FI 状态跟踪和预测市场价差，还没有人系统讲 oracle 相关 MEV (OEV)。

OEV 和 Day 6 的 Aave 清算研究直接衔接：清算人最稳定的收入来源之一，就是等到预言机价格更新后 backrun 清算。今天的问题不是“清算怎么触发”，而是：

预言机更新本来要公开广播给所有人，为什么它的信息价值会被 builder/validator 和 searcher 拿走？Chainlink SVR 用什么机制把这部分价值还给 DeFi 协议和预言机网络，代价又是什么？

可直接打卡草稿

OEV 回收：Chainlink SVR 如何把预言机触发的清算价值还给协议

今天研究 oracle extractable value (OEV) 和 Chainlink 的 Smart Value Recapture (SVR)。它属于 MEV 的一个特定子集，但和普通 DEX 套利、三明治不同：利润来源是预言机价格更新这条信息，而不是任意链上订单流。

一、OEV：预言机更新本身就是一种套利信号

OEV 的定义很窄：由预言机价格更新触发的 MEV。

典型场景是借贷协议清算。链下预言机把新价格提交到链上后，谁先看到这个更新，谁就知道哪些借款仓位 health factor 跌破阈值、可以被清算。于是出现固定的执行顺序：

text

新价格上链

- searcher 发现可清算仓位
- 同一区块内 backrun 预言机交易
- 清算执行，获得清算奖励

过去这笔收益基本被三拨人瓜分：

- searcher 通过竞拍或直接抢跑清算获利；
- builder 收取打包交易的费用；
- validator 获得优先权租金。

借款协议承担了坏账与价格风险，却拿不到清算奖励中的大部分价值。OEV 的“提取”就是从这条链上，把本该属于协议的这部分价值部分收回来。

二、SVR：一套价格报告，两条发布通道

Chainlink SVR 的核心设计是双聚合器，而不是把预言机改成私有。

每次链下计算出的价格报告，同时走两条通道：

text

同一份价格报告

- └─ Standard Aggregator：照旧进公共 mempool，所有人都能看
- └─ SVR Aggregator：先进私有拍卖通道 (MEV-Share / Titan / Atlas)

私有通道的流程类似：

text

价格报告发到私有通道

- searcher 看到报告，竞拍 backrun 清算的权利
- builder 选择最高出价打包
- 同一区块内完成 SVR 价格更新 + 清算执行

关键是 SVR 的价格更新和清算执行被设计成原子化：要么一起在同一个区块内发生，要么都不发生。这样 searcher 拍到的不是“看到价格”的权限，而是“在同一区块里执行清算”的完整机会。

三、收益怎么分，谁拿大头

这套机制把清算收益拆成两份：

text

清算奖励总额

- ≈ searcher 保留的利润 (竞拍后的净收益)
- + 拍卖出价

拍卖出价再由 DeFi 协议和 Chainlink 网络按约定比例分成。也就是说，SVR 不是把 MEV 消灭，而是把原本漏给 builder/validator 的部分，改造成协议和预言机网络的收入。

Chainlink 文档里没有给固定分成比例，说明是按协议定制的可变费率，需要看具体 feed 的部署参数。

四、失败回退：私有通道不是单点依赖

SVR 不是完全绕开公共预言机。如果私有路由失败或超时，系统会在一个可配置延迟后回退到 Standard Feed 的价格。

text

SVR 私有通道正常 → 使用 SVR 价格

SVR 私有通道失败/超时 → 延迟后回退 Standard Feed 价格

这个回退保证了预言机的可用性，代价是引入了一个小的、可配置的延迟。对清算场景来说，延迟可能让价格略旧，这是协议在选择 SVR 时需要接受的权衡。

五、SVR 回收什么，不回收什么

需要明确边界：

SVR 只回收预言机更新触发的 MEV，主要是清算 backrun；

它不消除三明治攻击、普通 DEX 抢跑或其他订单流价值；

实际回收率是动态的，取决于 searcher 竞争和拍卖深度，不是固定百分比；

它改变的是价值分配，不是价格本身。

所以“上 SVR”不等于“没有 MEV 了”，只等于“预言机触发的这部分价值，协议能拿回一部分”。

六、另一条路线：Api3 OEV 的独家更新权

Chainlink SVR 的思路是“同一份价格报告，公开 + 私有两条通道并行”。Api3 OEV 是不同的机制：把更新预言机的权利本身拿出来拍卖。

text

Api3 OEV

→ searcher 竞拍某个周期的独家价格更新权

→ 中标的 searcher 原子化更新 price feed 并执行清算

→ 拍卖收入回到协议

两种路径的对比：

| 维度 | Chainlink SVR | Api3 OEV |

|---|---|---|

| 价格通道 | 标准 feed + SVR 私有拍卖并行 | 独家更新权拍卖（合作 searcher），公开 Signed API 数据延迟 |

| 触发方式 | backrun 同一区块内的价格更新与清算 | 中标的 searcher 自己执行更新 + 清算 |

| 价值来源 | 预言机报告触发的清算 MEV | 预言机更新权利本身 |

| 主要风险 | 私有路由延迟/失败回退 | 独家更新方集中，公开数据路径延迟 |

两者的共同点：都承认预言机更新是稀缺信息，并通过拍卖把这部分价值引导回 DeFi 协议，而不是留给排序者。

七、今天的核心收获

OEV 不是一种新套利策略，而是一个价值分配问题：

text

预言机更新

→ 信息优势（先看到价格变化）

→ 排序者/清算人捕获价值

→ SVR/Api3 用拍卖把部分价值还给协议

对清算套利者来说，SVR 上线后竞争结构会变化：不再主要比拼谁更快抢到公共 mempool，而是比拼谁在私有拍卖里出价和执行的效率。对协议方来说，SVR 是把防御性成本转成收入的手段，但收益规模依赖 searcher 竞争，不保证恒定。

下一步可以做一次只读验证：找一个已部署 SVR feed（比如 Aave 专用 feed），记录 Standard 与 SVR aggregator 地址、回退延迟参数和拍卖通道，观察同一轮价格更新中清算发生的位置。只建立观察表，不连接钱包、不执行交易。

参考来源

Chainlink SVR Feeds 文档 (<https://docs.chain.link/data-feeds/svr-feeds>)

Api3 OEV 文档 (<https://docs.api3.org/oev/>)

证据边界

本文是协议机制整理，基于 Chainlink SVR 与 Api3 OEV 官方文档；没有读取具体 feed 的链上状态。

文中没有出现任何实时清算、拍卖出价或回收率数字，因为没有做链上快照。

SVR 与协议的分成比例按具体部署定制，官方文档未给统一固定值。

私有通道涉及 Flashbots MEV-Share/Titan（主网）与 Atlas（Base/Arbitrum/BNB/Monad），具体支持范围以部署时文档为准。

笔记 031: 三角套利的核心，是利用三个交易对之间短暂出现的汇率失衡，通过连续三次兑换，最后回到最初的资产，并且最

作者：OxPrint | 时间：2026-08-17 10:05 UTC | 字数：1509 | 评分：72
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

三角套利的核心，是利用三个交易对之间短暂出现的汇率失衡，通过连续三次兑换，最后回到最初的资产，并且最终数量大于初始数量。比如从 USDT 出发，先买 BTC，再用 BTC 买 ETH，最后把 ETH 换回 USDT。如果最终得到的 USDT 比起始金额多，理论上就存在套利空间。

实际计算时，不能直接拿三个“最新成交价”相乘。每一步都要根据真实交易方向使用买一或卖一价格。买入某个资产时使用 Ask，卖出某个资产时使用 Bid。更进一步，还要看盘口深度，因为第一档价格只代表很小的成交量。真正投入 1,000、5,000、10,000 USDT 时，需要模拟逐档吃单，得到加权成交均价。很多表面看起来有 1% 甚至 2% 的三角价差，在较大交易金额下会迅速消失。

单一中心化交易所里的三角套利成本相对简单，主要包括三腿交易手续费、买卖价差、滑点、价格冲击、延迟、数量精度和部分成交风险。比如每腿 taker 手续费是 0.1%，三次交易基础手续费就接近 0.3%。因此毛价差只有 0.2% 时基本没有研究价值；如果手续费等级比较低，比如每腿 0.02%，可利用空间会明显增加。第一版研究可以采用简单模型：三角毛收益减去三腿手续费，再减一个固定安全余量，用来快速过滤机会。

执行风险比公式本身更重要。第一腿成交后，市场可能已经变化；第二腿也可能只成交部分；第三腿又可能遇到价格变化。执行程序必须根据上一腿“实际成交数量”计算下一腿订单，而不是三笔订单都按照最初计划金额发送。适合使用 IOC 或 FOK 一类快速成交订单，尽量减少挂单等待。任何一腿异常，都要有补偿、退出或者人工处理逻辑。

单一 CEX 三角套利在技术上比较容易实现。可以把交易所所有现货交易对构造成一张资产图，资产是节点，交易方向是边，程序自动寻找 A→B→C→A 的闭环，然后实时计算每个闭环的可执行收益。Binance 这类成熟市场竞争非常激烈，BTC、ETH、USDT 等主流交易对的三角失衡往往持续时间很短，因此更值得观察新上市币、小币种、稳定币组合和非主流计价交易对。

DEX 三角套利的结构类似，例如 USDC→WETH→TOKEN→USDC。它最大的优势是可以通过智能合约把三次交换放进同一笔原子交易中。合约最后检查最终余额，利润没有达到最低要求就整笔回滚，因此不会出现第一腿完成、后两腿失败留下方向性仓位的问题。代价是多了三次池手续费、Gas、价格冲击、MEV 和失败交易 Gas。公开 mempool 中的明显套利还容易被其他机器人抢跑，因此真正做 DEX 三角套利时，还需要研究私有交易、Builder、Bundle 和 MEV 保护。

跨链三角套利复杂度更高。跨链桥带来的几十秒甚至几分钟延迟会破坏三腿原子性，桥费、跨链流动性和目标链价格变化也会快速吞噬利润。因此跨链更适合库存型套利：提前在两条链上准备好资金，同时完成两端交易，之后再慢慢做资金再平衡。

从可实施性来看，最适合先验证的是单一 CEX 三角套利。它没有 Gas、桥费、钱包和链上失败问题，只需要交易所 WebSocket、OrderBook、手续费模型和一个三角路径扫描器。第一阶段只做监控，不下单，连续记录 7~30 天的数据，重点看每天出现多少机会、持续多久、手续费后还剩多少、不同金额下最大可执行规模是多少，以及机会主要集中在哪些币和哪些时间段。

如果长期数据显示机会持续时间只有几十毫秒，而且手续费后几乎没有利润，就可以迅速停止投入。如果在新币、小币或特殊稳定币组合中反复出现持续数秒、并且有足够深度的机会，再进入模拟交易和小规模验证

笔记 032: 残酷共学打卡：原子 DEX 套利，也要先赢下区块排序

作者：xsl | 时间：2026-08-17 11:28 UTC | 字数：2632 | 评分：90
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

残酷共学打卡：原子 DEX 套利，也要先赢下区块排序

今日行动

今天借助 Agent 阅读了 Ethereum 当前 MEV 说明，以及 Flashbots 关于 bundle、定价和未上链排查的官方文档。我重点复核一个问题：两段 DEX 交易即使能在同一笔交易或 bundle 中原子执行，为什么屏幕上的价差仍不等于 searcher 能拿到的利润。

这与 8 月 13 日研究的 intent auction 不同：上次关注应用层订单执行权如何在 quoter、filler 和 solver 之间分配；今天继续向下追到区块构建层，研究已经形成的套利交易如何竞争纳入和排序。

这次只做文档复核和离线敏感性计算，没有连接钱包、签名、提交 bundle 或发送链上交易。

学到的执行链

MEV 不只是额外的一项 Gas 成本。它来自区块生产者对交易的纳入、排除和排序能力。DEX 套利是最典型、也最拥挤的 MEV 机会之一：多个 searcher 看到同一状态偏差后，会竞争有限的区块空间，并通过 priority fee 或直接支付把大部分机会价值让给 builder / validator。

Flashbots bundle 能解决一部分腿风险：bundle 内的交易按给定顺序执行，除非显式允许某些交易 revert，否则任一交易失败会使整个 bundle 无效；bundle 内部也不会被其他交易插入。但这不等于获得执行权：

bundle 可以被放在区块中的不同位置，top-of-block 不保证；

builder 优化的是整个区块利润，报价较低的 bundle 可能被普通 mempool 交易或竞争 bundle 挤掉；

其他 bundle 可以先改变池子状态，让原先的模拟结果失效、收益下降或发生 revert；

即使本地模拟正确，也可能因为到达 builder 太晚、目标 validator 未运行 mev-boost，或 builder 与 proposer 没有共同 relay 而无法纳入。

因此，“原子”只表示成交后不会只完成一条腿，不表示一定成交、一定排在竞争者前面，也不表示价差归自己。

修正后的收益模型

我把单次 DEX 套利拆成两个判断：

text

landed_net

= 毛价差

- DEX fee、slippage 与 price impact
- base fee 与实际 gas
- priority fee 或直接 builder / validator payment
- 融资成本

expected_net

= P(bundle 有效且被纳入) × landed_net

- 模拟、节点、数据与延迟基础设施成本
- 资金和机会占用成本

未上链的 private bundle 通常不会留下失败交易 Gas，但仍然会留下机会丢失、基础设施与时间成本；公开广播的失败交易则可能实际消耗 Gas。两者不能混成同一种“失败成本”。

离线敏感性检查

我用一个纯假设样本检查竞价对利润的压缩，不把它当成真实市场收益：毛价差 1.00 ETH，DEX、slippage、base fee 等执行成本合计 0.20 ETH，竞价前剩余 0.80 ETH。

text

竞价让渡占剩余机会价值 落地后剩余

50% 0.400 ETH

80% 0.160 ETH

95% 0.040 ETH

这个表说明：路径模拟为正还不够。竞价边界必须进入模型；否则在高度竞争的机会中，searcher 可能只是替 builder / validator 发现并搬运价值。

今日结论

今天修正的核心认识是：

原子性解决的是交易腿的一致性，区块排序市场决定的是机会能否落地以及利润归谁。

对个人研究者来说，先做出能算 DEX 毛差的脚本远远不够。没有逐 block 模拟、有效 gas price / direct payment 记录、到达时延、纳入率和冲突样本，就不能把回测中的价差视为可执行 edge。Ethereum 官方资料也明确提醒，成熟的 DEX 套利与清算已经高度竞争，不适合作为新 searcher 的默认盈利假设。

下一步

先做不连接钱包的 prospective 观察表。每个候选记录 state block、target block、模拟时间、gross spread、gas used、最大可接受竞价、预期净值和下一块是否仍存在；负样本按 revert / bid too low / conflict / late / proposer coverage 分类。

只有在加入竞价和纳入概率后仍持续为正，才进入 Sepolia 的签名 bundle 模拟；当前不进入主网，不发送真钱交易。

复核资料：

Ethereum: Maximal extractable value (<https://ethereum.org/developers/docs/mev/>)

Flashbots: Understanding Bundles (<https://docs.flashbots.net/flashbots-auction/advanced/understanding-bundles>)

Flashbots: Bundle Pricing (<https://docs.flashbots.net/flashbots-auction/advanced/bundle-pricing>)

Flashbots: Bundle Inclusion Troubleshooting (<https://docs.flashbots.net/flashbots-auction/advanced/troubleshooting>)

Flashbots: JSON-RPC Endpoints (<https://docs.flashbots.net/flashbots-auction/advanced/rpc-endpoint>)

笔记 033: Day 10 | MEV、Mempool、Searcher、Bundle 与 Block Buildin

作者：HerschelLi-31 | 时间：2026-08-17 11:36 UTC | 字数：19925 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10 | MEV、Mempool、Searcher、Bundle 与 Block Building

1. 今天的核心目标

Day 9 我们解决的是：

```
\[
\boxed{\[
IfFoundAnOpportunity,\ CanIExecuteBothLegsSafely?\[
]\[
]
```

Day 10 要继续追问：

$$\boxed{\text{EvenIfICanExecute, WhyWouldTheOpportunityBelongToMe?}}$$

因为链上套利不是只有你一个人能看到。

同一个机会可能同时被：

$$\bigcup_{i=1}^n \text{Searcher}_i$$

发现。

所以今天的核心认知是：

$$\begin{aligned} & \backslash \backslash \\ & \backslash \boxed{\backslash} \\ & \text{Discovery} \backslash \\ & \text{eq} \backslash \\ & \text{Execution} \backslash \\ & \text{eq} \backslash \\ & \text{Inclusion} \backslash \\ & \text{eq} \backslash \\ & \text{Profit} \backslash \\ & \} \backslash \\ &] \end{aligned}$$

你发现一个套利、构造出正确 Transaction，甚至成功广播，都不代表它最终能以你希望的顺序进入 Block。

Ethereum 对 MEV 的定义，本质上就是通过 包含、排除或改变 Block 中 Transaction 顺序，在标准 Block reward 和 Gas 之外可以提取的价值。(ethereum.org (https://ethereum.org/developers/docs/mev/?utm_source=chatgpt.com "Maximal extractable value (MEV")))

2. 什么是 MEV ?

MEV :

$$\boxed{\text{Maximal Extractable Value}}$$

可以理解成：

由于 Blockchain 中 Transaction 的 inclusion、exclusion 和 ordering 可以改变最终 State，因此 Transaction 排序本身具有经济价值。

假设当前状态：

```
\[
S\0\
]
```

存在两笔交易：

```
\[
TX\_A, TX\_B\
]
```

如果顺序：

```
\[
TX\_A \rightarrow TX\_B\
]
```

得到：

```
\[
S\1\
]
```

而：

```
\[
TX\_B \rightarrow TX\_A\
]
```

得到：

```
\[
S\2\
]
```

并且：

```
\[
S\1
eq S\2\
]
```

那么：

```
\[
\boxed{\
TransactionOrderingHasValue\
}\
]
```

这就是理解 MEV 的第一性原理。

3. 为什么 Transaction 顺序会改变结果？

继续使用 AMM。

原始：

```
\[
ETH=4000\
]
```

Alice 准备执行一个：

```
\[
$1,000,000\
]
```

ETH Buy。

由于 Day 2 学过：

```
\[
LargeSwap\
\rightarrow\
LargePriceImpact\
]
```

]

所以 Alice 交易以后：

$\backslash \backslash$

ETH:

$4000 \rightarrow 4050$

]

这意味着：

Alice 之前

$\backslash \backslash$

State=S₀

]

Alice 之后

$\backslash \backslash$

State=S₁

]

如果另一个 DEX 仍然：

$\backslash \backslash$

ETH=4005

]

那么 Alice 的交易刚刚创造：

$\backslash \backslash$

4050-4005

]

的 Price Dislocation。

Searcher 就可能：

$\backslash \backslash$

Buy_{cheap}

+ $\backslash$

Sell_{expensive}

]

赚取这个 State Change。

4. MEV 和 Arbitrage 到底是什么关系？

它们不是完全相同的概念。

Arbitrage

研究：

$\backslash \backslash$

P_A

= P_B

]

利用：

$\backslash \backslash$

Buy_{Cheap}+Sell_{Expensive}

]

赚价格收敛。

MEV

研究更高一层：

$\backslash \backslash$

$\boxed{\backslash$

TransactionOrdering $\backslash$

$\}\backslash$


```

]
如何创造或决定谁能获得这些机会。
所以：
\
\boxed{
ArbitrageCanBeAFormOfMEV
}\
]
但：
\
MEV
eq ArbitrageOnly
]
Ethereum 官方列出的典型 MEV activity 就包括 DEX arbitrage、liquidation 等。(ethereum.org
(https://ethereum.org/developers/docs/mev/?utm_source=chatgpt.com "Maximal extractable value (MEV)"))

```

5. 什么是 Mempool ?

一个非常简化的传统流程：

```

User
↓
Sign Transaction
↓
Broadcast
↓
Pending Transaction
↓
Mempool
↓
Block Inclusion
↓
Execution

```

可以暂时把 Public Mempool 理解为：

尚未正式进入 Block 的公开 Pending Transaction 集合。

如果 Searcher 能看到：

```

\
PendingTX
]

```

就可以模拟：

```

\
StateBeforeTX
]

```

和：

```

\
StateAfterTX
]

```

然后寻找：

```

\
Opportunity
]

```

6. Searcher 是谁？

Flashbots 当前把 Searcher 描述为专门寻找和提取 MEV、并通过 Bundle 等高级交易结构提交交易的高级 Ethereum participant。(Flashbots 文档 (https://docs.flashbots.net/flashbots-mev-boost/glossary?utm_source=chatgpt.com "Glossary |

Flashbots Docs"))

简单理解：

\[

\boxed{\

Searcher

=====

OpportunityDetection\

+ \

Simulation\

+ \

Execution\

+ \

Bidding\

}\

]

Searcher 的系统可能不断：

Listen

↓

Simulate

↓

Detect

↓

Construct

↓

Submit

↓

Repeat

7. 一个简单 Searcher Pipeline

例如：

New Pending Transaction

|

▼

Decoder

|

▼

Simulation

|

▼

State Difference

|

▼

Opportunity Search

|

▼

Profit Estimation

|

▼

Build TX

|

▼

Build Bundle

|

▼

Submit Bid

这说明：

```

\
\boxed{
Searcher\
eq\
SimplePriceScanner\
}\
]

```

它更接近：

```

\
RealTimeStateTransitionEngine\
]

```

8. 什么是 Front-run ?

假设 User TX :

```

\
TX\_U\
]

```

会使某个资产上涨。

Searcher 想让自己的：

```

\
TX\_S\
]

```

排在 User 前面：

```

\
TX\_S\
\rightarrow\
TX\_U\
]

```

这类依靠抢到目标交易之前的位置来获利的行为，一般称为：

```

\
\boxed{
FrontRunning\
}\
]

```

核心不是：

“交易速度快。”

而是：

```

\
\boxed{
DesiredPosition=BeforeTargetTX\
}\
]

```

9. 什么是 Backrun ?

Backrun 正好相反。

目标：

```

\
TX\_U\
]

```

执行以后创造新的 State：

```

\
S\_0\
\xrightarrow{TX\_U}\

```

```
S\1\
]
```

Searcher 希望紧跟：

```
\
TX\S\
]
```

所以：

```
\
TX\U\
\rightarrow\
TX\S\
]
```

这就是：

```
\
\boxed{\
Backrun\
}\
]
```

Flashbots 当前的 MEV-Share 教程就明确使用用户 Transaction + 后续 Searcher Transaction 的 backrun bundle 结构寻找套利。(Flashbots 文档

(https://docs.flashbots.net/flashbots-mev-share/searchers/tutorials/limit-order/introduction?utm_source=chatgpt.com "Introduction | Flashbots Docs"))

10. Backrun 为什么和套利特别相关？

假设：

Before

Uniswap：

```
\
ETH=4000\
]
```

Balancer：

```
\
ETH=4001\
]
```

几乎没有套利。

然后：

```
\
UserTX\
]
```

大量买 Uniswap ETH：

```
\
4000\rightarrow4050\
]
```

现在：

```
\
Uniswap=4050\
]
```

```
\
Balancer=4001\
]
```

Searcher：

```

\
Buy_{Balancer}\
]

```

```

\
Sell_{Uniswap}\
]

```

就是典型：

```

\
\boxed{
StateChangingTX\
\rightarrow\
BackrunArbitrage\
}\
]

```

这正是 Day 8 中：

```

\
TriggerTX\
\rightarrow\
ArbitrageTX\
]

```

结构的进一步解释。

11. 什么是 Sandwich？

Sandwich 的结构通常可以抽象为：

```

\
SearcherTX\_1\
]

```

↓

```

\
VictimTX\
]

```

↓

```

\
SearcherTX\_2\
]

```

即：

```

\
\boxed{
FrontRun+Victim+BackRun\
}\
]

```

Searcher 利用目标 Swap 的 Price Impact，把自己的交易夹在用户交易前后。

Day 10 你只需要理解其机制。

对于我们的套利路线，更重要的是优先研究：

```

\
\boxed{
BackrunArbitrage\
}\
]

```

而不是把 Sandwich 当成主要研究方向。

12. 三种结构对比

类型	Searcher 想要的位置	核心来源
:-----	:-----	:-----

Front-run	Target 前	提前利用信息
Backrun	Target 后	利用 Target 改变后的 State
Sandwich	Target 前+后	利用 Target Price Impact

对你现在最重要的是：

```
\\  
\boxed{\  
Backrun\  
}\  
]
```

因为它和：

```
\\  
DEXArbitrage\  
]
```

连接最直接。

13. 什么是 Bundle？

这是 Day 10 最重要的技术概念之一。

Flashbots 当前把 Bundle 描述为一个有序的 Transaction 数组；在 MEV-Share 语境下，Bundle 可以用于把目标 Transaction 和 Searcher 的 backrun Transaction 按指定关系组合执行。(Flashbots 文档 (https://docs.flashbots.net/flashbots-mev-share/searchers/understanding-bundles?utm_source=chatgpt.com "Understanding Bundles"))

概念上：

Bundle

TX 1

TX 2

TX 3

要求：

```
\\  
TX\_1\  
\rightarrow\  
TX\_2\  
\rightarrow\  
TX\_3\  
]
```

按照设计顺序执行。

14. 为什么 Bundle 对套利重要？

假设策略要求：

```
\\  
UserTX\  
]
```

之后立刻：

```
\\  
ArbitrageTX\  
]
```

如果只是普通广播：

你无法保证：

```
\\  
UserTX\  
\rightarrow\  
YourTX\  
]
```

中间可能插入：

```
\\  
SearcherA\  
]  
\\  
SearcherB\  
]
```

因此机会可能被别人拿走。

Bundle 的价值就在于让 Searcher 更明确地表达：

```
\\  
\boxed{\  
DesiredTransactionOrdering\  
}\
```

15. Atomicity

Day 9 我们学过：

```
\\  
ParallelOrders  
eq Atomic\  
]
```

Bundle 则引出了真正重要的：

```
\\  
\boxed{\  
AtomicExecution\  
}\
```

可以简单理解成：

这一组 Transaction 按指定规则整体执行，否则不会留下自己不希望出现的部分执行状态。

Flashbots 的 MEV-Share 文档将 Bundle 描述为有序、原子执行的一组 Transaction。(Flashbots 文档 (https://docs.flashbots.net/flashbots-mev-share/searchers/understanding-bundles?utm_source=chatgpt.com "Understanding Bundles"))

这对套利意义巨大。

因为你非常害怕：

```
\\  
LegA=Success\  
]  
\\  
LegB=Fail\  
]
```

Atomic structure 可以在某些链上策略中大幅减少这种风险。

16. CEX Arbitrage 与 On-chain Arbitrage 的重要区别

Day 9：

CEX A：

```
\\  
Buy\  
]
```

CEX B：

```
\\  
Sell\  
]
```

两条腿来自：

```
\[\nTwoIndependentSystems\
```

```
]
```

所以：

```
\[\nAtomicity\
```

```
]
```

非常困难。

但 Same-chain Smart Contract Arbitrage 可以尝试把：

```
\[\nSwap\_A\
```

```
]
```

```
\[\nSwap\_B\
```

```
]
```

放进同一 Transaction。

如果第二步失败：

```
\[\nWholeTransaction\
```

```
]
```

可以 Revert。

于是：

```
\[\n\boxed{\nSameChainAtomicity\
```

```
}\n
```

```
]
```

是链上套利很重要的结构性优势。

17. Flash Loan 为什么经常和 Atomic Arbitrage 一起出现？

假设：

```
\[\nBorrowUSDC\
```

```
]
```

↓

```
\[\nBuyETH\
```

```
]
```

↓

```
\[\nSellETH\
```

```
]
```

↓

```
\[\nRepayUSDC\
```

```
]
```

全部发生在：

```
\[\nSingleTransaction\
```

```
]
```

如果最终不能偿还：


```
\\  
TransactionReverts\  
]
```

所以：

```
\\  
FlashLoan\  
+\  
AtomicExecution\  
]
```

非常适合某些 Same-chain Arbitrage。

但它仍然需要考虑：

```
\\  
Gas\  
]  
  
\\  
FlashLoanFee\  
]
```

```
\\  
Competition\  
]
```

```
\\  
Inclusion\  
]
```

18. 为什么构造正确 Transaction 仍然不代表赚钱？

假设你算出：

```
\\  
GrossProfit=$1000\  
]
```

但是：

```
\\  
SearcherA\  
]
```

也发现了。

```
\\  
SearcherB\  
]
```

也发现了。

大家都希望获得同一个：

```
\\  
BackrunPosition\  
]
```

于是开始竞争。

最终：

```
\\  
\boxed{\  
MEVCompetition\  
}\  
]
```

会把一部分套利利润转移给：

```
\\  
BlockBuildingPipeline\  

```

]

19. Builder 是什么？

现代 Ethereum Block production 中，Block construction 和最终 Block proposal 可以分开。

MEV-Boost 当前就是 validator 使用的一套 middleware，让 validator 可以从一个竞争性的 block-building market 获取 Builder 提交的 Block。（Flashbots 文档 (https://docs.flashbots.net/flashbots-mev-boost/introduction?utm_source=chatgpt.com "Overview"))

简化解理解：

\

Searchers\

]

提交各种：

\

Bundles\

]

↓

\

Builders\

]

组合：

\

Transactions+Bundles\

]

构造候选 Block。

20. Proposer 是什么？

最终某个 Slot 会有 Validator：

\

Proposer\

]

负责提出 Block。

所以高度简化的 pipeline：

Users

|

▼

Transactions

Searchers

|

▼

Bundles

|

▼

Builders

|

▼

Candidate Blocks

|

▼

Proposer

|

▼

Ethereum Block

Ethereum 官方的 PBS 资料也把核心思想描述为分离：

\\
BlockBuilding\
]

和：

\\
BlockProposing\
]

两个角色。(ethereum.org (https://ethereum.org/roadmap/pbs/?utm_source=chatgpt.com "Proposer-builder separation"))

21. Relay 在哪里？

在 MEV-Boost 架构中，还会出现：

\\
Relay\
]

简化管理解：

Relay 帮助 Builder 和 Validator/Proposer 之间进行 Block bid 与 payload 的传递。

所以更完整一点：

\\
Searcher\
]

↓

\\
Builder\
]

↓

\\
Relay\
]

↓

\\
Proposer\
]

↓

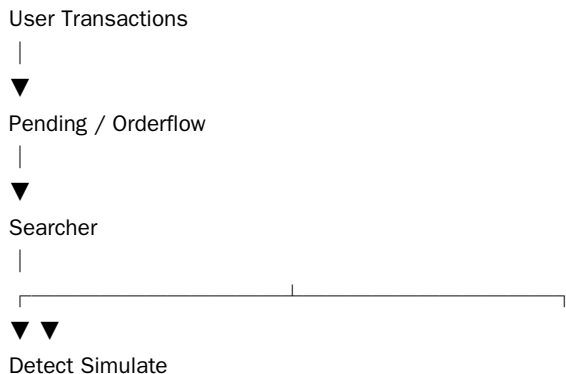
\\
Block\
]

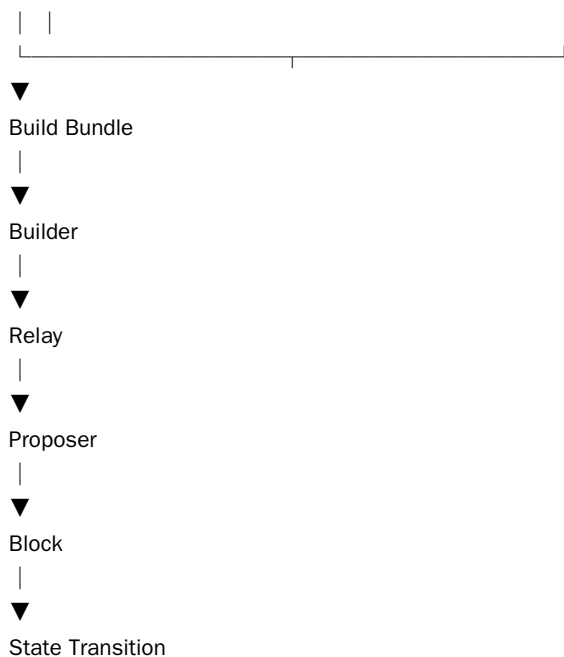
Day 10 不需要深入 Relay protocol。

先知道它在 Block building market 中承担连接角色即可。(Flashbots 文档
(https://docs.flashbots.net/flashbots-mev-boost/introduction?utm_source=chatgpt.com "Overview"))

22. MEV Pipeline

今天最重要的一张图：





这就是为什么：

```

\ \
 \boxed{ \
OpportunityDiscovery \
} \
 ]

```

只是整个系统最前面的一小部分。

23. Searcher 真正在竞争什么？

不是：

谁最早看到 ETH=4000。

而是：

```

\ \
 \boxed{ \
ProfitableBlockPosition \
} \
 ]

```

例如大家都想：

```

\ \
Position(TargetTX)+1 \
 ]

```

于是：

```

\ \
Searcher\_1 \
 ]

```

可能愿意支付：

```

\ \
$100 \
 ]

```

Searcher 2：

```

\ \
$200 \
 ]

```

Searcher 3：

```
\
$500\
]
```

竞争获得 inclusion。

24. MEV Competition 如何吃掉套利利润？

假设：

```
\
GrossArbitrage=$1000\
]
```

Searcher 需要支付：

```
\
Gas=$50\
]
```

为了提高 inclusion：

```
\
Bid=$700\
]
```

最终：

```
\
NetProfit
=====
1000-50-700
```

```
$250\
]
```

所以：

```
\
\boxed{
VisibleArbitrageValue\
eq\
SearcherProfit\
}\
]
```

Day 7 的 Cost Model 因此需要再增加：

```
\
MEVCost\
]
```

25. MEV Cost

OpportunityEvaluator 可以升级：

```
\
NetProfit
=====
GrossEdge\
]
\
TradingFees\
]
\
Gas\
]
\
```

```
Slippage\
]
\
PriceImpact\
]
\
Latency\
]
\
FailureCost\
]
\
\boxed{MEVCost}\
]
```

这里 MEV Cost 可以表现为：

```
Priority Fee
Builder payment
Opportunity lost to competitor
Adverse ordering
Bid competition
```

26. Profit Auction

如果所有 Searcher 都知道：

```
\
OpportunityValue=1000\
]
```

理性竞争可能让他们不断提高：

```
\
Bid\
]
```

直到：

```
\
SearcherProfit\
]
```

被压得越来越薄。

所以：

```
\
\boxed{\
PublicObviousOpportunity\
}\
]
```

通常很难长期保持巨大利润。

这是 Day 1 那个问题：

“为什么明显的 5% 套利还持续几个小时？”

进一步的答案。

27. 如果机会非常明显，你真正该怀疑什么？

现在已经可以形成更完整的 Checklist：

如果：

```
\
VisibleEdge=500bps\
]
```

并且长期存在，

先怀疑：

1. Price 不 executable；
2. Liquidity 很差；
3. Token 有风险；
4. Bridge 无法完成；
5. Capital constraint；
6. Non-atomic execution；
7. Opportunity 已被 MEV competition 捕获；
8. 你看到的是 stale data；
9. Strategy 本身存在隐藏 tail risk。

而不是：

所有人都没发现。

28. Private Transaction / Private Orderflow

Public Mempool 并不是唯一交易路径。

Transaction 也可以通过：

```
\\  
PrivateOrderflow\  
]
```

进入某些 builder / relay pipeline。

Flashbots Protect 就提供 private transaction submission 相关机制，而其文档说明这些 private transactions 可以与 builder / MEV-Share 流程发生联系。(Flashbots 文档
(https://docs.flashbots.net/flashbots-protect/additional-documentation/eth-sendPrivateTransaction?utm_source=chatgpt.com "eth_sendPrivateTransaction"))

这意味着：

```
\\  
\boxed{\  
NotAllOpportunitiesArePubliclyVisible\  
}\
```

因此 Data Access 本身也可能成为：

```
\\  
Edge\  
]
```

29. MEV-Share

MEV-Share 当前的目标之一，是让用户、wallet 和 application 可以分享或内部化其交易产生的 MEV，而不是把所有价值全部留给独立 Searcher。(Flashbots 文档
(https://docs.flashbots.net/flashbots-mev-share/introduction?utm_source=chatgpt.com "Introduction | Flashbots Docs"))

概念上：

```
\\  
UserTransaction\  
]
```

创造：

```
\\  
MEV\  
]
```

Searcher 捕获：

```
\\  
MEV\  
]
```

MEV-Share 则尝试让部分价值：

```
\\  
Searcher\  
\rightarrow\  
User\  
]
```

重新分配。

30. Searcher 获取的信息可能并不完整

这是一个很重要的现实。

你不能假定 Searcher 永远可以看到完整：

```
\\  
PendingTransaction\  
]
```

不同 orderflow / privacy mechanism 可能只暴露部分信息。

于是 Searcher 需要在：

```
\\  
IncompleteInformation\  
]
```

下：

```
\\  
Simulate\  
]
```

和：

```
\\  
Bid\  
]
```

这又创造：

```
\\  
InfrastructureEdge\  
]
```

和：

```
\\  
ModelingEdge\  
]
```

31. MEV-Share 中的 Backrun 思维

MEV-Share 的 Searcher workflow 可以粗略理解成：

```
\\  
ReceiveEvent\  
]  
↓  
\\  
UnderstandPossibleStateChange\  
]  
↓  
\\  
ConstructBackrun\  
]  
↓  
\\  
Simulate\  
]  
↓
```



```
\
SubmitBundle\
]
```

Flashbots 当前也提供专门教程演示如何根据 event data 找 backrun opportunity。(Flashbots 文档 (https://docs.flashbots.net/flashbots-mev-share/searchers/tutorials/limit-order/using-events?utm_source=chatgpt.com "Using Events"))

这和我们 Day 8 的：

```
\
TriggerTransaction\
\rightarrow\
Arbitrage\
]
```

完全连接起来了。

32. Simulation 为什么如此重要？

假设你看到：

```
\
GrossEdge=$500\
]
```

如果直接发送 Transaction：

可能：

```
\
Revert\
]
```

因为：

State 已变化；
AmountOut 不够；
Pool liquidity 变化；
Token balance 不够；
Gas estimation 错误；
前置 Transaction 没按预期执行。

所以专业 Searcher 会先尽可能：

```
\
\boxed{\
SimulateBundle\
}\
]
```

Flashbots 当前也提供 Bundle simulation / debugging 相关 endpoint 和文档。(Flashbots 文档 (https://docs.flashbots.net/flashbots-mev-share/searchers/debugging?utm_source=chatgpt.com "Debugging | Flashbots Docs"))

33. Searcher 的 Profit Model

今天可以把 Searcher 收益写成：

```
\
\boxed{\
SearcherEV
=====
p_{inclusion}\
\times\
p_{success}\
\times\
Profit_{included}
-----
ExpectedFailureCost\
}\
```

]

其中：

\[

Profit_{included}

=====

GrossMEV

Gas

BuilderBid

Fees\

]

所以一个机会是否值得 Searcher 抢，不只取决于：

\[

GrossMEV\

]

还取决于：

\[

p_{inclusion}\

]

34. Inclusion Probability

两个 Searcher：

Searcher A

发现机会：

\[

\$1000\

]

但 inclusion：

\[

10%\

]

Searcher B

发现：

\[

\$400\

]

但 inclusion：

\[

90%\

]

仅从 EV 看：

A：

\[

$0.1 \times 1000 = \$100$ \

]

B：

\[

$0.9 \times 400 = \$360$ \

]

所以：

```

\
\boxed{
BiggerOpportunity\
eq\
BetterStrategy\
}\
]

```

又一次回到：

```

\
RiskAdjustedExecutableEdge\
]

```

35. 为什么 Latency 在 MEV 中特别重要？

假设：

Searcher A：

```

\
Detect=20ms\
]

```

Searcher B：

```

\
Detect=300ms\
]

```

如果 opportunity lifetime：

```

\
150ms\
]

```

那么：

Searcher B：

```

\
OpportunityAvailableToMarket\
]

```

但：

```

\
NotAvailableToB\
]

```

这就是 Day 1 提过的：

```

\
\boxed{
MarketOpportunity\
eq\
MyOpportunity\
}\
]

```

在 MEV 中最极端的表现。

36. Infrastructure Edge

MEV 让我们第一次真正看到：

```

\
Infrastructure\
]

```

也可以成为 Alpha。

例如：

```

\
FastNode\
]

\
LowLatencyRPC\
]

\
EfficientSimulation\
]

\
PrivateOrderflow\
]

\
OptimizedContracts\
]

\
BuilderConnectivity\
]

```

都有可能直接决定：

```

\
PnL\
]

```

所以：

```

\
\boxed{
StrategyEdge\
+
InfrastructureEdge\
}\
]

```

必须分开研究。

37. Day 10 不建议现在做什么？

暂时不要尝试：

```

\
MainnetMEVBot\
]

```

尤其不要一开始就想着：

写一个抢跑 Bot。

原因不是 MEV 不值得研究，而是现代高竞争 MEV 的 difficulty 很大程度来自：

```

\
Latency\
]

\
Infrastructure\
]

\
Simulation\
]

\
Orderflow\
]

```

```

\
BuilderCompetition\
]

```

你的 Day 10 目标只是：

```

\
\boxed{\
UnderstandTheSystem\
}\
]

```

38. Day 10 Experiment 1 | 重新分析 Day 8 Case

拿 Day 8 的套利：

```

\
TX\_{arb}\
]

```

重新观察：

它前面是否有 Trigger TX？

如果：

```

\
TX\_{trigger}\
\rightarrow\
TX\_{arb}\
]

```

紧邻出现，

问：

这是否可能属于 Backrun structure？

然后记录：

```

Trigger Position
Arbitrage Position
Same Block?
Adjacent?

```

39. Experiment 2 | 画 Block Ordering

例如：

Block N

TX 0

TX 1

TX 2

TX 3 $\leftarrow$ Large Swap / Trigger

TX 4 $\leftarrow$ Arbitrage

TX 5

TX 6

然后描述：

```

\
S\_0\
\rightarrow{TX\_3}\
S\_1\
\rightarrow{TX\_4}\
S\_2\
]

```

分别回答：

```

\
S\_0\rightarrow S\_1\

```

```

]
创造了什么？
以及：
\\
S\_1\rightarrow S\_2\
]

```

套利者消除了什么？

40. Experiment 3 | 模拟 Searcher Competition

假设：

```

\\
GrossMEV=$1000\
]

```

三个 Searcher：

Searcher	Builder Bid	Gas	Net Profit
A	\$200	\$50	
B	\$500	\$50	
C	\$800	\$50	

分别计算：

```

\\
NetProfit
=====
1000-Bid-Gas\
]

```

A：

```

\\
750\
]

```

B：

```

\\
450\
]

```

C：

```

\\
150\
]

```

然后思考：

为什么 C 还可能愿意出更高 Bid？

答案：

因为：

```

\\
InclusionProbability\
]

```

也会上升。

真正比较的是：

```

\\
ExpectedValue\
]

```

而不是单次成功情况下 Profit 最大。

41. Experiment 4 | Opportunity Lifetime

选择一笔套利案例，估计：

```
\\  
Trigger\  
]
```

到：

```
\\  
Arbitrage\  
]
```

之间：

是否同一个 Block ?
相隔几笔 Transaction ?
相隔几个 Block ?

然后分类：

```
Slow  
  
\\  
Minutes\  
]
```

Medium

```
\\  
Seconds\  
]
```

Fast

```
\\  
SameBlock\  
]
```

Ultra-fast

依赖：

```
\\  
PendingOrderflow\  
]
```

这会直接影响第三周是否值得选择这个方向。

42. Day 10 OpportunityEvaluator 升级

Day 7：

Gross Edge
Fees
Gas
Slippage
Latency
Failure

Day 9 增加：

Legging Risk
Fill Probability

Day 10 再增加：

MEV Competition
Inclusion Probability
Builder Bid
Atomicity
Orderflow Visibility
Searcher Competition

于是：

```
\[\n\boxed{\nEV\n==\np\_{detect}\n\np\_{inclusion}\n\np\_{success}\n\n\Pi\_{success}\n\n-----\n\nExpectedCosts\n}\n\n]
```

这已经开始越来越接近真实链上套利。

43. Day 10 验收题

Q1：MEV 的本质是什么？

不是单纯：

Gas 很贵。

而是：

```
\[\n\boxed{\nTransactionOrderingAndInclusionHaveEconomicValue\n}\n\n]
```

Q2：为什么 Arbitrage 可以属于 MEV？

因为某笔 Transaction 改变市场 State 后：

```
\[\nP\_{A\n\neq P\_{B\n\n}]
```

Searcher 可以通过获得特定 Block position 捕获这个价格不一致。

Q3：Front-run 和 Backrun 最核心区别？

Front-run：

```
\[\nSearcherTX\n\n\rightarrow\n\nTargetTX\n\n]
```

Backrun：

```
\[\nTargetTX\n\n\rightarrow\n\nSearcherTX\n\n]
```

对当前套利学习更重要的是：

```
\[\nBackrunArbitrage\n\n]
```

Q4：Bundle 为什么重要？

因为 Searcher 不只是关心：


```
\\  
WhatTransactions\  
]
```

还关心：

```
\\  
\boxed{\  
InWhatOrder\  
}\
```

Bundle 可以表达特定 Transaction ordering，并支持原子化执行结构。(Flashbots 文档 (https://docs.flashbots.net/flashbots-mev-share/searchers/understanding-bundles?utm_source=chatgpt.com "Understanding Bundles"))

Q5：为什么发现 MEV 不等于获得 MEV？

因为其他 Searcher 也可能发现。

最终还需要竞争：

```
\\  
Inclusion\  
]
```

```
\\  
Ordering\  
]
```

因此：

```
\\  
Discovery  
eqCapture\  
]
```

Q6：为什么 MEV Competition 会降低 Searcher Profit？

Searcher 会为了提高 inclusion probability 支付：

```
\\  
PriorityFee\  
]
```

或其他 bid。

竞争越激烈：

```
\\  
MEV\  
]
```

越可能被分配给 Block-building pipeline，而不是全部留给 Searcher。

Q7：为什么 Public Opportunity 特别容易竞争？

因为多个 Searcher 可以同时观察相同信息：

```
\\  
SameState\  
]
```

```
\\  
SamePools\  
]
```

```
\\  
SameTransactions\  
]
```

所以：

```
\\  
Edge\  
]
```

```
]

```

很快被争抢。

Q8：为什么 Same-chain Arbitrage 通常比 CEX-CEX 更容易实现 Atomicity？

因为多个 Contract Call 可以被放进：

```
\[
SingleTransaction\
]

```

失败时整体 Revert。

而两个独立 CEX 的 Order Execution 无法天然共享同一个 Blockchain Transaction。

44. Day 10 最终 Task

今天至少完成四件事：

Task 1：画 MEV pipeline

```
\[
User\
\rightarrow\
Orderflow\
\rightarrow\
Searcher\
\rightarrow\
Bundle\
\rightarrow\
Builder\
\rightarrow\
Relay\
\rightarrow\
Proposer\
\rightarrow\
Block\
]

```

Task 2：重新分析 Day 8 的真实 Arbitrage Case

判断有没有：

```
\[
TriggerTX\
\rightarrow\
ArbitrageTX\
]

```

结构。

Task 3：画出 Block 内 Transaction Ordering

并写清：

```
\[
StateBefore\
]
\[
StateAfterTrigger\
]
\[
StateAfterArbitrage\
]

```

Task 4：给 Day 7 的 OpportunityEvaluator 新增：

```
mev_competition
inclusion_probability

```

builder_cost

atomicity

opportunity_lifetime

45. Day 10 最终产出

建议项目加入：

arbitrage-lab/

```
|—— reports/
|   |—— mev_execution_map.md
|
|—— notebooks/
|   |—— mev_case_reconstruction.ipynb
|
|—— src/
|—— models/
|—— mev_risk.py
```

其中 mev_execution_map.md 至少记录：

MEV Definition

Searcher

Mempool / Orderflow

Front-run

Backrun

Sandwich

Bundle

Atomicity

Builder

Relay

Proposer

Searcher Competition

Builder Bid

Inclusion Probability

Opportunity Lifetime

Infrastructure Requirement

46. Day 10 总结

Day 9 的核心问题是：

```
\[
  \boxed{
    CanIExecuteBothLegs?
  }
]
```

Day 10 进一步变成：

```
\[
  \boxed{
    CanIGetTheRightBlockPositionBeforeSomeoneElseDoes?
  }
]
```

因此，我们对套利的理解已经从最初的：

```
\[
  Price_A
  eq Price_B
]
```

一步一步升级成：

```
\[\nPriceDislocation\n]\n↓\n\[\nExecutableQuote\n]\n↓\n\[\nLiquidity\n]\n↓\n\[\nTransaction\n]\n↓\n\[\nStateTransition\n]\n↓\n\[\nOrderflow\n]\n↓\n\[\nSearcherCompetition\n]\n↓\n\[\nBlockOrdering\n]\n↓\n\[\nInclusion\n]\n↓\n\[\nNetProfit\n]\n]
```

笔记 034: 数据库：

作者：hello16 | 时间：2026-08-17 13:55 UTC | 字数：1705 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

数据库：

数据库实际设计成：

```
Opportunity\n|—— timestamp\n|—— chain\n|—— block_number
```

```

|
├── token
|   ├── token_in
|   └── token_out
|
├── pool
|   ├── pool_a
|   ├── pool_b
|   └── dex
|
├── route
|   └── token → pool → token → pool → token
|
├── opportunity
|   ├── price_a
|   ├── price_b
|   ├── spread_bps
|   └── amount_in
|
├── economics
|   ├── gross_profit
|   ├── gas
|   ├── builder_fee
|   ├── bribe
|   └── net_profit
|
├── execution
|   ├── simulated
|   ├── submitted
|   ├── landed
|   └── tx_hash
|
├── failure
├── status
└── reason

```

其中最重要的是把：

「价格机会」和「可执行利润」分开。

例如：

WETH/USDC

Pool A price: 3000.00

Pool B price: 3010.00

spread = 33.3 bps

这只能说明存在价格差。

真正应该记录的是：

amount_in = 100 ETH

gross_profit = \$42.18

gas = \$7.32

builder_fee = \$5.00

net_profit = \$29.86

status = SUCCESS

这样你的数据库以后才能回答真正有价值的问题：

「什么样的机会，在真实执行后最赚钱？」

failure reason 标准化

不要让 Agent 随便写自然语言，最好做成枚举：

NO_PRICE_DIFF
INSUFFICIENT_LIQUIDITY
SLIPPAGE_TOO_HIGH
PROFIT_TOO_LOW
GAS_TOO_HIGH
BUILDER_FEE_TOO_HIGH
SIMULATION_FAILED
REVERT
NONCE_CONFLICT
OPPORTUNITY_EXPIRED
OUTBID
NOT_LANDED
UNKNOWN

然后你后面可以直接统计：

发现机会 100,000

↓

有可执行路径 32,000

↓

扣 Gas 后盈利 8,500

↓

Simulation 成功 5,200

↓

提交 3,100

↓

成功上链 1,240

↓

真正盈利 980

这实际上就是你的 MEV Research Agent 的实验数据库。

而且结合你之前做的 Hot-State / PairSpread / Seed-Centric Route Composition，我尤其建议增加一个字段：

discovery_method

例如：

PAIR_SPREAD
HOT_STATE
SEED_CENTRIC
MEMPOOL_BACKRUN
CROSS_DEX
CROSS_POOL
LONG_TAIL

这样以后你可以直接比较：

PairSpread 找到的机会 vs Seed-Centric 找到的机会，哪个最终 EV 更高？

这比单纯记录 profit 有价值得多。

04 做市与LP（6 篇）

笔记 035: 学习记录

作者：capromka | 时间：2026-08-16 15:22 UTC | 字数：450 | 评分：48

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习记录

今天主要学习了 LP 中 V2 和 V3 的区别，对集中流动性和 V3 仓位管理有了更清晰的理解。

主要收获

V2 是全范围流动性，V3 可以自定义价格区间，V3 可以把资金集中到当前价格附近，提高区间内的资金效率。

V3 的核心特点是集中流动性：区间越窄，资金效率可能越高，但也越容易出区间。

理解了

V3

出区间后的变化：价格上涨出区间，仓位会逐渐变成稳定币；价格下跌出区间，会逐渐变成波动币，出区间后不再赚手续费。

出区间不是爆仓，也不是止损；如果价格重新回到区间，未撤出的仓位可以重新恢复活跃。

V2 使用 LP Token，V3 使用 NFT，因为 V3 每个仓位的价格区间、手续费档位和流动性等参数都可能不同。

今日总结

V2

简单稳定，V3

更灵活、资金效率更高，但需要自己选择区间并管理仓位；区间越窄，效率可能越高，同时管理难度和出区间风险也越高。

笔记 036: 今日学习：Lighter

取消免费账户做市积分，本质为创收而非抑刷——周末美股无波动时该平台仍成交近

作者：Jun Guo | 时间：2026-08-16 16:19 UTC | 字数：162 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习：Lighter 取消免费账户做市积分，本质为创收而非抑刷——周末美股无波动时该平台仍成交近 \$150M（同期 Hyperliquid 仅约 \$60M），因大量用户双账户 maker/taker 互刷积分。积分套利被堵后，做市激励的真实价值需重估。启发：平台激励规则本身即套利机会，规则变更等于套利窗口的关闭与重开。

笔记 037: Day 13：全自動套利系統架構 v1.2

作者：Web3Rason | 时间：2026-08-16 16:46 UTC | 字数：14469 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 13：全自動套利系統架構 v1.2

Day 12 的結論是「我只有一把尺，而且它只量一個方向」。

今天的問題更難看一點：我昨天拿來換掉舊讀數的那個「修正值」，比原值錯得更多。

而且我當時對它的信心特別高 —— 因為它在雙方向上一致到小數點後一位。

骨幹不動。但有一條 LO 紀律是今天才長出來的。

一、我的「修正」是把自己填的參數量了一遍

Day 8 我量到速度溢價約 1.0 bps。

Day 11 我把它推翻，改成 51 bps，理由是「舊值只在單腿 + 小額 + 只比 toAmount 的窄條件下成立」，並且我特別記下：雙方向 51.07 / 51.13，一致到小數點後一位。

今天我把 slippage 參數當自變量掃了一遍。同一條路徑、同一個金額、只動滑點設定：

| slippage 設定 | toAmount 差 | toAmountMin 差 |

| :--- | :--- | :--- |

| 0.1% | 1.02 bps | 11.02 bps |

| 0.3% | 1.02 bps | 30.94 bps |

| 0.5% | 1.02 bps | 51.02 bps |

左邊那欄是常數。右邊那欄就是我自己填的滑點。

根因是兩種報價類型混在一起比：

| 報價類型 | toAmountMin 的定義 |

| :--- | :--- |

| 意圖型（報價即承諾） | 恆等於 toAmount —— 完全不吃 slippage 參數 |

| 一般型 | toAmount × (1 - slippage) —— 扣滿 |

CHEAPEST 命中意圖型、FASTEST 命中一般型 → 我比的不是兩條路的品質，是「我的參數有沒有被套用」。

修正 L3.4：toAmountMin 跨報價類型完全不可比。

速度溢價回到 ≈1.0–1.1 bps（toAmount 口徑，單腿 1000 U 檔），

Day 11 的 51 bps 與由它推出的「四腿每腿 12.8 bps」（其實只是 51÷4）一併作廢。

最惡毒的一格在後面：FASTEST 命中哪一型是時變的。

同一條路徑，多數次命中一般型，但滑點設 1% 那一次命中了意圖型 —— 那一次「速度溢價」自己掉回 1.05 bps。

這個假訊號會間歇出現又消失，比一個恆定偏差難抓一個數量級。

那句「一致到小數點後一位」

我當時把它當成強證據。今天知道為什麼它會那麼一致：

因為兩邊量的都是同一個常數 —— 我的滑點設定，跟市場沒有關係。

升格為 L0 紀律：內部一致性不是外部效度。

一個量在多個「本來應該讓它變動」的條件下高度一致時，

第一件事是懷疑它根本沒在測量那些條件，而不是慶祝訊號乾淨。

落地判準：任何量測項上線前，要先跑一次已知應該讓它變的參數（滑點、金額、方向、路由偏好），確認它真的會變。不會變的維度，代表口徑接錯了。

同一天另一個報價欄位的疑慮也一起解掉了：有人回報 executionDuration 顯示「2ms」、懷疑它只算聚合器內部。實測 —— 單位是秒不是毫秒，值就是 2（另一條同時段是 7、反方向是 20），且頂層值等於各子步驟之和。那是單位誤讀，欄位語義沒變，硬過濾欄維持有效。

二、價差不是自變量，是應變量

我的掃描器把「價差」和「池深」當成兩個獨立輸入去組合枚舉。今天四組互不相干的資料指向同一件事：

| 證據 | 內容 |

| :--- | :--- |

| 長尾跨鏈 91 樣本 | 毛價差中位 50 bps（比主流三鏈對高兩個數量級），全成本淨利中位 307 bps，最好 150 bps |

| 生息／包裝幣掃描 | NAV 折價數字越大、對應 DEX 池反而越薄，容量被池深硬鎖 |

| 代幣化股票基差 | 中位 11.7 / p95 12.4 / max 12.6 bp —— 離散度趨近 0 的基差是 carry 不是機會 |

| 機制推論 | 螢幕上看得到的大價差，長在沒人填得平的淺池上：它之所以還活著，唯一原因就是它吃不動 |

價差與淺池不是兩個變量，是「尚未被填平」這同一個狀態的兩張臉。

修正 L2.2：掃描器不得把 價差 與 深度 當獨立自變量做笛卡兒積枚舉。

唯一的機會判準是淨值（扣完衝擊成本後），而不是先按價差排序再看深度。

附帶一個免費的分類器：先看毛價差分布的離散度 ——

中位 $\approx$ p95 $\approx$ max 代表這是持續存在的 carry，往返吃不掉，直接跳過。

同族還有一條把清單長度當機會量的老毛病又犯了一次：

「滿借溫床清單長度」 $\neq$ 機會量 —— 主流抵押品即使利用率 100% 通常沒有 DEX 折價，值得逐個驗的只有衍生／質押類長尾幣。這和先前的「可清算清單長度 $\neq$ 機會量」是同一個形狀。

三、v3 的 Quoter 帳單，付得起

Day 12 我把「v3 腿唯一可信報價源是 Quoter」寫進架構，但留了一個沒解的問題：

每週期約 8,000 次 eth_call，跟速率限制對不上。

今天有一份實跑把它拆開了：

| 階段 | 做法 | 成本 |

| :--- | :--- | :--- |

| 第一階段 | AMM 公式估算 + implied_ratio / depth / tick_aware 過濾 | 零 eth_call |

| 第二階段 | 只對含 v3 腿的路徑打 Quoter；v2-only 誤差 <0.4% 直接放行 | 只剩通過第一階段的那些 |

實跑數字：3,050 池 $\rightarrow$ 7,996 條三角路徑 $\rightarrow$ 第一階段通過 226 條 $\rightarrow$ Quoter 確認後真實機會 0 條。

而且那 226 個幻覺全部集中在含 v3 腿的穩定幣三角 —— 這同時坐實了 Day 12 的 v3 修正。

L1.7 補上執行方案：Quoter 不是全域取代估算公式，是只掛在 v3 腿上的第二階段。

「0 個機會」在這個架構下是正確答案，不是掃描失敗 —— 因為過濾層是有驗收點的。

同族的第二條：v3 歷史重放本身也有可行性閘門 ——

每池每塊 3 個 eth_call 會跑數小時且被限流，

正解是 slot0 + liquidity + TickLens 打包成單次 Multicall3 再落本地快取。

四、規模曲線：平方滑點模型再度被否認，可比窗口再收一級

同一天短時三窗口重測五個金額檔：

| 檔位 | 相對當輪 1k 基準的中位變化 |

| :--- | :--- |

| 1k | 0 |

| 5k | 0.81 bps |

| 10k | 1.06 bps |

| 30k | 0.72 bps |

| 50k | 0.93 bps |

兩件事：

1. 整條曲線幅度 < 1.1 bps，而平方滑點模型預測 50k 檔應惡化 24.5 bps → 該模型再度被否認。
2. 曲線非單調 —— 10k 檔比 30k 和 50k 都差。「規模越大單位報價越差」不成立，用單點推規模上界會錯。

而且 15 秒間隔的三輪之間，1k 基準自己就漂了 1.48 bps，已經大於整條曲線的幅度（0.72–1.06 bps）。

修正 L3.9：Day 12 寫的「規模曲線只能同窗口內比」還不夠嚴。

收緊成 只在同一輪連續採樣內可比，且結論一律用多輪中位數 + 區間表述，單輪數列不可當描述。

採樣時金額順序要輪換（拉丁方），否則時間漂移會被整段誤讀成金額效應。

五、幾條不改結論、但會改做法的

- 穩定幣套利的機會來源一直被切錯。「買折價的 A、賣溢價的 B」本身不賺錢 ——

按真實市價定價的池子已經把折價與溢價同時反映在兌換比例裡（10 萬 USDC 換出去再折回美元是倒虧的）。

真正的邊只來自單一池內部比例失衡造成的暫時偏離 → 監控對象是池的 balances 分佈，不是兩幣報價差。

- 「停充」與「停提」方向相反，不可合併成一個「充提異常」訊號。

停充 = 供給被切斷 → 傾向溢價，可用所內既有庫存鎖定（真 alpha 是事前就有庫存，看到停充才搬幣來不及）；

停提 = 出口被切斷 → 傾向折價，但買進去提不出來 —— 買到的其實是一張「提現恢復期權」，是事件風險交易不是套利。

- 同名幣跨鏈不是同一經濟對象，而且可以事先登錄成硬欄位。實測某鏈的 WETH 是本鏈原生 Wrapped，

另一鏈的是依賴 canonical bridge 的可升級 proxy → 贖回路徑與可升級性風險不同。

原生 USDC 也不是無風險現金（設計上就帶 Pause / Blacklist / Upgradeability）。

→ token registry 每筆強制記：chain_id + 合約位址（統一小寫）+ 發行人 + canonical/bridged/wrapped +

贖回路徑 + pause/blacklist/upgradeability + haircut + 排除決策與理由 + 證據 URL。

loader 不提供 default / fallback / guess，缺欄位或重複身分一律顯式失敗。

- 官方文件對「緩衝」的敘述不能當狀態讀。某包裝穩定幣 docs 稱超額抵押緩衝「只增不減」，

直讀鏈上：儲備 \$218.26M vs 流通 226.0M → 抵押率 96.57% (<100%)、贖回價值 \$0.9561。

緩衝不只被吃光，還倒欠 3.4%。→ 掃生息／包裝幣一律讀實際儲備／流通比。

- 清算的資金模型要按協議分流。有的協議是「先還債 → 再取抵押品」（要閃電貸墊款），

有的是倉位轉移模式（接管債務 + 接管折價抵押品，不需要預付債務資產）。把前者那套套到後者，

可行性判準從輸入層就是錯的。

- 負溢價結構最容易做反方向。統一寫成 B = 永續 現貨，B 從負往零走才賺錢；

吃收斂的正確結構是「做多永續 + 借券賣現貨」，反過來兩腿皆虧、還要付負資費，費率再高救不了方向錯誤。

兩個配套閘門：① 現貨可借量是獨立閘門（可借不到目標空頭 1.2 倍就不開）；

② ADL 會把中性倉直接剪成裸倉（多腿被減 40% → 瞬間淨空 400），

正解是按剩餘數量重算 Delta 用 reduce-only/IOC 回補多出來的那一腿，「補回被剪掉的腿」不能當預設。

- 跨所價差要先扣掉「天然價差」。兩所間長期存在結構性溢價（實例：某幣常態貴 0.04%），

開倉判準應為 當前價差 天然價差 雙邊手續費 ≥ 門檻；基準取最近約 24 小時的中位數而非平均。

把長期結構性溢價當成利潤，是跨所套利的系統性高估來源。

- 雙腿執行耗時要拆四段：31ms 兩腿已發送 / 480ms 私有 WS 收到成交 / 1,400ms 訂單終態 / 1,800ms 帳本完成。

真實成交約在 500ms。前兩段決定成交價與單腿風險，後兩段只決定「何時能安全開下一筆」 ——

混成「整筆 1.8 秒」會把優化火力打在錯的地方。

同族：私有 WS 的成交事件可能早於下單 API 返回 → API 返回後才註冊監聽會整個漏掉，早到事件必須先進暫存。

- 延遲優化要先按層歸因再決定投資順序。網路層 RTT 從 1–2ms 壓到 0.5ms 幾乎不影響捕獲率；

決定性的是應用層重寫（捕獲率 87.5% → 91.7% → 96.8%，路徑計算 40s → 3s）。

- 同區塊多機會並發受連續 nonce 硬約束。只能廣播「從 base nonce 起的連續成功前綴」，

中間任一候選準備失敗必須截斷其後全部以免 nonce 空洞；逐筆查 pending nonce 會把 RPC 延遲塞進熱路徑。

- 原子路徑要分「局部 settlement 成功」與「全部償債後淨增」兩態。實跑到達真實 v4 settlement、

完成 native ETH take、內層償債成功，卻在償還最外層債務時輸出不足而整筆回滾。

中間換幣成功 ≠ 閉環，狀態機不可把三種結果併成一個 REJECT。

- 遲到的舊狀態不得覆蓋較新狀態。已取消的訂單會從稍後才到的舊分頁裡「復活」，

正解是為刪除事件維護有時效的 tombstone。這與「WS 通知刪快取 → 再補讀 HTTP」是同一個坑。

- 速率限制的窗口口徑要實測。某聚合器是 2 小時滾動窗口（未認證約 75 次 / 帶 key 12,000 次），

不是每分鐘硬限 → 可以短時爆發後被整段封鎖。額度必須從回應標頭讀剩餘與重置時間，不可信文件預設值。

同族：交易所的封禁會續期（第一次剩 6.7 分鐘，再撞一次變 50.5 分鐘）

→ 418/429 立刻拋錯，絕不退避重試；只有連線層錯誤適用指數退避。

- 順序打兩個場所的 REST 會製造系統性 venue skew。整輪 145 秒，末行資料已陳舊 145 秒 → 生出幽靈套利。

改成逐標的成對抓取後，skew 中位從 1,678ms 降到 828ms。age / staleness 是判據不是裝飾。

- 跨場所的商品身分不能靠 symbol 對映。復現過「請求 XAG 回傳 BRENT OIL」。

完整市場身分只證明資料屬於誰，不證明兩個市場是同一個經濟對象。

- 自建的常數對照表也要用鏈上 symbol() / decimals() 回讀。有人的 alias 表把某鏈的 WETH/WBNB 寫反，

工具會一本正經地回報錯池的價格 —— 這比 stale state 更隱蔽，因為所有欄位看起來都健康。

- 聚合器回應裡沒有可綁定的區塊高度 / hash → 聚合器腿與鏈上 Quoter 讀數無法固定在同一鏈狀態比較，一律標 unknown 並拒絕，不要自行湊。
 - 聚合器的 tool 名稱不揭露下游流動性來源。嚴謹說法只能是「未顯式展示、來源未知」，不可據此推斷用了或沒用某個做市層。
 - 主網原子套利的分帳實測：毛差僅 1.14 bps 即可成立，但 builder 抽走 55.4%，發起者實得只有毛利的 21.6%。
 - 失敗率高不等於失敗成本高：某掃描型 bot 7,176 筆中閉環成功僅 3.09%、失敗 95.4%，但全部鏈費只佔毛利 0.32%。而成功高度時段驅動（單日 3,068 筆中 150+ 成功集中出現）→ 失敗率與機會量必須按窗口統計，禁用全期平均。
 - 零手續費不構成商業模式：某代幣化股票板塊 maker 費率 0 bp（同帳戶其他交易對是 10/10，屬專門補貼），但天花板已量到 —— 100% 吃滿全盤只有 \$18.87/小時，扣掉對沖腿後是 \$116.48/小時。補貼出現的地方，通常正是量小的地方。
 - 判斷端點鑑權級別可以不開交易權限：用格式非法的假 key 探錯誤碼躍遷（「格式非法」→「key 不存在」代表服務端已走到查 key 那步）即可確認是純行情級端點。
 - 逆選擇（markout）在稀疏成交市場會失效：窗口非空機率只有 7-22%，程式回落成「取下一筆」，於是 @1s / @5s / @30s 三個指標測的是同一件事，且買賣價跳動會造出假的正 markout。做市商不可能在價差外還系統性賺錢 —— 方向反常理時先查測量，正解是用 WS 錄盤口對 mid。
 - 自動化的第一道過濾可以是「槓桿比還原度」：能還原名目倍數（2×、3×）代表定價自洽，直接跳過；還原不了的才看。三個訊號同現（槓桿比失配 + 跨平台分歧擴大 + 週一不收斂）才值得深挖。
 - 生息代幣要多一格獨立維度：兌換率是鏈上公式，還是 offchain 裁量。有的只拿項目方每月自報的美元值（鏈上無法反推有沒有少報），有的公式與報價全在鏈上。
 - 摘要綁雜湊要做三方一致性審計：現場算的、產物綁的、文件裡留的三個 digest 互不相同 → 舊數字全部 STALE fail-close。被當成單一真相源的文件本身，就是第三個獨立漂移來源。
 - 本地 fork 驗收矩陣新增「報價老化實驗」：在報價與執行之間人為插入另一筆交易，驗嚴格保護會回滾、寬鬆保護只是利潤下降 —— 把「滑點容忍度是回滾邊界不是成本項」變成可跑的測試。
 - 閃電貸型抵押品估值差有明確上下界，而且借款端從來不是瓶頸：實際 709.93 < 最優約 1,350，超過約 2,750 轉虧；借款流動性 834 萬用不到，硬約束是 DEX 池深。池深 ×100 → 最優 ×100、峰值淨利 ×154。
 - MEV 價值歸因要先切「新增價值」與「內部轉付」再求和。searcher→builder、builder→proposer 都是轉付，加總會重複計算 → relay 報告的 bid / payout 既不是總 MEV 也不是淨利潤。
 - 同鏈跨 venue 三角是事件觸發型，不是常駐套利。歷史區塊重放上確實可原子獲利（成因是一筆外部交易打歪其中一池、另兩池未同步），但當前狀態數百次實鏈模擬 0 命中。→ 掃描器要等的是「哪個池被打歪」，不是輪詢價差。
 - 帳平不等於發現套利。錢扣損益歸因修掉三個污染源（債務代幣被當正資產、生息憑證的利息計提、跨鏈鑄造被誤判成憑空增發）後，residual 從 \$56,402 收斂到 1.76e-10 —— 但最終結論仍是 NO_EVIDENCE。
 - 代幣化美股永續要按場所配對切表：CEX↔CEX 已被抹平到 0.65-6 bps，CEX↔某永續場所是 5-20 倍（8.0 / 14.4 / 51.9 bps），方向穩定、std 只有 0.51-4.6 bps。
- 而且資金費在該組合下可能是收益不是成本（一邊恆 0.0000%/8h、另一邊 +0.0071%/8h，約 +7.8% APR）。
- 「有幾個人算過」不代表證據強度。同一筆交易六個人各自拆解，其中兩人對同一條腿給出 +869.65 與 130.35 的互斥歸因，連預言機口徑都對不上。採信判準是「有逐筆對帳 + 源碼坐實合約身分」那一份，且引用前必須自己重跑。
 - 清算距離不要引用任何人的口頭數字，一律現算： $(\text{抵押量} \times \text{oracle} \times \text{LLTV 債務}) \div (\text{抵押量} \times \text{oracle})$ 。同一筆交易上，口頭數字給出的「0.4%」與現算的約 0.02% 差了一個數量級。

六、參數驗證進度（接續 Day 12）

| # | 參數 | 值 | 狀態 |

| :- | :- | :- | :- |

| 162 | 速度溢價 | ~51 bps (Day 11) ~ → 回到 ≈1.02-1.11 bps (toAmount 口徑，單腿 1000 U，雙方向) | Day 11 的修正被推翻 |

| 163 | toAmountMin 的可比性 | 跨報價類型完全不可比：意圖型 = toAmount；一般型 = toAmount×(1slippage)。差值 11.02 / 30.94 / 51.02 bps @ 滑點 0.1 / 0.3 / 0.5% | 新增（一手實測，16 組）|

| 164 | 「每腿 12.8 bps」| 作廢 —— 那只是 51÷4 | 證偽 |

| 165 | 假訊號的間歇性 | FASTEST 命中哪一報價型是時變的 → 同一條路徑上「速度溢價」會自己出現又消失 | 新增 |

| 166 | 內部一致性 vs 外部效度 | 量測項上線前必須先確認「應該讓它變的參數真的會讓它變」；不變 = 口徑接錯 | 新增 (LO 紀律) |

| 167 | executionDuration 的 2ms 疑慮 | 單位是秒不是毫秒 (值 2 / 7 / 20)，頂層 = 各子步驟之和 → 硬過濾欄維持有效 | 解除 (他方單一觀測為單位誤讀) |

| 168 | 價差與池深的關係 | 反相關，是同一狀態的兩張臉，不得當獨立自變量枚舉 | 新增 (4 組獨立證據) |

| 169 | 基差離散度分類器 | 中位 $\approx p95 \approx \max(11.7 / 12.4 / 12.6 \text{ bp}) = \text{carry}$ 不是機會 | 新增 |

| 170 | 長尾跨鏈第三方複驗 | 毛價差中位 50 bps、淨利中位 307 bps，最好 150 bps (91 樣本 / 14 幣 $\times$ 5 鏈) | 既有結論第 N 個獨立樣本 |

| 171 | v3 Quoter 的成本解法 | 兩階段分流：一階零 eth_call 過濾，二階只對含 v3 腿打 Quoter；v2-only 誤差 $<0.4\%$ 放行 | 新增 (解掉 Day 12 的 8,000 call/週期) |

| 172 | 幻覺的集中度 | 226 個一階通過候選，Quoter 後真實機會 0，且幻覺 100% 集中在含 v3 腿 | 坐實 Day 12 的 v3 修正 |

| 173 | 平方滑點模型 | 再度否認：50k 檔預測惡化 24.5 bps，實測整條曲線幅度 $<1.1 \text{ bps}$ | 證偽 (第二次) |

| 174 | 規模曲線的單調性 | 非單調 (10k 1.06 比 30k 0.72、50k 0.93 更差) | 「越大越差」證偽 |

| 175 | 規模曲線可比窗口 | $\sim$ 同窗口內 $\sim$ $\rightarrow$ 同一輪連續採樣內，且採樣要輪換金額順序 (拉丁方) | Day 12 的紀律被收緊 |

| 176 | 同輪基準自漂 | 15 秒間隔三輪，1k 基準漂 1.48 bps ($\geq$ 整條曲線幅度 0.72–1.06 bps) | 新增 |

| 177 | L2 $\leftrightarrow$ L2 小額 fee 下限 | $\sim 0.27\% \sim \rightarrow 0.2500\%$ (+gas 0.67 bps = 0.2567%)，須標當日 gas 條件 | 下修 (極低 gas 檔位讀數) |

| 178 | 主網原子套利分帳 | 毛差 1.14 bps 即可成立；builder 抽 55.4%，發起者實得毛利 21.6% | 新增 |

| 179 | 失敗成本 vs 失敗率 | 閉環成功 3.09%、失敗 95.4%，但鏈費僅佔毛利 0.32%；成功時段驅動 | 新增 (禁用全期平均) |

| 180 | 零費率板塊的天花板 | maker 0 bp，但吃滿全盤僅 \$18.87/小時，扣對沖腿 \$116.48/小時 | 新增 |

| 181 | 代幣化美股永續價差 | CEX $\leftrightarrow$ CEX 0.65–6 bps vs CEX $\leftrightarrow$ 某永續場所 8.0–51.9 bps (5–20 倍)，std 0.51–4.6 bps | 新增 (按場所配對切表) |

| 182 | 該組合的資金費方向 | 一邊恆 0.0000%/8h、另一邊 +0.0071%/8h (約 +7.8% APR) $\rightarrow$ 是收益不是成本 | 新增 |

| 183 | 聚合器速率限制口徑 | 2 小時滾動窗口 (未認證約 75 次、帶 key 12,000 次)，非每分鐘硬限 | 新增 |

| 184 | 交易所封禁會續期 | 第一次剩 6.7 分鐘 $\rightarrow$ 再撞一次變 50.5 分鐘 $\rightarrow$ 418/429 立刻拋錯不退避 | 新增 |

| 185 | venue skew | 順序抓兩場所整輪 145 秒 $\rightarrow$ 幽靈套利；改成對抓後 skew 中位 1,678 $\rightarrow$ 828ms | 新增 |

| 186 | 延遲優化的層歸因 | 網路層 RTT 1–2ms $\rightarrow$ 0.5ms 幾乎無效；應用層重寫使捕獲率 87.5% $\rightarrow$ 91.7% $\rightarrow$ 96.8% | 新增 |

| 187 | 雙腿執行的四段耗時 | 31ms 送出 / 480ms 成交 / 1,400ms 終態 / 1,800ms 帳本；真實成交約 500ms | 新增 |

| 188 | 包裝穩定幣的實際抵押率 | 儲備 \$218.26M vs 流通 226.0M $\rightarrow$ 96.57%，贖回價值 \$0.9561 (docs 稱緩衝只增不減) | 文件敘述證偽 |

| 189 | 清算的資金模型 | 倉位轉移模式不需預付債務資產 (與「先還債再取抵押品」不同) $\rightarrow$ 掃描器要按協議分流 | 新增 |

| 190 | 現貨可借量 | 獨立開門：可借不到目標空頭 1.2 倍就不開倉 | 新增 |

| 191 | ADL 的中性倉破壞 | 多腿被減 40% $\rightarrow$ 1,000 空 vs 600 多 = 瞬間淨空 400；正解是重算 Delta 用 reduce-only 回補多出的腿 | 新增 |

| 192 | 跨所天然價差 | 兩所間常態結構性溢價 (實例 0.04%) 必須先扣；基準取 24 小時中位數非平均 | 新增 (系統性高估來源) |

| 193 | 閃電貸估值差的硬約束 | 是 DEX 池深不是借款流動性 (834 萬用不到)；最優約 1,350、 $>2,750$ 轉虧；池深 $\times 100 \rightarrow$ 淨利峰值 $\times 154$ | 新增 (池深為假設值) |

| 194 | 同鏈跨 venue 三角的性質 | 事件觸發型：歷史重放可原子獲利，當前狀態數百次模擬 0 命中 | 新增 |

| 195 | 清算距離的算法 | 一律現算 (抵押 $\times$ oracle $\times$ LLTV 債務) $\div$ (抵押 $\times$ oracle)；口頭數字 0.4% vs 現算約 0.02% 差一個數量級 | 新增 (改成公式) |

| 196 | token registry 的必填欄位 | chain_id + 位址 + 發行人 + canonical/bridged/wrapped + 贖回路徑 + pause/blacklist/upgradeability + haircut + 證據 URL；loader 禁 default/fallback/guess | 新增 |

| 197 | 常數對照表的驗證範圍 | 內建 alias 表也要鏈上 symbol()/decimals() 回讀 (有人把某鏈 WETH/WBNB 寫反，回報錯池價格) | 新增 |

| 198 | 聚合器報價的鏈狀態綁定 | 回應無 block hash/高度 $\rightarrow$ 與鏈上 Quoter 無法固定同一狀態，一律標 unknown 並拒絕 | 新增 |

| 199 | 穩定幣套利的機會來源 | 不是「買折價賣溢價」 (那是倒虧)，是單一池內部比例失衡 $\rightarrow$ 監控 balances 分佈非報價差 | 新增 (切分修正) |

| 200 | 停充 vs 停提 | 方向相反：停充傾向溢價 (需事前庫存)、停提傾向折價 (買到的是提現恢復期權) | 新增 (不可合併成一個訊號) |

| 201 | 同名幣的跨鏈身分 | 某鏈 WETH 為原生 Wrapped，另一鏈為 canonical bridge 的可升級 proxy；原生 USDC 帶 Pause/Blacklist/Upgradeability | 新增 (basis risk 可事先登錄) |

| 202 | markout 在稀疏市場 | 窗口非空機率僅 7–22% $\rightarrow$ @1s/@5s/@30s 測同一件事，且價跳造出假正值 | 新增 (方向反常理先查測量) |

| 203 | 端點鑑權級別的偵察 | 假 key 探錯誤碼躍遷 (格式非法 $\rightarrow$ key 不存在) 即可判定為純行情級 | 新增 |

| 204 | 生息代幣的兌換率來源 | 鏈上公式 vs offchain 裁量 (後者由項目方每月自報，鏈上無法反推少報) | 新增維度 |

| 205 | digest 審計的方數 | 三方 (現場算 / 產物綁 / 文件留) —— 被當單一真相源的文件本身是第三個漂移源 | 新增 |

| 206 | 滑點容忍度的定位 | 是回滾邊界不是成本項 $\rightarrow$ 用「報價老化實驗」變成可跑測試 | 新增 (驗收項) |

| 207 | 並發送單的 nonce 約束 | 只能廣播連續成功前綴，中間失敗必須截斷其後全部 | 新增 |

| 208 | 原子路徑的成功定義 | 局部 settlement 成功 $\neq$ 全部償債後淨增 (實跑卡在最外層償債，整筆回滾) | 新增 (三態不可併成 REJECT) |

209	私有 WS 的事件時序	成交事件可能早於下單 API 返回 → 早到事件必須先進暫存	新增
210	舊狀態的覆蓋規則	遲到的舊狀態不得覆蓋較新狀態 (已取消訂單會從舊分頁復活) → 有時效的 tombstone	新增
211	免費節點的事件接口	BSC 免費層日誌/事件接口整體不可用 (拒單/403/SSL 斷連三種形態, 2 組獨立樣本)	新增
212	v3 歷史重放的取數	每池每塊 3 個 eth_call 不可行 → slot0+liquidity+TickLens 打包 Multicall3 + 本地快取	新增
213	MEV 價值歸因	先切「新增價值」與「內部轉付」再求和 → relay 的 bid/payout 既非總 MEV 也非淨利潤	新增
214	帳平的意義	residual 收斂到 1.76e-10 仍是 NO_EVIDENCE —— 帳平不等於發現套利	新增
215	「幾個人算過」的證據值	6 人拆同一筆, 兩人同一條腿給出 +869.65 vs 130.35 互斥歸因 → 共識人數不是證據	新增 (採信判準)
216	長尾抵押幣的價源	沒有可信現貨價源 (某資料源命中 0/30); 預言機會回離譜價值 (NAV 到 1e21), 需合理性區間	新增
217	「滿借溫床清單長度」	≠ 機會量 (主流抵押品即使 100% 利用率通常無 DEX 折價)	新增 (同「可清算清單長度」形狀)

七、一句話

Day 8: 量具本身的誤差, 比我要找的東西還大。

Day 9: 我把「我當時的實驗設定」寫成了「世界的性質」。

Day 10: 我的量具可能整台是壞的, 而它會回報一切正常。

Day 11: 儀器沒壞、數字也對, 錯的是我把讀數搬出實驗室那一步。

Day 12: 我一直在把尺調準, 但我只有一把尺, 而且它只量一個方向。

Day 13: 我把自己的手指伸進畫面裡, 然後量了它十二天。

昨天那條「速度溢價 51 bps」不是市場的性質, 是我自己填在請求裡的 0.5%。

它從頭到尾就在那個欄位裡, 我只是換了個名字把它讀回來。

最刺的不是錯, 是我當時對它比對舊值更有信心 ——

因為「雙方向 51.07 / 51.13, 一致到小數點後一位」。

那份一致性是真的。它一致, 正是因為它跟市場無關。

兩邊量的都是同一個常數, 所以不管方向、不管標的、不管時間, 它都會給我同一個答案。

一個「怎麼都不會變」的量, 不是穩健, 是沒接上。

十三天下來我修過口徑、修過作用域、修過維度。今天要修的是更前面一格:

我一直用「數字乾不乾淨」當可信度的代理指標。

但乾淨可以來自兩個相反的原因 —— 訊號真的強, 或者根本沒在測。

而後者的數字更乾淨, 因為它連噪聲都沒有。

所以驗收條件要反過來寫。以前我問「這個量測穩不穩」,

現在要先問:「我知道有哪些條件應該讓它變 —— 它真的變了嗎?」

不會變的維度, 不是控制得好, 是線根本沒接上去。

笔记 038: markdown

作者: darven | 时间: 2026-08-17 06:48 UTC | 字数: 2086 | 评分: 71

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

markdown

Neutr1 停贖事故研習: 把「市場中立」框架用在鏈下風險落點

今天的重點

新開了一個單項目研習 topic: Neutr1 (NUSD 市場中立合成美元)。從官推 @Neutr1 的釘選停贖公告出發, 用 CDP 繞過被封鎖的搜尋引擎與官方網站, 抓到 docs 全文、整理事故始末。最後收斂出一個可以拿去檢驗其他項目的框架:「市場中立」在 DeFi 語境下, 從一開始就把風險的落點遷移到鏈下。

我學到了什麼

1. Neutr1 是「市場中立合成美元」, 不是一般的穩定幣。docs 錨定句寫得很直白: NUSD 靠 OTC 套利、funding-rate 價差、DeFi-native 市場中立策略給你收益。資金流是: 用戶存 USDC/USDT/USDe → 1:1 鑄 NUSD → 抵押品進機構託管帳戶 (Fireblocks/Copper/Ceffu) → 部署到 duration-matched 市場中立策略。質押 NUSD→sNUSD 收統一的「Neutr1 rate」。這不是 USD 的錨定協議, 是「拿你的美元去做市場中性策略、分收益給你」的合成美元。

2. docs 自己就寫了 Counterparty Risk。它的風險頁列了六類: Funding / Liquidity / Counterparty / Exchange / Stablecoin / Margin。其中 Counterparty Risk 明寫「Default risk from OTC trading partners and third-party service

providers」。所以這根本不是隱藏風險，是白紙黑字在風險文件裡的自認——只是用戶在存入那一刻不會去看那頁。

3. 事故是「停贖 + 法律顧問」，這兩個訊號合起來就是鏈下風險的實錘。 8/13 官方釘選：因影響協議儲備，暫停鑄造與贖回及其他功能，經法律顧問建議。規模約 \$53.7M USD 被凍結。肇因線索來自研究者 @systematicls：主策略是「買高度非流動（鎖定）的資產」，結合官方 8/5、8/10 文章（tokenized equities / pre-liquidity IPO 前 shares）→ 指向儲備被拿去買非流動的 IPO 前/鎖定股權。純鏈上事故不需要法律顧問——法律顧問只在要跟真實世界的實體（對手方、託管方、發行人）對簿公堂時才需要。這條推論我覺得是今天最有力量的一手。

4. mark-to-market ≠ 爆雷 Apyx STRC 跌價是市價波動，機制內可消化；Neutr1 是儲備受損直接停贖+走法律程序——兩者本質不同。我之前把它們並排成「連環暴雷」是分類錯誤，也是想要的結論（「這赛道全在爆」）污染了證據。

5. 修正後反而得到更準的判準：問題不在「neutral」，在底層標的的流動性。為什麼 Apyx 沒爆而 Neutr1 爆了？不是 Apyx 更安全，是它的標的（STRC，Nasdaq 上市優先股）是流動的，跌價可以 mark-to-market 消化、不需要停贖；而 Neutr1 把標的換成非流動的 IPO 前/鎖定股權，一旦出事就變成現金擠兌式的硬停贖。風險不是消失了，是換了位置。

6. 用這把尺反推 Apyx，發現它的鏈下死穴不在標的、在股息收款鏈。使用者接著拋出「那用這框架檢驗 Apyx 呢？」——我拆了它：STRC 崩盤時 Apyx 安然（標的流動），但它的股息收集節點完全在鏈下：STRC 股息從 Nasdaq 公司 → 法定實體（BVI fund/經紀商帳戶）→ offchain treasury 轉成 apxUSD → 才進鏈上 vault。而且 apyUSD 的月 yield rate 是項目方 offchain 裁量、鏈上無法反推。所以 Apyx 的鏈下風險落點是「誰在忠心收股息、忠心派息」+「20 天贖回 cooldown 會在出事時變成卡贖窗口」。這正是 Neutr1 教訓的延伸：審計保得了代碼，保不了鏈下的對賭對象。

收斂

「市場中立」這個詞聽起來像是在消掉風險，實際上是把風險換了一個更難驗證的落點——鏈下。delta 削掉之後，剩下的收益（funding / basis / OTC 折扣）全都是「把資產借給或交割給某個對手方的服務費」，純信用暴露；再加上機構託管這條第二層鏈下鏈。而「停贖 + 法律顧問」就是鏈下風險已經爆穿的招牌訊號。這段跟 08-15 講的 Inverse PSM「理論地板存在但實際為空」是同一種警覺：別信機制表層的承諾，要追風險真正落腳在哪、誰在對面。一句話：賣「無風險市場中性收益」的協議，賣點越穩，越依賴鏈下某個能做空前後的實體把收益「無風險」地做出來——而那正是最不透明的一環。

笔记 039: 学习记录

作者：capromka | 时间：2026-08-17 07:16 UTC | 字数：383 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习记录

今天继续学习LP 相关内容，进一步理解了集中流动性下的仓位管理和收益逻辑。

主要收获

理解了 V3 LP 的核心是选择合适的价格区间，资金不是平均分布在所有价格上。

区间越窄，资金效率越高，但价格更容易跑出区间，需要更加频繁地管理。

当价格在区间内时，LP 可以持续赚取交易手续费；价格跑出区间后，仓位会逐渐变成单边资产，同时停止赚手续费。

LP 的收益不能只看手续费，还需要考虑 无常损失、价格变化、Gas 和调仓成本。

做 LP 本质上是在权衡：资金效率、手续费收入、价格风险和管理成本。

今日总结

V3

LP

的关键不是单纯“把钱放进池子”，而是选择合适的价格区间，并根据市场变化管理仓位，在手续费收益和价格风险之间找到平衡。

笔记 040: 参考

作者：mookixiao | 时间：2026-08-17 11:39 UTC | 字数：1317 | 评分：77

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

参考

Gemini

问题

无

笔记

1. CCTP (Cross Chain Transfer Protocol, 即跨链传输协议)：是Circle官方为USDC设计的跨链基础设施协议，基于“销毁-铸造 (Burn & Mint)”的通信原语实现USDC的跨链，工作流程：

1. 源链销毁：在原链调用CCTP合约，销毁 (Burn) USDC

2. 链下验证：Circle验证节点确认销毁后，签署密码学证明 (Attestation)

1.

链下 (off-chain)：指不运行在区块链智能合约内部、不占用区块链虚拟机 (EVM) 计算资源、也不直接消耗Gas费的外部软件系统
此处的“链下”指的是Circle官方部署在云端服务器上的API与监控程序

3. 目标链铸造：凭证明在目标链直接凭空铸造 (Mint) 等额原生USDC

2. 查询USDC在不同链上的Token Address：<https://developers.circle.com/stablecoins/usdc-contract-addresses>

3. AMM (Auto Market Maker, 自动做市商)

1. 核心恒定乘积公式

1. 假设在资金池中， x 表示ETH储备量， y 表示USDC储备量， k 表示恒定乘积，有： $x \times y = k$

2. 此时，交易者输入 Δy USDC 换取 Δx ETH ，有： $(x - \Delta x) \times (y + \Delta y) = k$

3. 推导得： $\Delta x = x - \frac{k}{y + \Delta y} = x - \frac{x \times y}{y + \Delta y} = \frac{x \times \Delta y}{y + \Delta y}$

2. 举例，原来在资金池中 $x = 100 \mathrm{ETH}$ ， $y = 200000 \mathrm{USDC}$ ， $\Delta y = 2000 \mathrm{USDC}$

1. 原ETH边际价格（边际价格指订单无限小、不产生价格冲击时的价格）为 $\frac{y}{x} = \frac{200000}{100} = 2000 \mathrm{USDC/ETH}$ ，理论ETH兑换量 $\frac{\Delta y}{2000} = \frac{2000}{2000} = 1 \mathrm{ETH}$

2. 实际ETH兑换量 $\Delta x = \frac{100}{\frac{2000}{0.990099}} \approx 200 \mathrm{USDC/ETH}$ ，平均成交价 $\frac{2000}{0.990099} \approx 2020 \mathrm{USDC/ETH}$

05 其他 (1 篇)

笔记 041: 1\ 闭环资产验证 (M1 验收项)

作者：Wang Hua | 时间：2026-08-17 07:04 UTC | 字数：761 | 评分：59

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1\ 闭环资产验证 (M1 验收项)

\ 编写 `scripts/verify\_case01\_loops.py`：逐笔检查 222 笔成功交易的 signer 资产 delta 结构

\ 结果：8 笔 `strict\_loop` + 214 笔 `loop\_with\_residual`，全部通过 signer 层闭环验证

\ 用“base delta > 当日中位数 10 倍”标记出 25 笔 `dominance\_anomaly`

2\ 重大修正发现

\ 此前的“USDC 闭环毛利 989.70”中，956.78 (96.7%) 来自单笔异常交易 5vjumj2MaDEJKQ

\ 已从交易日志验证它是真实 Raydium→Orca 闭环 (5,997 USDC → 63,450 Saber2g → Orca 卖出)，但规模是当日中位数 50+ 倍

\ 剔除异常笔后，4 日 USDC 闭环毛利约 32.9 USDC——此前汇总数字严重高估

\ 这个修正证明：单笔大额交易主导的汇总必须逐笔验证，不能直接加总

3\ Burst 聚合 (120 秒预注册规则)

\ 编写 `scripts/aggregate\_case01\_bursts.py`：7,130 笔聚成 537 个 burst

\ 92 个 burst (17.1%) 含成功交易，32 个含 ≥2 笔成功

\ 最大 burst：506 笔 / 19 成功 (01-18 15:09 起)

\ 失败费用只在 burst 内聚合，不摊给单笔成功

06 套利框架 (31 篇)

笔记 042: 课程 Day 11 | 暂停旧课程路径，重新定义研究问题

作者：shiyang07ca | 时间：2026-08-16 15:02 UTC | 字数：801 | 评分：73

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

课程 Day 11 | 暂停旧课程路径，重新定义研究问题

今日实际完成

重新运行当前仓库测试，确认 lab.test_audit_lighter_rwa 仍是 4/4 通过；同时复现请求 XAG 却返回 BRENT OIL/159 的身份错配。这个缺陷真实存在，但它只是研究系统的一处局部边界，不应继续占用整天课程。

通过公开只读 API 重新获取两边市场身份：Lighter 当前包含 XAU/92、XAG/93、WTI/145、BRENT OIL/159；Hyperliquid HIP-3 当前使用完整名称 xyz:GOLD/110003、xyz:SILVER/110026、xyz:CL/110029、xyz:BRENT OIL/110049。

核对官方说明后确认：完整市场身份只能证明“数据属于谁”，不能证明两个市场代表相同经济对象，更不能证明价差扣除深度、费用、funding、延迟与双腿成交风险后仍可交易。

今天最重要的判断

当前课程把过多时间花在孤立字段、网页问答和局部代码缺陷上，没有持续推动最终能力：独立发现候选、快速证伪、确定性重放、实时模拟与运行复盘。因此暂停沿现有课表机械推进，重新从下面的问题设计后续学习：

给定两个真实市场，能否用可复现数据证明或否定一个净收益假设，并明确下一项最便宜、最能证伪的实验？

下一步

先研究最近的共学笔记与官方一手资料，找出值得迁移的机会发现方法、成本模型和失败案例；再审查仓库现有文件，只保留能支持“发现 → 证伪 → 重放 → 模拟 → 复盘”的代码和证据。Day 12 到结营将改为少量、完整、可运行的研究任务，而不是继续堆概念页。

边界

本次只使用公开行情和文档，没有读取或保存交易密钥，没有发送订单，也没有宣称任何候选存在可执行套利收益。

笔记 043: markdown

作者：y-cat-van | 时间：2026-08-16 15:21 UTC | 字数：491 | 评分：52
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

markdown

学了什么

案例还原不是“复盘赚了多少钱”，是把这笔的盈亏拆出归因：靠策略 / 靠基础设施 / 只是时间窗口的运气（计划 D9-11 原话）。这三者天差地别——靠策略的可复制、靠基础设施的要看你有没有那个设施、靠运气的根本不能进 Agent。一笔赚钱的案例如果归因错了，比亏钱还危险，因为你会去复制一个复制不了的东西。

而群里今天沉在最底下的那句“有价差的地方没有流动性”，正是我这一整周（D3 价差≠利润 → D5 滑点二次 → D7 池深是分布）用数学一路逼近、却一直没能一句话说破的东西。被幻☆精灵一次实测、群里一句共识给点破了。

点破之后我立刻看见自己前一周藏得最深的一个错：我把“价差”和“流动性（池深 L）”当成两个独立的输入，分别喂进引擎去扫。错。它俩反相关——屏幕上你能看到的大价差，恰恰长在没人能填平的浅池上。价差之所以还活着没被抹平，唯一的原因就是它吃不动：谁去填它，谁的单量就吃穿池子，滑点比价差还大。价差和浅池，是“未被套利填平”这同一个状态的两张脸，不是两个可以自由组合的自变量。

笔记 044: 打卡笔记：周末效应与杠杆 ETF 偏离规律（监控第 20-22 次）

作者：Wave | 时间：2026-08-16 15:38 UTC | 字数：1236 | 评分：83
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

打卡笔记：周末效应与杠杆 ETF 偏离规律（监控第 20-22 次）

链上套利残酷共学 · 2026-08-16

今天做了什么

系统性偏离监控自动化恢复全量运行（代理修复后 6 平台全通），连续执行 3 次（20:45/21:43/22:41），专门观察周末休市期间杠杆 ETF 永续合约的偏离行为。

一、关键环境变化

代理 127.0.0.1:6864 恢复：自 8/12 起连续 10 次执行宕机后首次恢复，Binance/OKX/MEXC/Bybit 六平台全通、首轮零重试基准切换：8/13+8/14 美股连续暴涨后，偏离已从 8/12 的极端值（SNXX +1731bps）回归正常区间

二、三次监控的核心观察

| 执行 | SNXX | KORU | SNDK | 特征 |
|---|---|---|---|
| 20:45 | +181.5 | +226.2 | +75 | 杠杆 ETF 仍>150bps 异常区 |
| 21:43 | +165.7 | +189.5 | +86.2 | KORU 触发趋势收敛 |
| 22:41 | +175.7 | +201.5 | +81.5 | 全部平稳，0 个趋势变化 |

规律：偏离全为正，且随杠杆倍数放大（SNXX 2x≈2xSNDK、KORU 3x≈3xEWY），跨平台 spread 极小 → 周末杠杆溢价是常态，无套利空间。

三、核心洞察

1. 周末效应机制：周日美股休市，基准冻结在 8/14 周五收盘，而永续合约继续交易——偏离 = 杠杆 ETF 的"周末时间价值 + 周一开盘预期定价"，天然随杠杆倍数放大
2. 杠杆比是试金石：SNXX/KORU 杠杆比精确还原 (2x/3x) = 合约定价自治；MUU 杠杆比异常 (1.27x-1.41x 而非 2x) 且跨平台分歧最大 (Gate vs Binance 差 21.6bps) ——唯一值得留意的信号，可能是 MUU 流动性差导致
3. 验证方式固定：周一美股开盘后基准更新，周末偏离预期全部收敛；若开盘后仍不收敛才是真信号

四、方法论沉淀

周末监控的正确姿势：不必对杠杆 ETF 的正偏离紧张——先查杠杆比能否还原，再查跨平台 spread，最后等开盘验证异常信号清单：杠杆比失配（如 MUU 1.4x）、跨平台分歧扩大、周一开盘后不收敛——三者同时出现才值得深挖
代理恢复的意义：6 平台全通后，单平台标的（SNXX/MU/MUU）的跨平台 spread 验证终于可行，数据可信度大幅提升
下一步

周一（8/17）开盘验证周末偏离收敛

重点盯 MUU 的杠杆比异常是否持续

继续完善监控：把"杠杆比还原度"作为自动化判定的第一道过滤

笔记 045: 本地做了下面这些测试，逐渐熟悉所有流程

作者：yhzongc | 时间：2026-08-16 15:42 UTC | 字数：1020 | 评分：60

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

本地做了下面这些测试，逐渐熟悉所有流程

1. 固定 Fork 部署验证：确认 Arbitrum 链 ID、历史区块、Token、Router、Factory、Pair 地址及池状态正确。
2. Token 精度验证：确认程序读取到 USDC 为 6 位小数、WETH 为 18 位小数。
3. 池状态报告：显示 Uniswap、Camelot 的储备、现货价格、10 USDC 报价和报价差。
4. 交易金额扫描：分别测试投入 1、5、10、20、50 USDC 时的输出、折价、毛利润、Gas 和模型净利润。
5. 储备变化报告：比较套利前后两个池的 WETH、USDC 储备及价格差变化。
6. 历史区块重放：比较三个固定 Arbitrum 历史区块的储备和完整往返报价。
7. 旧报价实验：在报价与执行之间插入另一笔交易，验证旧报价失效、严格保护回滚和宽松保护利润下降。
8. Gas 敏感性测试：比较 0.01、0.1、0.5、1 gwei 下的模型 Gas 成本和净利润。
9. 单档 Gas 模型报告：输出 10 USDC 成功交易的 Gas Used、Gas 成本、毛利润和模型净利润。
10. 原子套利成功路径：验证 10 USDC 能在一笔本地交易中完成 Uniswap 买入和 Camelot 卖出。
11. 第一跳失败回滚：当 Uniswap 最低 WETH 输出不满足时，验证所有余额和池储备不变。
12. 第二跳失败回滚：当 Camelot 最低 USDC 输出不满足时，验证第一跳也被整体撤销。
13. 零本金测试：amountIn 为零时必须使用明确的自定义错误回滚。
14. 零第一跳最低输出测试：minimumWethOut 为零时必须回滚。
15. 零第二跳最低输出测试：minimumUsdcOut 为零时必须回滚。
16. 零最低利润测试：minimumProfitUsdc 为零时必须回滚。
17. 本金未收回测试：最终 USDC 少于投入本金时必须回滚。
18. 毛利润不足测试：本金已收回但利润低于最低门槛时必须回滚。
19. transferFrom 返回失败测试：ERC-20 拉取本金返回 false 时必须转换为明确的自定义错误。
20. approve 返回失败测试：ERC-20 授权 Router 返回 false 时必须转换为明确的自定义错误。

笔记 046: Day 9 | 比價工具邁向多鏈：一個地址寫錯的教訓，與「單一事實來源」重構

作者：HenryChang | 时间：2026-08-16 15:45 UTC | 字数：1070 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 | 比價工具邁向多鏈：一個地址寫錯的教訓，與「單一事實來源」重構

比價專案繼續推進 (+3,324 行)：為五鏈擴充做地基重構，過程中抓到一個很有教育意義的 bug。

一個高危 bug：BSC 上的 WETH 地址寫錯了

alias 表裡 BSC 鏈的 WETH/WBNB 地址標錯，修正前查價會讀到錯的池。學習點有三層：

1. 多鏈世界裡「同名 token ≠ 同一 token」：每條鏈的合約地址完全不同，BSC 上的 WETH 是橋接資產、WBNB 才是原生包裝幣——複製地址時搞混，工具會一本正經地回報錯誤池子的價格

2. 這正是 Bruce 第一性原理說的「起點終點可比性」問題的工程版：資產 $\times$ 鏈 $\times$ 形態，任何一個維度錯了，算出來的價差都是幻影——比 stale state 更隱蔽，因為數字看起來完全合理

3. 防禦方法：地址入表前用鏈上 `symbol()/decimals()` 回讀驗證（我們的 `TokenResolver` 本來就有這能力，教訓是連「內建 alias 表」也要過同一道驗證）

重構：chain registry 單一事實來源

原本鏈相關資料（RPC 端點、原生幣、gas 參數、瀏覽器 URL...）散落在各模組。這次收攏成一個 chain registry，所有模組從同一處取鏈資料。動機：接下來要從 2 鏈擴到 5 鏈，散落的鏈常數會讓每次加鏈都變成全 repo 找碴。先把資料歸一、再做規模擴張——和 Day 6 學的「分層是為了讓引擎搬得動」是同一課的續集。

其他工程進度

server 生命週期遷移到 FastAPI lifespan（新版標準寫法，資源啟停更可控）

錯誤訊息人性化：輸入非合約地址時給出友善提示（含「是不是 BSC 地址？」的引導）——工具給人用，錯誤訊息就是 UX

五鏈交易對頁面實作計畫完成（docs），下一步執行

collector 持續蓄水：spread-log 新增一天記錄（8/12、8/13、8/15 三天資料在手）

收穫總結

今天最值錢的是那個地址 bug——它把「配置也是程式碼、配置錯誤最難察覺」具象化了。比價工具的可信度建立在每一個 token 地址、每一個 decimals、每一條鏈參數都對的前提下；驗證機制（鏈上回讀）比人眼複查可靠。

明日計畫

執行五鏈擴充計畫；spread-log 出軌分析（頻率/幅度/存活時間）持續排隊中

笔记 047: 分析 Gas、滑点与手续费

作者：Marco | 时间：2026-08-16 15:46 UTC | 字数：736 | 评分：54

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

分析 Gas、滑点与手续费

第八天的学习重点是分析 Gas、滑点与手续费对套利结果的影响。建立净收益模型之后，还需要进一步观察各项成本如何随着交易规模、网络状态和路线变化，才能判断策略在哪些条件下成立。

Gas 通常更接近固定执行成本，但会受到网络拥堵、交易复杂度和优先级影响。交易规模较小时，Gas 占比可能较高；规模增加后，Gas 的相对占比下降，但更大的交易通常会带来更明显的滑点和价格冲击。

滑点和价格影响更多与交易金额和池子流动性相关。相同交易对在不同输入金额下，最终输出可能出现明显差异。报价中的预估输出不能脱离交易规模理解，还应记录流动性、价格影响、最小可接受输出和滑点保护，避免把理论价格误认为实际成交价格。

手续费也需要拆分记录，包括 DEX 交易费、桥费、聚合器费用以及借贷或资金成本。不同协议、路线和金额区间的收费方式可能不同，不能用一个固定费率覆盖所有情况。对每项成本单独建字段，才能判断利润被哪一部分消耗。

分析这些成本时，可以设计低、中、高 Gas，以及不同交易金额和滑点水平的情景，比较毛收益、总成本、预期净收益和盈亏平衡价差。若策略只有在最理想条件下才盈利，就不具备足够的安全边际。

今天的关键知识点是：成本不是收益模型的附属项，而是决定套利是否可执行的核心变量。固定成本影响小额交易的可行性，滑点和价格冲击影响大额交易的边界，手续费则决定不同路线之间的真实优劣。

下一步可以选择几条样本路线，分别测试不同金额和成本情景，并将结果记录到敏感性分析表中。第九天将进一步研究执行延迟、失败原因和桥风险，把“理论可行”推进到“执行风险可量化”。

今日小结：价差决定机会的上限，Gas、滑点和手续费决定最终能否留下利润。

笔记 048: 具体任务

作者：derenjie | 时间：2026-08-16 15:49 UTC | 字数：312 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

具体任务

1. 选择一种简单策略

同一资产在两个 DEX 之间的价差

同一资产在两条链之间的价差

2. 写出进入条件

例如：

只有当价差大于 Gas、桥费、手续费、滑点和安全缓冲后，才记录为候选机会。

3. 建立模拟交易表

记录：

时间

资产和链

买入价格

卖出价格

总成本

预计净收益

是否真的可执行

最后失败或忽略的原因

4. 观察并记录 3 个案例

不需要真实下单，只判断：

这是表面价差，还是可执行价差？

扣除成本后是否还有利润？

执行过程中最大的风险是什么？

笔记 049: D09 | 稳定币套利

作者：averywexler | 时间：2026-08-16 15:52 UTC | 字数：676 | 评分：69

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D09 | 稳定币套利

偏离 1 美元可能是暂时失衡，也可能是信用、赎回或流动性风险的信号。

今日术语

锚定：资产价格围绕某个目标值运行；

脱锚：市场价格明显偏离目标值；

赎回机制：按规则将稳定币换回储备资产的路径；

发行方风险：由中心化发行、托管、冻结或合规带来的风险；

池子失衡：稳定币池中一边资产占比显著上升；

StableSwap：针对价格接近资产设计的 AMM 曲线。

参考

Circle：What is USDC? (<https://developers.circle.com/stablecoins/what-is-usdc>)

Curve StableSwap 白皮书 (https://docs.curve.finance/assets/pdf/whitepaper_stableswap.pdf)

例子：0.985 不一定是“白捡 1.5%”

某稳定币显示 0.985 USDC。买入 10,000 单位并假设回到 1.000，屏幕毛利为 150 USDC。但你还要回答：

1. 是否能按 1 美元赎回，谁可以赎回，要多久？

2. 10,000 单位的实际买入均价是多少？

3. 池子是否正在快速单边失衡？

4. 发行方、桥接版本和合约地址是否一致？

5. 回锚前资金占用和尾部风险值多少钱？

这些问题最后都会影响实际的结果。总的来说其实稳定币套利这个方向也是不可行的，很难去找到足够的价差空间。

笔记 050:

作者：Ox2077 | 时间：2026-08-16 15:53 UTC | 字数：780 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

| 策略 | 描述 | 为什么适合你 |

| :----- | :----- | :----- |

| 长尾资产套利 | 避开主流的 ETH/USDT，关注冷门小币种或新协议的价差。 | 竞争压力小，对“网感”和“观察力”要求高。 |

| 统计套利 (统计学套利) | 不追求单笔暴利，通过大量的小额、低风险统计模型获利。 | 考查架构的稳健性和多点监控能力。 |

| Jit Liquidity (即时流动性) | 预测交易路径，提前在特定价格区间部署流动性。 | 属于“降维打击”，需要对 DeFi

协议底层逻辑有极深理解。|

在主流视野之外，存在大量的“微小机会”，大机构看不上，小散户看不懂：

微量清算 (Micro-Liquidation)：某个小借贷协议里有人抵押了 50U 的代币，跌破了阈值。大机器人不跑这种单子（因为 Gas 不划算），但如果你在低 Gas 的 L2 上，一笔清算可能赚 0.5U。

长尾套利 (Niche Arbitrage)：某个新公链上的两个小 DEX 之间存在 2% 的价差。

代币领取/复投自动化：很多协议的收益需要手动 Claim 才能复投。你可以写脚本帮自己（或小规模社区）自动化这个过程，利用复利效应或节省下来的 Gas 费作为收益。

笔记 051: 今天做了什么

作者：zenkai | 时间：2026-08-16 16:10 UTC | 字数：428 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天做了什么

D7 成本模型收尾:①机会证据库更新到 D7 视角——OPP-005 清算假设由 D6 数据(7 天仅 3 笔)验证后降级为"事件监控项";OPP-006 新链价差条件化,新增规则"新链 TVL<\$1 亿直接跳过";新增 OPP-007 成本模型模板(净收益 = 毛价差 - Σ七项成本)。②D7 成本模型讲解视频启动生成(NotebookLM,台湾住宅节点,源=成本模型模板)。

证据在哪

机会证据库.md(OPP-005/006/007 状态与数据);成本模型模板.md(Scroll 案例:Σ0.96% vs 价差 0.5%);D7 视频 notebook 8d2b8f54(生成中,预计 10-20 分钟)

卡点

视频生成待完成;补卡由 Hermes 整理,内容本人核实

明天任务(D8)

用成本模型验证新链机会:LI.FI 实测一条 TVL>\$1亿 新链的价差 vs 全成本(或直接进回测脚本)

笔记 052: Day 13 | 2026.08.17 | Monad 成本模型：为什么 gasLimit 不能乐观

作者：xzone911 | 时间：2026-08-16 18:34 UTC | 字数：2202 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 13 | 2026.08.17 | Monad 成本模型：为什么 gasLimit 不能乐观估算

背景

在 Monad 上做 MEV，第一件事不是写搜索，而是先搞清成本模型。它有几个和以太坊、Solana 都不同的硬约束，直接决定利润公式怎么写、以及「什么时候才允许签名」。

Monad 的四个特殊规则

1. gasLimit 不退回

这是最反直觉的一条。以太坊上你习惯「多给点 gas 反正用不完会退」，Monad 上成本 = 提交的 gasLimit × gasPrice，估多了就是实打实多付。所以 gasLimit 的估算必须精确——但精确估算又有「估少了 out of gas」的风险，这个矛盾需要单独处理。

2. 最低 gas price 100 gwei

低于 100 gwei 会被 clamp/reject。默认提交用 121 gwei，100 只是硬下限，不是推荐值。

3. 先确认顺序、下一块才执行

Monad 出块不是「这一块就执行」，而是先确定交易顺序，下一个块才真正执行。这意味着从「读到状态」到「交易真正生效」，中间隔着至少一个块的延迟。这个延迟直接决定了状态新鲜度和 submit gate 必须保守。

4. gasLimit 不够 = 直接 out of gas

和「不退回」叠加起来，形成一个双重约束：估多亏钱，估少失败。两边的代价都是实打实的。

三层利润公式

text

```
gas_cost = gas_limit_submitted × gas_price_used
net_profit = gross_profit - flash_loan_fee - fastlane_bid_amount - gas_cost
profit_ok = net_profit >= min_profit
```

关键设计决策：gas 成本按「提交的 gasLimit」全额计，不按实际 gas used。因为 Monad gasLimit 不退回，按实际 gas used 乐观估算会系统性高估利润——你会在「以为赚」的情况下签名，实际亏掉 gas 差额。

gasLimit 的估算链路

text

$\text{eth_estimateGas} \times \text{safety_factor}$ (默认 12000 bps = 1.2x) → 向上取整

为什么是 1.2x 而不是 1.5x 或 1.1x？这是一个「估算误差」和「多付成本」之间的平衡：

系数太小：estimateGas 本身有误差（尤其跨多个 DEX 的多跳路径），估少了直接 out of gas，整笔交易失败

系数太大：Monad gasLimit 不退回，放大越多实付越多，吃掉本来就不厚的套利润

1.2x 是「覆盖估算误差」和「不浪费太多成本」的折中

但有个容易被忽略的细节：放大后的

要重新全额计入成本。不能「先放大拿到更安全的执行，又只按原估算算利润」——这等于自己骗自己，放大带来的额外成本必须反映到 net_profit 里。

gasLimit

gas

FastLane 提交的经济决策

FastLane 是「贿赂 builder 优先打包」。默认阈值 10 MON：

text

预计净利润 ≤ 10 MON → 走 direct (普通提交，不付 bid)

预计净利润 > 10 MON 且 $\text{bid} \neq 0$ → 走 FastLane (付 bid 换优先级)

几个硬约束：

bid 默认按利润的万分之 100 (1%)，可配置覆盖

payBidOnFail 固定禁用——不允许用 shMONAD 抵押物补 bid，因为失败时 bid 是纯损失，不能用抵押物兜底

最终 live submit 前仍按 $\text{gross} - \text{flash_fee} - \text{fastlane_bid} - \text{gas_cost} - \text{buffer}$ 判断

这里有一个关键的会计陷阱（后面 Day 21 会详细展开）：FastLane bid 通过 msg.value 预付时，链上利润守卫必须把「预付的 bid」和「route 自己赚的利润」分开算，否则会出现「route 实际没赚到 bid，但因为预付的 bid 被算进 profit，守卫误判通过」的 bug。

今日认知

成本模型不是「扣多少手续费」，而是「在什么边界下才签名」。Monad 的三个规则 (gasLimit 不退回 + 最低 100 gwei + 先确认后执行) 叠加起来，意味着：

1. gasLimit 估错 = 实打实亏钱，不能乐观
2. gasLimit 估少 = 交易 out of gas，也失败
3. 状态延迟 = 机会可能在你提交时就消失了

所以「精确估算 + 1.2x 安全系数 + 全额重算成本」三件事缺一不可。这和我 Solana 项目里「保守费用预算从 5,000 调到 19,000 lamports」是同一个思路：用最坏情况做决策，不用期望值做决策。套利的利润很薄，经不起任何「乐观估算」。

笔记 053: 探讨在 cex-dex 套利中我一直存疑的问题，Day 11：【股票/RWA】

作者：ninanren05 | 时间：2026-08-17 00:19 UTC | 字数：400 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

探讨在 cex-dex 套利中我一直存疑的问题，Day 11：【股票/RWA】

今天主要研究了山寨市场降温后，股票/RWA 是否可能成为新的套利方向。

几个收获：

股票代币化、股票永续正在快速进入 Crypto 交易所，未来可能形成新的跨市场价差机会

股票 Spot / Perp、CEX / DEX、Tokenized Stock 之间，本质上仍然可以复用 Crypto 套利的很多思路

传统山寨市场确实明显降温：相比去年 BTC 高点附近，山寨成交活跃度已经大幅下降

与其赌“下一轮山寨季什么时候来”，不如让套利系统具备扩展不同资产类别的能力

最大的认知变化是：

以后想做的可能不应该只是一套“山寨币套利系统”，而是一套寻找多市场异常定价的系统。

先从熟悉的 Crypto 市场做起，但给股票/RWA 留好扩展空间，让真实数据决定以后主攻哪里。

笔记 054: 今天结束时的真实状态

作者：blimnwind-bot | 时间：2026-08-17 02:42 UTC | 字数：1355 | 评分：74

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天结束时的真实状态

RWAT 001

Evidence Pipeline 5/5

Strategy Evidence MEASURED

Edge State MEASURED_POSITIVE_EDGE

Eligible Cycles 6

Net Measured 6

Net Profitable 6

Total Net PnL +\$328.99

Pattern Score 90

Evidence Confidence 90

ARBITRAGE CANDIDATE

USD Accounting PASS

Asset Accounting REVIEW

Asset Closure REVIEW_BOUNDED_ACCEPTED

RWAT Acceptance REVIEW

Smart Money false

PERP_TRADER NO_EVIDENCE

所以今天最大的成果不是：

找到了一个赚钱钱包。

而是建立了一条可以不断证伪自己的链：

RawEvidence→EconomicAccounting→ExecutionProvenance→FeeInclusion→NetEdge→Repeatability→StrategyVerdict

最后得到：

6条真实可观察交易→6条净收益可测→6条全部为正→ARBITRAGE=CANDIDATE

今日最大的认知

今天最大的变化，是我越来越不想问：

这个地址是不是聪明钱？

因为这句话太容易把结果先写好，再去找证据。

现在更合理的顺序是：

先证明发生了什么→再证明钱如何变化→再证明收益从哪里来→最后才讨论它是不是SmartMoney

一个 0x0 → wallet 的 mint 可以只是跨链资产到账。

一个 debt token 可以是负债，不是资产。

一条路径闭环可以只是 protocol transformation，不是套利。

一个正 Gross PnL 也可能因为成本未知而完全没有 Net Edge 证据。

所以 Smart Money Engine 真正应该做的，不是“给钱包打高分”，而是：

把钱包行为重建成一份可审计的经济事实。

下一步

冻结路线：

CANDIDATE→AssetClosure→SoftGapScope→SmartMoneyPromotion

其中 Asset Closure 今天已经完成。

下一刀只审：

REWARD_CLASSIFICATION_UNAVAILABLE

判断它究竟是钱包全局缺口，还是确实会影响这 6 条套利证据。

不扩钱包。

不继续找新的 Edge。

不碰 Perp。

不为了 SUPPORTED 修改冻结规则。

笔记 055: CEX—CEX 套利赚的是同一枚币在两所的可成交价差，属于库存套利，非原子。

作者：Montaire | 时间：2026-08-17 03:19 UTC | 字数：199 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

CEX—CEX 套利赚的是同一枚币在两所的可成交价差，属于库存套利，非原子。

价差方向随机，必须两边各压 USDT 和 BTC（四份库存），本金占用翻倍。

核心分界：成交阶段赚差价，再平衡阶段为下一次备货；价差到手不等于结束。

主流币跨所价差薄到约 0.003%，双边手续费约 0.2%，净亏，散户无利可图。

三个亏钱原因：手续费、时间差收敛、深度滑点（盘口价 $\neq$ 成交均价）。

笔记 056: https://github.com/ZeroNullNone/onchain_arbitrage_

作者：Nothing | 时间：2026-08-17 05:06 UTC | 字数：1246 | 评分：71

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/docs/daily/day_13.md

Day 13 — Transaction Simulation

状态与结论

状态：Day 13 paper-only acceptance scope 已完成。

在 Base 的 same-chain USDC→WETH 任务链上构建了 eth_call 仿真 evidence 的解析与比较。

成功仿真、Allowance 不足、Min-output Revert 三类证据均可通过原始 fixtures 重建。

完成项

- 新增 src/onchain_arb/simulation.py。
- load_raw_simulation 从 append-only raw envelope 重建仿真记录。
- compare_quote_and_simulation 输出 SimulationComparison，包含执行判断与 reject 原因。
- 明确记录：gasUsed、ERC20/ETH 级别余额变化、revert reason、allowance、block number、raw reference。
- 为 quote 与 simulation 的可执行性判断实现：
- token 匹配
- quote/minimum output 覆盖
- allowance 覆盖
- stale block 检查
- 新增 3 个测试 fixture 和 Day 13 acceptance 测试。

Evidence 与验证

- Simulation fixtures (raw)：
- tests/fixtures/simulation/day13_success.json
- tests/fixtures/simulation/day13_allowance_reject.json
- tests/fixtures/simulation/day13_min_output_revert.json
- Acceptance tests：tests/test_simulation.py

Gate 边界

本日只实现 eth_call 单链仿真；未集成 Tenderly/local fork。

未实现任何 wallet 操作、签名、广播。

仿真输出只用于决定是否 executable，不改变基础策略定义。

下一步

Day	14	将把	Simulation	比较结果并入	Scanner
gate：DETECTED→REQUOTING→COSTED→INVENTORY_CHECKED→SIMULATED→PAPER_READY。					

笔记 057: 资金费对冲:从零到「我该从哪开始」

作者：starkxun | 时间：2026-08-17 06:45 UTC | 字数：11018 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

资金费对冲:从零到「我该从哪开始」

2026-08-11 | 配套脚本:funding_probe.py (https://vscode-remote+wsl-002bubuntu-002d22-002e04.vscode-resource.vscode-cdn.net/home/starkxun/MEV-LIFI/funding_probe.py)(只读公开行情,不需要 API key)\

前置调研:套利工具调研-TaoliTools与Backpack.md (https://vscode-remote+ws1-002bubuntu-002d22-002e04.vscode-resource.vscode-cdn.net/home/starkxun/MEV-LIFI/docs/week_2/%E5%A5%97%E5%88%A9%E5%B7%A5%E5%85%B7%E8%B0%83%E7%A0%94-TaoliTools%E4%B8%8EBackpack.md)

如果你只想知道从哪开始,直接跳到最后一课。但前面的坑建议先看,\n
因为我在写这份文档的过程中,第一次测就错了 9.3 倍,还把 3 个币的方向搞反了。

第 0 课:为什么这个方向和前面四周都不一样

我们这四周否掉了四个方向,死因只有两种:

- ① 延迟竞赛赢不了 原子套利、公开喂价清算
- ② 价值被拍卖回收 SVR 吃掉 89%,Atlas 上 Top1 占 84.6% 的中标

两种都是赢家通吃:机会是一份,谁先拿到就没别人的了。\
小而中立的玩家在这种结构下天生吃亏,这不是努力能改变的。

资金费不是这个结构。

资金费是按未平仓量、按比例、付给所有空头的。\
没有人能"抢在你前面拿走一笔资金费"——你持有多少仓位,就按比例拿多少。

这是四周以来第一个不存在赢家通吃的方向。\
结构不同,不是程度不同。

代价是:它也不是套利,是 carry(持有报酬)。第 7 课细说。

第 1 课:资金费到底是什么

永续合约没有交割日。那怎么让它的价格贴着现货走?

靠一笔周期性的转账,叫资金费。

永续价格 > 现货价格 → 说明多头太热
→ 多头付钱给空头 (资金费为正)

永续价格 < 现货价格 → 说明空头太热
→ 空头付钱给多头 (资金费为负)

它不是交易所收的手续费,是多空之间的转账。交易所只是搬运工。

Backpack 上每小时结算一次(注意:不是常见的 8 小时,别写死):

```
python3 funding_probe.py floor
```

结算间隔 1 小时,一年 8760 次

第 2 课:carry 怎么组

要收资金费(而不是付),在资金费为正时:

现货买入 1 个 ETH ← 你持有 ETH,涨跌都跟你有关
+ 永续做空 1 个 ETH ← 反向对冲掉涨跌

= 净价格敞口 ≈ 0

但你是空头 → 每小时收一笔资金费

两条腿的价格波动互相抵消,剩下的就是资金费。

这就是为什么它满足我们的闭合检查器 (https://vscode-remote+ws1-002bubuntu-002d22-002e04.vscode-resource.vscode-cdn.net/home/starkxun/MEV-LIFI/docs/week_3/%E9%97%AD%E5%90%88%E6%A3%80%E6%9F%A5%E5%99%A8%E8%AE%BE%E8%AE%A1.md)第 ① 问\

——起点是 USDC,终点也是 USDC,token 口径闭环。\
(对比 DOS 那个池子:买进去之后你持有的是 DOS,闭不上环,那是投机。)

第 3 课:动手 —— 先量地板

任何判断之前,先知道要跨过多高的门槛。

```
python3 funding_probe.py floor
```

```
=== Backpack Tier 1 费率(官方文档,bps)===
```

```
spot_maker 8 spot_taker 10
```

```
perp_maker 2 perp_taker 5
```

```
=== 往返手续费地板(开仓 + 平仓,现货腿 + 永续腿)===
```

```
全 maker 20 bps
```

```
全 taker 30 bps
```

为什么是 4 笔手续费:开仓时现货买一笔、永续开空一笔;平仓时各再来一笔。

然后换算成"需要多高的年化才回得了本":

持仓 1 天回本 → 需要年化 73.00%

持仓 3 天回本 → 需要年化 24.33%

持仓 7 天回本 → 需要年化 10.43%

持仓 30 天回本 → 需要年化 2.43%

持仓 90 天回本 → 需要年化 0.81%

这张表就是整个策略的形状

持仓时间越长,手续费越不重要。 持 90 天的话,0.81% 年化就够回本。 \

反过来说:这不是一个"快进快出"的策略,越频繁调仓,地板越吃人。

第 4 课:动手 —— 短窗口会骗你

先跑一个"快照",看当前哪些币资金费高:

```
python3 funding_probe.py rates --limit 100
```

我第一次跑的结果(100 期 = 约 4 天):

BILL_USDC_PERP +126.01% 同号 100%

SKR_USDC_PERP +70.60% 同号 97%

WLFI_USDC_PERP +16.05% 同号 100%

XPL_USDC_PERP +14.05% 同号 99%

WLD_USDC_PERP +13.23% 同号 100%

看起来太太好了:年化 126%,而且 100 期里 100 期都是同一个方向。

我当时差点就写"BILL 是最佳候选"。

第 5 课:把窗口拉长 —— 真相

```
python3 funding_probe.py history BILL_USDC_PERP
```

对同样这 5 个币,拉满历史再看一遍:

```
| 币 | 4 天窗口 | 长窗口(真相) | <br /> |
| :--- | :----- | :----- | :----- |
| BILL | +126.01%,同号 100% | +13.57%(88 天),同号 81% | 高估 9.3 倍 |
| SKR | +70.60%,同号 97% | 33.44%(201 天),两半反号 | 符号反了 |
| WLFI | +16.05%,同号 100% | 2.89%(344 天),两半反号 | 符号反了 |
| WLD | +13.23%,同号 100% | 11.25%(344 天),同号 55% | 符号反了 |
| XPL | +14.05%,同号 99% | +9.74%(320 天),同号 85% | 一致 |
```

5 个"100% 稳定"的候选,3 个在长窗口下符号是反的。

如果按短窗口下单做空 SKR 收资金费,你实际上会一直在付钱。

这一课是整份文档最重要的

这是我在这个项目里第二次犯同一个错。 上一次是清算:\

用 14 天窗口算年化,得出"市场一年只值 18,682",扫了160天才发现真值是18,682",扫了160天才发现真值是721,542 —— 低估 39 倍。

病根一样:这类数据的分布是偏的,短窗口测到的是"最近碰巧发生了什么", \ 不是"通常是什么"。

可迁移的判据:任何"年化"数字,先问它是从多长的样本外推的。 \

4 天外推一年 = 放大 91 倍,任何噪音都会被放大 91 倍。

怎么防

脚本里加了两个硬指标,单看年化不够:

同号占比 全部样本里,资金费保持同一方向的比例

两半对比 把历史劈成近一半 / 远一半,分别算年化

如果两半符号相反,直接出局 —— 说明这个币的资金费在换方向

第 6 课:筛选结果

```
python3 funding_probe.py screen --top 24
```

判据:同号 ≥ 70%、两半不反号、30 天内回本。

=== 通过长窗口筛选的 ===

LIT_USDC_PERP 年化 +19.28% 同号 86% 224 天样本 回本 3.8 天

LINK_USDC_PERP 年化 +15.78% 同号 92% 380 天样本 回本 4.6 天
BILL_USDC_PERP 年化 +13.62% 同号 81% 88 天样本 回本 5.4 天
ASTER_USDC_PERP 年化 +13.08% 同号 90% 324 天样本 回本 5.6 天
UNI_USDC_PERP 年化 +9.03% 同号 87% 358 天样本 回本 8.1 天
ETH_USDC_PERP 年化 +7.01% 同号 72% 384 天样本 回本 10.4 天
PAXG_USDC_PERP 年化 +6.30% 同号 85% 281 天样本 回本 11.6 天
XLM_USDC_PERP 年化 +4.43% 同号 79% 356 天样本 回本 16.5 天

8/24 通过

LINK 是里面最扎实的一个:380 天样本、92% 同号、两半都为正、年化 15.78%。

对比一下被筛掉的:

BERA 年化 -99.96% 但同号只有 38% ← 数字大但方向乱,不可用

KMNO 年化 -36.09% 同号 35% ← 同上

年化最大的往往是最不稳的。这和我们在清算上看到的一样:\n中位 0.03、均值0.03、均值81 —— 极端值决定均值,但极端值不可重复。

第 7 课:它不是套利,是 carry

这个定性必须说清楚,因为它改变了"成功"的定义。

套利:两个场子价格不一致,你同时买卖,锁定差价。风险来自执行。

carry:你持有一个头寸,因为承担了某种风险而被支付。风险来自持有。

资金费 7~15% 的年化,是在为下面这些风险付你钱:

| 风险 | 说明 |

| :----- | :----- |

| 交易所对手方 | 钱放在 CEX 上。FTX 的教训 |

| ADL / 爆仓 | 极端行情下被强制减仓,对冲断腿 |

| 稳定币脱锚 | 整个 U 本位假设失效 |

| 资金费反向 | 即使 92% 同号,也有 8% 的时段你在付钱 |

| 单腿 | 入场、出场各一次暴露窗口 |

所以不存在"这个 edge 存不存在"的问题 —— 钱明摆着在付。\\

真正的问题是:付给我的,够不够抵这些风险?

这是风险偏好问题,不是研究问题。研究到此为止,决策归你。

还有一个全新的东西:托管风险。\\

前四周我们做的一切要么纯只读,要么押 0.1 ETH(\$192)。\\

这次是真金白银放在别人服务器上,是个我们没碰过的风险类别。

第 8 课:Backpack 的 100ms 减速带

调研文档里挖到的,中文圈几乎没人提,但它直接影响你怎么下单:

Backpack 对所有 taker 订单(现货 + 期货)统一施加 100ms 减速带。\\

只有带 postOnly 的订单免疫。撤单不受影响。

三条后果:

① 吃单入场的成本 > 30 bps —— 还要加上那 100ms 里盘口的漂移

② 如果 Backpack 这条腿是 taker,它必然比另一条腿晚 100ms

→ 单腿暴露从"网络抖动的随机量"变成"至少 100ms 的确定量"

③ 唯一解法是 postOnly(纯挂单),但那可能不成交 → 又回到单腿问题

这是明确的取舍,没有免费午餐。

有一个至今没有答案的问题(调研文档里的 Q7):\\

Taoli Tools 的 VA/VB 限价单,下的是普通限价单还是 postOnly?\\

如果是普通限价,价格穿过盘口时照样会吃到那 100ms。\\

这条在真下单之前必须确认。

第 9 课:还有一项没测,所以不出总数

按我们自己在成本模型 (https://vscode-remote+wsl-002bubuntu-002d22-002e04.vscode-resource.vscode-cdn.net/home/starkxun/MEV-LIFI/docs/week_2/%E6%88%90%E6%9C%AC%E6%A8%A1%E5%9E%8B%E6%95%99%E5%AD%A6.md)里定的规矩:\\

主项没测出来,就拒绝打印合计。

手续费地板 20 bps(maker)/ 30 bps(taker) ← 已测,官方文档
资金费收入 长窗口筛选后 7~19% 年化 ← 已测,380 天样本
单腿发生率 待定 ← 没测
减速带滑点 待定 ← 需要采样盘口

单腿发生率是唯一能杀死这个策略的未知数。\
如果它是 50 bps 量级,7% 的年化直接被吃光。
而它只能靠小额真跑测出来 —— 没有公开数据,链上也查不到。\
这就是为什么下一课的第 4 步必须真下单。

第 10 课:★ 我该从哪开始

五步,前三步零成本零风险,做完再决定要不要花钱。

第 1 步:自己跑一遍筛选(今天,10 分钟,零成本)

```
python3 funding_probe.py floor  
python3 funding_probe.py screen --top 40 --out shadow/funding_screen.json
```

目的不是找币,是让这几个数字变成你自己的,而不是我告诉你的。\
特别注意看:年化最大的那几个,同号占比是多少。

第 2 步:亲手复现那个陷阱(今天,5 分钟,零成本)

```
python3 funding_probe.py rates --limit 100 # 短窗口,好看  
python3 funding_probe.py history SKR_USDC_PERP # 长窗口,反号
```

亲眼看一次 +70.60% 变成 33.44%。\
这个体感比读十遍"要注意样本长度"有用。

第 3 步:挂服务器跑两周,验证筛选稳不稳(零成本)

这一步不能跳。 我们已经在两个方向上栽在"用一次快照当结论"了。

挂上去

前提:先进入项目目录。tmux 会继承你启动它时的工作目录,\
下面的命令用的是相对路径,所以 pwd 必须是 /root/MEV-LIFI。

```
cd /root/MEV-LIFI  
pwd # 确认输出 /root/MEV-LIFI 再往下  
mkdir -p logs shadow  
tmux new -s funding -d '\nwhile true; do  
T0=$(date +%s); D=$(date +%F)  
echo "[$(date -u +%FT%TZ)] start" >> logs/funding_$D.log  
python3 funding_probe.py screen --top 40 --show 40 \n--out shadow/funding_$D.json >> logs/funding_$D.log 2>&1  
echo "[$(date -u +%FT%TZ)] done rc=$? 用时 $(( $(date +%s) - T0 ))s" >> logs/funding_$D.log  
S=$(( 86400 - ($(date +%s) - T0 )); [ $S -lt 60 ] && S=60  
sleep $S  
done'
```

两个容易踩的地方:

启动前必须在项目目录里 —— 命令用的是相对路径 logs/ 和 shadow/,\
tmux 继承启动时的目录。如果从别处启动,文件会写到错地方,而且不报错 ——\
你要等到跑 compare 找不到快照才发现。所以上面先 pwd 确认一次。
不是简单的 sleep 86400:screen --top 40 实测要跑 ~23 分钟\
—— screen --top 40 实测要跑 ~23 分钟(每个币要分页拉满一年历史)。\
直接睡 86400 秒的话,每天执行时间往后漂 23 分钟,两周漂 5 个多小时。\
上面这个写法从本轮开始时刻算起扣掉耗时再睡,执行时间就锚住了。

日常查看

```
tmux ls # 确认会话还在  
tmux attach -t funding # 进去看(Ctrl-B 然后 D 退出,别 Ctrl-C)  
tail -f logs/funding_$(date +%F).log # 看当天日志  
ls -la shadow/funding_2.json # 看攒了几个快照
```

停掉

做跨所 carry 钱要躺在两个所里,KYC 真正的意义是「能把钱拿回来」。

cross 和 hours 是后加的,起因是 taoli.tools 排名页上\

同一个币在不同所的费率差 —— 那才是排名页上真正有意思的结构,\

而 1Y 那一列是单点外推(实测 COTI:下次 $+0.6898\% \times 1095 = +755.3\%$,完全对上)。

详见 永续与资金费-概念入门.md (https://vscode-remote+wsl-002bubuntu-002d22-002e04.vscode-resource.vscode-cdn.net/home/starkxun/MEV-LIFI/docs/week_3/%E6%B0%B8%E7%BB%AD%E4%B8%8E%E8%B5%84%E9%87%91%E8%B4%B9-%E6%A6%82%E5%BF%B5%E5%85%A5%E9%97%A8.md) 第 11、12 课。

附:这一课学到的方法

1. 任何"年化"先问样本多长 —— 4 天外推一年 = 放大 91 倍
2. 看方向稳定性,不只看幅度 —— 年化最大的往往最不稳
3. 把历史劈两半对比 —— 两半反号是最便宜的证伪手段
4. 有待定项就不出总数 —— 这条已经救过我们两次

笔记 058: RobinHood Lighter ↔ Lighter 双市场价差套利：可行性与复现路线

作者：QQQ | 时间：2026-08-17 06:49 UTC | 字数：8250 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

RobinHood Lighter ↔ Lighter 双市场价差套利：可行性与复现路线

19.1 项目定位：不是搬币，而是两个独立 Perp 市场之间做市场中性价差

项目：lihanyu81/RobinHoodLighter---Lighter-Arbitrage-Tool。

该策略利用 Robinhood Chain 上的 RBLighter 与原生 Lighter 两套独立永续合约订单簿之间的短暂价格差。两边拥有独立的 Order Book、账户、API Key、Market Index、仓位和流动性，因此同一个 BTC/ETH Perp 在短时间内出现不同的 Bid/Ask 是合理的。

典型结构：

RBLighter BTC Ask = 100,000

Lighter BTC Bid = 100,100

→ RBLighter 做多 BTC

→ Lighter 做空 BTC

→ 两边 Token 数量匹配

→ 等待价差收敛后平仓

策略不主要押注 BTC 的绝对涨跌，而是交易两个 Venue 之间的 Spread/Basis 收敛。因此它属于 Cross-Venue Market-Neutral Arbitrage。

19.2 真正应该计算的是 Executable Edge，而不是页面价格差

最简单但错误的做法是直接比较两边 Last Price 或 Mid Price。真正可执行的两个方向应分别计算：

Direction A：BUY RBLighter + SELL Lighter

$Edge_A = (Lighter\ Bid - RBLighter\ Ask) / RBLighter\ Ask$

Direction B：BUY Lighter + SELL RBLighter

$Edge_B = (RBLighter\ Bid - Lighter\ Ask) / Lighter\ Ask$

例如：

RBLighter Bid/Ask = 99,990 / 100,000

Lighter Bid/Ask = 100,100 / 100,110

可执行方向为 BUY RBLighter @100,000 + SELL Lighter @100,100，Top-of-book Edge 约 10 bps。

但 Top-of-book 仍不等于最终利润。生产系统应进一步计算：

$Net\ Executable\ Edge = Executable\ Edge - Fee - Slippage - Funding - Latency\ Loss - Execution\ Risk\ Buffer。$

这与前面 LI.FI 跨链研究得到的结论完全一致：Displayed Price < Executable Quote < Real Execution。看到价差只是 Candidate Signal。

19.3 为什么这种“蚂蚁搬家”在数学上可能成立

作者称一次实盘中约 200U 本金盈利约 2U。这个数字等于约 1% 的阶段收益，但不能直接解释为“每天稳定 1%”。

如果策略每次只捕捉很小的价差，例如 5–10 bps：

$20U \times 5 \text{ bps} \approx 0.01U$;

$20U \times 10 \text{ bps} \approx 0.02U$ 。

如果一天能够完成约 100 次真正有净 Edge 的小额交易，累计收益数量级可以接近 1-2U。因此作者数字在数量级上并不离谱。真正需要验证的不是算术，而是：一天到底有多少次 5-10 bps 以上、持续时间足够、深度足够、两腿可以真实成交的机会。

19.4 Lighter 手续费结构为什么对小价差策略重要

Lighter 的账户类型和费率/延迟结构直接决定最小可交易 Edge。若账户具有极低或零交易手续费，小到数个 bps 的价差仍可能具有研究价值；若账户需要支付较高 Taker Fee，则必须把双边开仓、平仓成本全部加入阈值。

因此系统不能写死“Spread > 5 bps 就交易”，而应该根据实际账户费率动态计算 Minimum Executable Edge。

19.5 最大技术问题之一：Latency

套利的竞争点不是“能不能看到价差”，而是从看到价差到两腿真正成交时，价差是否仍然存在。

完整延迟包括：

WebSocket 数据到达 → 本地计算 → 订单构造/签名 → 网络 RTT → 平台处理延迟 → Matching Engine → Fill 回报。

例如 TO 时看到 +10 bps，几百毫秒后可能只剩 +1 bp，甚至变成负值。因此需要统计 Opportunity Lifetime，而不能只统计出现次数。

作者建议使用日本 VPS，本质上是在降低与交易基础设施之间的网络延迟和抖动。服务器位置不是盈利来源，但它可能决定短寿命 Edge 是否还能成交。

19.6 双腿成交是核心风险：套利可能瞬间变成方向仓位

计划：

RBLighter BUY 1 ETH

Lighter SELL 1 ETH

如果 BUY 成交而 SELL 没成交，账户会留下裸 Long ETH；反之则留下裸 Short。此时策略已经不再是市场中性套利。

因此作者程序使用 IOC (Immediate-or-Cancel) 类型的执行思路，并在两腿发送后进行成交对账。

IOC 的含义是：当前能在限定价格内成交的数量立即成交，剩余部分取消。例如 BUY 1 BTC、最高接受 100,010；盘口只有 0.7 BTC 位于这个价格以内，则成交 0.7 BTC，剩余 0.3 BTC 取消。

随后系统必须检查：

$RBLighter \text{ Filled Qty} == \text{Lighter Filled Qty} ?$

若不相等，差值就是裸露 Delta，需要立即补偿、Reduce-only 平仓或进入暂停状态。

19.7 一个可实盘的执行器至少需要的模块

Market Data Layer：同时连接 RBLighter 与 Lighter WebSocket，维护实时 BBO/L2 Order Book。

Normalization Layer：统一 Symbol、Market Index、价格精度、数量精度和时间戳。

Opportunity Engine：计算两个方向的 Executable Edge，并维护滑动统计窗口。

Depth/VWAP Engine：按指定资金规模穿透订单簿，计算真实平均成交价。

Execution Engine：两腿并行 IOC/Aggressive Limit，控制最大可接受价格和最大滑点。

Reconciliation Engine：检查订单状态、Filled Qty、Average Fill Price 和最终 Position。

Compensation Engine：部分成交、单腿成交、超时或状态不确定时快速处理裸露仓位。

Risk Engine：控制 Max Position、Max Delta、Max Slippage、Funding、Margin、Liquidation Distance、Daily Loss、Kill Switch。

Persistence/Recovery：保存订单、成交、仓位和状态机，程序重启后能够恢复，而不是重复下单。

19.8 为什么必须从 BBO 升级到 VWAP

例如页面显示：

RBLighter Ask = 100,000

Lighter Bid = 100,100

表面 Edge = +10 bps。

但最优价可能只有极小数量。交易 20U 时能够获得接近 10 bps；交易 2,000U 时需要吃多档订单，实际 BUY VWAP 可能升至 100,070，SELL VWAP 降至 100,040，原本的正 Edge 已经消失。

因此 Scanner 应同时测试多个资金档，例如：20 / 50 / 100 / 200 / 500 / 1,000 USDT，并输出每档的 BUY VWAP、SELL VWAP、Gross Edge、Net Edge 和 Executable Capacity。

这也解释了为什么“200U 能跑”并不代表“20,000U 可以按相同比例放大”。套利容量由订单簿深度决定。

19.9 Funding 必须进入模型

因为两边都是 Perpetual，持仓期间还存在 Funding。短时间持仓时 Funding 影响可能较小，但如果 Spread 长时间不收敛，Funding Difference 会逐渐成为重要成本或收入。

应记录：

RBLighter Funding Rate；

Lighter Funding Rate；

Next Funding Time；

预计持仓时间；

Funding Differential。

净收益模型应加入 Expected Funding Cost，而不是只看开仓 Spread。

19.10 最关键的第一阶段验证：只读 Arbitrage Scanner

不建议一开始运行作者闭源程序或投入资金。第一版完全不需要交易 API 私钥，只连接公开行情接口，连续运行 24–72 小时。

Scanner 记录：

timestamp、symbol、RH_bid、RH_ask、Lighter_bid、Lighter_ask、各档 size、L2 depth、funding、数据延迟。

实时计算两个方向：

$\text{Edge_A} = \text{LighterBid} / \text{RHAsk} - 1$ ；

$\text{Edge_B} = \text{RHBid} / \text{LighterAsk} - 1$ 。

并统计：

\>2 bps、>5 bps、>10 bps、>20 bps 的事件次数；

每个事件持续时间；

Peak Edge；

Median Edge；

不同资金规模下的 VWAP Edge；

可执行容量。

19.11 Opportunity Lifetime：判断普通服务器到底能不能吃到

必须把连续价差识别成一个 Opportunity Event，而不是每个 Tick 都算一次新机会。

每个事件记录：Opportunity ID、Start Time、Peak Edge、End Time、Duration。

如果结果显示 >10 bps 的价差很多，但中位寿命只有 50–100ms，那么普通 API 用户可能几乎无法成交；如果 5–15 bps 的 Edge 经常持续 1–5 秒，则策略的现实可执行性显著提高。

因此“价差频率 × 价差寿命 × 可执行容量”比单纯的平均 Spread 更重要。

19.12 复现作者“200U → 2U”应该怎样验证

不要直接问“能不能一天赚 1%”，而应把作者结果拆成可证伪的统计问题：

① 200U 资金规模下，每天出现多少个 Net Edge > 0 的事件？

② >5 bps、>10 bps 的事件分别多少？

③ 这些事件的中位持续时间是多久？

④ 200U 的订单是否能在两边以目标 VWAP 成交？

⑤ 加入假设执行延迟后 Edge 还剩多少？

⑥ Funding 和实际费率后 Net PnL 是多少？

⑦ 如果模拟 IOC，两腿 Fill Ratio 是多少？

⑧ 单腿失败/部分成交造成的 Expected Loss 是多少？

最终应计算：

$\text{Expected Daily PnL} = \sum \text{Successful Arbitrage PnL} - \text{Slippage} - \text{Fees} - \text{Funding} - \text{Failed Hedge Loss} - \text{Abort Cost}$ 。

只有这组结果长期接近作者描述，才能说“博主内容可复现”。

19.13 推荐四阶段技术路线

Phase 1：Market Observer / Scanner。

只读 WebSocket，抓取 24–72 小时订单簿，不签名、不交易。目标：证明价差是否真实存在，统计频率、寿命、深度和容量。

Phase 2：Arbitrage Simulator。

保存 Order Book Replay, 加入 50/100/200/300ms
等延迟模型、VWAP、Slippage、Fee、Funding, 模拟不同阈值和不同资金规模下的理论 Fill/PnL。

Phase 3 : Paper Trading。

在实时行情上运行完整状态机, 但不提交真钱订单。模拟 IOC、Partial Fill、双腿成交、Abort、PnL, 并统计 Fill Rate、Hedge Success Rate、Average Leg Exposure Time。

Phase 4 : Small Live。

只有前三阶段证明长期 Expected PnL 为正后, 才考虑 20-50U 等可承受损失的小额真实测试, 用真实 Fill 数据校准模拟器, 再决定是否扩大。

19.14 推荐的系统架构

RBLighter WebSocket + Lighter WebSocket

- Order Book Manager
- Market Normalizer
- VWAP / Depth Engine
- Two-way Spread Engine
- Opportunity Event Detector
- Fee/Funding/Latency Cost Model
- Net Executable Edge
- Risk Engine
- Paper Execution / IOC Simulator
- Reconciliation
- PnL Database
- Threshold Calibration。

进入实盘后再增加: Signer、Order Gateway、Position Reconciliation、Compensation Engine、Kill Switch 和 Restart Recovery。

19.15 作者闭源 Binary 的安全问题

公开仓库主要提供编译后的程序而非完整可审计源码, 而真实交易需要配置 Lighter 与 RBLighter 的 API Private Key。SHA-256 校验只能证明下载文件与作者发布文件一致, 不能证明程序内部不存在恶意或高风险逻辑。

因此安全原则应是: 验证策略阶段不提供私钥; 优先自己实现只读

Scanner; 若未来使用第三方案程序, 只使用独立小额账户、最小权限 API、限制资金和提款权限, 并做好服务器隔离和日志审计。

作者程序还包含推荐/邀请关系验证门禁, 这属于软件运行条件, 不是套利机制本身。自己实现 Scanner 和执行器可以把策略研究与作者商业门禁解耦。

19.16 当前可行性评级

两套独立市场与订单簿: 高可行。

公开行情与 WebSocket 数据采集: 高可行。

实时双向 Spread 计算: 高可行。

VWAP/Depth Scanner: 高可行。

自动双腿 IOC: 技术可实现, 但工程复杂度中等。

200U → 2U 的数量级: 数学上合理, 但未独立验证。

每天稳定 1%: 没有证据支持。

机会频率: 待实测。

Opportunity Lifetime: 待实测, 是关键指标。

200U 可执行深度: 待实测。

双腿真实 Fill Rate: 待实测。

Funding 影响: 待实测。

延迟竞争: 重要风险。

第三方 Binary 安全性: 无源码情况下不可完整审计。

自行复现 Scanner: 高可行。

自行复现生产级交易机器人: 中等偏高工程难度。

19.17 与前面 TUT Basis 套利的关系

两者底层思想相同：都不是预测币价，而是锁定两个独立市场之间的相对价格差。

TUT 案例属于极端异常 Basis, Spread 可以达到 10%、20%
甚至更高，因此执行速度不是唯一矛盾，重点是强平、Mark/Index、深度和 Hedge。

RBLighter ↔ Lighter 更像高频微小 Spread，可能只有几个到几十个 bps，因此 Fee、Latency、Order Book Depth 和 Fill Probability 更重要。

可以统一成：

Market A Order Book + Market B Order Book

→ Same-Qty VWAP

→ Gross Spread

→ Cost/Risk Model

→ Net Executable Spread

→ Two-leg Execution

→ Reconciliation

→ Convergence Exit。

19.18 最值得继续做的实验

第一优先级不是安装闭源 Bot，而是自己实现 RBLighter ↔ Lighter Read-only Scanner。

第一轮建议只选 BTC、ETH 和其他两边共同存在且流动性较好的 Perp，连续采集至少 24–72 小时。输出一份报告：

Spread Distribution；

Opportunity Count；

Opportunity Lifetime Distribution；

20/50/100/200/500/1000U VWAP Edge；

Executable Capacity；

Funding Differential；

假设 50/100/200/300ms 延迟后的 Edge Decay；

模拟交易次数；

Expected PnL。

如果数据证明 200U 规模下每天存在足够多持续时间可执行的 5–10 bps 净机会，再进入 Paper Trading；如果大部分正 Edge 在 100ms 内消失，或者 200U 深度已经吃掉利润，则应直接判定普通用户难以复现，不必承担真钱风险。

19.19 本节核心结论

RobinHood Lighter ↔ Lighter 套利不是明显不成立的“宣传故事”。两个独立 Perp Venue
之间产生短暂价格差的机制合理，公开行情接口也使核心假设可以独立验证；作者描述的 200U 赚约 2U 在数量级上存在可能。

但真正决定可复现性的不是截图里的收益率，而是四个变量：Net Executable Edge、Opportunity Lifetime、Executable
Depth、Two-leg Fill Probability。

因此正确研究顺序应该是：

Observe → Measure → Replay → Paper Trade → Small Live，

而不是：看到 200U→2U → 下载闭源程序 → 直接放入 API 私钥和真钱。

最终目标是把这个案例从“博主收益展示”转化为一个可以被数据证伪或证实的 Execution-aware Cross-Venue Arbitrage 实验。

资料：GitHub 项目 lihanyu81/RobinHoodLighter---Lighter-Arbitrage-Tool；Lighter API / WebSocket / Trading Fee
相关公开文档；Robinhood Chain 相关公开资料。

笔记 059: 2026-08-17 學習打卡

作者：cht+ | 时间：2026-08-17 09:03 UTC | 字数：838 | 评分：48

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-17 學習打卡

今天完成了什麼

完成跨市場套利監控器的狀態機修正：將新機會掃描與既有事件追蹤解耦，避免已收斂的舊事件阻擋同一商品後續的新套利窗口。加入價差收斂後重新武裝機制，透過低價差持續門檻形成遲滯，降低永久去重與門檻附近通知震盪的風險。

補強事件證據保存：保留觸發前一分鐘的雙腿報價，並在觸發後持續記錄十五分鐘，以利日後還原報價新鮮度、跨市場時間偏差與資料品質閘門。

釐清商品能力標示：區分官方已有交易能力文件、帳戶與商品資格尚未驗證，以及本地監控器仍不具自動下單能力。

完成測試、靜態檢查、程式審查與部署驗收；正式監控服務持續正常掃描，未新增任何實盤交易權限。

證據在哪裡

完整自動測試、程式規範檢查、格式檢查與差異檢查皆通過。

新增的回歸測試已覆蓋掃描與追蹤解耦、低價差重新武裝、事件前後證據保存及商品能力邊界。

部署後監控服務維持正常運作，連續掃描期間未觀察到執行錯誤；另一套獨立監控服務未受本次部署影響。

正式環境目前沒有自然出現新的合格高價差事件，因此新根通知行為已有自動測試證據，但尚無今日正式行情事件可供實地驗證。

尚未完成／下一步

持續等待自然高價差事件，驗證重新武裝後確實建立新的獨立機會通知，並檢查事件前後證據是否足以完整還原行情。

在任何執行能力評估前，仍需另外驗證特定帳戶、地區、商品資格、股票腿真實可成交深度與數量單位。

研究如何在不放寬報價新鮮度與跨市場時間偏差安全門檻的前提下，降低資料品質拒絕造成的有效訊號流失。

今日學習心得

套利監控不只是在算價差，更重要的是把「新機會偵測」與「舊事件覆盤」的生命週期拆開。去重若沒有明確的重置條件，會從防止噪音變成永久漏報；而沒有觸發前後的同步行情證據，就無法分辨漏報、資料過期或時間錯位。這次再次確認，安全門檻不應為了增加通知量而放寬，應先改善狀態機與可觀測性，讓每個判斷都能被驗證與重現。

笔记 060: 链上套利残酷共学 · Day 12 心得

作者：kako | 时间：2026-08-17 09:53 UTC | 字数：638 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 12 心得

今日主题：信号工程

前两天的调研都是在“一次性观察”的层面——发现一个机会、记录、存档就结束了。今天的任务是把这个过程升级成“可重复运行的信号检查”，让机会发现从手动变成自动。

把之前的候选机会转换成可重复查询时，核心要解决的是“查什么”和“怎么查”两个问题。查什么决定了信号的价值——比如我设定的是监控 ETH/Arbitrum 之间的稳定币价差是否超过阈值 0.5%，并同时检查目标链的 Gas 费用是否低于预设上限。怎么查决定了信号的可靠性——直接用 LI.FI (<https://li.fi/>) 的报价接口定时拉取，而不是依赖手工刷新页面。

设置好之后让它实际跑了一次，输出了一份包含价差、滑点预估、两端 Gas 和时间戳的结构化报告。看到数据自动刷新出来的那一刻，感觉整个前期的学习积累开始形成闭环：从看报价到拆成本，从手动录数据到脚本自动算净收益，现在再到自动监控价差触发条件，一步步把“人盯盘”变成了“程序盯盘”。

今天的自查问题很有挑战性：信号检查会不会报“假阳性”？怎么降低？假阳性最常发生在价差瞬间跳变又迅速消失的情况——报价接口返回了一个大于阈值的数据，但实际执行时这个价差已经不存在了。降低方法是引入持续性确认机制：比如连续两次查询都超过阈值才触发告警，而不是第一次超过就报。另一个简单有效的做法是剔除流动性极低的路径，因为这类路径上的报价波动异常剧烈，最容易被误判为“机会”。

笔记 061: Day 12：两次 Quote 只差 2 秒，但回程条件在预计到达时已过去 1,078 秒。

作者：daohuang1982-beep | 时间：2026-08-17 10:21 UTC | 字数：661 | 评分：59

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12：两次 Quote 只差 2 秒，但回程条件在预计到达时已过去 1,078 秒。

今天没有刷新市场报价，而是把 Day 10-11 已保存的 LI.FI 证据整理成“候选机会时间门槛 V0”。LI.FI 的 executionDuration 是预计执行秒数，不是到账承诺；transactionRequest 也只是可供签名的交易请求，不代表已经结算。

用 Day 10 样本填写：第一腿 Quote 采集于 12:24:57，预计执行 1,080 秒，推算到达时间为 12:42:57；回程 Quote 采集于 12:24:59，所以到预计到达时已经过去 1,078 秒。这个数字只是时间差，没有官方有效期或到达后重新报价证据，因此届时能否获得同等回程条件仍为 unknown。

同时，Day 10 的闭环最低输出只有 995.00625 USDC，相对 1,000 USDC 已减少 4.99375 USDC，两腿 Gas 估算另有 0.1042 USD。经济门槛已经失败，所以结果应为 economicGate=fail、timingGate=blocked_by_economics、candidateState=quote_rejected

。正确顺序是先验证闭环最低净收益，再评估时间风险；更快不能挽救负收益候选。

今天没有请求新 Quote，没有保存或生成新的可执行 calldata，也没有连接钱包、授权、签名、广播或成交；当前没有发现或确认任何可执行套利机会。

笔记 062: 2026-08-17 | 共学 Day 13 | 深度案例研究2：资金费率套利 | 投入约 1.

作者：wuyuandao | 时间：2026-08-17 10:43 UTC | 字数：740 | 评分：75
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-17 | 共学 Day 13 | 深度案例研究2：资金费率套利 | 投入约 1.5 小时

今天学了什么：

研究了第二个套利案例——资金费率套利（期现对冲）。原理：买入现货 + 做空等量永续合约，价格涨跌完全对冲，每 8 小时坐收资金费率。查了 Binance 全市场 874 个永续合约的费率，从年化 -1592% 到 +514%。

得到什么结果：

1. 完整成本计算：BTC 资金费率套利，理论年化 4.93%，扣 0.3% 开平仓成本后真实年化 4.63%。短期持有（7天）因成本未摊薄反而净亏 -10.7%。

2. 发现第三个坑：费率最极端的 BSP (+514%) 和 BNC (-1592%) 是 TRADIFI 纯合约币，没有现货市场，根本无法做「买现货+做空永续」的对冲套利——高收益是「不可执行的幻觉」。

3. 纠正了自己的错误结论：一开始误以为「只有 BTC/ETH 能做」，重新交叉对比「有现货+正费率」后发现 266 个币可执行，甜区年化 10-11% (XLM、MATIC、ETC、LDO、AR、DYDX 等一大批主流/次主流币)，最高可执行是 TST 年化 32%。

核心认知：

套利的正确筛选逻辑不是「抓极端费率」，而是「排除不可执行项，在甜区里选」。所有「看起来能暴富」的信号都藏着三个致命问题之一：假数据、扣成本后亏损、无法执行。真正能做的是排除不可执行项后，在年化 10% 左右的甜区持续监控。

困难：

资金费率历史数据路径 (data.binance.vision) 404，只能用实时 API。费率是动态的，今天的数据不代表明天。

下一步：

Day 14 第二周总结——Agent 能力盘点 + 策略候选清单 + 工具链回顾。

笔记 063: Day 13 | 观测缓冲重算净收益门槛

作者：chrise | 时间：2026-08-17 11:04 UTC | 字数：647 | 评分：73
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 13 | 观测缓冲重算净收益门槛

今日研究

把昨日三窗口数据接回成本模型：每个规模档采用最差单位报价降幅，并用小额参考价格的最大漂移速度乘以该档最长响应时间，形成暂定报价陈旧度缓冲。

套利逻辑

比较时沿用 35 bps 假设毛边际及相同的协议费、失败期望损失、资金成本和固定费用，只替换滑点与延迟预算。这样可以观察 29,166 USD 上界对平方滑点假设的敏感程度。

数据与结果

小额参考价格的最大短时漂移速度为 0.0215 bps/秒。平方压力模型在五个已采样档位中仅有 10,000 USDC 通过双重利润门槛；暂定观测缓冲模型通过 10,000、30,000、50,000 USDC。30,000 USDC 的门槛余量分别为 -0.9479 与 +40.5287 USD，50,000 USDC 分别为 -44.5132 与 +74.1644 USD。

成本与风险

观测缓冲使用相对 1,000 USDC 档的规模变化，未计入小额档自身的基础价格冲击。数据只覆盖约 103 秒，存在路由切换；同链报价也不能代表完整跨链执行成本。

今日结论

原 29,166 USD 上界主要由平方滑点压力假设决定。当前重算说明模型结论对滑点曲线高度敏感，尚不能证明 50,000 USDC 真实可盈利，也不能建立新的规模上限。

下一步验证

把同链交换成本与跨链再平衡成本拆成两套预算，避免桥费、双链 gas 与同链报价发生口径混用。

笔记 064: 链上套利共学 · Day13 打卡笔记 (08/17)

作者: haibian-slowly | 时间: 2026-08-17 11:29 UTC | 字数: 881 | 评分: 77

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 · Day13 打卡笔记 (08/17)

今日主题

还原一个公开套利案例：从发现信号，到验证成本，再到决定是否执行。

今天学到的东西

今天把前几天学到的内容串起来看了一遍。一个看起来很诱人的价差，至少要经过下面几道检查：

1. 价差是不是真的：确认两个市场的资产、链和计价单位一致，避免把价格方向看反。
2. 能不能走通：确认兑换路径、桥、授权对象和预计到账时间，不把“屏幕上的价格”当成已经到手的钱。
3. 扣掉全部成本：Gas、协议费、桥费、滑点、失败交易成本和资金占用时间都要算进去。
4. 有没有安全垫：如果报价在等待期间变差，或者桥的执行时间变长，剩余利润是否还能覆盖损失。
5. 先模拟，后执行：没有足够证据时，只做报价采集和 Paper Trading，不签名、不转账。

用一个简化例子复盘

假设看到一笔 1000 USDC 的跨链机会，屏幕毛价差为 0.8%。如果已知跨链和兑换成本约为 0.32%，表面上还剩约 0.48%。

但这 0.48% 还不是利润，还要继续问：

报价是否会在执行前失效？

路径是 7 秒，还是可能变成 18 分钟？

失败时要损失多少 Gas？

到账后的资产能不能按预期价格卖出？

这个机会能否重复出现，还是只是偶然插针？

因此，今天的判断是：先放进观察名单，不进入真实执行。真正值得做的不是毛价差最大的机会，而是扣完全部成本、风险可控、执行链条能够闭合的机会。

今日结论

套利案例的复盘重点，不是记住某一笔赚了多少钱，而是还原它为什么成立、哪些条件必须同时满足，以及今天还能不能重复。

价差只是线索，成本核算决定利润，执行闭环决定机会是否真实。

明天计划

把今天的检查清单整理成一个简单的候选机会表，字段包括：链、资产、输入金额、毛价差、路径、预计时间、手续费、Gas、滑点、失败成本、净收益和最终决定。

说明：今天只做研究和模拟，没有签名、转账或真实资金操作。

笔记 065: Day 13 | MEV 生态分层、原子交易与对冲

作者: fangliaofan | 时间: 2026-08-17 11:41 UTC | 字数: 543 | 评分: 60

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 13 | MEV 生态分层、原子交易与对冲

今日学习

今天重点把 MEV 供应链的分层结构理顺：Searcher 负责发现并构造套利/清算交易，通过 bundle 提交给 Builder；Builder 竞标打包区块，Relayer 再把区块传递给 Validator。三者构成分工明确的管道，谁离“排序权”越近，谁分到的利润越高——这也是 PBS（提议者-建造者分离）重塑 MEV 格局的核心。

原子交易是理解闪电贷的钥匙：一笔交易要么全部执行成功，要么整体回滚，不存在中间状态。闪电贷正是借助“借→套利→还”在同一个原子交易里完成，任一环节失败则整笔回滚，放贷方零风险，套利者只损失 gas。

关键认知

1. MEV 的价值分配遵循“入口决定论”：Searcher 提供信号、Builder 提供打包能力、Validator 提供区块权，三者缺一不可。
2. 对冲不是消除风险，而是把“方向性风险”换成“价差/基差风险”，代价是更低的净收益。

下一步

实测一笔测试网闪电贷套利，观察原子交易的回滚行为与 gas 消耗。

用 Flashbots bundle 提交一次模拟 Searcher 交易，理解 bundle 落块逻辑。

笔记 066: Day 13 (08-17) : 90 篇增量笔记批次消化 + 执行层两连

作者: Paxon Qiao 乔鹏军 | 时间: 2026-08-17 12:19 UTC | 字数: 1881 | 评分: 90

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 13 (08-17) : 90 篇增量笔记批次消化 + 执行层两连

主线: 残酷共学增量笔记 90 篇批次消化 (08-16 23:01 生成)

为何做: 用户发来最新增量批次 PDF (90 篇/179 页), 按「记录优先」SOP 归档后再消化; 本批同学笔记已进入执行纪律/成本模型阶段, 与我们的五阶段管线 (回测/纸面) 同频。

怎么做: pymupdf 提取全文 (21.8 万字符) → 脚本定位 9 章节行号区间 → 3 子代理并行逐篇精读 (每篇带具体数字) → 人工抽读 4 篇交叉校验 → 合成 digest。

结果: 归档 PDF+文本进 sources/; digest 五强信号——044 状态跳变=第10类机会 (ERC-4626 accrue 重估 +0.0031%×2772万闪电贷=+\$869.65; v3 腿 x-y=k 高估 7.6% 必须换 Quoter)、023 LI.FI 路由每周翻转+AcrossV4 一夜接管 13/14 条+L2 native USDC deny list (code 1011)、075 Hyperliquid 250 天费率回测证伪「躺平收租」(ETH 5.0% vs SOL -1.2%, 翻转 70-90 次/30天)、079 HL 币股结构性基差 (CRWD 中位 52bps 为 CEX↔CEX 的 5-20 倍)、062/053 OI 过滤+ADL 剪刀仓风控。产出 notes/icl-incremental-notes-digest-20260816.md + README/links/daily 更新, commit 6e8f776。

执行层: Robinhood Chain FCFS 查证 (冷知识)

官方文档确认 Robinhood Chain 排序=先到先得 (FCFS), 无 priority auction, 付费不能插队→竞争形态从「拼 tip」变「拼延迟」; 互证 0xd7121208 研究线 (价格侧无信息差+排序侧无竞价=纯延迟竞赛); 未来变量=Arbitrum Timeboost (付费提前 0.5s) 若接入需重新评估。

广度: 闪赚对冲策略评估 (白云 issue #6)

OKX 闪赚+现货对冲 (现货买入吃 100%+ 年化活动收益+永续空对冲): 用户实测 RE 单币 8500U 5 天净 40U≈年化 34% (宣传 120% 的 1/3.5) → gross vs net 实证; 结论: 机制成立但资金占用高、收益事件性, 不进主攻方向。

产出: notes/icl-incremental-notes-digest-20260816.md、daily/2026-08-17.md

发布线: D11 公众号 + X thread (晚上)

公众号《Web3 套利第 11 天: 第一笔实弹交易 10 秒落地, 8 轮排查两座坑》已发布 (mp.weixin.qq.com/s/SAC6RNYi1XWL8Yxrd9MZmA): 合规自查正文 0 违规 → preflight 全过 → ship (media_id 100011704; cookie 会话过期重登一次) → 手机预览 → 用户发布; X thread 发布 9 条 (x.com/qiaopengjun/status/2089341248033501244, 草稿 2/8 拆两条, 浏览器验证全完整)。公众号 D1-D11、X thread D1-D11 全部发布完成。

执行层 (晚间追加)

- jito_arb_pipeline 链过滤修复: DexScreener 拉池加 chainId==solana 过滤 (原混入 Base/BSC/ETH 池, rx/ry 数量级不一致污染跨池判定); 修复后只留 3 个 Radium 深池、9 条路径全负 (主流池价差已磨平=正确输出), 管线达「可复跑+正利润自动接 Jupiter quote→swap→bundle」状态

- 归档李胜利套利软件安全实操手册: 一键套利=私钥即控制权/开源≠安全/GPT 七问拆解代码/签名类攻击 (Permit、setApprovalForAll 无 gas 授权) /剪贴板地址替换/独立助记词隔离

- 归档 Lighter×Robinhood Chain 价差评估: RH Chain 7/1 主网+Lighter 默认 perp DEX+1100 万 LIT 激励+积分 2x=新订单簿补贴期; 工具未开源按推广处理; Points 禁机器人风险

笔记 067: 2026-08-17 学习打卡

作者: UU20220213 | 时间: 2026-08-17 12:22 UTC | 字数: 1303 | 评分: 78

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-17 学习打卡

今日完成

1. 学习套利监控为什么要同时测试多个交易金额。交易金额增大后可能吃到更差的盘口档位, 滑点和手续费增加, 因此投入最多不等于利润最高。应同时比较绝对净利润与净收益率, 并设置双重门槛。
2. 完成交易金额选择题: 在净利润至少 10 USDT、净收益率至少 0.20% 的条件下, 2,000 USDT 方案利润不足, 8,000 USDT 方案收益率不足, 5,000 USDT 方案以 15 USDT 净利润和 0.30% 净收益率满足条件, 因此选择 5,000 USDT。
3. 学习报价新鲜度。套利必须比较同一时刻的可成交价格; 如果两家交易所的报价时间相差过大, 旧报价可能制造假价差。监控记录需要包含交易所时间、程序接收时间和报价年龄, 并过滤超过时限的数据。完成判断: A 报价 0.4 秒、B 报价 3 秒, 在 2 秒上下限下应丢弃。
4. 学习连续确认规则。机会只有连续多次同时满足净利润、净收益率、报价新鲜度和可执行数量要求后才提醒。若中间一次不达标, 连续计数应清零; 示例中第 3、4、5 次连续达标, 因此应在第 5 次后提醒。
5. 学习提醒去重与冷却时间。同一币种、同一买卖交易所、同一策略方向持续存在的价差, 应记录为一次机会及其持续时间、最高

和最低净利润，而不是把每秒数据都统计成独立机会。完成判断：连续 20 秒、20 条采样但没有中断，应统计为 1 次独立机会。

6. 学习合格提醒的字段：交易对、买卖方向、建议数量、两边加权平均成交价、毛利润、手续费、滑点、风险预留、保守净利润、净收益率、报价年龄、连续达标次数、持续时间和单腿处理方案。

7. 完成净利润判断：毛利润 30 USDT，扣除手续费 8、滑点 5、风险预留 4 后，预计净利润为 13 USDT；低于 15 USDT 门槛，不应提醒。

8. 理解风险预留不是交易所实际收取的费用，而是为价格变化、部分成交、接口延迟、模型偏差和单腿补救损失保留的安全垫。初期可设置保守比例，积累模拟或真实数据后再按历史偏差校准。完成计算： $20,000 \text{ USDT} \times 0.06\% = 12 \text{ USDT}$ 。

今日关键认识

监控器不是看到价差就提醒，而要先选择在盘口深度、成本和风险门槛下更合理的交易规模。

旧报价、短暂报价和重复采样会夸大机会数量，必须通过报价年龄、连续确认和提醒去重进行过滤。

保守净利润比毛价差更重要；风险预留的作用是避免把不确定性错误地当成利润。

学习证据

正确选择 5,000 USDT 作为同时满足净利润与净收益率门槛的方案。

正确判断过期报价应丢弃、连续 20 秒的相同价差只算 1 次机会。

正确完成 13 USDT 保守净利润和 12 USDT 风险预留计算，并能据此判断是否提醒。

未完成与下一步

下一步学习如何把一条提醒转化为 Paper Trading 记录，对比信号时的预计价格与数秒后的模拟成交价格，累计报价偏差、机会持续时间、成功率和单腿风险数据。继续只读研究，不自动下单，不使用真实资金验证。

笔记 068: day13

作者：lunateng | 时间：2026-08-17 13:08 UTC | 字数：736 | 评分：81

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

day13

二元期权的交易机制

在 Polymarket 上，针对任何二元事件（如“15分钟后 BTC 会涨到 \$90,077 以上吗？”），市场都会生成两种对立的代币：YES 和 NO。

固定赔付（结算规则）：

事件发生（判断正确）：代币最终按 \$1.00 结算兑现。

事件未发生（判断错误）：代币最终归零 \$0.00。

价格即概率：

期权代币在事件未结束前可以在订单簿上随意买卖。

代币的价格介于 \$0.00 到 \$1.00 之间。如果 YES 的价格是 \$0.42，代表市场目前认为该事件发生的概率是 42%。

理论锚定：由于结果非黑即白，理论上在任何时刻 $\text{price}(\text{yes}) + \text{price}(\text{no}) = 1$

短期价格二元期权是如何产生“定价错误”的？

像 5分钟 / 15分钟 BTC 涨跌这种超短期二元期权，市场常常出现瞬时低效：

交易所数据延迟：Polymarket 盘面上的普通散户或流动性提供者反应较慢，而现货交易所（如 Binance/Coinbase）的价格已经快速反弹。

情绪化恐慌抛售：当现货瞬间跌了 \$50，有人出于恐慌在 Polymarket 上以 35¢ 挂单砸盘 YES，但实际上现货离目标价依然非常近，结合剩余时间计算，YES 的真实获胜概率依然高达 42%。

套利空间（Arbitrage/EV+）：程序监控到 35¢ 与 42¢ 的偏差后，买入被低估的 YES，如果市场随后修正价格回到 42¢，即可提前卖出平仓套利，或持仓到期获取正期望收益（EV）。

笔记 069: Day12 跨所、跨期、展期套利

作者：tutu | 时间：2026-08-17 13:50 UTC | 字数：1013 | 评分：66

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day12 跨所、跨期、展期套利

今天把“屏幕上的价差”重新按可执行净收益计算了一遍。价差大只能说明市场被分割，不能直接当成利润。跨所赚的是不同交易场所之间的价格差，跨期赚的是近远月价差收敛，最后都要扣掉手续费、滑点、资金费、借币、转账和资金占用成本。

展期纸面案例

CL 近月 74.80，远月 75.50，定义 $S = \text{远月} - \text{近月}$ 。预期价差从 0.70 收窄到 0.30，做多近月、做空远月，1,000 桶毛收益为 $(0.70 - 0.30) \times 1,000 = 400$ USDT 等值。手续费和滑点合计 420 USDT，预期净亏损 20 USDT，直接放弃。0.28 只是覆盖成本的盈亏平衡点，没有给价差扩大和模型误差留下缓冲。

近月临近到期通常向现货靠拢，远月仍会受现货预期、融资成本、风险溢价和流动性影响。多头展期的卖近买远、空头展期的买近卖远会暂时改变价差，但不能当作必然方向。

跨所纸面案例

A 所 BTC 永续可执行买价 100,000，B 所可执行卖价 100,600，数量 0.049 BTC，预计平仓价差 100，全部开平成本 18 USDT。A 所做多、B 所做空，预期净收益为 $(600 - 100) \times 0.049 - 18 = 6.5$ USDT。

压力情景下价差扩大到 3,500，按盘口退出的净盈亏为 $(600 - 3,500) \times 0.049 - 18 = -160.1$ USDT，触发纸面账户 -160 USDT 红色预警。正常赚 6.5，压力亏 160.1，赔率约为 1:24.6，因此即使账面预期为正也应放弃。

跨所价差长期没人抹平，可能是盘口薄、滑点和费用过高、价差没有收敛驱动、提币或借币受限、合约指数和结算规则不同、两所保证金隔离，或者存在 ADL 与平台信用风险。提前在两所放资金能减少转账延迟，也会增加资金占用和平台风险。

今日入场规则

只有当扣除全部成本后的预期净收益有足够安全边际，收益风险比达到预设标准，压力情景下可执行净亏损控制在 -140 USDT 以内以给 -160 USDT

红线预留误差，价差有可验证的收敛驱动和持有期限，且双腿能同步成交和退出时，才允许入场。任何一项答不上来，直接放弃。

明日进入 Day13，画清打新或拍卖从参与、获配、上交易所到退出的路径，先找最终退出流动性。

笔记 070: Day 10 | 中場盤點：九天學到的東西，拼成一張可用的地圖

作者：HenryChang | 时间：2026-08-17 13:59 UTC | 字数：1361 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 10 | 中場盤點：九天學到的東西，拼成一張可用的地圖

今天是課程第 13 天（21 天過半），沒有新實作——就把這天用來做一件同樣重要的事：把前九篇筆記的碎片攤開重拼，看看哪些已經內化成可用的判斷力，哪些還是待辦。

一、成本模型：五塊拼圖都有實測數據了

看到任何價差，先過這五關（每關都親手量過）：

| 成本項 | 學於 | 實測錨點 |

| --- | --- | --- |

| 手續費 | Day 4/5 | 兩腿 60 bps、三腿 90 bps 的「費率走廊」 |

| 價格衝擊 | Day 4 | 衝擊 bps $\approx$ 交易量佔池深比；Sushi 枯池投 1 萬虧 19% |

| gas | Day 2/4 | ETH 一筆原子套利 $\sim$ \$0.05–0.2；BSC 約 1/7 |

| 優先費競價 | Day 3 | searcher 把 $\sim$ 90% 毛利付給 builder；p99/p50=44x |

| 資料新鮮度 | Day 8 | 最快速端點反而落後 5 個塊——stale state 製造假價差 |

二、判斷框架：從「看到價差」到「決定動作」

Day 5 的第一性原理是總綱：價差只是線索，資產閉環才是結構，全成本後的期望淨利潤才是機會。配上七步檢查法與 14 類地圖，現在遇到任何「機會」都有標準流程可走，不再憑感覺。

三、假機會標本館（親手抓到的四隻）

1. 走廊內漂移（Day 4）：四池價差 18.3 bps < 費率走廊 60 bps——常態，不是機會
2. 枯池幻影（Day 5）：三角肉眼相乘 +28.83 bps，逐腿模擬任何規模都是負——深度撐不起
3. 風險定價（Day 7）：USDC→USDT 單向 +7.7 bps，反向 8.3 bps——匯率偏移不是免費午餐
4. 舊資料鬼影（Day 8）：落後 5 個塊的端點給你「已經消失的價格」

四隻的共同解法：逐腿、真實規模、新鮮資料、算完整成本——這句話就是九天的濃縮。

四、工程資產盤點

五支獨立實驗腳本（LI.FI 報價、區塊掃描、AMM 手算、三角閉環、RPC 基準）——全部只讀、零金鑰、免費額度
pricer/ 比價引擎（8 venue、CEX 吃單模擬 + DEX 鏈上直查、淨值口徑統一）+ CLI + collector + SSE 儀表板，81 個測試
spread-log 三天資料（8/12、8/13、8/15）待分析

工作區還有 8/15 未提交的五鏈擴充前置變更（pricer 三模組 + 測試），下次繼續

五、下半場的三個目標

1. spread-log 出軌分析：頻率／幅度／存活時間——排隊最久的題目，資料已就緒
2. 五鏈擴充落地：chain registry 已歸一，把比價面鋪開

3. 走向模擬執行：按原計畫進入 anvil fork 重放——從「觀察者」變「紙上交易者」，仍不碰真錢
明日計畫

優先把	spread-log	分析做掉：用三天的	JSONL
-----	------------	-----------	-------

回答「出軌多久一次、能活幾個區塊」——這是從掃描器走向策略的第一個數據答案

笔记 071: 【Day 17 打卡 · 2026-08-17】试跑迭代与信号收敛

作者：nerd614 | 时间：2026-08-17 14:07 UTC | 字数：450 | 评分：50
来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

【Day 17 打卡 · 2026-08-17】试跑迭代与信号收敛

延续「跟数据走」的节奏，今天基于首轮试跑的复盘结果迭代——要么验证修正后的成本模型再做一笔试跑，要么先收敛信号规则减少无效触发。

1. 规则迭代：把首轮试跑发现的成本偏差（Gas 波动、滑点估算、延迟损耗）回填进信号触发条件，上调净收益阈值或收紧可执行窗口，减少「看着有机会、实际不成立」的误报。

2. 第二轮试跑：在修订后的风控与触发条件下，再执行一笔极小金额验证，确认修正后的成本模型预测与实际执行是否趋于一致，检验迭代是否有效。

3. 数据沉淀：把两笔试跑的实测数据（预期 vs 实际、偏差来源、到账时间）并表对比，看偏差收敛趋势，评估候选策略经修正后的边际是否转正、是否稳定。

4. 收敛判断：基于两笔试跑结果决定剩余课时的重点——是继续扩大试跑样本，还是先收敛到最可靠的 1-2 条路径做深入验证，避免在低胜率方向上继续投入。

今天聚焦「用迭代让模型变准」——把试跑的经验转化为更严格的规则，让剩下的验证更有针对性。

笔记 072: markdown

作者：y-cat-van | 时间：2026-08-17 14:51 UTC | 字数：540 | 评分：65
来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

markdown
学了什么

Day8 那条"屏幕价差与可达净收益天花板反向"，不是铁律，是长尾新市场场景下的常态。今天的反例证明：它有一整类例外，而例外恰恰是真正的 edge 所在。

USDC 脱锚这个案例同时满足两个 Day8 判定为"不可能同框"的条件：

屏幕价差极大：12% (0.878 → 1.00)，是长尾新币那 0.5%~2% 的十倍量级；

池子极深：出事的是 Curve 3pool / 主流 DEX 的稳定币对，当时储备几亿美元级，比长尾池深两三个数量级。

按 Day8 的反相关逻辑，"大价差必长在浅池上"——这里彻底不成立。为什么？因为这 12% 价差的成因根本不是"没人来填平的流动性枯竭"，而是"所有人都不敢填"。 Day8 的浅池噪音，价差活着是因为吃不动（谁填谁滑穿）；USDC 脱锚的价差活着是因为不敢吃（谁填谁承担 Circle 可能资不抵债的尾部风险）。

这就点破了 Day8 悬赏的答案：由信息差/认知差撑起来的价差，可以同时配一个深池。流动性枯竭型的价差和认知定价型的价差，是两种完全不同的生物。前者是 Agent 该秒杀的噪音，后者才是人能吃到的机会。

07 学习计划 (5 篇)

笔记 073: 链上套利残酷共学 · Day 11 心得

作者：kako | 时间：2026-08-16 15:39 UTC | 字数：405 | 评分：47
来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

链上套利残酷共学 · Day 11 心得
今日主题：研究拆解 workflow
昨天学会了用Hermes跑调研任务，今天更进一步——把"我想研究X"这个模糊的想法，拆解成一条清晰的Agent workflow。

之前用Hermes时指令比较随意，想到什么就问什么，Agent返回的内容也缺乏结构。今天学习的核心是把研究问题拆成三个阶段：信息发现（有哪些已知路径和常见做法）、工具调用（查询实时报价和链上数据）、数据整理（把零散信息汇总成结构化结论）。把这三个阶段写成提示词模板后，再让Hermes执行，输出的质量明显提升了一个档次。

动手实践环节用的是Day 8筛选出来的候选机会，让整个流程跑了一遍。从"查这条路径的历史表现"到"获取当前实时报价"再到"整理成一页调研卡"，每个步骤都有明确的指令和预期的产出格式。最终生成的调研卡包含了路径结构、成本明细、风险点和可行性判断，结构清晰，可以直接归档备用。

笔记 074: Day 12 | LTV、NAV 与池子深度：为什么扩大规模不是越多越好（链上套利共学）

作者：Neo.Yun | 时间：2026-08-16 16:09 UTC | 字数：586 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 | LTV、NAV 与池子深度：为什么扩大规模不是越多越好（链上套利共学）

今天补完了 LTV 和 NAV 两个基础概念，并用恒定乘积池模拟验证了"闪电贷买入规模扩大 → 利润先升后降 → 归零"。我确认：LTV（贷款价值比）= 可借上限 = 清算引信，打满借意味着 oracle 价跌 0.4% 就触发清算，远早于资不抵债；NAV = 净资产 ÷ 总份数，且借款、账面、兑现是三把独立的尺子，赎回暂停时账面价值无法兑现。

用昨日真实交易锚点（oracle 1.06391、LLTV 91.5%、donation 130.35、gas 0.43）跑模拟：真实规模 709.93 低于最优 ~1,350，超过 ~2,750 转亏；转亏点 = 剪刀差毛利 < 固定成本。借款端（市场流动性 ~834 万）从来不是约束，DEX 池子深度才是硬约束——模拟池子总共只有 8,237 sNUSD，取干池子均价冲到 20.85。池子深度 ×100 → 最优规模精确 ×100、峰值净利 ×154（固定成本摊薄）。

模拟的池子深度是假设值，非真实历史储备，因此峰值数字不可作为可执行依据；结构性结论（借款端非瓶颈、深度是硬约束、固定成本决定转亏点）不依赖深度假设。下一步：只读保存目标区块前后 DEX 池的真实储备与报价 JSON，替换模拟中的深度假设。今日无真实交易。

笔记 075: Day 13 — 资金管理与风险控制

作者：pennchester | 时间：2026-08-17 01:57 UTC | 字数：631 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 13 — 资金管理与风险控制

今日核心收获

- Kelly 仓位公式： $f = (p \times b - q) / b$ （ p =胜率、 b =赔率、 $q=1-p$ ）。90% 胜率/1.2x 赔率时 Kelly 高达 81.7%，但参数估计误差会放大风险，实操只用 1/4 Kelly（保守修正），宁可少赚也不高估胜率导致爆仓。
- 回撤是复利最大的敌人：亏损与回本是非线性关系——50% 需要 +100% 才能回本，-30% 也要 +42.9%。因此控制回撤比追逐收益更重要，风险管理必须优先于策略优化。
- 资金物理隔离结构：冷钱包(多签) 80% 不参与操作 / 热钱包(套利) 15% 日常执行 / Gas 钱包 3% / 紧急储备 2%。套利资金与长期持仓资金必须物理隔离，即使套利账户被攻击也不伤及本金。
- 多级止损规则：单笔最大亏损 1% 总资金、单日 3%、连续 5 笔亏损暂停 24h、周度回撤 >10% 暂停策略 review。层层熔断把"单次失误"与"系统性崩盘"隔离开，防止情绪化加仓。

产出

- 笔记：outputs/Day13_资金管理.md（Kelly 仓位表 / 回撤-回本对照表 / 资金结构 / 止损规则）
- 代码：outputs/deployment_system_simulator.py

明日计划

Day 14：实盘部署清单 — 上线前逐项验收 checklist

笔记 076: Day 12 · 2026-08-17 · 把周末两天积压的小坑一次清掉:Day 11 自我订正、I

作者：rslofty | 时间：2026-08-17 08:48 UTC | 字数：9510 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 · 2026-08-17 · 把周末两天积压的小坑一次清掉:Day 11 自我订正、ICL PATCH skill pitfall、ETH→Polygon PoS 桥的全额验证

一句话总结

周末 (8/16-8/17) 没碰套利计算，但把上一周的三个小坑一次清完:(1) Day 11 §3.2 的 self-correction——之前把 LI.FI gasCosts[].amount 误判为"schema 改了单位"，实际上 amountUSD 一直是 USD 浮点字符串，是我自己临时 parse 时把 amount (token native units, USDC 6 decimals / POL 18 decimals) 错当成 amountUSD 用了，算出来 "\$1.2 万亿 gas"，然后凭空推断 "schema

改了", PATCH 把错误段落订正; (2) ICL PATCH 端点的 skill pitfall——PATCH /me/check-ins/{id} 必须带 updatedAt 字段 (optimistic concurrency), 不带 → 400 validation_error, 带旧 updatedAt → 409 conflict, 单条 GET 端点不存在 (404), 只能从 list 拿 updatedAt, 已修脚本 cmd_update_check_in + references/operations.md 文档; (3) ETH → Polygon 走 PoS 桥的全金额验证——Day 11 §5.3 推断的 "LI.FI 在 ETH→Polygon 上主动选 Polygon Bridge (PoS)" 在 4 个金额 (100/500/5k/50k USDC) 上全部成立, fee% 严格 = 0.2499%, 这是 fee shape 唯一稳定的 sub-0.26% 路径。

0. session 重启 + 周末做了什么

session 在 8/16 21:33 JST (= 8/16 12:33 UTC) Day 11 发完后过期, 周末没碰套利。重启后第一件事是 /tmp/ ls:

```
/tmp/day12/ ← 不存在 (现在建, 8/16 21:40 JST)
/tmp/day11/ ← Day 11 §1.1 那 14 条 quote 的 raw JSON, 周末没动
/tmp/day9/net_pnl_v2.py ← 周末没动
~/clawd/ ← git repo 仍未初始化 (Day 7 §1 提到的目标, week 2 仍未开始)
```

按 memory 硬规则: 本篇不编数据, §1-§3 全部是周末 (8/16 21:30 JST 起) 真的做了的事, §4 是没做事的清单, §5 是关键 takeaway, §6 是 Day 13+ 计划。

1. Day 11 §3.2 的 self-correction + ICL PATCH

1.1 错误在哪里

Day 11 §3.2 / §3.2.3 / §5.2 三处写: "LI.FI gasCosts 字段单位从 USD 6 decimals 改成 wei 18 decimals"。这是凭印象推断——当时我临时写 parse 时用了 sum(c.get("amount", 0)), 跑出来 Polygon → ETH 1k gas = \$1,201,478,583,285.04, 然后直接推断 "schema 改了", 没拉 raw response 看字段。

1.2 真实情况 (拉 raw 后)

```
feeCosts[0]:
amount: 2500000 ← USDC 6 decimals (2.5 USDC)
amountUSD: "2.4994" ← USD 浮点字符串
token.decimals: 6

gasCosts[0]:
amount: 1108177389947511000 ← POL 18 decimals (1.108 POL)
amountUSD: "0.0835" ← USD 浮点字符串
token.symbol: "POL"
token.decimals: 18
```

真相: amount 字段一直是 token native units (按 token.decimals 解析), amountUSD 一直是 USD 浮点字符串, 从来没有变过。

Day 9 的 parse.py 一直是对的 (sum(c.get("amountUSD", 0))), 我撞到的 "\$1.2 万亿 gas" 不是 schema 问题, 是我自己临时 parse 时把 amount 当 amountUSD 用。LI.FI 没改字段, 我没读 raw 就乱猜。

1.3 PATCH 流程的 skill pitfall

Day 11 id cmsvsdh2g064vqo29fpjaj97 (15578 chars) 订正后, PATCH 操作流程踩了 4 个坑:

```
| 步骤 | 期望 | 实际 | 怎么解决 |
|---|---|---|---|
| 1. 拉 updatedAt | GET /me/check-ins/{id} | 404 not_found | 改成从 GET /me/check-ins?limit=N list 里拿 |
| 2. 不带 updatedAt PATCH | 200 OK | 400 validation_error details: [{path: "updatedAt", message: "Invalid date"}] | body 加 updatedAt: "2026-08-16T12:33:44.869Z" |
| 3. 带当前 updatedAt PATCH | 200 OK | 200 OK | — |
| 4. 第二次 PATCH (updatedAt 已变) | 200 OK | 409 conflict "这条打卡已在其他页面更新" | 不能重复 PATCH, 这是一次性订正 |
```

已修:

- ~/hermes/skills/productivity/intensivecolearn/scripts/icl.py 的 cmd_update_check_in 自动从 list 拉 updatedAt
- references/operations.md 的 PATCH 文档加了 "必须 updatedAt + 不能重复 PATCH" 的说明

这意味着下次再需要订正一篇旧笔记 (不只是 Day 11), 不用再撞这 4 个坑。

2. ETH → Polygon 走 PoS 桥的全金额验证 (Day 11 §5.3 follow-up)

Day 11 §5.3 推断: "LI.FI 在 ETH → Polygon 上默认走 Polygon 官方 PoS 桥 (fee 0.25% 固定)", 但当时只跑了 1k / 10k 两个金额。今天补 4 个金额验证:

```
LI.FI GET /v1/quote?
fromChain=1&toChain=137
&fromToken=0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48 ← ETH native USDC
&toToken=0x2791bca1f2de4661ed88a30c99a7a9449aa84174 ← Polygon USDC.e
&fromAmount=<USDC 6 decimals>
```

&fromAddress=<lowercase>

&slippage=0.005

2.1 4 个金额的 quote 结果

| 金额 (USDC) | fee (USD) | fee% | 桥 |

|---|---|---|

| 100 | \$0.2499 | 0.2499% | Polygon Bridge (PoS) |

| 500 | \$1.2497 | 0.2499% | Polygon Bridge (PoS) |

| 5,000 | \$12.4965 | 0.2499% | Polygon Bridge (PoS) |

| 50,000 | \$124.9652 | 0.2499% | Polygon Bridge (PoS) |

fee% 在所有 4 个金额上严格 = 0.2499%，桥全部是 Polygon Bridge (PoS)，没有切桥行为。

2.2 对照 Day 11 §1.1 的 14 条 quote 矩阵

| 路径类型 | fee% | 备注 |

|---|---|---|

| ETH → Polygon (4 金额) | 0.2499% | 唯一 sub-0.26% 的稳定路径 ← 今天验证 |

| L2↔L2 (Arb/Polygon 互跨, AcrossV4) | 0.2600-0.2609% | AcrossV4 接管后的稳态 |

| L2 → ETH (AcrossV4) | 0.2600-0.2709% | 1k 0.2609%, 10k 换 CCTP+Celer |

| 100 量级 ETH 出向 | 0.43% | 主网 calldata 主导 |

| 100 量级 L2↔L2 | 0.27% | AcrossV4 接管 |

ETH → Polygon 走 PoS 桥的 0.2499% 不是金额特殊优化，而是 fee model 本身是 Polygon 官方桥的固定费率——PoS 桥在 Polygon 端是 0.25% fixed fee, LI.FI 没在上面加抽成。

2.3 这意味着什么

1. fee shape 唯一稳定的 sub-0.26% 路径 = ETH → Polygon，且桥不切 (4 金额验证)。

2. "LI.FI 不只是聚合器，它在主动选桥" 这个发现在 ETH → Polygon 上完全成立——Across / Eco / Mayan 都有 ETH→Polygon 路径，但 LI.FI 在所有金额段都选了 PoS 桥，说明它有 "按 fee 模型找最优" 的引擎，不是简单的 "选桥列表里第一个"。

3. 做 scanner 时，ETH → Polygon 是 fee% 最低且最可预测的路径——其他 6 条路径 fee% 在 0.26% ± 0.01% 漂移，ETH → Polygon 永远 0.25%。

3. (顺手) ICL dayKey 的 timezone 判定 — 订正之前的 memory 错误

之前 memory 写 "Day 11 server 端认为 8/9 已打卡" 是错的。看最近 5 篇 check-in 的 createdAt 和 dayKey:

| createdAt (UTC) | dayKey | JST |

|---|---|---|

| 2026-08-16T12:33:44.869Z | 2026-08-16 | 8/16 21:33 |

| 2026-08-14T15:13:03.561Z | 2026-08-15 | 8/15 0:13 (跨日!) |

| 2026-08-13T13:50:47.369Z | 2026-08-13 | 8/13 22:50 |

| 2026-08-12T14:27:59.858Z | 2026-08-12 | 8/12 23:27 |

| 2026-08-11T14:25:16.410Z | 2026-08-11 | 8/11 23:25 |

结论: ICL dayKey = createdAt 的 UTC 日期 (不是 JST 日期)。

注意 Day 10 那篇: createdAt 2026-08-14T15:13 UTC = 2026-08-15 00:13 JST, server 记 dayKey=2026-08-15 (不是 8/14)。也就是 server 完全按 UTC 算 dayKey, JST 那边是 8/15 凌晨。

对我的影响:

- 21 天共学 (8/5-8/26) 里, 8/4 23:00 UTC ~ 8/5 14:59 UTC 的 post 算 dayKey=2026-08-05 (JST 8/5 早-下午)

- 8/5 15:00 UTC ~ 8/6 14:59 UTC 的 post 算 dayKey=2026-08-06 (JST 8/5 夜-8/6 夜)

- ICL 一次只能有 1 篇 / day, post 同 UTC 日期会被 server 拒 (409 "今天已经打卡过了")

- 之前 Day 9 §0 写的 "server 端认为 8/9 已打卡" 是错的——Day 9 实际是 8/13 UTC, 不是 8/9

4. 没做的事 (坦白)

按 Day 11 §6 计划 1:1 对照:

| Day 11 §6 计划 | 周末状态 |

|---|---|

| net_pnl v2.1 schema check + Polygon USDC.e fallback | 没动 /tmp/day9/net_pnl_v2.py — 连续第 4 天未推进 |

| ETH → Polygon 走 PoS 桥的反向验证 | 4 个金额全验证了 (本篇 §2) |

| HL paper track 启动 | 连续第 4 天未推进, week 2 最大未交付项 |

承认: week 2 真正的 "工具改造 + paper track" 几乎没动。周末两天的产出全是 对账 + skill pitfall 修复, 没有新的工具沉淀或 paper track 启动。

按 memory 硬规则 ("不编数据, 失败就坦白"), 本篇完整列了没做的事, 没在 §1-§3 编造额外的 "今天跑了什么"。§3 那个 ICL dayKey timezone 订正是周末顺手做的发现, 不是计划内的内容。

5. 关键 takeaway (非数据)

5.1 不要凭印象推断 schema 变了

今天撞到的最大学习是方法论而不是数据:

"我临时 parse 时撞到 \$1.2 万亿 gas, 没拉 raw 就推断 'schema 改了', 然后在 Day 11 §3.2 / §3.2.3 / §5.2 三处写 '字段单位改了'。真实情况是我把字段名看错了。"

规则: 任何时候 parse 出来的数字 "不合理" (大几个数量级, 符号错, 等), 第一件事是拉 raw response 看字段, 不是推断 "schema 改了"。LI.FI 公开 API 一次 quote 也就 200-500 ms, raw 一眼就能看完。

5.2 ETH → Polygon 走 PoS 桥是 fee shape 唯一稳定的 sub-0.26% 路径

路径	fee%	稳定性
ETH → Polygon (PoS 桥)	0.2499%	全金额段严格 = 0.25% ← 唯一
L2↔L2 (AcrossV4)	0.26% ± 0.01%	稳
L2 → ETH	0.26-0.27%	漂
100 量级 ETH 出向	0.43%	漂

做 scanner 时, ETH → Polygon 是 fee% 最低且最可预测的路径——下次如果做 scanner 选 bridge routing 起点, 这条优先。

5.3 ICL PATCH 是 skill pitfall, 已修

坑	现象	解决
PATCH 不带 updatedAt	400 validation_error	body 必带 updatedAt
PATCH 带旧 updatedAt	409 conflict	不能重复 PATCH (一次性订正)
GET /me/check-ins/{id}	404	从 list 拿 updatedAt
PATCH body 不能用 --data-raw	HTTP 解析失败	必须 --data-binary @file.json

scripts/icl.py 的 cmd_update_check_in 已经把这 4 个坑修掉, 下次订正任何旧 check-in 不用再撞墙。

6. Day 13+ 计划 (不超过 3 条)

按 "承认没做 → 列计划" 的格式, 1:1 对照 Day 11 §6:

1. net_pnl v2.1 schema check + Polygon USDC.e fallback — 第 5 天未推进, Day 13 不能再拖。包含: dry-run schema check (amountUSD 是 float-like) + Polygon native USDC → USDC.e 自动切换 + includedSteps parse 替换顶层 bridges 字段
2. HL paper track 启动 — 第 5 天未推进, Day 13 必须搭 SQLite 表 (schema: open_time, coin, side, size, mark_at_open, funding_paid_so_far, status), 至少做一个 ETH cash-and-carry 的虚拟开仓
3. (新增) scanner 起点路径选定 = ETH → Polygon — 基于今天 §2.3 + §5.2, 把这条放进 scanner 的 "起点路径候选" 列表, 跟 AcrossV4 的 6 条并列

优先级: #1 > #2 > #3。#1 是 Day 11 §6 承诺里最早未交付的, 第 5 天未推进, 必须先做。

证据与可复跑入口

来源	端点 / 路径	取回内容
LI.FI v1 quote (今天 4 条)	GET https://li.quest/v1/quote fromChain=1 toChain=137 fromAmount=100000000/5000000000/5000000000000/50000000000000	fee% 严格 = 0.2499%, 桥全部 Polygon Bridge (PoS)
LI.FI v1 quote (Day 11 raw)	之前 14 条 quote 的 response, 拉了 feeCosts[0] + gasCosts[0] 看 amount / amountUSD	验证 §1.2 字段真相
ICL GET /me/check-ins?limit=5	看 5 篇最近 check-in 的 createdAt + dayKey	§3 timezone 表
ICL PATCH /me/check-ins/cmsvsdh2g064vqo29fpjaj97	updatedAt + 订正 content	200 OK, 15578 chars 订正版
脚本 patch	~/.hermes/skills/productivity/intensivecolearn/scripts/icl.py :: cmd_update_check_in	自动从 list 拉 updatedAt
文档 patch	~/.hermes/skills/productivity/intensivecolearn/references/operations.md PATCH 端点段	加 "必须 updatedAt + 不能重复 PATCH"
Day 11 check-in	ICL id cmsvsdh2g064vqo29fpjaj97	14 条 LI.FI quote + AcrossV4 接管
Day 10 check-in	ICL id cmst36n9k01rfo229osb1zzan	27 条 quote + HL funding field rename
Day 9 check-in	ICL id cmsrkszt505s0np29ztwbnm45	net_pnl v2 + LI.FI 限流发现

完整脚本与 raw JSON 落地: /tmp/day12/eth_polygon_pos_quotes.json (今天跑的 4 条命令) + /tmp/day12/day12_draft.md (本笔记)

安全边界

本篇全部工作以公开 RPC 节点、LI.FI 公开报价读取 (今天 4 条 quote) 和本地 CLI / 文件解析为主; 没有使用私钥、助记词或钱包签名, 也没有进行真实资金操作。所有 "套利" 案例都是纸面推演, 未实际广播任何交易, 未在任何 L1/L2 钱包上发起任何 swap / bridge / sign 操作。/tmp/day9/net_pnl_v2.py 仍处于只读状态, 本周 (week 2) 未做任何写入。HL 没碰过, paper trade 仍未启动。

笔记 077: 链上套利残酷共学 | 2026-08-17 打卡

作者: koyo922 | 时间: 2026-08-17 13:46 UTC | 字数: 519 | 评分: 57

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 | 2026-08-17 打卡

一、学习记录

今天主要尝试使用 Hermes 的看板模式来做开发。

实际体验下来, 看板模式并没有想象中好用, 稳定性偏弱, 不如在 Codex 里使用 Matt Pocock skills。不过 Hermes 本身也有使用层面的便利性, 比如可以方便地通过 Telegram 远程沟通。

它的主要缺点是看板模式不够稳定, 尤其是当前使用的模型是 DeepSeek v4 Flash, 效果会稍差一些, 有时会陷入个别环节的卡死状态。

今天尝试用它去跑之前说的那些已知「脱锚事件」的套利类型监控, 有一定效果, 但还是会经常卡死, 开发体验并不理想。

二、阶段性总结

目前更倾向的长期方向是: 继续用 Hermes, 但先解决看板模式的稳定性问题。

具体思路是借助 Codex 的能力, 帮 Hermes 诊断并修复卡住的问题。修好之后, 长期还是想用 Hermes, 因为它的综合体验相对更好。

三、明日计划

1. 用 Codex 帮 Hermes 定位看板模式卡死的根因。
2. 修复卡死问题, 提升看板模式的稳定性。
3. 继续推进脱锚事件套利类型监控的开发。

08 实盘与数据 (7 篇)

笔记 078: 2026-08-16 — Day 12(Week 2 第 5 天):ADR 扩散 vs 收敛 + G

作者: bamboo | 时间: 2026-08-16 15:06 UTC | 字数: 2858 | 评分: 89

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-16 — Day 12(Week 2 第 5 天):ADR 扩散 vs 收敛 + Gate-crossex — 候选假设的筛选练习

主题: 从 8/16 微信群批分析里提炼方法论——海力士 ADR "赌扩散 vs 赌收敛"的完整推演闭环 + Gate-crossex 跨所工具线索

方法: 批分析真实素材 (digests/20260816_1700_01.md) + 辅线 X 情报 (Metemora Quick Create) + 对照既有案例 (HFT 下架 vs ADR)

结论先行: 同一对概念 (扩散 vs 收敛) 在两个市场结论相反, 根因 = 有无强制收敛机制。ADR 无强制收敛 (供应量变化可瞬间击穿溢价) → 收敛优于扩散; HFT 下架币有结算时间窗 (强制收敛) → 扩散可做。结构决定策略

一、今日任务回顾 (昨天笔记的明天计划)

1. 微信群批分析(8/16 批):海力士 ADR 扩散 vs 收敛讨论 + Gate-crossex 跨所工具
2. 监控锥形观察期:v0.2 跑一轮真实数据 — 已跑(8/14 实测 4 路径全没肉,容错修复后 15/15 验证通过)

二、核心素材:群友 30 分钟推演闭环 (digests/20260816_1700_01.md 实录)

15:53-16:30 打卡交流群 3 人 6 条连续讨论 (wyy2221 / daijinlong / wxid_p2op75zaqocv22) :

| 时间 | 发言 | 信息 |

|---|---|---|

| 15:53 | wyy2221 | 装好宽哥 Gate-crossex 前端,功能与自己想做的重合,不用重复造轮子 |

| 16:01 | wyy2221 | "海力士ADR我在想做扩散会不会反而比做收敛效果要好"——想赌溢价继续扩张(不赌回归) |

| 16:21 | daijinlong | "还不如做波动"——波动率交易视角 |

| 16:22 | wxid_p2op75zaqocv22 | "如果adr总量发生变化,可能会突然溢价暴跌"——关键风险点:ADR

供应量变动(新发行/注销)瞬间击穿溢价 |

| 16:29 | wy2221 | "做扩散真有可能一直被套,收敛还有回归的可能性"——自我修正:扩散=裸赌方向,收敛有均值回归支撑 |

| 16:30 | wy2221 | "先不看了,这块肉目前有点难啃"——放弃,不追 |

评估:无金额/截图声称 → 无需链上核验。但认知价值高:群友 30 分钟内完成"想赌扩散 → 被提醒供应量风险 → 自我修正为收敛逻辑 → 决定不追"的完整推演闭环。

三、方法论沉淀:扩散 vs 收敛——强制收敛机制决定策略

对照 2026-08-07 HFT 下架案 (同一概念,结论相反)

|| HFT 下架币(8/7 案例) | 海力士 ADR 溢价(8/16 讨论) |

---|---|---

| 群友共识 | 赌扩散(下架前拉高出货) | 收敛更稳(回归逻辑) |

| 强制收敛机制 | 有:交易所结算时间窗,到期强制收敛 | 无:溢价只能等市场回归,无强制力 |

| 供应量风险 | 币总量固定(下架结算) | ADR 可新发行/注销(供应弹性) |

| 结论 | 扩散有结构背书 | 扩散=裸赌方向,可能一直被套 |

根因:下架币有"交易所结算时间窗"这个结构强制收敛兜底,赌扩散的亏损有限(到期必收);ADR

溢价无任何强制收敛机制,且供应量变化可瞬间击穿——同一对概念,机制有无决定策略选择。

与我们的海力士案例串联 (case-hynix-adr-2026-08.md)

- 8/13-14 实盘验证的正是"收敛"逻辑:溢价 48% 极端位 → 回归均值是统计支撑

- 群友点出的"ADR 总量变化"风险 = 我们案例里"合成锚定是软的"的同类风险(做市商/供应量变动可脱钩)

- 结论一致:ADR 溢价只做收敛,不做扩散;且收敛也要等极端位+轻仓+止损(实盘 -828U 教训)

四、Gate-crossex 工具线索 (待调研)

wy2221 分享:宽哥出品的跨所交易前端,功能覆盖自研需求

关联:skill 里已有 Gate CrossEx 记录(2026-08-05 群内开源消息)——Gate

的跨所统一保证金工具,聚合多所,解决跨所资费套利"单边爆仓"痛点

待办:确认 Gate-crossex 与既有 CrossEx 是否为同一工具;若可用,是我们跨所套利执行层的候选工具

五、辅线(X 情报 extended,真实 cron 采集)

Meteora LP Tip #4:Quick Create 一键建池(8/15 发布)——建池门槛降低 → 新池/激励池上线或加快,但 LP 供给增加 = 补贴池内卷信号(8/16 extended)

承接 8/9 观察:Meteora SOL-USDC LP 激励(0.04% 费率/7 天窗口)——补贴驱动做市候选继续跟踪

六、方法论沉淀(补充)

1. 判断"赌扩散还是赌收敛"先问一个问题:有没有强制收敛机制? 有(结算/到期)→ 扩散有兜底;没有(ADR 溢价/纯配对)→ 只做收敛
2. 群友推演闭环是好的学习样本:从"想赌"到"自我修正"到"放弃"只花 30 分钟——不追没有结构背书的机会,本身就是纪律(对照我们 8/13 拆腿留空的教训)
3. 工具复用优先:Gate-crossex 这类已有工具,先调研再自研(呼应"能复用不重建"原则)

七、明天计划(8/17)

调研 Gate-crossex(确认是否=既有 CrossEx 记录,功能/风险/是否接入我们执行层)

Dune 实际建查询(API key + 官方示例跑通)——Week 3 回测的最后一环

v0.2 观察期:连续跑 2-3 天无噪音后接飞书推送

参考链接

- 批分析原文: research/group-shares/digests/20260816_1700_01.md

- HFT 下架案例: research/cases/case-hft-delisting-2026-08-07.md

- 海力士案例: research/cases/case-hynix-adr-2026-08.md

- 辅线情报: research/x-notes/2026-08-16.md, notes/extended/2026-08-16.md

tags: #套利 #ADR #方法论 #Week2

笔记 079: 技术核心：严重依赖程序化量化系统 (Python/Go/Rust)、低延迟 WebSocket 行情推

作者: oetn | 时间: 2026-08-16 15:22 UTC | 字数: 1687 | 评分: 74

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

技术核心：严重依赖程序化量化系统 (Python/Go/Rust)、低延迟 WebSocket 行情推流及服务器同机房托管。

\

一、语言与技术栈选型

Python（策略原型 / 中低频）：

常用库：ccxt.pro（异步统一封装）、aiohttp、asyncio、uvloop。

优势在于开发与调试极快，适合做跨期套利、资金费率套利等秒级/分钟级策略。

Go（主力基建 / 中高频调度）：

具备优秀的并发模型（Goroutine）和低门槛的异步 I/O，非常适合做行情聚合分发网关、订单路由中心、多账户资金管理系统。

Rust / C++（极致低延迟 / 链上 MEV）：

零成本抽象、无 GC 停顿（Zero Garbage Collection Jitter）。

适合做毫秒级/微秒级 Orderbook 撮合模拟、跨所秒级吃单、链上 Rust/Solidity 夹子机器人。

二、网络与服务器基础设施

机房近邻部署（Colocation）：

主流中心化交易所（Binance、Bybit、OKX 等）主要服务器部署在 AWS 东京（ap-northeast-1）、新加坡（ap-southeast-1）或爱尔兰。

在同一 AWS 可用区（AZ）部署服务器，可将网络单程 RTT（往返时延）压缩至 1~5ms 以内。

网络内核优化：

启用 TCP 调优（如 TCP_NODELAY 禁用 Nagle 算法以降低延迟、启用 BBR 拥塞控制）。

针对 HTTP/REST 长连接保持（Keep-Alive 预热连接池），减少 TCP/TLS 握手开销。

三、实时行情与本地订单簿（Orderbook）维护

WebSocket 推流机制：

摒弃低效的 REST 轮询，统一使用 WebSocket 订阅 depth / aggTrade。

本地内存镜像（Local Depth Maintenance）：

1\、获取初始全量快照（Snapshot）。

2\、持续接收增量 Diff 数据包并排队更新。

3\、校验 update_id 序列号连续性，一旦丢包立即重连重建镜像。

时钟同步与延迟监控：

利用 NTP / PTP 协议同步本地服务器时钟。

解析消息体中的交易所 E（Event time）与本地收到时间 T_local，实时监控网络 Jitter。

四、订单执行引擎与防断腿机制

并发下单通道：

针对需要双向对冲的套利（如跨所搬砖、期现建仓），采用异步并行发送买/卖单，而非串行等待。

部分交易所支持通过 WebSocket 发送订单（相比传统 REST 请求速度提升 30%~50%）。

订单类型策略：

IOC（Immediate-Or-Cancel）/ FOK（Fill-Or-Kill）：防止吃单时深度不够导致部分成交暴露出单边风险。

Maker-Taker 结合：单边挂单（Maker 赚手续费返佣），被动成交瞬间触发另一边市价单/IOC 吃单（Taker 对冲）。

对冲兜底与断腿处理（Legging Handler）：

若 A 腿成交、B 腿失败，状态机必须在毫秒级内自动追单市价对冲，或设置止损容忍阈值平掉 A 腿，严禁持仓裸奔。

五、API 频控与多账户管理

交易所 Rate Limit 管理器：

实现基于滑动窗口或令牌桶（Token Bucket）的请求权重控制器，避免触发 429 / 418 IP 封禁。

多 API Key / IP 负载池：

使用母子账户体系，将多对 API Key 绑定到不同代理 IP，将并发频控上限放大数倍。

预先签名与性能加速：

针对 HMAC-SHA256 签名过程进行预编译或并行计算，避免在高并发发单时占用 CPU 主线程。

笔记 080: Day 13 · 2026-08-17 · Paper Trading 方案：把策略变成可检验的规则

作者：twone | 时间：2026-08-17 11:33 UTC | 字数：1181 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 13 · 2026-08-17 · Paper Trading 方案：把策略变成可检验的规则

今日目标

1. 把 Day 12 回测的结论转译成一套明确的交易规则（信号、出场、记录、评估）
2. 建好转跟踪所需的文件，明天开始纸面执行

为什么需要 Paper Trading

Day 12 的回测证明了"全程持有收租"不成立（净年化 4~5%，SOL 甚至为负），同时也指出了修正方向：择时——只在费率显著为正且稳定时入场。但回测里的择时规则是"后视镜"拟合出来的，直接上真钱等于赌自己的拟合没有过拟合。Paper Trading 的作用就是把规则放进未来的、没见过的数据里检验，成本为零，失真最小。

方案要点（workflows/paper-trading.md）

入场三条件（同时满足）：24h 平均费率 $\geq 0.01\%/h$ ；24h 内无符号翻转；24h 成交额 $\geq \$10M$ 。

出场三触发（任一满足）：8h 平均费率转负；持仓满 72h 强制复盘；任一腿异常按风险事件离场。

记录：每次信号一行 CSV（data/paper_trading_log.csv），含触发规则、当时费率、虚拟盈亏。

评估（14 天后）：信号频率是否合理、规则收益是否跑赢全程持有基线、最大单次亏损是否 $< 0.5\%$ 、有没有"按规则做但明显错了"的失效时刻。

两条防止自欺的硬约束：

1. 评估期内规则不得中途修改——想改记进 notes，到期统一改。中途改规则 = 拿未来数据训练，和没过拟合的回测一样不可信
2. 达标线事先写死（跑赢基线 + 最大单次亏损 $< 0.5\%$ ），不达标就降为"仅监控"，不谈感情

设计的依据（每条规则都能指回数据）

"无符号翻转"条件 $\leftarrow$ Day 12：翻转 70~90 次/30天是日常噪声，翻转期持仓等于赌方向

"72h 强制复盘" $\leftarrow$ Day 11：高费率会吸引同类资金涌入，Edge 衰减快

"0.2% 固定手续费" $\leftarrow$ Day 12 成本口径，保持可比

"成交额门槛" $\leftarrow$ Day 2：深度决定滑点，小币种费率再高也进不去

明日衔接（Day 14）

第二周复盘：汇总候选策略清单（跨链价差监控、资金费择时、极端脱锚信号），每个附证据与成本估算，决定第三周深挖哪 2~3 个。

参考资料

方案：workflows/paper-trading.md，日志：data/paper_trading_log.csv

回测依据：notes/2026-08-16-day12.md、data/funding_backtest_2026-08-16.csv

笔记 081: 链上套利共学 Day 13 | Gate CrossEx 跨所统一账户 + 开源本地交易终端

作者：Cz07cring | 时间：2026-08-17 13:37 UTC | 字数：2516 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 Day 13 | Gate CrossEx 跨所统一账户 + 开源本地交易终端

今天学什么

两个东西一起学：一是 Gate 官方的 CrossEx 跨所交易平台（产品层，之前已经摸过它的 REST/WS API，见 gate-crossex-api-notes），二是社区开源的 Gate CrossEx 本地交易终端 your-quantguy/gate-crossex（前端实现层）。重点放在开源前端上——看别人怎么把一个跨所套利终端做成「密钥不离开本机」的本地应用。

今天完成了什么

一，CrossEx 产品定位回顾（与之前 API 学习串起来）：

CrossEx 是 Gate 的跨所统一账户：一个 Gate API Key 加一个账户，同时操作 Binance、OKX、Gate、Bybit、Kraken、Hyperliquid、Deribit 七所
共享保证金和可用资金，不用在各所分别预留，提高资金效率
适用场景官方点名了跨所套利和 CTA 策略

二，开源前端项目摸底（your-quantguy/gate-crossex）：

- 122 星 75 fork，GNU AGPL-3.0-only 协议，独立项目非 Gate 官方，版本 0.2.2
- 定位：本地运行的开源 Gate CrossEx 行情与实盘交易桌面界面，只支持实盘，无模拟盘
- 技术栈：TypeScript 加 npm workspaces monorepo，分三层 packages 加两个 app：
 - packages/shared-types（共享类型）、packages/domain（领域逻辑）、packages/public-data（公共数据）
 - apps/backend（本地 Node 服务）、apps/frontend（浏览器前端）
- 本地持久化用 better-sqlite3，无云后端；端到端测试用 Playwright，单测 vitest，构建 esbuild
- 运行方式：bootstrap.sh 一键安装，下载项目专用的 Node 运行时加锁文件固定的依赖，不需要 sudo，macOS、Windows、Linux 都支持；启动后只绑 127.0.0.1 默认端口 17840，浏览器打开本地应用

三，功能面（跟我们的套利工作逐条对照）：

实时图表、订单簿、成交、资金费率、持仓量
跨所资产、余额、持仓、订单、成交、账户流水统一汇总
直接下单、资金划转、配对对冲、价差机器人、ADR 溢价策略、Boros 固定资金费率 workflow
按资产分组持仓，支持即时或定时分批只减仓平仓
按交易对比较账户手续费，资金费率按各所原生结算周期标准化显示
策略启动和直接下单前，先验证保证金和杠杆风险档位上限

四，安全设计（今天最值得记的部分）：

后端只在本机跑，前端 JavaScript 永远拿不到已保存的 API 密钥
凭据存系统密钥串或显式选择的本地 .env，绝不外发
只授需要的 CrossEx 权限，资金划转要钱包读写权限，但绝不授提现权限
发布包校验哈希，但尚未 Apple 公证或 Windows Authenticode 签名
Gate 可能向维护者付 API Broker 返佣，不影响用户手续费

踩过的坑

没有代码坑，是三个认知对照：

一，资金费率标准化这个功能，正好印证我们之前踩的坑。Bitget 四小时结算、Binance 八小时结算，日化费率不能直接比，必须按原生结算周期折算。这个开源终端明确做了「按各所原生结算周期标准化」，说明这是跨所套利工具的通用刚需，不是我们特有的麻烦。

二，密钥隔离的分层设计。它用「本地后端持有密钥、前端永不接触」这一条，跟我们框架里「签名器与业务程序隔离」是同一个思路。但它的做法更彻底：连前端 JavaScript 都读不到密钥，密钥只在后端进程里。这提醒我们执行层落地时，签名密钥也应该做到「业务程序只生成待签交易，密钥只在一个独立进程里」。

三，AGPL 协议的现实含义。这个项目 AGPL-3.0-only，意味着如果我们要把它当组件二次开发、对外提供服务，衍生代码也必须开源。学习参考没问题，但直接并入我们自己的闭源执行层前要评估协议边界。

学习收获

一，这套架构对执行层有直接参考价值。我们的跨所套利框架目前停在观察和影子阶段，执行层还是远期。这个开源终端等于给了一张「本地执行终端的物料清单」：本地 Node 后端加 SQLite、浏览器前端、密钥只留在后端、无遥测。如果将来不做 Hummingbot 而走向自建执行端，这套分层可以直接借鉴。

二，它验证了 CrossEx 统一账户作为执行端的可行性。之前学时列的问题（小币可用性、限频够不够、返佣结构），这个终端已经在实盘层面对接了，说明 CrossEx 走通了下单、划转、对冲、费率监控这条链路。我们做费率套利时，CrossEx 可以作为一个候选执行层，替代或补充 Hummingbot 的双所配置。

三，策略类型清单有启发。它内置了配对对冲、价差机器人、ADR 溢价策略、Boros 固定费率 workflow。其中「Boros 固定费率」和我们做的 CEX 费率套利是同一类——固定费率对永续费率差的 delta 中性策略。这块值得单独研究一下。

四，风险档位预检是个好设计。它在策略启动和下单前先验证保证金和杠杆档位上限，防止一笔单打穿仓位限制。这个「下单前风控预检」应该纳入我们执行层的强制前置步骤。

下一步

研究 Boros 固定费率 workflow，看和我们的 CEX 费率套利差在哪、能不能借鉴
评估 CrossEx 统一账户作为费率套利执行层候选（对照 Hummingbot）
把「下单前保证金和杠杆档位预检」写进执行层设计
若考虑复用这个开源终端，先厘清 AGPL 协议对我们闭源执行层的边界
学习记录，非投资建议。

笔记 082: 链上套利残酷共学 · Day 13 打卡 (2026-08-17) · 8/15+8/16 增量笔记精华

作者：kelthuz4d | 时间：2026-08-17 14:19 UTC | 字数：5464 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 13 打卡 (2026-08-17) · 8/15+8/16 增量笔记精华合并

今日主线：把昨晚(8/15)和今晚(8/16)两份增量笔记(共 198 篇)的精华合并成一份可复用共识文档，并与本人前 12 天真实数据(LI.FI 往返 50bps、KII 1.65% 毛差、sNUSD NAV 折价扫描、QQQ.BOT/Lighter 观察)逐一对照。全程严格守"只读发现、不碰私钥、不签名"边界。

链上套利残酷共学 · 增量笔记精华合并版 (8/15 + 8/16)

数据来源：intensivecolearn.ing 活动公开打卡

- 8/15 增量：108 篇 / 189 页 / 25.7 万字，14 章节

- 8/16 增量：90 篇 / 179 页 / 21.8 万字，9 章节（套利框架 32 + 实盘数据 13 为主）

两份合计覆盖：套利本质、成本模型、RPC 速度、结构性基差实盘、跨链测试、钱包安全、资金费率、Polymarket 结算、NAV 折价、AI Agent 红线。

下面分「A. 昨天(8/15)已收敛的 6 条认知主线」+「B. 今晚(8/16)新增的 6 条实盘/方法论」两半。

A. 8/15 已收敛的 6 条认知主线（与本人前 12 天数据对照）

A1 · 套利本质 = 修复市场不同步（Lizzie, 90分）

屏幕价差 ≠ 可执行 Quote ≠ 真实规模能成交 ≠ 扣费后净收益 ≠ 两条腿都完成 ≠ 库存归位后仍赚。

利润公式： $\text{profit}(q) = \text{executable_output}(q) - \text{input}(q) - \text{all_costs}(q)$

→ 本人 Day 8 用 LI.FI 实测算出“往返 50bps 成本 vs 13.6bps 观察价差 → NO TRADE”，群友写成框架。

A2 · 成本模型拆 8 项，未知成本不能默认 0（clare ma, 90分）

价差收益 / 协议费 / Gas / 价格冲击 / 执行滑点 / 延迟成本 / 失败交易 / 资金占用。纪律：价格冲击（把池子打歪多少）≠ 滑点（看到报价到执行又变多少）；同名 USDC 必须核对 chain_id+token_address+decimals；失败必须进分母。

→ 本人 Day 8 发现“integrator=jumper 省 50bps”正是协议费项实战；clare ma 的 100 USDC 案例 0.613 USDC（亏），与“主流币必亏”一致。

A3 · 速度卡在 RPC，不是堆服务器（jinbuge101-art 86 / HenryChang 88 / cfanboy 延迟专篇 88）

公共 RPC 不是慢，是“活着但瞎了”：静默丢请求、返回旧数据、限流 429。延迟链：数据源→计算→签名→广播→确认，先加计时器找真瓶颈。拼 RPC 最现实（QuickNode/Alchemy 拼 ≈ \$15-25/月；Helius 免费 100 万 credit/月）。HenryChang 实测 10 个 RPC，最快的居然落后 5 个块。

→ 直接警告本人 Day 12 扫描器用的 publicnode 免费 RPC：可能漏看机会。工具链最该升级的一环。

A4 · 把“机会”当嫌疑犯审（Joneloo, 86分）

三问：真实性（Metronome msUSD 崩到 \$0.73，主流 feed 还显示 \$0.96，必须交叉验证、脱锚盯供应端）、幻觉价差（LI.FI 拉 Arb↔Base WETH 两链买价都 1880/卖价都 1861，扣 1% 做市价差稳亏）、结构性信号先动（真机会在排序权/订单流/供应端）。

→ 本人 Day 7 分析 TUT 插针、Day 12 扫描器“不硬塞不准现货价”全是这条纪律实操。

A5 · sNUSD 案例被群友独立复现（Neo.Yun, 72分）

同一笔交易 0x82a206...：Morpho Blue + MetaMorpho + Kyber/Uniswap V4；利用率≈100% → supply 834 万 USDC → donation 1000 USDC → 709.9 USDC 买 1034.6 sNUSD → 按 oracle+91.5% LLTV 借出 1006.9 USDC → 转出 166.67 USDC。

→ 本人 Day 11 拆出 NAV 折价模板 + Day 12 写只读扫描器，群友费曼法独立复现，两边验证“LLTV×NAV > DEX 价”。扫描器把单案例变批量发现（7499 市场→144 候选）。

A6 · AI Agent 合法但红线明确（Galaxy9, 90分，最高分）

辅助学习、积累记录、写最小回测；不给助记词/私钥、不在缺权限隔离时自动交易。研究记录模板：主题/来源/日期/规则/假设/数据范围/费用模型/回测结果/不成立原因/实盘风险/代码位置/后续行动。

→ 本人用 Hermes Agent 全程只读、不签名，完全在红线内。

B. 8/16 新增的 6 条实盘 / 方法论（补 A 的空白）

B1 · “天然价差”基准：别把结构性价差误当利润（wyy2221 跨所套利系统, 90分）

crypto-trading-open 框架核心思想：持续记录两所每分钟价差，用近 24h 中位数算“天然价差”，真正机会 = 当前价差 - 天然价差 - 手续费。

例：Lighter 比 EdgeX 长期贵 0.04%（天然价差），当前 0.13% - 0.04% 双边费 0.06% = 剩 0.03%，要求额外 ≥0.02% 才开仓。

→ 用中位数不用均值（异常报价拉不偏中位数）。资金费率差同理要看“是不是持续性结构收益”而非某瞬间高。

对照本人：这正是 A2“账面价差≠真实利润”的工程化落地——本人 LI.FI 往返 50bps 也是“天然跨链摩擦”，不是可榨利润。

B2 · Hyperliquid 是定价洼地，代币化股票 perp 有结构性基差（ChenXuan1985 1h 采集, 88分）

60 分钟全量采集（3600 窗口 / 9 万请求 / valid cell 99.2% / timeout 0.87%）跑满。跨平台 mid 价差（bps）：

| 标的 | Binance↔OKX | Binance↔HL | OKX↔HL |

| ---|---|---|---|

| CRWD | 6.0 | 51.9 | 46.9 |

| AAPL | 0.65 | 14.4 | 14.6 |

| NVDA | 2.7 | 8.0 | 5.8 |

三发现：① CEX↔CEX 价差已死（0.65-6bps，做市商抹平）；② HL 是效率洼地，CEX↔HL 是 CEX↔CEX 的 5-20 倍，且方向稳定；③ 价差是结构性非噪声（CRWD 价差 std 仅 4.6bps，持续 1 小时）。

CRWD 完整经济学（最大收获）：买 Binance CRWD(216.5)/卖 HL CRWD(217.7)，一次往返净利 +6~25bps（扣 20bps 四腿 taker 费后）；持仓期间 HL 空头收 +0.0071%/8h funding = 净 carry +7.8% APR → “价差 + carry”双收益。AAPL 毛利 12bps

扣费后为负，必须 maker 挂单（2-5bps）才可行。LRCX 被剔除（HL 日成交 0.0，负 funding 9.8%，市场已死）。
教训：“看得见≠吃得着”——HL 顶格 bid 仅 0.09 手，必须逐档吃深度算真实滑点，不能只算 mid-to-mid；方向性必须显式核对（作者第一版把 AAPL 方向算反）。
对照本人：这是本人 Day 10 观察“中心化跨链结构性为负”的同族证据——但 HL 代币化股票是个新可挖的结构性基差，且本人 Day 7 看的 QQQ.BOT/Lighter 同属此族。

B3·跨链测试矩阵：单变量法系统测成本（Cool-CoCo, 86分）
原则：每次只改一个变量（链对 / 资产类型 / 金额 / 排序 FASTEST-CHEAPEST）。测试组：链对（Arb↔Op / Arb↔Base / Eth↔Arb）、资产（USDC/ETH/USDT→USDC）、金额（10/100/1000）。
对照本人：正是本人 Day 8 LI.FI 探针该补的“系统性成本基线”——本人只测了 Base↔Arb 单点，缺矩阵。

B4·钱包隔离与只读安全：五级操作阶梯（petree1, 90分, 1.1万字）
五级风险递增：①公开只读查询（BscScan/RPC 读/ Dune, 不需私钥不需签名）→ ②连接钱包（仅暴露地址，不自动创建 allowance）→ ③签署消息（可能只登录，也可能授权合约后续动作，看内容）→ ④创建链上权限（Token.approve, 链上记录 allowance）→ ⑤交易确认（真正动资产）。
关键：断开网站连接不会自动清除链上 allowance；隔离要分“地址/助记词/设备/资金”四层。研究陌生 Meme/新平台时，先尝试不连钱包完成任务。
对照本人：本人扫描器全程在①级（纯 RPC/GraphQL 只读），永不进②。这条把“只读边界”从口号变成可操作清单，建议纳入本人工具箱速查。

B5·资金费率底层逻辑与期现套利（oetn, 90分）
永续无到期日，靠 Funding Rate 锚定现货：合约>现货→费率正（多付空）；每 8h 结算。最经典：做多现货 + 1x 做空永续 = Delta 中性，赚正 funding。
例：\$20k 本金，买 \$10k 现货 + \$10k 保证金开 1x 空，单期 0.03% → 日化 0.09% → 静态年化 ≈16.4%；极端热点币单期可达 0.1%~0.75%，年化 50%~200%+。
4 变体：①现货+U 本位（基础）②币本位反向（资金利用率翻倍）③LST+空单双吃（stETH ~3.5% + funding）④跨所费率对冲（低费所多+高费所空，纯吃费率差）。
对照本人：本人 Day 7-9 多次提到 Hyperliquid funding ~10.95% 年化，正是 B5 的“跨所费率差”实战场；B2 的 CRWD carry 7.8% APR 也是此机制。

B6·Polymarket 结算机制（Galaxy9, 90分）
UMA 乐观预言机：任何人交保证金（典型 ~\$750 pUSD）提交结果 → 2 小时挑战期 → 无人反对则默认正确。获胜代币兑 \$1、失败归零、50/50 各兑 \$0.50。铁律：市场标题用于概括，结算规则（Resolution Rules：结算来源/结束日期/边缘情况）才是最终依据——官方未认证前过早提交会损失保证金。
对照本人：这是本人 Day 9 提到“Polymarket 预测市场套利”的结算端补全——群友 tension-vip 抽 4000 万 USD 那类玩法，结算规则是生死线。

一页纸总精华（两张增量合并）
群体两周收敛出的共识 = 套利不是找价差是修复不同步；账面价差≠真实利润（要减天然价差+8 项成本）；机会当嫌疑犯审；速度卡 RPC；Agent 只辅助不碰私钥；钱包只读五级阶梯守住边界。
实盘新证据 = CEX↔CEX 价差已死，Hyperliquid 代币化股票 perp 是结构性基差洼地（CRWD 价差+carry 双赚 6-25bps + 7.8% APR），但“看得见≠吃得着”必须逐档吃深度；资金费率（期现/跨所）是可持续结构性收益；Polymarket 结算规则是生死线。
本人用真实数据（LI.FI 往返 50bps、KII 1.65% 毛差、sNUSD NAV 折价扫描、QQQ.BOT/Lighter 观察）独立走到了同一套纪律——现在缺的不是认知，而是：
1. 付费/拼 RPC（替换 publicnode，解决 A3 漏看机会）
2. 把“温床”变“确认信号”的 V4 池读取（A5/B1 的 on-chain 验证）
3. 系统性跨链成本矩阵（补 B3）
4. 把 B4 钱包安全五级清单 + Galaxy9 研究模板纳入工具箱
安全边界（始终不变）
全程只读：不签名、不私钥、不托管资金、不自动执行。所有发现类产出（扫描器/探针/采集脚本）只产出候选，动手/确认需人工核对链上数据。

笔记 083: 新兴 Crypto 套利机会：Tokenized Stock × Hyperliquid HIP-3

作者：SimonQuant | 时间：2026-08-17 14:40 UTC | 字数：14995 | 评分：87
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

新兴 Crypto 套利机会：Tokenized Stock × Hyperliquid HIP-3 永续合约的 RWA 基差套利
核心结论：

2026 年我认为比较值得系统研究的一类新兴套利，是：

做多链上 Tokenized Stock 现货 + 做空 Hyperliquid HIP-3 对应股票永续合约，赚取「永续溢价收敛 + Funding + 不同交易时段造成的价格错位」。

例如：

Long NVDAx 链上 Nvidia Tokenized Stock

Short xyz:NVDA Hyperliquid / TradeXYZ Nvidia Perp

这不是传统意义上的完全无风险套利，更准确地说属于：

delta-neutral cross-venue relative-value arbitrage / soft arbitrage。

我对这个方向的评价是：

新颖程度：★★★★★

竞争程度：★★★

技术门槛：★★★★★

资金门槛：★★

容量：★★★

纯套利程度：★★★

个人量化可执行性：★★★★★

未来 6–12 个月研究价值：★★★★★

机会存在性的置信度：高，约 90%。 \

目前小中等资金仍可能存在可交易 edge 的判断置信度：中高，约 70%。 \

未来能否长期保持高收益的置信度：中等，约 55%–65%。

一、为什么现在会出现这种机会？

核心原因不是“Hyperliquid 有股票”。

真正重要的是：

传统股票正在被拆成多个彼此不完全同步的 24/7 市场。

现在同一只 NVDA 可以同时存在：

NASDAQ NVDA

↓

Tokenized Stock : NVDAx

↓

Hyperliquid HIP-3 : xyz:NVDA

↓

其他股票 Perp / CFD / Tokenized Equity

这些市场：

底层资产类似

但

交易者不同

流动性不同

资金费率不同

预言机不同

交易时间不同

杠杆需求不同

清算机制不同

做市商不同

于是：

一个 Nvidia，实际上出现了多个价格。

这就是套利的来源。

HIP-3 已经允许第三方在 Hyperliquid 上 permissionlessly 部署永续市场，部署者负责市场定义、oracle、杠杆限制等参数。TradeXYZ 已经在其中部署股票、指数、商品等市场；Hyperliquid 当前前端本身已经展示 300+ 个现货和永续市场。

另一方面，xStocks 已将真实股票经济敞口 tokenization。官方资料显示 xStocks 由底层股票 1:1 支持，可以在链上自由转移，并已经进入 Solana、Ethereum、Arbitrum 等多条链；链上二级市场可以 24/7 交易，而一级发行/赎回主要保持 24/5。

事实判断置信度：高，95%。

二、套利最基本的结构

假设：

NVDAx 价格 = 100.00

xyz:NVDA Perp = 100.80

那么：

Perp Premium = 0.80%

建立：

买入 100,000 USDC NVDAx

+

做空 100,000 USDC xyz:NVDA

理论 Delta：

+100,000 NVDA exposure

-100,000 NVDA exposure

≈ 0

于是 Nvidia 本身涨跌不再是主要收益来源。

真正赚的是三个东西：

1. Basis Convergence
2. Funding
3. Microstructure Dislocation

三、第一层收益：Basis Convergence

定义：

S = Tokenized Stock 现货价格

P = HIP-3 Perp 价格

Basis = $P / S - 1$

例如：

S = 100

P = 100.80

Basis = 0.80%

之后价格变成：

S = 103

P = 103.15

此时：

Basis $\approx 0.146\%$

股票涨了 3%。

但我们并不太关心。

因为：

Long Spot：

100 $\rightarrow$ 103

赚 +3

Short Perp：

100.8 $\rightarrow$ 103.15

亏 -2.35

合计：

+0.65

这正好对应：

0.80% - 0.15%

$\approx 0.65\%$

所以：

$\text{Price PnL} \approx \text{Basis}_{\text{entry}} - \text{Basis}_{\text{exit}}$

这是整个策略最重要的第一性原理。

四、第二层收益：Funding

如果 HIP-3 股票 Perp 中：

Long Demand > Short Demand

Perp 很容易长期存在：

Perp > Oracle

进而形成正 Funding。

如果定义：

$\text{Funding} > 0$

=

Long 支付 Short

那么我们的：

Long NVDAx

Short NVDA Perp

不仅赚 Basis 收敛，还能：

持续收 Funding

Hyperliquid 的 funding 机制本身就是用来推动 perpetual 和 underlying/reference price 收敛的；官方定义中 funding 由多空双方直接支付。

因此完整收益是：

PnL

$\approx \text{Basis Convergence}$

+ Funding Received

- Trading Fees

- Slippage

- Market Impact

- Inventory Cost

- Tracking Error

五、真正有意思的地方：这里其实存在三个价格

不要只监控：

HIP-3 Perp

vs

xStock

最好的模型应该监控：

Traditional Stock

Tokenized Stock

HIP-3 Oracle

HIP-3 Perp

定义：

C = 传统股票参考价格

X = xStock

O = HIP-3 Oracle

P = HIP-3 Perp

那么：

P / X

可以进一步拆成：

P / X

=

$(P / O) \times (O / X)$

取 log：

$\log(P/X)$

=

$\log(P/O)$

+

$\log(O/X)$

这个分解极其重要。

因为两个套利来源完全不一样。

第一部分

$\log(P/O)$

代表：

Hyperliquid 永续自身的多空失衡。

主要受到：

Funding

杠杆多头

清算

订单簿

投机资金

影响。

第二部分

$\log(O/X)$

代表：

TradeXYZ oracle 与 Tokenized Stock 市场之间的价格发现差异。

主要受到：

传统市场是否开盘

Tokenized Stock 流动性

HIP-3 oracle mechanism

盘后新闻

周末新闻

做市商库存

影响。

所以：

不能简单地做 Perp - Spot。

应该拆解 edge 来源。

六、为什么盘后和周末尤其值得研究？

这里是整个策略最有价值的地方。

传统 NVDA：

不是 24/7 连续交易

但：

NVDAx 可以链上持续交易

xyz:NVDA 也可以持续交易

xStocks 官方明确描述其链上资产可以在支持的 DeFi 市场全天候交易，但一级发行/赎回仍主要与传统市场的 24/5 周期连接。

TradeXYZ 的关键设计同样是：

让股票 Perp 的 oracle 在传统股票交易时段之外继续工作。

其 2026 年已经把 equity perpetual 的 internal oracle time constant 调整到约 30 分钟，并专门设置 discovery bounds 处理周末、假日等 internal pricing session。

于是周末会形成非常有意思的结构：

Friday NVDA Close

|

| 传统市场关闭

|
 |—— NVDAx 自己价格发现
 |
 |—— xyz:NVDA 自己价格发现

假设周六出现 Nvidia 重大 AI 新闻。

可能出现：

Friday Close = 100

NVDAx = 104

xyz:NVDA = 107

那么：

P/X - 1

≈ 2.88%

这时候传统股票根本不存在可以直接观察的“正确价格”。

两个 crypto-native 市场实际上在：

争夺价格发现权。

这就是新兴 RWA 市场非常特殊的 alpha。

七、但周末套利和正常时段套利必须分开

这是一个非常重要的区别。

正常交易时段

如果：

股票现货 = 100

NVDAx ≈ 100

HIP-3 = 101

因为 xStock 可以通过一级市场及套利商更紧密地锚定真实股票：

Long NVDAx

Short HIP-3

比较接近传统：

cash-and-carry

周末

如果：

Traditional Market Closed

那么：

NVDAx = 104

HIP-3 = 107

你不知道：

Monday 真 NVDA

到底会：

102

104

107

110

所以这时候并非严格无风险套利。

它更接近：

Cross-Venue Relative Value

+

Convergence Trade

这一点的判断置信度：高，95%。

八、为什么 Tokenized Stock 这一腿现在越来越可用了？

这也是为什么我认为这个机会是“2026 新兴机会”，而不是以前就能轻松做的东西。

xStocks 官方已经在 Solana 上重点扩大：

SPYx

QQQx

NVDAx

TSLAx

CRCLx

等资产的链上流动性，并明确表示对这些市场进行了更大规模的流动性部署；Jupiter、Raydium、Kamino 等 DeFi 基础设施已经接入。

而 NVDA 的 HIP-3 perpetual 目前也已经可以直接在 Hyperliquid 的 xyz:NVDA 市场交易。

也就是说：

以前：

股票

↓

证券账户

Crypto perp

↓

Crypto exchange

两套世界完全分离

现在：

Tokenized Stock

↑

DeFi

↑

HIP-3 Perp

开始真正连接起来。

生态成熟度判断置信度：中高，约 80%。

九、实际策略不应该直接用百分比阈值

最简单的策略可能写成：

Basis > 1%

→ 做空 Perp

→ 做多 xStock

这个设计太粗糙。

正确做法应该使用：

Executable Basis

而不是 midpoint basis。

假设交易量 q：

BuyPrice_xstock(q)

ShortPrice_perp(q)

那么：

ExecutableBasis(q)

=

ShortPrice_perp(q)

/

BuyPrice_xstock(q)

- 1

注意：

必须用真实可成交价格。

不能使用：

mid_perp - mid_spot

因为很多 RWA 套利：

屏幕：

+70 bps

真正打进去：

+15 bps

甚至：

-10 bps

十、我会怎样定义完整 Alpha

定义：

EB(q,h)

=

Expected Basis Convergence

+

Expected Funding(h)

-

Execution Cost(q)

-

Inventory Cost

-

Tail Risk Reserve

其中：

Execution Cost

=

Spot Fee

+

Perp Fee

+

Spot Slippage

+

Perp Slippage

+

AMM Price Impact

+

Gas

因此只有：

EB > Threshold

才执行。

比如：

预计 Basis 收敛 +45 bps

预计 Funding +18 bps

Perp fee/slippage -9 bps

xStock fee/impact -11 bps

链上/库存成本 -3 bps

风险缓冲 -5 bps

Expected Net Edge +35 bps

这才是一笔交易。

而不是：

“两个价格差 0.6%，赶紧套利”

十一、一个完整的实际例子

假设：

NVDAx executable ask = 180.00

xyz:NVDA executable bid = 181.08

因此：

Basis = 0.60%

准备交易：

Notional = \$50,000

建立：

Long NVDAx \$50,000

Short NVDA perp \$50,000

24 小时后：

NVDAx = 184.00

Perp = 184.28

Basis：

$184.28 / 184 - 1$

$\approx 0.152\%$

Basis 收益：

60 bps - 15.2 bps

≈ 44.8 bps

如果期间收到：

Funding = 13 bps

那么：

Gross PnL

$\approx 44.8 + 13$

= 57.8 bps

假设：

Trading + Slippage

= 21 bps

最终：

Net PnL ≈ 36.8 bps

\$50,000：

PnL $\approx \$184$

这类机会单笔并不一定非常夸张。

真正价值在于：

NVDA

TSLA

SPY

QQQ

COIN

MSTR

其他股票

指数

商品

国际股票

不断扫描。

最后做成：

RWA basis arbitrage universe。

上面的收益数字是示例而非当前市场实测值。\
示例代表性的置信度：中等，约 60%。

十二、信号模型可以怎么设计？

第一版完全没必要机器学习。

我反而建议：

Signal 1：Robust Basis Z-score

计算：

$b_t = \log(P_{\text{perp}} / P_{\text{xstock}})$

然后：

Median = rolling median(b)

MAD = median(|b - Median|)

定义：

$Z = (b - \text{Median}) / (1.4826 \times \text{MAD})$

相比普通：

mean/std

MAD 对：

周末 jump

新闻事件

流动性瞬间抽空

更稳健。

第一版可以实验：

$Z > 2.5$

且

Funding > 0

且

Executable Edge > 25bps

且

Depth 足够

才：

Long xStock

Short HIP-3

这些阈值是研究初值，不是经过实盘验证的最优参数。\
参数合理性置信度：中低，约 50%。

十三、再加入 Funding 预测

不要只看：

Current Funding

应该预测：

Expected Funding over holding period

例如：

EF_12h

EF_24h

EF_48h

输入可以包括：

current funding

funding EMA

premium index

perp/oracle basis

OI change

long/short aggressor imbalance

CVD

orderbook imbalance

basis momentum

得到：

Expected Carry

然后真正信号变成：

Expected Edge

=

Expected Basis Compression

+

Expected Funding

-

Expected Execution Cost

这比简单 Funding Arbitrage 要强很多。

十四、我尤其建议研究“周末效应”

我会单独建立：

Session Regime

例如：

1 = US Cash Session

2 = Pre-market

3 = After-hours

4 = Overnight

5 = Weekend

6 = Holiday

7 = Earnings Window

然后分别统计：

Basis Mean

Basis Std

Basis Half-life

Funding Mean

Max Adverse Excursion

Convergence Probability

我怀疑真正比较大的 edge 很可能集中在：

Friday Close → Weekend

Sunday → Monday Open

重大新闻后的 0-3h

财报后的非传统交易时段

原因不是简单的季节性，而是：

传统价格锚暂时消失后，不同 24/7 市场独立进行价格发现。

这是我的策略判断而非已验证统计事实。\\

判断置信度：中高，约 70%。

十五、甚至可以研究一个更高级的信号：谁的价格发现更准？

这是我认为最值得深入挖的部分。

例如周日：

Friday NVDA = 100

NVDAx = 104

HIP-3 = 107

Monday open：

NVDA = 105

那么：

$x\text{Stock error} = |104 - 105| = 1$

$\text{HIP3 error} = |107 - 105| = 2$

长期统计：

MondayOpen - FridayClose

vs

Weekend xStock Return

vs

Weekend HIP3 Return

建立：

MondayGap

=

$\beta_1 \times x\text{StockWeekendReturn}$

+

$\beta_2 \times \text{HIP3WeekendReturn}$

+

ϵ

如果发现：

$\beta_1 > \beta_2$

意味着：

xStocks 的价格发现更准确。

于是：

HIP3 偏离 xStock

就是非常有价值的 mean-reversion signal。

反过来也一样。

这就从：

机械套利

升级成：

Price Discovery Arbitrage

十六、数据系统应该怎么搭

第一版至少抓：

Hyperliquid

bid/ask

L2 book

trades

markPx

oraclePx

funding

OI

volume

liquidations

Hyperliquid 官方提供公开 Info API 和 WebSocket 实时数据接口。

xStocks

不要只抓所谓：

NVDAx price

应该模拟真实成交：

Jupiter quote

\$1k

\$5k

\$10k

\$25k

\$50k

然后记录：

effective ask

effective bid

price impact

route

liquidity

xStocks 已接入 Jupiter、Raydium 等 Solana DeFi 基础设施。

Traditional Market

记录：

NVDA spot

pre-market

after-hours

market status

earnings

dividends

splits

halts

最终得到：

Traditional

↓

Tokenized Stock

↓

HIP3 Oracle

↓

HIP3 Perp

四层价格数据库。

十七、实盘最重要的问题：不要临时跨链

真正执行的时候，绝对不要：

发现套利

↓

Hyperliquid 转钱

↓

Bridge

↓

Solana

↓

买 xStock

到这里机会基本没了。

正确模式是：

Solana：

预先放

USDC + xStocks inventory

Hyperliquid：

预先放

USDC margin

发现机会：

同时执行两腿

套利完成后再：

低频 rebalance inventory

因此本质上属于：

inventory-based arbitrage。

和 CEX-DEX 套利非常类似。

十八、执行引擎我会这样设计

状态机：

IDLE

↓

SIGNAL_DETECTED

↓

CHECK_EXECUTABLE_EDGE

↓

RESERVE_INVENTORY

↓

SEND_BOTH_LEGS

↓

HEDGE_MISMATCH

↓

POSITION_OPEN

↓

MONITOR_BASIS

↓

EXIT

↓

REBALANCE

最危险的是：

Leg Risk

比如：

买入 NVDAx 成功

但

Short HIP3 失败

此时瞬间变成：

裸多 NVDA

所以需要：

max_unhedged_time

max_unhedged_notional

IOC

slippage guard

emergency hedge

例如：

max_unhedged_time = 1–3 sec

超过：

立即 unwind

具体秒数取决于 API、网络和盘口数据，需要实测。\\

设计原则置信度：高；1–3 秒参数置信度：中低。

十九、最大的风险不是方向风险

很多人看到：

Long NVDA

Short NVDA

会觉得：

没有风险

这是错误的。

真正的风险是：

1. Basis Expansion

0.5%

→

3%

虽然最终可能收敛，但你可能先爆仓。

2. Oracle Risk

HIP-3 的 oracle 并不等于：

NASDAQ 实时股票价格

尤其是在非传统交易时段。

TradeXYZ 本身明确采用了针对 internal sessions 的 oracle/discovery 机制。

3. Tokenized Stock Tracking Error

你持有的是：

NVDAX

不是 Nasdaq order book 中的一张普通 NVDA 股票。

虽然 xStocks 官方表示其资产由底层证券 1:1 支持并处理公司行为，但其链上价格仍由链上流动性决定。

4. Primary Market Closed

周末：

secondary xStock market

虽然继续交易，但：

issuance / redemption anchor

可能弱于正常交易时间。

这也是周末 Basis 更可能扩大的原因之一。

5. Funding Flip

进入时：

Funding = +0.02%

几小时后：

Funding = -0.04%

那么原本：

收 Funding

变成：

付 Funding

6. Corporate Actions

例如：

dividend

stock split

merger

halt

special dividend

可能造成不同系统处理时间差。

xStocks 使用链上机制反映 dividends、stock splits 等 corporate actions。

7. Liquidation Risk

这是最容易被忽略的。

例如：

Long Spot \$100k

Short Perp \$100k

整体 Delta：

≈ 0

但 Hyperliquid margin account 不知道你的 Solana 上还有 NVDAx。

因此：

HIP3 Perp 单边浮亏

仍然可能把你的：

Short

清算。

所以这种策略：

绝对不应该使用高杠杆。

二十、回测时最容易犯的错误

绝对不要用：

1分钟 close

vs

1分钟 close

进行回测。

会严重高估收益。

正确的数据至少应该达到：

1s

或

tick

因为需要恢复：

bid

ask

depth

AMM quote

funding timestamp

oracle

mark

每笔历史套利信号都应该重新模拟：

如果当时真的下 \$10,000

能不能成交？

而不是：

midpoint 有没有价差？

二十一、我最关心的几个回测指标

不要优先看 Sharpe。

首先看：

Median Net Edge

其次：

P5 Net Edge

然后：

Basis Half-Life

再看：

Max Adverse Basis Excursion

简称可以自己定义：

MABE

例如：

Entry Basis = 0.7%

最终 Exit = 0.1%

但过程中曾扩大到 3.4%

那么这笔套利虽然最终赚钱：

+0.6%

但实际上资金管理非常危险。

二十二、容量测试

应该逐级模拟：

\$1k

\$5k

\$10k

\$25k

\$50k

\$100k

\$250k

计算：

NetEdge(q)

最终很可能出现：

Edge

|

| \

| \

| \

| \

| \

| \

Notional

然后寻找：

q

=

argmax

Expected Dollar PnL

而不是：

看到 80bps

直接打一百万

这类 RWA 市场目前最大的限制之一仍然是不同市场之间的流动性碎片化；tokenized RWA 的流动性问题也一直是这一类别的重要结构性限制。

容量受限判断置信度：高，约 85%。

二十三、这个策略最适合什么资金规模？

我目前会把它定位为：

\$10k–\$50k

非常适合研究和验证

↓

\$50k–\$250k

开始具备比较现实的收益意义

↓

\$250k–\$1m

需要认真研究市场容量和 inventory

\$1m / single name

这不是基于完整市场盘口历史得出的容量结论，而是基于当前市场结构的研究优先级判断。

二十四、为什么我认为这个方向值得你研究？

新市场

市场碎片化

交易基础设施快速增长

套利最舒服的时候往往不是：

市场完全没人

因为那时候根本不能交易。

也不是：

市场完全成熟

因为所有价差都已经被专业 MM 压掉。

比较好的阶段恰恰是：

开始有真实成交量

开始有多个市场

但定价系统尚未完全统一

我认为 Tokenized Equities × HIP-3 正处在这个阶段。

xStocks 在 2026 年继续扩大链上流动性及资产覆盖，而 Hyperliquid HIP-3 同时快速扩展股票、商品、指数等传统资产 perpetual 市场。

这一阶段判断的置信度：中高，约 75%。

二十五、真正应该建立的系统

最终不要只做：

NVDAx vs xyz:NVDA

而应该建立：

RWA Arbitrage Monitor

架构：

Traditional Market

Reference Price

Tokenized Spot HIP-3 Oracle

| HIP-3 Perp

Basis Engine



▼
Expected Net Edge
|

▼
Execution Engine

然后扫描：

NVDA
TSLA
COIN
MSTR
SPY
QQQ
其他股票
全球指数
黄金
原油
其他商品

最终形成的是：

Crypto-Native Global Cross-Asset Arbitrage Engine。

最后的判断

这类策略最吸引我的地方，并不是单纯的：

Funding Arbitrage

而是一个更加结构性的变化：

传统资产正在从“一个主要交易市场”，变成“传统市场 + Tokenized Spot + Crypto Perp + 24/7 Oracle”的多价格体系。

市场一旦碎片化：

价格发现速度不同

+

交易者结构不同

+

杠杆需求不同

+

交易时间不同

+

资金费率不同

就天然容易产生：

Basis

Funding

Lead-Lag

Weekend Dislocation

Oracle Dislocation

Liquidity Dislocation

这些才是真正值得量化研究的 edge。

所以我会把这个方向定义成：

2026–2027 比较值得个人 Quant 提前布局的 RWA Microstructure Arbitrage，而不是已经被完全商品化的 BTC/ETH 跨所 Funding Arbitrage。

综合判断置信度：中高，约 70%。

最值得优先做的 MVP 很明确：

第一阶段只做：

NVDAx

vs

xyz:NVDA

采集 1s 数据

↓

拆解：

Perp / Oracle

Oracle / xStock

Perp / xStock

↓

按：

Cash Session

After Hours

Weekend

Earnings

分别统计

↓

最后做：

Executable Basis

+

Funding

-

Real Slippage

-

Fees

如果这个单品种能够证明存在稳定的净

edge，再横向扩到几十个

RWA

标的。这样比一开始做一个很大的“全市场套利扫描器”更容易真正找到可实盘的 alpha。

笔记 084: 2026-08-17 — Day 13(Week 2 第 6 天):Dune 数据管道上手 + Ga

作者：bamboo | 时间：2026-08-17 14:57 UTC | 字数：3069 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-17 — Day 13(Week 2 第 6 天):Dune 数据管道上手 + Gate-crossex 调研

主题: Week 2 收官主线——Dune 查询平台学习 + 链上数据管道雏形验证 + 候选工具调研

方法: 官方文档直读(docs.dune.com/llms.txt 全套索引)+ 免 key 真实数据源验证 + 群消息/仓库溯源

结论先行: Dune 查询骨架已就绪(等 API key 即跑);真实数据管道已用 Hyperliquid 免 key API 验证跑通;Gate-crossex 确认=既有 CrossEx 记录,但开源仓库当前 404

一、今日任务回顾(昨天笔记的明天计划)

1. 调研 Gate-crossex(确认是否=既有 CrossEx 记录)
2. Dune 实际建查询(API key + 官方示例跑通) 骨架完成,key 未配置
3. v0.2 观察期: 连续跑 2-3 天无噪音后接飞书推送(已在观察,明后天验收)

二、Dune 平台学习(主线,官方文档直读)

学到的核心知识

- DuneSQL: TrinoSQL 兼容方言,门槛低;核心资产是 curated data(跨链标准化的现成表),不用碰原始合约数据
- 三张回测关键表:
 - dex.trades: 全 DEX 逐笔交易(block_date/blockchain/project/token_pair/amount_usd/taker/maker),官方入门示例=按链日成交量
 - prices.hour: 小时级 token 价格(Coinpaprika 前 ~2000 大币 + DEX 衍生价补长尾,按量加权,前向填充 7 天过期)——跨链价差回测的核心数据
 - prices.day / prices.minute: 日/分钟粒度
- API 五件套: executions+results(执行取数)/ query management(建改查询)/ data uploads(上传自定义表)/ materialized views / pipelines
- 免费额度小,执行查询注意 performance 档位

产出(代码)

- scripts/dune_query.py: execute/status/results/run 四个子命令,从 .env 读 DUNE_API_KEY,一键"执行→轮询→拉结果"
- research/dune/queries.md: 3 个回测 SQL 模板
- T1 跨链价差回测(同 token 双链 hourly 价差,LI.FI 候选 #1 的回测核心)
- T2 DEX 日成交量按钮(官方示例)
- T3 单池深挖(收费站/流动性候选)
- 卡点: DUNE_API_KEY 未配置
- 昨天计划写"API key + 官方示例跑通",但 .env 里只有 ICL 三键,没有 Dune key
- 需要用户去 dune.com 右上角 Settings → API Keys 创建(免费版够用),加进 .env 后 python3 scripts/dune_query.py run <query_id> 即可跑
- 今天先完成"建查询"这半边(骨架+模板),执行半边等 key

三、真实数据管道验证(免 key,已跑通)

Hyperliquid 公开 API 无需 key,直接拉 Week 3 回测需要的 SKHX/SKHY 历史数据:

| 数据 | 结果 |

|---|---|

| SKHX 费率 | 168 期/7天,均值 +0.0048%/h = +42.4%/yr(多头付费) |

| SKHY 费率 | 168 期/7天,均值 -0.0015%/h = -13.6%/yr(空头付费) |

| SKHX 1h K线 | 49 根/48h,最新 close 1244.2 |

与 8/13 实测口径一致(SKHX 正费率、SKHY 负费率),数据管道可用

结论重申: 配对双腿都付资金费(合计 ~56%/yr),只适合短期持有,等溢价极端位回归

四、Gate-crossex 调研(确认=既有 CrossEx 记录)

溯源结论

- 确认同一工具: 8/5 群消息(yourQuantGuy 推文,OCR 存档)描述的 Gate CrossEx = 8/16 笔记"待确认"的 Gate-crossex
- 定义: Gate 的跨所统一保证金工具,聚合 币安/OKX/Hyperliquid/Bybit/Kraken/Gate 等
- 解决传统跨所资费套利两大痛点: ① 避免单边爆仓(5x 杠杆下普通方案波动 20% 单边爆仓;CrossEx 两边浮盈亏抵消,唯一风险=两所价差拉开 20%) ② 无需频繁平衡保证金
- 开源物=前端(CrossEx 本身只有 API 无官方 UI),yourQuantGuy 做了开源前端+费率监控+一键开对冲仓位
- 关联: QQQ.BOT v0.0.33 已支持 CrossEx(8/7 群消息)

观察点: 开源仓库当前 404

- GitHub search 能搜到 your-quantguy/gate-crossex,但直接访问返回 404 页(可能改名/私有/删除)
- 影响: 若想接入,需找替代入口(QQQ.BOT 已内置支持是现成路径);不阻塞——我们执行层目前走 HL 手动,候选工具而已
- 待办: 后续遇到群友再分享时追新入口

五、明天计划(8/18)

决策节点: PLAN 里 8/18 = 从 2-3 个候选假设中选出 1 个进回测决赛。今天先把候选清单过一遍,明天定

Dune: 等用户给 API key 后跑通 T1 跨链价差回测

v0.2 观察期验收: 2-3 天无噪音 → 接飞书推送

参考链接

- Dune 官方文档: docs.dune.com/llms.txt(全套索引)
- Dune 入门查询: https://docs.dune.com/learning/how-tos/create-your-first-query.md
- dex.trades 表: https://docs.dune.com/data-catalog/curated/dex-trades/evm/dex-trades.md
- prices.hour 表: https://docs.dune.com/data-catalog/curated/prices/prices_hour.md
- CrossEx 群消息: research/group-shares/tg_img_ocr_20260806.txt:3278
- CrossEx 资源跟踪: research/group-shares/RESOURCES_TRACKING.md #5

tags: #套利 #Dune #数据管道 #Week2 #CrossEx

09 风险与安全 (7 篇)

笔记 085: binance三角套利延迟优化

作者: Zhihui Zhang | 时间: 2026-08-16 15:50 UTC | 字数: 2351 | 评分: 89

来源: https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

binance三角套利延迟优化

优化分三层递进：网络层 → 应用层 → OS/内核层，每层改动逐步叠加。

一、网络层优化（改动最小，收益确定）

1.1 确定最优可用区（AZ）

Binance 的 AZ 不公开，需要实测。在同一 Region 的三个 AZ 各开一台 EC2，分别测量延迟：

```
bash
for az in ap-northeast-1a ap-northeast-1b ap-northeast-1c; do
# 在该 AZ 的机器上执行
nping -c 100 --tcp -p 443 stream.binance.com
nping -c 100 --tcp -p 443 data-stream.binance.com
nping -c 100 --tcp -p 443 data-stream.binance.vision
done
```

效果：RTT 从未精调的 $\sim 1\text{-}2\text{ms}$ 降至 $\sim 0.5\text{ms}$ (min 0.513ms, max 1.009ms)

1.2 筛选最优 IP 并硬编码

同一个 hostname 解析出多个 IP，逐一测量选最快的：

```
bash
dig stream.binance.com +short
dig data-stream.binance.com +short

for ip in <ip1> <ip2> <ip3>; do
nping -c 50 --tcp -p 443 $ip
done
```

硬编码到 /etc/hosts：

```
<best_ip> stream.binance.com
<best_ip> data-stream.binance.com
```

1.3 协议选择：FIX API

如果有 FIX API 权限，行情和下单都走 FIX，去掉 WebSocket 中间层开销。

二、应用层优化（收益最大）

2.1 语言迁移路径

版本	实现方式	测试网捕获率	问题
Elixir 1.0	纯 Elixir	7/8 = 87.5%	CPU 100%，事件积压导致 OOM
Elixir+C++ 1.1	套利计算用 C++ port	11/12 = 91.7%	JSON 序列化开销
Full C++ 2.x	全量 C++ 重写	30/31 = 96.8%	—

路径计算耗时：40+ 秒 → 3 秒

2.2 C++ 关键优化点

Lock-free 设计：无锁并发

热路径零内存分配：避免 malloc 开销

激进编译优化：-O3 -march=native -fno

移除安全加固：去掉 Spectre/Meltdown mitigation（需评估安全风险）

2.3 去容器化

不用 Docker，改用 Nix 直接部署裸进程，消除容器层开销。

三、OS/内核层调优

五项关键调优（参考 Harjus 的 Terraform 配置）：

```
bash
```

1. 实时内核

```
sudo amazon-linux-extras install kernel-rt
```

2. Socket busy polling（消除中断唤醒延迟）

```
echo 50000 > /proc/sys/net/core/busy_read
echo 50000 > /proc/sys/net/core/busy_poll
```

3. 禁用 iptables

```
sudo systemctl stop iptables
```

4. 禁用中断合并 (ENA 网卡)

```
sudo ethtool -C eth0 adaptive-rx off
```

```
sudo ethtool -C eth0 rx-usecs 0
```

5. 禁用 Spectre/Meltdown 缓解

GRUB 添加 : mitigations=off

6. 应用 tuned profile

```
sudo tuned-adm profile latency-performance
```

四、验证方法

每步优化后测量 :

```
bash
```

RTT 验证

```
nping -c 1000 --tcp -p 443 <best_ip>
```

应用层 : 测试网机会捕获率

成功捕获 / 检测到的总机会数

五、优化路线图

AZ选择 → IP选择 → FIX协议 → C++/Rust重写 → 编译优化 → 去容器 → 内核调优

↓ ↓ ↓ ↓ ↓ ↓ ↓

1-2ms → 0.5ms 减少协议层 40s→3s计算 +10-20% 消除开销 降低抖动

笔记 086: Day 12 · 把机会评分变成可审计的决策记录

作者 : Junfan Chen | 时间 : 2026-08-16 15:54 UTC | 字数 : 1655 | 评分 : 88

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 · 把机会评分变成可审计的决策记录

日期 : 2026-08-16

主题 : 从信号排序推进到可解释、可复盘的决策记录

今日整理 / 推进

前几天已经把研究链路从 collector、告警和执行前闸门逐步拆开 : 先确认数据质量, 再判断机会是否值得关注, 最后才考虑是否允许占用真实资金。今天继续整理一个配套问题 : 当多个候选同时出现时, 系统应如何解释为什么选中一个、放弃另一个 ?

今天没有新的链上交易、代码执行、报价采样或收益数据, 因此本次打卡记录的是指标设计和复盘框架, 不把计划写成已经完成的实操结果。

1. 分数不能替代证据

可以为候选机会计算一个排序分数, 但分数只负责排列优先级, 不能覆盖关键证据。每条候选仍应保留 :

text

signal_key

quote_age_ms

executable_bps

capital_lock_seconds

path_type

failure_risk

exit_route_available

如果某个字段缺失, 应该降低置信度或直接进入人工复核, 而不是用默认值把候选伪装成可比较数据。特别是跨链路径, 时间和退出能力往往比表面边际更能决定最终风险。

2. 将排序分数拆成可解释分项

初步把排序逻辑拆成四类因素 :

text

edge_score

freshness_score

capital_efficiency_score

execution_confidence_score

edge_score 反映扣除已知成本后的边际；freshness_score 反映报价距离失效还有多久；capital_efficiency_score 关注资金占用时间；execution_confidence_score 则记录路径、余额、授权和退出方案是否完整。最终权重应通过 paper decision 的历史记录复盘，而不是先设一个看似精确的固定系数。

3. 记录“为什么没有选它”

复盘不能只保存最终选中的机会，还要保留被淘汰候选及原因，例如：

```
text
lower_net_edge
stale_quote
capital_lock_too_long
non_atomic_path
exit_route_missing
insufficient_evidence
```

这样才能判断策略是在正确地过滤风险，还是因为某个阈值过严而错过了大量仍有价值的信号。拒绝记录也应带上当时的输入快照，避免事后用最新报价倒推当时的判断。

4. 决策记录需要版本化

同一候选在不同策略版本下可能得到不同结论，因此 decision record 还应保存：

```
text
strategy_version
cost_model_version
pricing_snapshot_at
decision_at
decision
```

如果未来调整 gas、桥费、安全垫或延迟惩罚，可以用旧版本重新回放，区分“市场变化”和“策略变化”造成的结果差异。这比只保存一个最终布尔值更适合长期迭代。

关键判断

套利系统的可用性不只取决于能否找到正边际，更取决于能否解释每次选择和拒绝。排序分数是导航，不是证据；决策记录必须保留输入、版本、时间和拒绝原因。在没有真实执行样本前，优先建设可审计的 paper decision 流程，比追求虚构的收益率更稳妥。

下一步

- [] 定义候选评分各分项及缺失字段处理规则
- [] 保存入选与淘汰候选的同一时刻输入快照
- [] 为策略、成本模型和价格快照增加版本号
- [] 用 paper decision 统计不同拒绝原因的占比
- [] 检查排序是否被单一指标长期主导
- [] 没有真实样本前，不把排序结果描述为实际盈利

笔记 087: 2026-08-16 学习打卡

作者：UU20220213 | 时间：2026-08-16 15:55 UTC | 字数：1208 | 评分：84

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-16 学习打卡

今日完成

- 分析 Astro 套利工具的定位与风险：它属于人工配置策略、程序自动监控和开平仓的执行工具，不等于自动发现必赚机会。公开仓库主要是文档、安装脚本和 SDK，核心交易程序以 Docker 镜像等形式交付，不能因为有 GitHub 仓库就认定核心完全开源或已经过安全审计。
- 理解跨交易所套利不是原子交易。两边订单可能出现一边成交、另一边未成交或部分成交；自动补救也可能失败，因此不能把自动化工具当成稳赚或无人值守系统。
- 学习 Maker 与 Taker：Taker 主动吃单、成交更快但手续费通常更高；Maker 挂单等待、费用可能更低，但存在不成交、部分成交和单腿风险。套利不能只比较手续费，还要权衡成交确定性。
- 讨论是否自己编写类似程序，明确当前更适合先写只读监控，而不是直接写全自动交易程序。只读监控只获取行情、计算成本和发出提醒，不持有交易或提现权限；自动下单还需要处理超时、重复下单、部分成交、精度、限频、断线、仓位失衡和紧急停止。
- 学习跨交易所监控器应比较低价交易所的卖盘 Ask 与高价交易所的买盘 Bid，而不是比较两个页面的最新成交价。方向应明确为低价端买入、高价端卖出。
- 掌握净价差公式：净价差率 = 毛价差率 - 两边手续费 - 两边滑点 - 资金费率影响 - 其他成本。完成例题：0.35% - 0.10% - 0.08% -

0.02% = 0.15% ; 投入 20,000 USDT 时, 理论净利润为 30 USDT。

7. 学习盘口深度与可执行数量。卖一价和买一价只对应有有限数量; 金额增大后会吃到更差档位, 必须按各档数量计算加权平均买价和卖价, 不能用首页价格乘以任意金额。

今日关键认识

自己编写不等于天然安全; 优点是逻辑可见, 缺点是所有程序错误都要自己承担。当前应先验证只读监控和成本计算, 再考虑模拟、半自动及极小资金测试。

套利工具只能提高监控和执行效率, 不能消除单腿、滑点、费用、接口异常和程序错误。

真正的机会必须同时满足: 方向正确、盘口有足够数量、按加权成交价计算后净利润仍为正, 并且风险收益值得执行。

学习证据

对照 Astro 的项目说明、GitHub、安装脚本和版本记录完成工具风险分析。

独立完成净价差率 0.15% 和 20,000 USDT 对应 30 USDT 理论净利润的计算。

能说明监控器为什么应比较 Ask 与 Bid, 以及交易规模增大后为什么必须计算盘口加权平均价。

未完成与下一步

下一步使用模拟盘口制作一张跨交易所机会核算表, 记录买入档位、卖出档位、可执行数量、加权平均成交价、Maker/Taker 手续费、滑点、预估净利润和单腿处理方案。继续保持只读研究, 不连接主账户, 不进行真实资金自动交易。

笔记 088: 今天最大的进展, 是把研究从“固定历史路径是否还能成立”, 推进到了“出现信号的新候选路径, 是否值得独立

作者: Quinn | 时间: 2026-08-17 02:35 UTC | 字数: 888 | 评分: 64

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天最大的进展, 是把研究从“固定历史路径是否还能成立”, 推进到了“出现信号的新候选路径, 是否值得独立验证”。

前面完成了 P7、P8、P9:

P7: 在真实 local Fork 中, 将 V3→V3→V4 的 callback、settlement、native ETH take、WETH 会计和偿债顺序串起来;

P8: 记录到达外层偿债失败边界的 fail-closed attempt Gas;

P9: 把 native ETH 与 WETH 的会计边界写成显式、可测试的安全 policy。

关键结论依旧没有变:

中间换币成功

≠ 套利成功

拿到 native ETH

≠ 可以偿还全部债务

只有完成所有偿债后,

终点资产仍高于起点,

才可能是可执行利润。

今天的 bounded Paper Trading 还发现了一条新的受控候选路径: 3 个固定 snapshot 中有 2 次出现 PAPER_TRADE_CANDIDATE, 但另有 1 次 UNKNOWN_QUOTE_FAILURE。

所以不能因为“2 次看起来有利润”就下结论。当前已经启动 P11, 对这条候选路径单独进行:

\

quote failure diagnostics

→ callback / direction inventory

→ 独立 local-Fork lifecycle probe

→ 独立 settlement、repayment 与 Gas evidence

今天的工程收获是:

候选信号不是结论，
旧路径的验证证据也不能外推到新路径。
每条路径都必须独立回答：

资产能否闭环？
债务能否偿还？
失败会不会原子回滚？
Gas 和收益证据是否路径专属？
全程保持只读、受控 manifest、temporary local Fork；没有私钥、签名、授权、广播或真实资金。

笔记 089: Day 11 (8/17) 打卡

作者：littlefyer007-igtm | 时间：2026-08-17 12:23 UTC | 字数：450 | 评分：48
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 (8/17) 打卡

今日完成

捡尸体策略学习

- 理解废弃项目残留价值的机会与风险
- 建立了6项检查清单：停运时间/资金进出/合约开源/审计/能否转出/社区死寂

实战检验 (jaredfromsubway蜜罐案)

- 用6项清单跑了一遍真实蜜罐案例
- 前3项主观判断正确避开了陷阱
- 但发现关键问题：6项清单里只有前2项是真检查

重要认知升级

- 发现自己的判断框架有一半是纸面检查
- 避开蜜罐靠的是运气（主观判断蒙对），不是能力
- 第3项读代码、第5项验证转出，我都做不到=安慰剂

AI与人的边界思考

- 明确了AI能代替执行，不能代替判断
- 我（AI）可以帮你读代码查数据，但交易决定必须自己长出来
- 否则自己就变成只会执行命令的机器人

核心收获

今天最有价值的不是学策略，而是发现自己判断框架里的一半是假检查。真正的成长=发现我以为我懂其实我做不到。

明天

开始学读Solidity合约代码（最紧急缺口）

笔记 090: 学习笔记：模块 5 第 2 天，安全执行框架

作者：Milli | 时间：2026-08-17 14:12 UTC | 字数：763 | 评分：55
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习笔记：模块 5 第 2 天，安全执行框架

第 13 天，阶段三：捕获、实施与复盘。主题是把「能跑」变成「敢跑」，权限最小化、session key、白名单、限额四道闸门。产出 strategies/security.md，新增 87 行安全执行框架，是明天 Paper Trading 的前置。

一句话总结：签名是一次性授权，approve 是持续性授权，私钥是终极授权，三者都要最小化。安全执行框架就是在 executor.py 链路外面套上白名单、限额、dry-run 三道闸门，让「会执行」升级成「安全地执行」。

威胁模型按执行链路拆攻击面：1 私钥存储，明文、复用、冷热不分，资金全丢；2 授权 approve，无限额度或授权错合约，余额被抽走；3 交易构造，恶意 to/data 钓鱼报价，签了不该签的交易；4 广播，无审查即发，滑点扩大或交易失败；5 合约信任，依赖未审计桥合约，漏洞直接吞钱。

五道防线：1 权限最小化，只授权本次报价金额，不设无限额度，executor.py 的 build_approve_tx 已实现，approve 目标是 quote.estimate.approvalAddress (LI.FI Diamond)，native 币无需 approve，进阶可 approve(0) 回收授权；2 session key 隔离，不给主私钥永久权限，单独派生执行专用热钱包，只放够跑当次策略的资金，主钱包金库分离，类比 Safe Session

笔记 091: Day 12 · 把机会评分变成可审计的决策记录

作者: Junfan Chen | 时间: 2026-08-17 14:20 UTC | 字数: 1655 | 评分: 88

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 · 把机会评分变成可审计的决策记录

日期: 2026-08-17

主题: 从信号排序推进到可解释、可复盘的决策记录

今日整理 / 推进

前几天已经把研究链路从 collector、告警和执行前闸门逐步拆开: 先确认数据质量, 再判断机会是否值得关注, 最后才考虑是否允许占用真实资金。今天继续整理一个配套问题: 当多个候选同时出现时, 系统应如何解释为什么选中一个、放弃另一个?

今天没有新的链上交易、代码执行、报价采样或收益数据, 因此本次打卡记录的是指标设计和复盘框架, 不把计划写成已经完成的实操结果。

1. 分数不能替代证据

可以为候选机会计算一个排序分数, 但分数只负责排列优先级, 不能覆盖关键证据。每条候选仍应保留:

text

signal_key

quote_age_ms

executable_bps

capital_lock_seconds

path_type

failure_risk

exit_route_available

如果某个字段缺失, 应该降低置信度或直接进入人工复核, 而不是用默认值把候选伪装成可比较数据。特别是跨链路径, 时间和退出能力往往比表面边际更能决定最终风险。

2. 将排序分数拆成可解释分项

初步把排序逻辑拆成四类因素:

text

edge_score

freshness_score

capital_efficiency_score

execution_confidence_score

edge_score 反映扣除已知成本后的边际; freshness_score 反映报价距离失效还有多久; capital_efficiency_score 关注资金占用时间; execution_confidence_score 则记录路径、余额、授权和退出方案是否完整。最终权重应通过 paper decision 的历史记录复盘, 而不是先设一个看似精确的固定系数。

3. 记录“为什么没有选它”

复盘不能只保存最终选中的机会, 还要保留被淘汰候选及原因, 例如:

text

lower_net_edge

stale_quote

capital_lock_too_long

non_atomic_path

exit_route_missing

insufficient_evidence

这样才能判断策略是在正确地过滤风险, 还是因为某个阈值过严而错过了大量仍有价值的信号。拒绝记录也应带上当时的输入快照, 避免事后用最新报价倒推当时的判断。

4. 决策记录需要版本化

同一候选在不同策略版本下可能得到不同结论, 因此 decision record 还应保存:

text

strategy_version

cost_model_version
pricing_snapshot_at
decision_at
decision

如果未来调整 gas、桥费、安全垫或延迟惩罚，可以用旧版本重新回放，区分“市场变化”和“策略变化”造成的结果差异。这比只保存一个最终布尔值更适合长期迭代。

关键判断

套利系统的可用性不只取决于能否找到正边际，更取决于能否解释每次选择和拒绝。排序分数是导航，不是证据；决策记录必须保留输入、版本、时间和拒绝原因。在没有真实执行样本前，优先建设可审计的 paper decision 流程，比追求虚构的收益率更稳妥。

下一步

- [] 定义候选评分各分项及缺失字段处理规则
- [] 保存入选与淘汰候选的同一时刻输入快照
- [] 为策略、成本模型和价格快照增加版本号
- [] 用 paper decision 统计不同拒绝原因的占比
- [] 检查排序是否被单一指标长期主导
- [] 没有真实样本前，不把排序结果描述为实际盈利